



1ST EDITION

Data Science with .NET and Polyglot Notebooks

Programmer's guide to data science using ML.NET,
OpenAI, and Semantic Kernel

A decorative orange geometric shape in the bottom left corner, consisting of several parallel lines forming a stylized, abstract shape.

MATT ELAND

Data Science with .NET and Polyglot Notebooks

Programmer's guide to data science using ML.NET, OpenAI, and Semantic Kernel

Matt Eland



Data Science with .NET and Polyglot Notebooks

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Kunal Sawant

Publishing Product Manager: Debadrita Chatterjee

Book Project Manager: Manisha Singh

Senior Editor: Esha Banerjee

Technical Editor: Jubit Pincy

Copy Editor: Safis Editing

Proofreader: Esha Banerjee

Indexer: Subalakshmi Govindhan

Production Designer: Joshua Misquitta

DevRel Marketing Coordinator: Sonia Chauhan

First published: August 2024

Production reference: 1230824

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK

ISBN 978-1-83588-296-2

www.packtpub.com

To Jon, Diego, Brett, Aleksei, Michael, Luis, Cesar, Bruno, and all the others who made the things described in this book possible.

To Sam and Sadukie, who encouraged, supported, and mercilessly heckled me as I began to publicly teach the topics captured in this book.

To Heather, for her endless love and patience, and for telling me to “just go get a master’s degree already!”

Contributors

About the author

Matt Eland is a senior software engineering and data science consultant at Leading EDJE in Columbus, Ohio. He loves sharing his journey by teaching software engineering, AI, and data science concepts in the most engaging ways possible. Matt has used machine learning to settle debates over whether certain movies are Christmas movies, reinforcement learning to create digital attack squirrels, data analytics to suggest improvements to his favorite TV show, and AI agents to play board games and create an AI agent with the personality of a dog. He is the author of multiple books and courses, helps organize a user group and regional conferences, holds a Master's of Science in Data Analytics, and is a 2x Microsoft MVP in AI and .NET.

Thank you to my friends and family for their continued support, as well as my coworkers at Leading EDJE and my client sites who smile and nod at my evening and weekend writing endeavors. Special thanks to Heather, Sam, Sadukie, Eddie, Victor, Mike, James, Kanan, and Stephanie for celebrating little milestones with me. Extra special thanks to Sam Gomez and Sam Nasr for being my fellow MVPs and technical reviewers. Finally, thank you to Debadrita, Esha, and Manisha for your work on our second project together at Packt.

About the reviewers

Sam Nasr has been a software developer since 1995, focusing primarily on Microsoft technologies. He is a Senior Software Engineer with NIS Technologies where he consults and teaches clients about the latest .NET technologies. Sam has achieved multiple certifications from Microsoft, such as MCSA, MCAD, MCTS, and MCT, and he has been the leader of the Cleveland C# since 2003. He also holds leadership roles for the .NET Study Group and Azure Cleveland User Group.

When not coding, Sam loves spending time with his family and friends or volunteering at his local church. You can learn more about Sam by visiting <https://linktr.ee/samnasr>.

Samuel Gomez has worked in software development for 15+ years (mostly Microsoft technologies). He is deeply passionate about the problem-solving aspect of his work. Recently, he has dedicated himself to exploring AI and machine learning technologies and has been working on understanding how these technologies can be applied to different aspects of our lives.

Beyond coding, Sam enjoys spending time with his family. As a soccer enthusiast, he loves to play, watch, and coach the sport.

Table of Contents

Preface

xv

Part 1: Data Analysis in Polyglot Notebooks

1

Data Science, Notebooks, and Kernels 3

Exploring the field of data science	3	Extending notebooks	
The rise of big data	4	with kernels	16
Data analytics	4	Polyglot Notebooks and .NET	
Machine learning	5	Interactive	17
Artificial intelligence	8	Summary	19
Data science notebooks and		Further reading	20
Project Jupyter	13		

2

Exploring Polyglot Notebooks 21

Technical requirements	21	Declaring classes and methods	33
Installing Polyglot Notebooks	22	Declaring methods	33
Creating your first notebook	24	Declaring classes	36
Executing notebook cells	27	Working with other languages	37
Adding code cells	29	Sharing variables	
Working with variables	30	between languages	39
The Variables view	31	Exporting variables	40
Markdown cells	32		

Troubleshooting notebook execution	43	Issues and the Polyglot Notebooks repository	45
Resolving compiler errors	43	Summary	46
Problems with notebook execution	44	Further reading	47
Diagnostic output for Polyglot Notebooks errors	45		

3

Getting Data and Code into Your Notebooks 49

Technical requirements	49	Getting JSON data with PowerShell	66
Importing code and NuGet packages	50	Building DataFrames from objects	69
Importing code files	50	Connecting to databases with SQL	71
Importing NuGet packages	56	Connecting to a SQL database	72
Importing project files	61	Executing SQL from SQL kernels	73
Reading CSV data	61	Sharing SQL results with other kernels	74
Understanding CSV data	61	Alternative ways of connecting to the Database	77
Reading CSV data into a DataFrame	62	Querying Kusto clusters with KQL	77
Specialized CSV loading scenarios	63	Summary	79
Troubleshooting CSV loading errors	64	Further reading	80
Loading TSV and other delimited file formats	65		

4

Working with Tabular Data and DataFrames 81

Technical requirements	81	Rows	89
Understanding data cleaning and data wrangling	82	Getting and setting cell values	89
Where unclean data comes from	82	Iterating over rows	90
The impact of unclean data	83	Working with columns	91
Data cleaning and data wrangling	84	Columns	91
Working with DataFrames in C#	85	Analyzing columns	93
Viewing and sampling data	86	Removing columns	95
		Renaming columns	96

Adding a new column	96	Merging DataFrames	103
Handling missing values	98	Filtering DataFrames	106
Sorting, filtering, grouping, and merging data	100	DataFrames in other languages	108
Sorting DataFrames	100	Summary	109
Grouping and aggregating DataFrames	101	Further reading	109

5

Visualizing Data 111

Technical requirements	111	Creating a box plot with ScottPlot	120
Understanding exploratory data analysis	112	Performing univariate analysis with Plotly.NET	123
Data visualization's role in exploratory data analysis	112	Plotly and Plotly.NET	123
Descriptive statistics for EDA	114	Box plots in Plotly.NET	124
Extracting insights with descriptive statistics	114	Violin plots with Plotly.NET	127
Using DataFrame.Description to generate descriptive statistics	114	Histograms with Plotly.NET	129
Descriptive statistics with MathNet.Numerics	117	Summary	131
		Further reading	132

6

Variable Correlations 133

Technical requirements	133	3D scatter plots with Plotly.NET	142
Performing multivariate analysis with Plotly.NET	134	Identifying variable correlations	144
Loading data and dependencies	134	Calculating variable correlations	145
Multivariate analysis with box and violin plots	135	Building feature correlation matrixes	146
Plotting multiple values with scatter plots	137	Summary	150
Adding color to a scatter plot	141	Further reading	150

Part 2: Machine Learning with Polyglot Notebooks and ML.NET

7

Classification Experiments with ML.NET AutoML 153

Technical requirements	153	Training a classification model with ML.NET AutoML	163
Understanding machine learning	154	Evaluating binary classification models	166
Supervised learning	154	Evaluating our model	169
Classification and regression	155	Calculating feature importance	170
Introducing ML.NET and AutoML	156	Predicting values with binary classification models	172
Understanding AutoML	157	Summary	174
AutoML and data pre-processing	158	Further reading	175
Creating training and testing datasets	159		

8

Regression Experiments with ML.NET AutoML 177

Technical requirements	177	Interpreting cross-validation results	187
Understanding regression	178	Evaluating regression metrics	189
Our regression task	178	Predicting values for outliers	191
Regression as a numerical formula	178	Applying PFI to regression models	194
Our regression dataset	180	Applying a regression model	195
Performing a regression experiment	182	Summary	199
Understanding cross-validation	185	Further reading	199

9

Beyond AutoML: Pipelines, Trainers, and Transforms 201

Technical requirements	202	Features and pipelines	203
Performing regression without AutoML	202	Creating an AutoML pipeline	206

Controlling AutoML pipelines	209	Customizing the hyperparameter tuner	216
Customizing the Featurizer	209	Scaling numeric columns	219
Customizing the model trainer selector	211	Selecting regression algorithms	221
Customizing hyperparameter tuning	212	Selecting binary classification algorithms	222
Understanding the search space	212	Summary	224
Customizing the search space	215	Further reading	225

10

Deploying Machine Learning Models 227

Technical requirements	227	Using the PredictionEnginePool	237
Introducing our multi-class classification model	228	Understanding model performance, data drift, and MLOps	238
Training our model	229	Detecting model drift	240
Evaluating multi-class classification models	230	MLOps and updating models	240
Generating test predictions	232	Surveying additional ML.NET capabilities	242
Exporting ML.NET models	234	ONNX and TensorFlow models in ML.NET	243
Hosting ML.NET models in ASP.NET web applications	235	Summary	243
Configuring a PredictionEnginePool	235	Further reading	244

Part 3: Exploring Generative AI with Polyglot Notebooks

11

Generative AI in Polyglot Notebooks 249

Technical requirements	250	Deploying models on Azure OpenAI Service	254
Understanding Generative AI	250	Getting access credentials for Azure OpenAI	257
Deploying generative AI models on Azure	251	Connecting to an Azure OpenAI Service	258
Creating an Azure OpenAI Service	252		

Chatting with a deployed model	259	Using text embeddings	264
Customizing model behavior with prompt engineering	262	Generating images with DALL-E	266
Zero-shot, one-shot, and few-shot inferencing	263	Summary	270
		Further reading	271

12

AI Orchestration with Semantic Kernel		273	
Technical requirements	274	Accepting KernelFunction parameters	284
Understanding RAG and AI orchestration	274	Defining a memory function	285
Introducing Semantic Kernel	276	Calling multiple functions using plugins	286
Chatting with Semantic Kernel functions	277	Examining FunctionResult objects	288
Building the Kernel	277	Azure OpenAI content filtering	290
Creating a prompt function	278	Handling complex requests with planners	291
Adding memory to Semantic Kernel	281	Knowing where to go from here	295
Defining complex functions	283	Summary	297
Creating functions from methods	283	Further reading	297

Part 4: Polyglot Notebooks in the Enterprise

13

Enriching Documentation with Mermaid Diagrams			301
Technical requirements	301	Communicating data with	
Introducing Mermaid diagrams	302	Entity Relationship Diagrams	309
Communicating logic		Communicating behavior	
with flowcharts	303	with state diagrams	313
Communicating structure		Communicating flow	
with class diagrams	306	with sequence diagrams	316

Communicating workflow with Git graphs	321	Summary	324
		Further reading	324

14

Extending Polyglot Notebooks 325

Technical requirements	325	Exploring magic commands	331
Understanding default formatting behavior	326	Creating a Polyglot Notebook extension	334
Default object formatting	327	Working with parameters	335
Default collection formatting	328	Invoking code on kernels	339
Styling output with custom formatters	329	Summary	343
		Further reading	343

15

Adopting and Deploying Polyglot Notebooks 345

Technical requirements	345	Deploying Polyglot Notebooks to GitHub Codespaces	355
Integrating Polyglot Notebooks into your day job	346	Configuring GitHub codespaces	356
Enabling rapid experimentation	346	Creating a codespace on GitHub	357
Supporting AI and analytics workloads	347	Advancing into machine learning and AI	360
Assisting testing workloads	348	Adding data science to your day job	361
Training new team members with Polyglot Notebooks	349	Getting into data science	363
Sharing Polyglot Notebooks with your team	351	Succeeding in data science	365
Integrating Polyglot Notebooks into Jupyter or JupyterLab	352	Summary	366
Storing Notebooks in source control	354	Further reading	367

Index 369

Other Books You May Enjoy 380

Preface

Polyglot Notebooks and its .NET Interactive kernels let you mix together code and markdown cells in an interactive and highly visual notebook experience. By supporting rich documentation and putting the results of each code cell below the cell, Polyglot Notebooks makes it easy for developers to quickly execute code and see results, then make adjustments and try again.

This makes Polyglot Notebooks ideal for experimentation, documentation, and iterative and analytical tasks such as data analysis, machine learning, and artificial intelligence applications.

This book will teach you the Polyglot Notebooks environment and then use that as a basis to explore and learn data analysis, machine learning, and generative AI.

By the time you complete this book, you'll have learned the basics of machine learning in .NET with ML.NET, data analysis and visualization with DataFrames and Plotly.NET, generative AI with Azure OpenAI and Semantic Kernel, and how these things can tie together to improve and expand your capabilities as a growing developer or data scientist.

Who this book is for

This book is for developers who are curious about augmenting their existing .NET development skills with new AI and machine learning capabilities and growing their knowledge and career prospects in the process.

This book teaches data analytics, machine learning, and generative artificial intelligence to experienced software engineers with .NET backgrounds through a series of small targeted experiments in an interactive notebook environment.

If you're a .NET developer wanting to learn machine learning and AI, this book will help you expand your skills into those new areas.

If you're thinking about making the same transition I did and specializing in AI and Machine Learning or even becoming a data scientist, this book will help you as you begin your journey into the unknown by starting you in familiar ground with familiar tools and languages.

This book is also for engineers and engineering managers who are looking to build interactive documentation for their development teams, marry together code and documentation, and better equip other developers for working with their organization's code.

If you're looking to learn something new, find a new way of writing .NET code, gain some new tools in building applications, explore new ways of sharing code and concepts, this book will help you on that journey.

What this book covers

Chapter 1, Data Science, Notebooks, and Kernels, introduces the idea of data science and interactive notebooks – and the scenarios notebooks help developers, data analysts, and data scientists tackle.

Chapter 2, Exploring Polyglot Notebooks, introduces the Polyglot Notebooks environment and .NET Interactive kernel to the reader by showing how interactive code cells and variable sharing work.

Chapter 3, Getting Data and Code into Your Notebooks, covers different ways of pulling data into a notebook environment including importing from CSV and TSV files, executing SQL and KQL queries, and pulling data in from external APIs.

Chapter 4, Working with Tabular Data and DataFrames, explores data analysis, data cleaning, and feature engineering using the ML.NET `DataFrame` object.

Chapter 5, Visualizing Data, shows how variable distributions can be explored, both in terms of descriptive statistics and in terms of various distribution plots in Plotly.NET and ScottPlot.

Chapter 6, Variable Correlations, takes data visualization to the next level by exploring how variable relationships can be visualized and introduces the idea of correlation scores and correlation matrixes.

Chapter 7, Classification Experiments with ML.NET AutoML, introduces machine learning through ML.NET by using ML.NET's automated machine learning capabilities to perform our first classification experiment.

Chapter 8, Regression Experiments with ML.NET AutoML, expands our exploration into ML.NET by covering the prediction of numerical values with regression.

Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms, moves away from automated machine learning and shows how ML.NET can be customized by using pipelines, transforms, model trainers, and hyperparameter tuning.

Chapter 10, Deploying Machine Learning Models, concludes our exploration of ML.NET by showing how models can be saved and loaded and embedded into other applications, such as an ASP .NET web application.

Chapter 11, Generative AI in Polyglot Notebooks, introduces generative AI and prompt engineering by using Azure OpenAI models for chat completions, image generation, and text embeddings.

Chapter 12, AI Orchestration with Semantic Kernel, expands on our generative AI knowledge by introducing retrieval-augmented generation (RAG), AI orchestration, and Microsoft Semantic Kernel to achieve complex tasks with AI agents.

Chapter 13, Enriching Documentation with Mermaid Diagrams, shows how Mermaid diagrams serve as simple maintainable markdown-based diagrams that help illustrate common engineering workflows.

Chapter 14, Extending Polyglot Notebooks, talks about how the Polyglot Notebooks experience can be customized and extended by providing custom formatters and magic commands.

Chapter 15, Adopting and Deploying Polyglot Notebooks, discusses ways of integrating Polyglot Notebooks into your workflows and sharing notebooks with others – including deploying them to a Jupyter Notebook server or hosting them online in a GitHub Codespace.

To get the most out of this book

This book is intended for software developers familiar with the basics of .NET development in C# trying to learn more about data science in the .NET ecosystem. The ideal reader will be familiar with basic programming concepts and syntax related to C# and curious about learning more about data science.

Although the book covers some F#, I do not expect most readers to be familiar with the language and its syntax, so all F# syntax will be explained.

The following software has been covered in the book:

- VS Code
- .NET (C# and some F#)
- ML.NET
- Other languages covered in brief: PowerShell, SQL, and KQL

And, you can use any of the following operating systems for this book:

- Windows
- MacOS
- Linux

In order to run code online in GitHub Codespaces, you will need to create a free GitHub account.

Chapters 11 and 12 refer to resources available in Azure and OpenAI. These resources may involve minimal costs and require creating an account. However, these chapters can still be understood and experienced without running the code.

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks>. If there's an update to the code, it will be updated in the GitHub repository.

This book's GitHub repository also features GitHub Codespaces that allow you to execute the code in your browser using a GitHub account. Using GitHub Codespaces is explained more in *Chapter 15*.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

Code in text: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and X (formerly Twitter) handles.

Here is an example: “When the experiment is finished executing we can grab our best model, an `ITransformer`, from the result's best run via `result.BestRun.Model`.”

A block of code, notebook cell, or other command you might enter is set as follows:

```
ExperimentResult<MulticlassClassificationMetrics> result =
    exp.Execute(split.TrainSet, split.TestSet);
ITransformer model = result.BestRun.Model;
var metrics = result.BestRun.ValidationMetrics;
metrics.ConfusionMatrix.GetFormattedConfusionTable()
```

The output of any command is written as follows:

```
Confusion table
| |=====
PREDICTED | | Win | Loss | Draw | Recall
TRUTH     | |=====
Win       | | 6,537 | 3 | 438 | 0.9368
Loss      | | 1 | 6,544 | 432 | 0.9379
Draw      | | 171 | 220 | 4,271 | 0.9161
| |=====
Precision | | 0.9744 | 0.9670 | 0.8308 |
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: “A **Variables** pane to view variables in the kernel.”

Tips or important notes

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at customer care@packtpub.com and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share your thoughts

Once you've read *Data Science in .NET with Polyglot Notebooks*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/978-1-83588-296-2>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1:

Data Analysis in Polyglot Notebooks

We'll start our journey by introducing Polyglot Notebooks and their role in software engineering, data analysis, and machine learning workflows. We'll cover the Polyglot Notebooks technology and user interface as well as the basic decisions and actions you'll make working with notebooks.

The remainder of this part focuses on loading data into a notebook and then performing data analysis in Polyglot Notebooks using .NET tooling using C# and F#.

Whether you're an experienced data analyst or have no prior knowledge, you'll learn how to load up, analyze, clean, and manipulate data using .NET technologies like the DataFrame.

You'll also see how you can effectively understand data distributions and create helpful visuals using libraries like Plotly.NET, Microsoft.Data.Analysis, and MathNet.Numerics.

This part has the following chapters:

- *Chapter 1, Data Science, Notebooks, and Kernels*
- *Chapter 2, Exploring Polyglot Notebooks*
- *Chapter 3, Getting Data and Code into Your Notebooks*
- *Chapter 4, Working with Tabular Data and DataFrames*
- *Chapter 5, Visualizing Data*
- *Chapter 6, Variable Correlations*

1

Data Science, Notebooks, and Kernels

Data science seems like it is at an all-time high in popularity as advances in computing, storage, and data analysis have made new types of applications not just possible, but accessible to people who traditionally wouldn't call themselves data scientists.

This book aims to help developers expand their existing knowledge and capabilities into the fields of data science, machine learning, **artificial intelligence (AI)**, and data analysis.

In this opening chapter, we'll cover these broad topics and explore what the field of data science includes, how the various parts of data science relate to each other, and how data science notebooks enable you to perform new tasks in new ways.

This chapter covers the following topics:

- Exploring the field of data science
- Data science notebooks and Project Jupyter
- Extending notebooks with kernels
- Polyglot Notebooks and .NET Interactive

Exploring the field of data science

Let's start by defining what data science is and isn't.

I define data science as the discipline of preparing and analyzing large amounts of data to extract insights and determine future behavior through machine learning and predictive modeling.

In other words, data science is all about gathering insights from the large amounts of data organizations amass every day. In fact, this is part of why data science has experienced an increase in popularity in the past decade.

Over the past several decades, organizations have moved more of their applications to be hosted on cloud computing providers such as Microsoft Azure, Amazon Web Services (AWS), and Google Cloud. This shift to cloud hosting had several benefits, including the following:

- Ease in scaling web applications and databases as usage grows
- Ease in adding new capabilities such as cloud storage and machine learning
- Automated tools for collecting, storing, and analyzing application telemetry

While there are many other advantages (and some disadvantages) of cloud hosting, these three capabilities highlight an interesting trend: organizations are collecting and retaining more data in their applications than they were a decade ago.

The rise of big data

Most businesses and technology executives I advise agree that the data their organizations collect is a major advantage for their business.

Market trends support this, with a report from Fortune Business Insights sharing that the global data storage market was valued at 217 billion US dollars in 2021. This figure grew to 247 billion in 2023 and is projected to grow to nearly 800 billion by 2030. See *Further reading* for more information.

This need to store and analyze data is so common that we use the term big data to refer to the systemic collection and analysis of large amounts of data to extract and apply insights hidden in that data.

There are entire books dedicated to the storage of data in various databases, data warehouses, data marts, data lakes, and data lakehouses. This book isn't focused on how data are stored, but rather on how to analyze data in a systematic way and use data to train and deploy machine learning models and generative AI solutions.

Because data science is often confused with other fields, let's explore the related terms of data analytics, machine learning, and AI.

Data analytics

Data analytics is the field of using statistics and exploratory data analysis techniques to identify, explore, and communicate the significant trends, patterns, or characteristics of our datasets.

Data analytics tasks are often performed by data analysts whose job focuses on identifying and communicating trends in data in compelling ways. Depending on the data analyst and their organization, this may be done using a variety of tools, from spreadsheets such as Microsoft Excel to data visualization tools such as Power BI or Tableau, or by working with raw data in data science notebooks using programming languages such as Python or R.

Data analysis is not just for data analysts but has value for other roles, including the following:

- Developers looking for patterns in errors or usage data
- Managers looking to improve processes and products
- Data scientists looking to understand correlations for machine learning

I've personally found a great deal of value in identifying and communicating trends in data. As a developer, I was able to analyze data to identify patterns in rare software bugs or slow application performance.

As an engineering manager, I was able to identify the parts of our code that encountered the most problems to prioritize improving code in that area and avoiding common problems. Any technologist in any role can use data analysis to communicate facts to leaders, and to shift conversations from gut feelings toward data-driven decision making.

As a data scientist and AI specialist, I routinely use data analysis to identify unusual outliers and missing values in data, identify data features that correlate with each other, and guide questions that I can ask to further understand the data and its implications.

I've had the opportunity to lead data analysis and data science workshops in the community, and what I've found is almost universal: people with numerous job titles see being able to identify and communicate challenges in their day-to-day work with data as a way of obtaining the support they need.

We'll explore data analysis throughout the rest of *Part 1* of this book as we look at using Polyglot Notebooks, tabular data analysis tools, and various charting libraries to identify trends in data.

Machine learning

Machine learning involves using various mathematical techniques to identify patterns in data and preserve them in equations or models.

Machine learning is an umbrella term that encompasses several sub-fields:

- Supervised learning, which seeks to predict new values based on historical trends using a trained machine learning model
- Unsupervised learning, which looks for patterns, groups, and anomalies in existing data
- Reinforcement learning, which seeks to optimize its choices by learning from prior successes and mistakes

Supervised learning

Most of the time, when I hear people talking about machine learning, they are referring to supervised learning. Supervised learning involves training a machine learning model from a dataset with historical data that includes values we will want to predict in the future.

This model training process takes time. For small datasets, model training may be near instantaneous or occur within a minute. However, larger datasets will require hours or potentially even days to train. The training process is important because it generates a trained machine learning model that internalizes trends observed in the training data. These associations allow it to predict values for new data the model hasn't seen before to a certain degree of accuracy.

Thankfully, once a model is trained, it can be used for multiple predictions without having to repeat the training process. This means that the slow model training process is a required up-front time investment that allows fast predictions from the trained model.

Let's consider a practical machine learning example.

Imagine you had data on historical car sales. Your dataset would include information such as the make, model, year, color, mileage, and condition of the vehicle. This dataset would also include the amount of money a customer paid when they bought the car.

You could use this data to train a model to predict the price that another car on your lot will ultimately sell for. A good model would predict a selling price close to the price the car will later sell for while less accurate models might predict a value much higher or much lower than the actual selling price.

To illustrate this, examine the scatter plot in *Figure 1.1* showing the relationship between the distance a car has driven and its ultimate resale price:

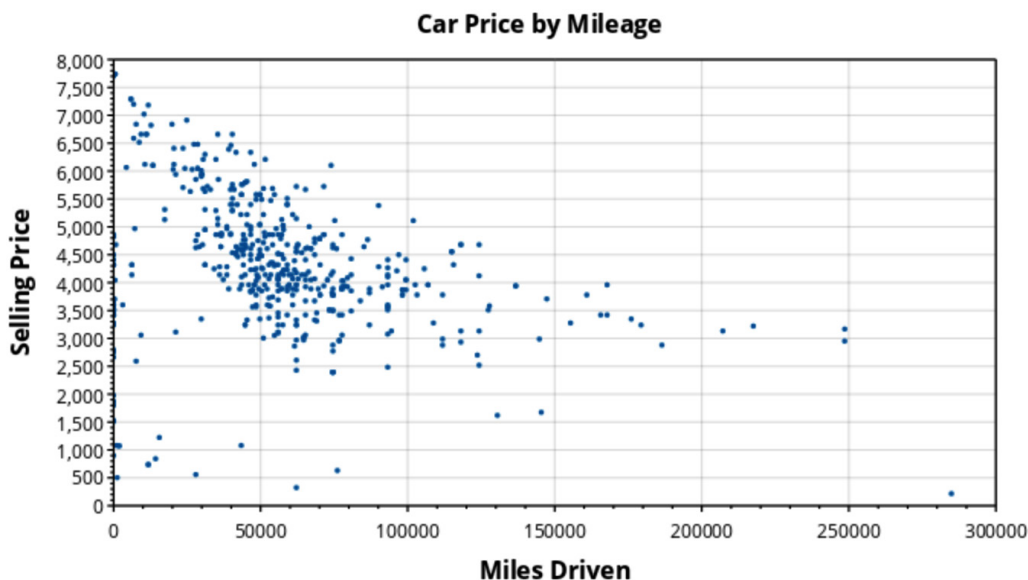


Figure 1.1 – A scatter plot of car selling prices and mileage

Here the mileage of the car is only one factor in the selling price of the car, but we can see a clear negative trend where the more a car has been driven, the lower its ultimate selling point typically is.

Data visualization

This scatter plot is one of the types of visuals we'll cover in *Chapter 6, Visualizing Variable Relationships*, using Polyglot Notebooks and the Plotly.NET charting library. We'll talk more about interpreting and customizing these and other chart types in that chapter.

In machine learning terminology, we refer to the value we're trying to predict as a label (sometimes referred to as a target), and the values that factor into this value are referred to as features. In our car example, the label would be the selling price of the car while its features would include the mileage, year, make, color, and other contributing factors. We'll discuss this terminology more in *Part 2* of this book, but it's important to know that supervised learning requires this labeled dataset – a dataset with historical labels that we're trying to predict.

We'll discuss supervised learning more in *Part 2* of this book using Microsoft's open source ML.NET machine learning library.

Unsupervised learning

Unsupervised learning is generally used for clustering and anomaly detection applications such as identifying different types of users or flagging suspicious network traffic. A key distinction of unsupervised learning is that it does not need a dataset with values that you'll try to predict in the future. Instead, unsupervised learning can be used to group data points into distinct groups or clusters or to identify anomalies in data.

Unsupervised learning is supported by ML.NET but is less frequently used in software development organizations in my experience, and as a result, is beyond the scope of this book. See the *Further reading* section at the end of this chapter for a link to more information on unsupervised learning with ML.NET.

Reinforcement learning

Like unsupervised learning, reinforcement learning does not require a labeled dataset with the values you're trying to predict. Instead, reinforcement learning tries different approaches and grades its performance using a fitness function. This fitness function provides a numeric score illustrating how effective the reinforcement learning routine was at its task. Based on this score, reinforcement learning can learn behaviors that lead to better or worse scores and gradually find an optimal solution to a problem.

Reinforcement learning's ability to find optimal solutions to problems is highly useful for specialized tasks such as robotics and training AI agents to play video games.

What's an agent?

The term AI agent refers to an individual entity in a simulation or the real world that can perceive and interact with its environment to accomplish a specific task or goal. Examples of AI agents include non-player characters in video games and individual robots interacting in a larger environment where they must coordinate with each other.

Because reinforcement learning doesn't require a labeled dataset like supervised learning but does require a way of grading its performance, reinforcement learning is sometimes referred to as a form of semi-supervised learning.

Reinforcement learning is outside of the scope of this book, but there are many books that cover that topic, including a number from Packt.

Let's close this section with our final topic of interest: AI.

Artificial intelligence

Artificial intelligence, commonly referred to as AI, is a broad field of computer science that seeks to emulate intelligent behaviors observed in humans or animals.

Note

It may surprise you to see animals included in this definition, but AI researchers study flocking behavior in animals such as schools of fish, birds, and ants. While interesting, these studies have yielded real-world insights into coordinating large groups of AI agents.

There are many different fields of AI, just as there are many different aspects of computer science. Here are just some of the types of systems AI can help simulate:

- Conversational AI agents such as chatbots
- Self-driving cars capable of navigating streets safely and reliably
- Game opponents in video games, including opponents and teammates
- Autonomous robots capable of manipulating packages in warehouses
- Logistical systems responsible for managing supply chains for stores

Academically, AI has been considered as a set of optimization techniques, such as routines that seek to identify the shortest route a salesperson must take to visit every location in a list of locations.

Other applications of AI involve game theory, which is a branch of AI and mathematics focused on studying how rational players interact with each other and the game to maximize their chances of winning.

Game AI agents typically look to optimize the AI agent's score while minimizing its opponent's score in games as simple as tic-tac-toe or rock paper scissors, or as complex as chess. A recent major achievement of AI in game theory is Google DeepMind's AlphaGo system. AlphaGo famously defeated the world's greatest Go player, Lee Sedol, by winning by four games to one in a highly televised series of matches in 2016.

It's worth noting that AlphaGo worked by using neural networks and reinforcement learning. Specifically, AlphaGo was able to train itself by playing against different versions of itself and learning from its mistakes over time. This led to AlphaGo learning new strategies by experimenting against itself to see which strategies were effective and which weren't.

AI can be viewed as an overarching category that encompasses many different aspects of trying to emulate aspects of human and animal intelligence, as illustrated in *Figure 1.2*:

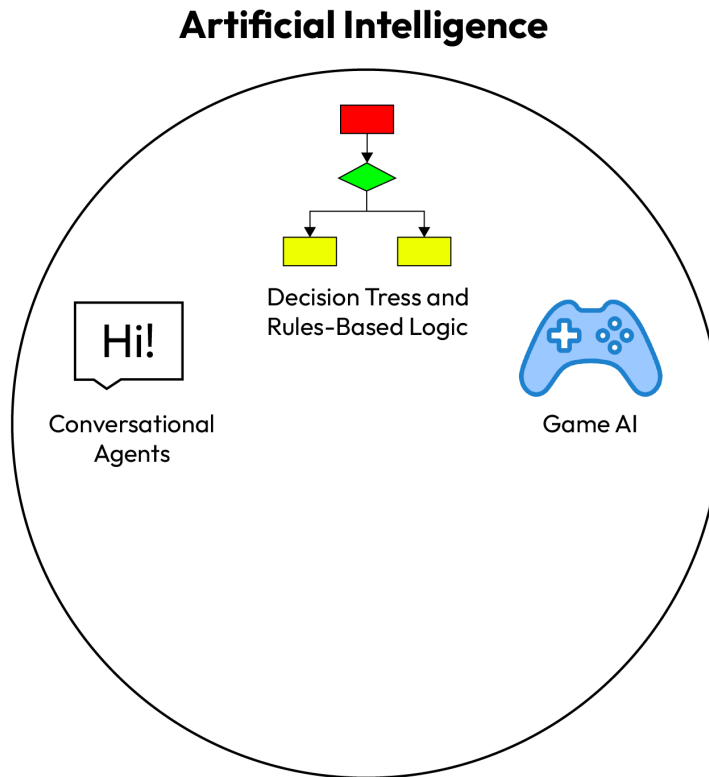


Figure 1.2 – AI encompasses many disciplines

Machine learning can be viewed as a subset of AI, which helps differentiate the disciplines, as shown in *Figure 1.3*:

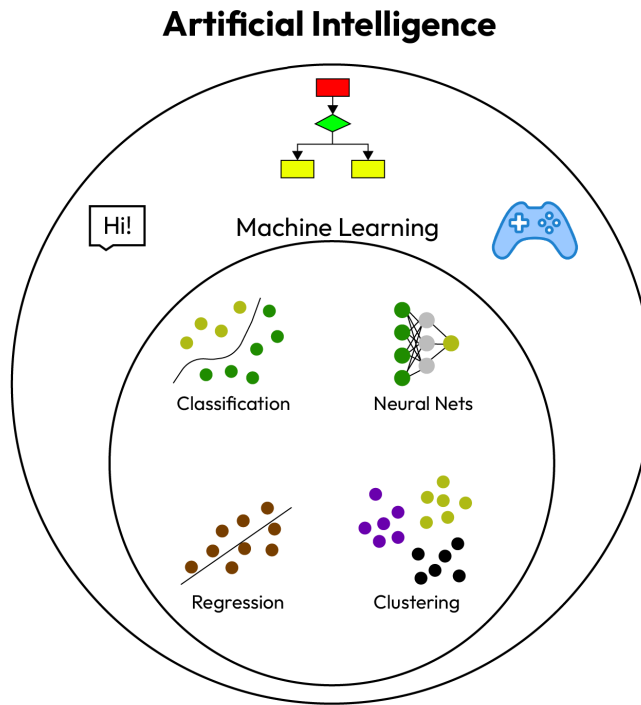


Figure 1.3 – Machine learning as a subset of AI

As we saw earlier, machine learning is itself a field with many different applications and specializations. Some machine learning involves mathematical equations, such as linear regression and logistic regression, which follow the basic concepts of linear algebra that model how changes in one variable affect other variables.

Not feeling mathy? Don't panic!

Some of you may see the phrase linear algebra and want to run screaming from the book. This can be a normal reaction to mathematics, even for professional software engineers. This book aims to provide an overview to help experienced developers understand data science in .NET. Wherever possible, we'll focus on the concepts and avoid the mathematical details. In the few places where details are needed, we'll limit the math to a few lines of C# syntax.

Other forms of machine learning can be significantly more complicated and use more advanced mathematical algorithms. We'll cover some of these algorithms in *Part 2* of this book.

Neural networks

Some of the most advanced forms of machine learning involve neural networks, which try to emulate the biological systems found in our brains through a collection of interconnected layers of simulated neurons that together calculate output values from a set of inputs, as illustrated in *Figure 1.4*.

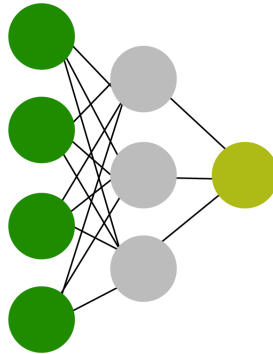


Figure 1.4 – A neural network as a series of connected layers of neurons

The exact details of how neural networks work are best left to books specifically on neural networks or their applied tasks, such as computer vision, speech synthesis, or speech recognition.

Keeping things at a high level, you can think of neural networks as complex calculators: they receive a series of values, perform a series of calculations, and output a series of values based on which neurons are connected to each other and how strong those connections are.

While neural networks are complex to interpret, they can generate very nuanced results based on their input data. This allows properly configured neural networks to perform very well at certain tasks.

Deep learning

Deep learning involves neural networks of sufficient depth and complexity. With deep learning, the different layers in a neural network can perform specific tasks in sequence, such as taking image data and identifying edges in the image or scaling a large image down to a small image.

These deep learning systems perform remarkably well at image and speech tasks because they have enough depth to perform a sequence of processing steps and enough complexity to have room to internalize many different rules for understanding the different types of inputs they might encounter.

Because of this, you can look at deep learning as a subset of machine learning that, in turn, is a subset of AI, as pictured in *Figure 1.5*:

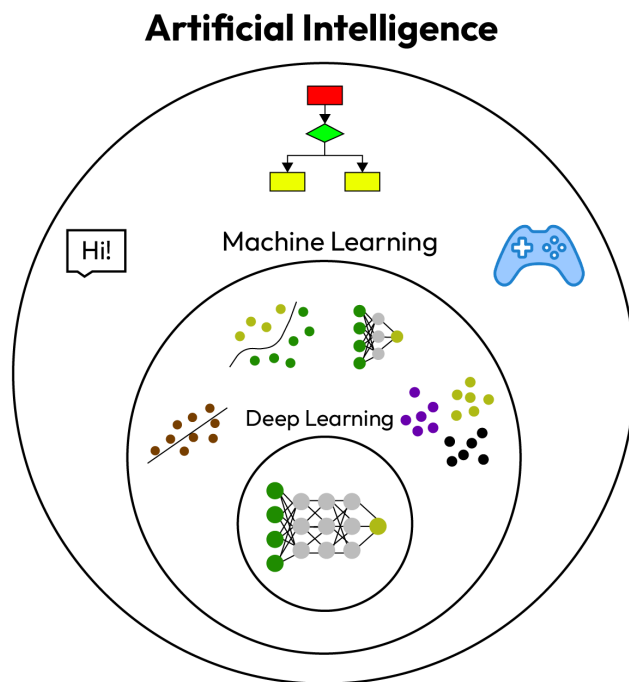


Figure 1.5 – Deep learning as a specialization of machine learning

Data science encompasses all aspects of machine learning and deep learning, but not all aspects of AI. For example, there are branches of AI related to rules-based systems or game AI that are of little interest or use for historical data and their patterns. However, almost all systems that need to use past information and trends to make inferences on new data involve machine learning.

What about LLMs?

Astute readers might notice that we've not covered **large language models (LLMs)** such as ChatGPT, GPT-4, Meta Llama, and others. These LLMs use complex neural networks to represent the internal relationships between different words and can be considered a specific application of deep learning. These LLMs can be used to generate new content from an input and are commonly referred to as generative AI systems as a result. We'll cover LLMs and generative AI more in *Part 3* of this book.

Now that we've established what areas data science and AI encompass and how these fields relate to each other, let's discuss one of the core tools of data scientists and data analysts: notebooks.

Data science notebooks and Project Jupyter

In programming, when you want to illustrate a concept or explore new coding techniques, it's normal to create a small application such as a console program and write the code you need there. This can be helpful for validating and communicating ideas, making small examples, reproducing errors, or building small demos.

Like programmers, data scientists also sometimes need to perform small experiments. While it's entirely possible to perform data science experiments by creating a new Python or .NET program, a far more common approach for data scientists and data analysts is to create a notebook.

A notebook is a combination of documentation in the form of Markdown cells mixed together with code cells. This combination of code and documentation allows you to provide rich formatted documentation via Markdown while also providing live executable code in code cells.

For years, when people have talked about notebooks in data science or data analytics contexts, they've largely been talking about a technology called Jupyter Notebooks.

Jupyter Notebooks came about as Project Jupyter in 2014, by Fernando Pérez and Brian Granger. Project Jupyter was a direct offshoot from the IPython interactive Python kernel and made it possible to effectively mix together Markdown and code and run small iterative experiments in a visual way.

Why Jupyter?

The name Jupyter is short for the three different programming languages that Jupyter Notebooks supports: Julia, Python, and R.

The impact of Jupyter Notebooks and the iterative and visual approaches it allows has been tremendous – particularly in the data science community. Jupyter Notebooks allows data analysts and data scientists to make small changes and get immediate feedback on their actions. When experiments are ready to share, the presence of Markdown documentation allows others to easily follow the logic and reasoning of the experiment.

While this book is not about Jupyter Notebooks, the Polyglot Notebooks technology we'll discuss is built on top of Jupyter Notebooks, so it is important to understand the basics and impacts of Project Jupyter before we talk about what Polyglot Notebooks adds to this equation.

Let's start by touring the code and Markdown cells in more depth.

Markdown cells

Most developers have worked with Markdown in the form of documentation in `readme.md` files in their code repositories on GitHub, GitLab, Azure DevOps, or other code-hosting sites.

Markdown allows you to write richly formatted documentation in simple text format. Using Markdown, you can denote text that should be bolded, italicized, or rendered as a hyperlink. You can also use various header styles and include numbered and ordered lists, quotes, images, and even inline code and blocks of formatted code.

A sample Markdown block looks like this:

```
### Notebooks have Cells
#### Markdown Cells
This is a markdown cell.
It supports standard [GitHub-flavored markdown] (https://github.github.com/gfm/).
```

When the Markdown isn't being edited, it will be rendered in a notebook as shown in *Figure 1.6*:

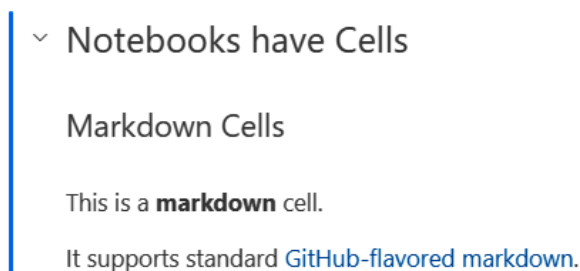


Figure 1.6 – The Markdown rendered as formatted text

You can learn more about Markdown in the *Further reading* section of this chapter, but a few of the main benefits are as follows:

- Markdown doesn't require using an application such as Microsoft Word
- Markdown is easy to store in plain text in version control
- Most developers work with Markdown or have at least encountered it before

Markdown may initially seem complex, but in practice, it serves as a simple tool-free way of formatting user-facing documentation.

Let's move on to the more exciting part of notebooks: the code cells.

Code cells

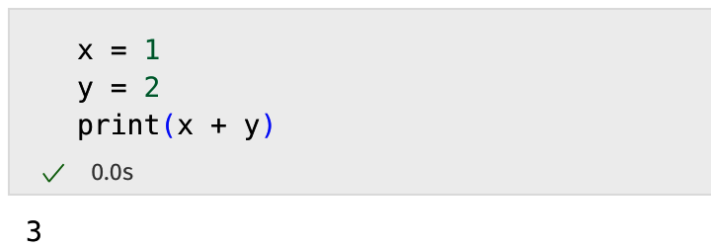
Code cells let you display code that can be executed inside the notebook. When a code cell is executed, a kernel interprets it, validates the code, and then carries out its actions.

For example, the following Python cell will add two numbers together and then print out the result of the addition:

```
x = 1
y = 2
print(x + y)
```

When this cell is run, the Python kernel will run the code sequentially, declare `x` and `y` variables, and then add them together and print the result.

When values in notebook cells are printed, their results are rendered below the cell when the cell is executed as shown in *Figure 1.7*:



```
x = 1
y = 2
print(x + y)
```

✓ 0.0s

3

Figure 1.7 – A cell in Jupyter Notebooks displaying its results

Because the code is stored in the notebook, you can run it and re-run it as many times as you like. Additionally, if you're curious about how the cell would work with different values for `x` and `y` or other logic changes, you can make those changes and then re-run the cell.

Notebooks can have as many code and Markdown cells as you like. Furthermore, when you run a cell, its variables become available to other cells you run later. This makes it easy to have small blocks of code mixed with markdown documentation, but also to have all the code work together to accomplish a larger task.

Trying Jupyter Notebooks

In the next chapter, we'll cover installing Polyglot Notebooks, which will allow you to work with Jupyter Notebooks as well. However, if you just can't wait and want to try out Jupyter Notebooks on your own, I recommend downloading Anaconda, which is a data science environment that includes Jupyter Notebooks, a version of Python, and many common libraries used for data science. See the *Further reading* section for more information.

There's more to cells and variable sharing than what I've covered in this section, but we'll cover variables in more detail in *Chapter 2* as we dive into Polyglot Notebooks and its own unique rules for variable sharing.

Before we explore Polyglot Notebooks, let's talk briefly about an important concept: kernels.

Extending notebooks with kernels

In our example in the previous section, we saw how code cells could be executed to produce a result.

This work is carried out by a kernel designed for the programming language you're working with. In the earlier example, we saw a code cell written in Python. Since this is Python code, it uses the Python kernel.

In this way, the Jupyter Notebooks environment can support multiple languages, including Julia, Python, and R, as shown in *Figure 1.8*:

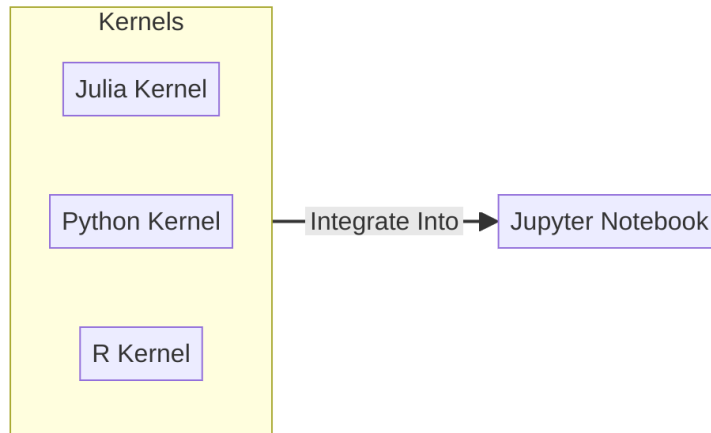


Figure 1.8 – Different kernels available to the notebook

When you execute a Python cell, the code is sent to the Python kernel, which interprets it, executes it, and produces a result as illustrated in *Figure 1.9*:

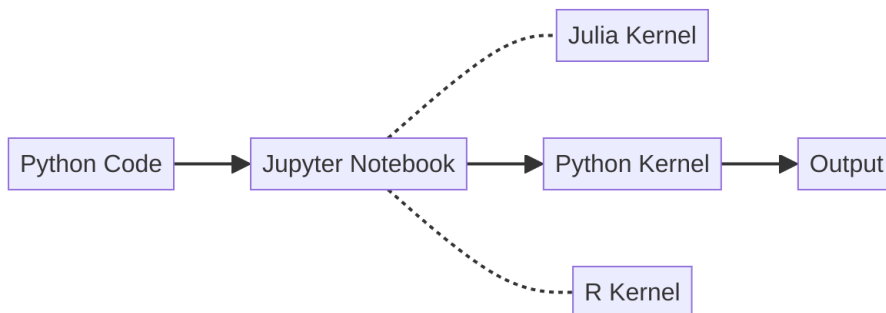


Figure 1.9 – Jupyter Notebooks executing the Python kernel

This extensible design allows Jupyter Notebooks to support additional languages by adding additional kernels for those languages.

This is exactly what Polyglot Notebooks does, as we'll see next.

Polyglot Notebooks and .NET Interactive

Polyglot Notebooks is a VS Code extension that integrates into Jupyter Notebooks to provide additional capabilities and language support through the kernel mechanism.

The goal of Polyglot Notebooks is to enrich the notebook experience by moving the supported languages beyond just the original languages. Polyglot Notebooks accomplishes this by adding its own kernel, called .NET Interactive, which supports the following languages:

- C#
- F#
- JavaScript
- Python
- R
- PowerShell
- HTML
- HTTP Requests
- Mermaid
- SQL
- KQL

The philosophy of Polyglot Notebooks is that you should be able to choose the languages and tools you want to use for the task you're trying to accomplish.

In other words, Polyglot Notebooks wants to allow you to pull data into your notebook via SQL, KQL, or a REST request, and then use C#, F#, or JavaScript to manipulate it before allowing you to visualize it using the capabilities of HTML, Python, F#, or another language of your choosing.

What's a polyglot anyway?

Oxford Languages defines a polyglot as someone who knows or uses several languages. In the context of programming, the word polyglot is often used to refer to developers who are comfortable coding in several different programming languages and pick the most appropriate one for the project they're working on.

Polyglot Notebooks is more than just a collection of new languages supported in Jupyter Notebooks.

.NET Interactive shines the most through the Polyglot Notebooks extension to VS Code, but it can also execute in other environments, including in a Jupyter Notebook and in your browser, via Codespaces, as we'll see in *Chapter 15, Adopting and Deploying Polyglot Notebooks*. This extension integrates the .NET Interactive kernel into Jupyter Notebooks so you can write, run, and analyze your notebooks directly from VS Code as shown in *Figure 1.10*:

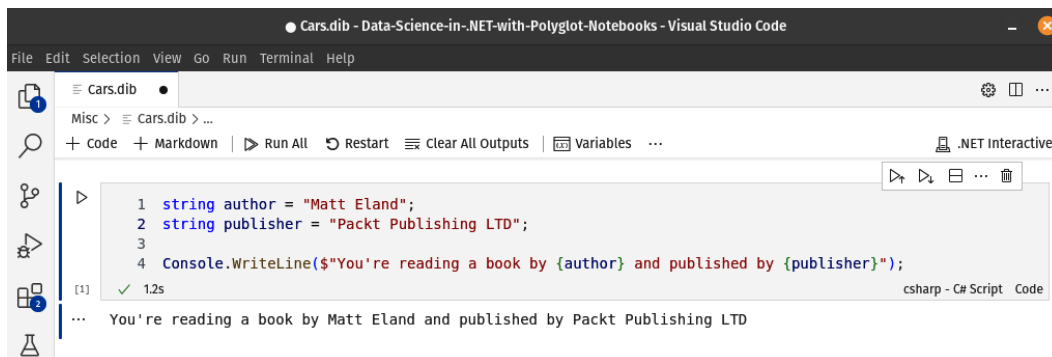


Figure 1.10 – VS Code with C# code in Polyglot Notebooks

The Polyglot Notebooks extension provides additional capabilities to the development experience:

- Advanced formatting of cell outputs
- A **Variables** pane to view variables in the kernel
- Variable sharing shortcuts to help share variables between languages

We'll talk more about the Polyglot Notebooks extension, variable sharing, and getting started with Polyglot Notebooks in the next chapter.

Polyglot Notebooks gives data scientists and data analysts with .NET backgrounds more languages at their fingertips to perform these experiments in whatever language they prefer.

There are many developers out there who want to follow the path I took by adding data science and AI skills to their existing .NET knowledge. For these developers, Polyglot Notebooks makes the learning curve easier by allowing them to use languages they're comfortable with to learn new things.

Keep in mind that Polyglot Notebooks is not just for data science and data analytics. Software engineering teams can use Polyglot Notebooks for living documentation with code examples integrated into the documentation. Polyglot Notebooks can also replace many scenarios where developers might have made a small console application to illustrate a proof of concept or read and display some data.

As a former teacher, I can tell you that being able to provide immediate feedback on whether a small piece of code works to new learners is invaluable, and so I'm convinced that Polyglot Notebooks has a strong role in the education field as well as in interactive documentation.

I've also enjoyed the ability to embed a Codespace in my .NET repositories on GitHub. This option allows others to try out my projects and libraries without having to clone or install them locally. If someone is curious, they click a button to create a Codespace around my code, and they can then experiment with the Polyglot Notebooks setup to help them explore my projects.

Personally, I find myself using Polyglot Notebooks for the following types of tasks:

- Analyzing data from production or QA environments in a visual manner
- Training and evaluating machine learning models
- Putting together interactive examples I can share with others, including on GitHub
- Infrequent calculations or analysis work I might want to do on an irregular basis or share with others
- Creating small utilities such as ways to convert data from one format to another
- Public or team-facing documentation on how to work with APIs
- Small demos for articles, conference talks, or lunch and learns

Your use cases for Polyglot Notebooks will be unique to your own skills, interests, and needs. Additionally, not every problem you encounter will benefit from Polyglot Notebooks. Still, in my experience, most developers and development managers I talk to can find ways in which Polyglot Notebooks can help their existing workflows.

Summary

In this chapter, we discussed the high-level fields of data science, machine learning, and AI.

While we covered these earlier in more depth, here is a consolidated set of definitions:

- **Artificial intelligence** is the broadest field and revolves around emulating aspects of behaviors found in humans and animals.
- **Data science** is the discipline of preparing and analyzing large amounts of data to extract insights and determine future behavior through machine learning and predictive modeling.
- **Machine learning** is a broad field involving applying mathematics and statistics to solve data problems. Machine learning includes supervised learning, unsupervised learning, and semi-supervised learning including reinforcement learning.
- **Supervised learning** involves applying statistical and mathematical techniques to model trends and relationships found in datasets.

In this chapter, we also discussed the role of notebooks in data science for conducting iterative experiments that mix together code and Markdown. We talked about how the concept of notebooks was popularized by Project Jupyter and discussed some of the excellent tooling around Jupyter Notebooks. We also discussed how Jupyter Notebooks uses a kernel to interpret and execute its code cells.

We closed the chapter by transitioning from Jupyter Notebooks to our main topic: Polyglot Notebooks. Polyglot Notebooks extends Jupyter Notebooks to offer the .NET Interactive kernel with support for a wide range of languages, including C# and F# in the .NET family. This provides developers with a new way to run code in languages they're comfortable with – and the option of embedding notebooks and possibly Codespaces into their repositories.

Polyglot Notebooks also presents data scientists and data analysts with new options for running their experiments if they would prefer to use languages supported by the .NET Interactive kernel or .NET-specific libraries.

In the next chapter, we'll dive into installing Polyglot Notebooks and interacting with cells and variables across different programming languages.

Throughout the rest of *Part 1* of this book, we'll explore the data analysis workflow through Polyglot Notebooks before transitioning to discussing data science workflows in *Part 2* and then to AI workflows in *Part 3*.

Further reading

See the following resources for more information on the topics covered in this chapter:

- *Polyglot Notebooks and .NET Interactive history*: <https://devblogs.microsoft.com/dotnet/dotnet-interactive-notebooks-is-now-polyglot-notebooks/>
- *Machine learning tasks – ML.NET*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/tasks#clustering>
- *GitHub CodeSpaces*: <https://github.com/features/codespaces>
- *Project Jupyter*: <https://jupyter.org/>
- *AlphaGo*: <https://deepmind.google/technologies/alphago/>
- *A computer code that transformed science*: <https://cs.lbl.gov/news-media/news/2021/project-jupyter-a-computer-code-that-transformed-science/>
- *Anaconda*: <https://www.anaconda.com/download/>
- *Markdown cells*: <https://packt.link/U9OUX>
- *Data storage market size trends*: <https://www.fortunebusinessinsights.com/data-storage-market-102991>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/introduction/>

Exploring Polyglot Notebooks

In the last chapter, we covered what notebooks are and how they're helpful to developers, data analysts, data scientists, and technology managers.

In this chapter, we'll dive in and see Polyglot Notebooks in action. We'll start by installing Polyglot Notebooks and then build and execute our first notebook. By the time we're done with this chapter, you should have a much clearer idea of what the notebook development process looks like so we can drill into more specific uses in future chapters.

In this chapter, we'll cover the following:

- Installing Polyglot Notebooks
- Creating your first notebook
- Executing notebook cells
- Declaring classes and methods
- Working with other languages
- Sharing variables between kernels
- Troubleshooting notebook execution

Technical requirements

In order to work with Polyglot Notebooks, you will need to first install **Visual Studio Code (VS Code)** and the version of .NET that Polyglot Notebooks currently requires.

VS Code is freely available at <https://code.visualstudio.com> for Windows 10 and above, macOS 10.15 and above, and various distributions of Linux, including Debian, Ubuntu, Red Hat, Fedora, and SUSE.

Polyglot Notebooks requires a specific version of .NET in order to operate. At the point of this book's release, that was .NET 8, but the Polyglot Notebooks team periodically updates to new versions of .NET as new major versions are released.

<https://dotnet.microsoft.com/en-us/download/visual-studio-sdks> VS Code and Polyglot Notebooks can also be run in your browser via GitHub Codespaces. We'll explore this more in *Chapter 15, Adopting and Deploying Polyglot Notebooks*. In fact, you can always check the required .NET version by looking at the `devcontainer.json` file in the `.devcontainer` folder of the official Polyglot Notebooks repository at <https://github.com/dotnet/interactive>.

Installing Polyglot Notebooks

Once you have VS Code and the required version of .NET installed, open VS Code and let's walk through the one-time process of installing Polyglot Notebooks. First, let's clear our workspace by opening a folder where you'd like to store your notebooks.

Do this by using the **File** menu followed by **Open Folder...** and then selecting a folder on your machine to open. I typically select an empty folder to keep my projects focused. You'll now need to install Polyglot Notebooks through the **Extensions Marketplace** in VS Code.

You can get to the extensions view by pressing `Ctrl + Shift + X` or by clicking on the four-boxes icon on the sidebar, as shown here:

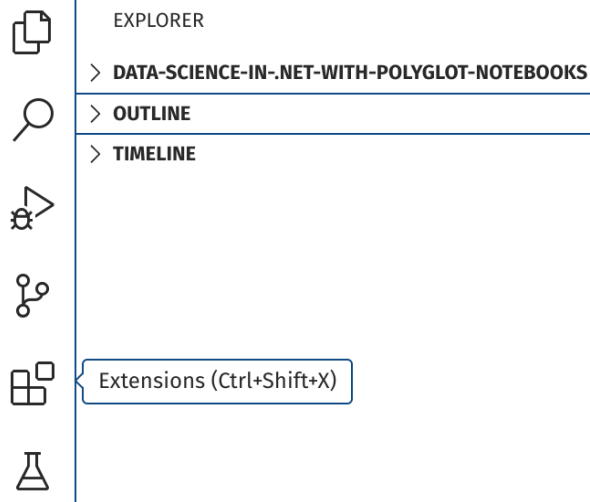


Figure 2.1 – The Extensions view location

The **Extensions** view allows you to install a wide variety of extensions in VS Code. Extensions can do a wide variety of things, including the following:

- Adding compiler support for a new language
- Adding a word count to your status bar
- Adding a new view for looking at resources on Azure or AWS
- Adding new editor color themes
- Responding to key press events by causing digital explosions, fireworks, and screen shaking (yes, this really is a thing – check out the **PowerMode** extension)

Polyglot Notebooks is packaged and installed as an extension. In order to get started, we'll need to find it in the Extensions Marketplace.

Type **Polyglot Notebooks** in the search bar and you should see the list of extensions updated to include the correct result, shown in *Figure 2.2* by Microsoft:



Figure 2.2 – The Polyglot Notebooks extension

Click **Install** and you should now see the extension download and install on your copy of VS Code.

A warning on private NuGet feeds

I once encountered a puzzling failure where Polyglot Notebooks would not install on my laptop and would not indicate a failure message. I ultimately discovered that Polyglot Notebooks was failing because my machine had a private NuGet feed registered from a control vendor and the install was failing because it was unable to check that particular NuGet feed during installation. See <https://github.com/dotnet/interactive/issues/3319> for more details on this particular issue.

It's possible that during installation or while creating your first notebook, you'll encounter an error message indicating you do not have the correct version of .NET installed. The error message will look something like *Figure 2.3*:

.NET 8 SDK REQUIRED

The Polyglot Notebooks extension now requires .NET SDK Version 8.0.

1. Install [here](#)

2. Restart VS Code

Figure 2.3 – A sample failure when lacking the correct .NET SDK version

Thankfully, this message contains details on what dependencies your machine is missing and how to get them.

If you encounter other issues during installation, you may want to skip ahead in this chapter and see the *Troubleshooting notebook execution* section for more steps on getting information about failures or submitting issues for additional assistance.

Once you have Polyglot Notebooks installed, you're ready to create your first notebook.

Creating your first notebook

There is no button in the user interface for adding your first workbook. Instead, we'll need to access this feature by using VS Code's **Command Palette**.

The **Command Palette** is where VS Code keeps its internal commands and settings, along with those registered by extensions you've installed, including Polyglot Notebooks. This helps the user interface stay minimal while keeping commands easily accessible.

Press *Ctrl + Shift + P* to open the **Command Palette** and list the available commands.

Ctrl + Shift + P versus Ctrl + P

It's worth noting that VS Code supports both the *Ctrl + Shift + P* shortcut for the Command Palette and the *Ctrl + P* shortcut for navigation. The user interface for both of them is very similar, but the navigator lists files in your editing session while *Ctrl + Shift + P* lists available commands. See this chapter's *Further reading* section for additional VS Code user interface resources.

Next, type in `Polyglot Notebook` to filter down the command list and choose **Create new blank notebook**, as shown in *Figure 2.4*:

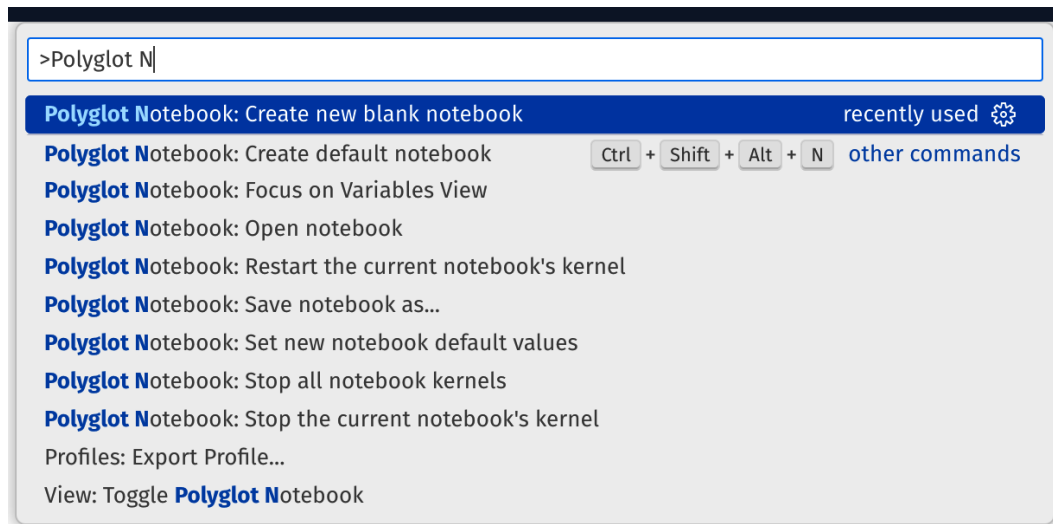


Figure 2.4 – The Command Palette

Next, VS Code will ask you if you want to create a new `.dib` or `.ipynb` file via a dialog, as shown in *Figure 2.5*:

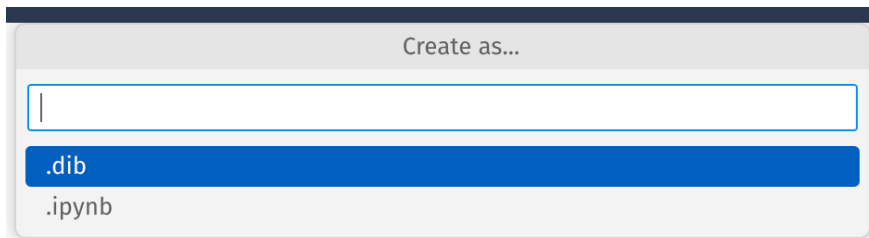


Figure 2.5 – Specifying the notebook format

If you've used Jupyter Notebooks before, you may recognize the `.ipynb` extension as short for an interactive Python notebook, otherwise known as a Jupyter notebook.

`.ipynb` files store both code and execution history, including the output of notebook cells. This means that you can share an `.ipynb` file with others and they can generally follow along with the notebook without needing to run it themselves.

The downside of this is that `.ipynb` files are larger and have a more complex file format. This makes storing `.ipynb` in version control more challenging than storing smaller code files.

`.dib` files are a new file format introduced by Polyglot Notebooks. `.dib` files store the notebook structure and code but do not retain the results of the last time cells were executed. This results in `.dib` files being smaller and easier to manage in version control but they do require the opener to rerun the notebook upon opening it if they want to see past results.

Both approaches are valid, with different advantages and disadvantages. There's an excellent GitHub discussion in the Polyglot Notebooks repository on the new `.dib` file format. Those curious can find the link in the *Further reading* section at the end of the chapter.

For this book, I'll be sticking with `.dib` files due to their smaller size and easier ability to track changes to them in version control since most chapters in this book will have code associated with them in a GitHub repository.

Once you select a file format, you'll be asked to select the default language of your notebook, as shown in *Figure 2.6*:

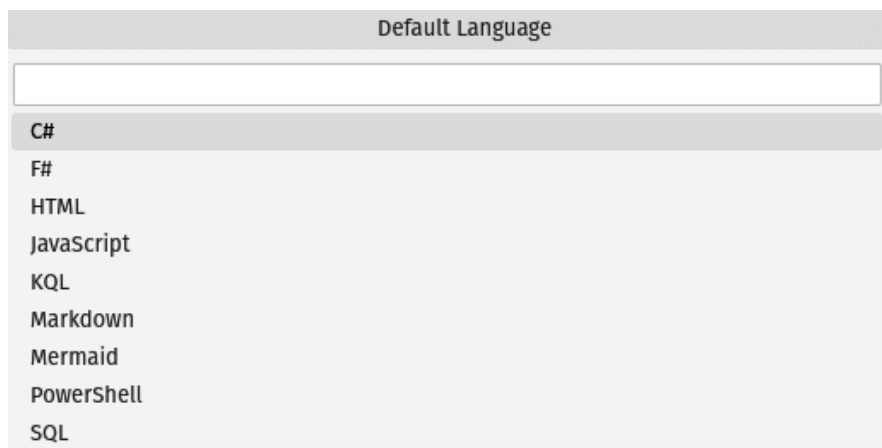


Figure 2.6 – Selecting a default language

The default language is the programming language that will be used for the first cell. Select **C#** and press *Enter* to create your first notebook. You should now see a blank notebook with a single empty cell. Let's put some code in there.

Executing notebook cells

When you create your new notebook, you should see something like *Figure 2.7*:

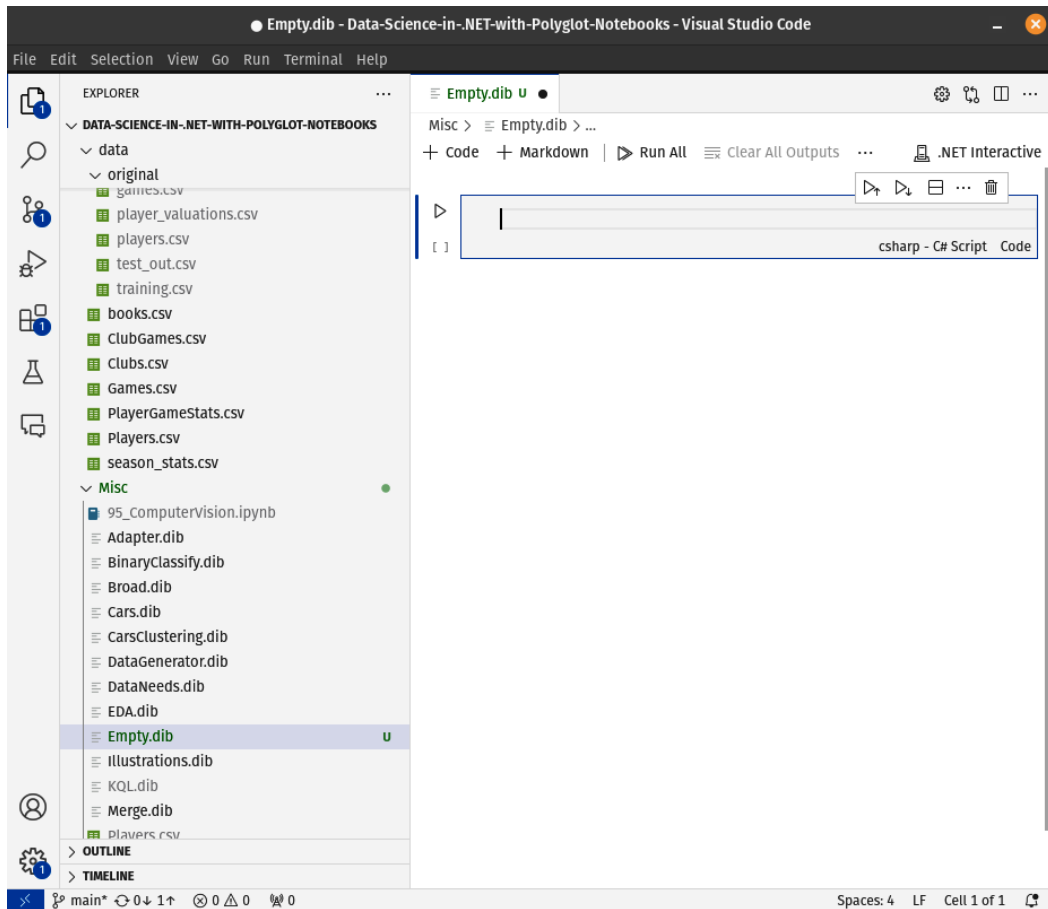


Figure 2.7 – An empty C# code cell

This view consists of a few key parts:

- A toolbar at the upper left with a few commands – we'll discuss these throughout this chapter
- The selected kernel at the upper right – this should be **.NET Interactive**.

- A gray box representing an empty **code cell**
- Indicators at the lower right telling you this is a code cell that will be interpreted as C#
- An **Execute Cell** button with a “play” icon on the left that allows you to run the cell
- A small toolbar at the upper right of the cell with additional commands

Click into your code cell and write the following line:

```
Console.WriteLine("Hello Polyglot Notebooks!");
```

Next, click the **Execute Cell** button to the left of the cell or press *Ctrl + Enter* on your keyboard.

This will send your code to .NET Interactive’s C# kernel for execution and you should see the result below your cell, as shown in *Figure 2.8*:

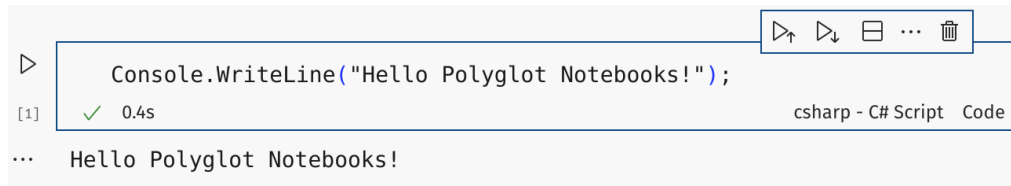


Figure 2.8 – The results of executing a C# code cell

First, you’ll notice that the values you write to the console appear below the cell. This makes it easy to see the output of your cells as you run your notebook.

Next, note that the `[ ]` on the left, below the **Execute Cell** button now shows a `[1]`. This indicates the cell’s execution order since the notebook started.

Execution order

Unlike code, notebooks do not need to be run sequentially. It is perfectly valid to open up a notebook and execute a random cell in the middle of the notebook. However, many cells will rely on earlier cells being executed first, so it’s usually advisable to run cells in order, repeating the execution of cells as you need to. As you experiment more with notebooks, you’ll see advantages in rerunning earlier cells as you tweak values.

Let’s add another cell to our notebook and see how variables work.

Adding code cells

Hover your mouse below the cell until you see the **Add Code Cell** and **Add Markdown Cell** buttons, as shown in *Figure 2.9*:

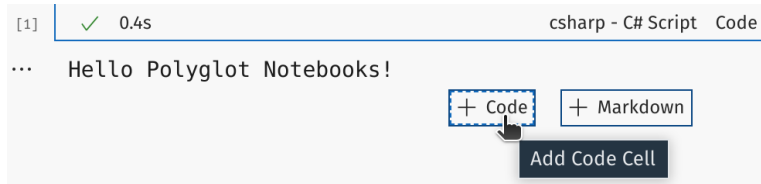


Figure 2.9 – Buttons (shown on hover) to add code or markdown cells

Click the **Add Code Cell** button to insert a new code cell below your current cell. This cell will use the last language used by a code cell or the notebook's default language you specified earlier if no code cells exist.

In our new code cell, type the following code:

```
int i = 1;  
i
```

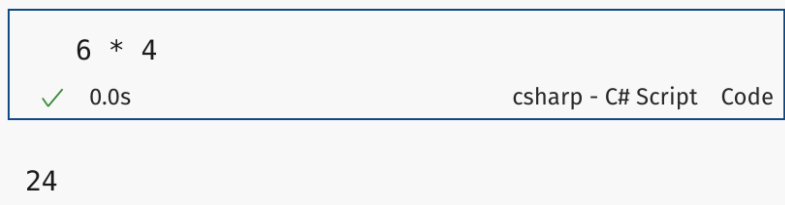
The first line defines a new integer variable named `i` and sets its value to 1. The second line doesn't look like valid C# syntax since it doesn't end in a semicolon, doesn't execute a method call, and doesn't declare or assign anything, or even perform a mathematical operation.

Nevertheless, when you run your cell, you should see the following output:

```
1
```

While the `i` statement isn't valid C# code, it's valid in a notebook. When you end a cell with a single variable name or an expression that returns a value but *do not put a semicolon at the end of the line*, Polyglot Notebooks treats that as an *implicit* `Console.WriteLine` of that variable and displays its value below the cell. In other words, our `i` statement was identical to if we had written `Console.WriteLine(i);`.

You can try this yourself by adding a single cell with the contents of `6 * 4`, or some other mathematical expression, and seeing the results displayed below the cell when the cell is executed, as shown in *Figure 2.10*:



The screenshot shows a code cell in a Polyglot Notebook. The cell contains the code `6 * 4`. Below the code, there is a green checkmark and the text `0.0s`, indicating successful execution. To the right of the code, the text `csharp - C# Script` and a `Code` button are visible. Below the code cell, the output `24` is displayed.

Figure 2.10 – The last line of a code cell results in the output of that variable by default

Note that this only works for the last value in a cell and only if a semicolon isn't present. Now that we've declared a variable, let's explore how variables work in Polyglot Notebooks.

Working with variables

First, variables inside of cells work the same way they would in another C# program, as we saw with our use of `i` earlier. However, it's important to know that variables defined in a notebook cell don't normally go away after the cell executes. This means cells that execute later on can refer to variables in cells that executed earlier.

Let's try this now. Add a new code cell with the following contents:

```
++i
```

This uses the C# **prefix increment operator** to increase the value stored in `i` by 1, then evaluate the current value of `i` after the value was increased.

In practice, this cell will increase the value of `i` by 1 every time the cell runs. Since the value is the last value in the cell and doesn't end in a semicolon, you'll see the value of `i` every time you run the cell. Run the cell a few times and you should see the value increase by 1 each time.

Of course, this example is a little trivial and not something you'd really do beyond teaching someone how Polyglot Notebooks works. In subsequent chapters, we'll do operations such as loading data in one cell, analyzing its distribution in another, generating a visualization in a third cell, and performing mathematical operations in another cell.

While the cells I've shown you thus far have been only a few lines of code, you're not required to keep your individual cells brief. If you needed to and wanted to, you could declare a single cell with hundreds of lines of code in it to accomplish some objective.

I would certainly argue that just because you *can* do something doesn't mean you *should*. In this case, Polyglot Notebooks' main benefits would be diminished, such as the ability to see the results of small blocks of code in isolation and make iterative tweaks.

Good practice: cell sizes

Your individual cells in a notebook should be easy to understand, related to a single task you're trying to accomplish, and short enough that you can easily think about the entire cell. I try for cells less than 10 lines of code long, though there may be cases for longer cells, particularly when you're defining classes and methods, as we'll see later in this chapter.

With large notebooks, it can be hard to manage all the variables you've defined.

Thankfully, Polyglot Notebooks has a tool to help keep track of the globally available variables in your notebook: the **Variables** view.

The Variables view

You can open the **Variables** view by clicking on the **Variables** button in the toolbar, as shown in *Figure 2.11*:

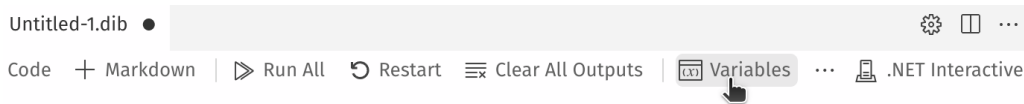


Figure 2.11 – The notebook toolbar with the Variables button

This will open a debugging pane at the bottom of VS Code with information on the currently defined variables, as shown in *Figure 2.12*:

 A screenshot of the 'POLYGLOT NOTEBOOK' tab in the VS Code interface. It shows a table of variables. The table has columns for Name, Value, Type, Kernel, and Actions. There is a search filter box at the top right of the table area.

Name	Value	Type	Kernel	Actions
i	12	System.Int32	csharp - C# Script	

Figure 2.12 – The Variables view lists all variables tracked by kernels

This table contains the following information:

- The name of the variable
- The current value of the variable
- The data type the variable represents
- Which language kernel the variable belongs to

We'll talk more about this information in the *Sharing variables between languages* section later in this chapter. For now, let's conclude this section with a brief discussion of Markdown cells.

Markdown cells

While code cells allow you to execute values against a language kernel, **Markdown cells** let you declare Markdown syntax that gets transformed into formatted documentation.

Add a new Markdown cell below any cell and add in the following contents:

```
# Main Heading
## A subheading
This is a paragraph. This is bold text and this is [a hyperlink]
(https://packt.com).
```

Unlike code cells, Markdown cells do not have an **Execute Cell** button. Instead, press *Ctrl + Enter* or click the **Stop Editing Cell** button to complete your cell, as shown in *Figure 2.13*:

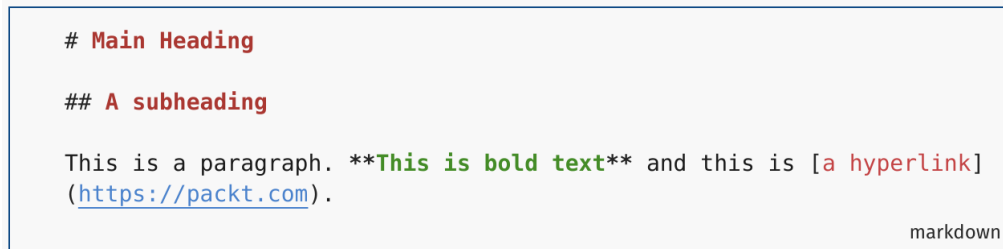


Figure 2.13 – A Markdown cell in edit mode

This will change the cell's display to a formatted rendering of the Markdown, as shown in *Figure 2.14*:

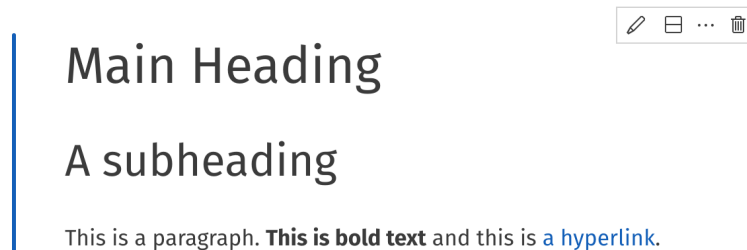


Figure 2.14 – A Markdown cell

If you ever want to edit the cell later, you can click on the cell and then click on the **Edit Cell** button with a pencil icon. Markdown is minimal to write, understandable even when viewed in code form, generates attractive documentation, and is commonly used in programming circles. If you're not familiar with Markdown, be sure to look into the links mentioned in the *Further reading* section of this chapter.

When you mix together Markdown and interactive code cells the user can edit and run, you get a powerful combination of documentation and logic. This allows for some interesting uses of Polyglot Notebooks, as we'll explore in *Part 4* of this book.

For now, let's move back into code to explore more complex scenarios with classes and methods.

Declaring classes and methods

We've now seen that you can declare variables and execute simple operations such as incrementing a variable, assigning values, and invoking common methods such as `Console.WriteLine`.

If these capabilities were the extent of Polyglot Notebooks, it would still be an impressive tool, but perhaps limited in what it could achieve. Thankfully, we're able to build much more complex notebooks by defining our own classes and methods inside of a notebook.

Declaring methods

Let's start by adding a method. Add a new code cell that declares a `CountOddNumbers` method using the following code:

```
// This method counts all odd numbers leading up to the number
// For example, 5 would return 3 counting the numbers 1, 3, and 5.
int CountOddNumbers(int input)
{
    double divResult = input / 2.0;
    return (int)Math.Ceiling(divResult);
}
```

This code uses division to get a remainder from dividing by two and then rounds up to the nearest whole number. In theory, this should give us the number of odd numbers leading up to our number.

When you execute this cell, you should see a *success* check mark and an indicator of the number of seconds spent executing the cell (for example, **0.0s**), but no output below the cell. This is because our cell defines a `CountOddNumbers` method, but doesn't invoke it.

Let's add another code cell below that calls the `CountOddNumbers` method:

```
CountOddNumbers(5)
```

When we execute this, you should see the output of **3** below your cell from Polyglot Notebooks executing the `CountOddNumbers` method and displaying the returned value of **3**. This **3** is calculated since $5 / 2.0$ is 2.5 and then 2.5 rounded up becomes **3**, indicating the count of the odd numbers (1, 3, and 5 in this case).

Local variables and the Variables view

If you're following along with the code, you may be surprised to notice that the `divResult` variable doesn't show up in the **Variables** view after running the `CountOddNumbers` method. This is because `divResult` is scoped to the `CountOddNumbers` method, just as it is in normal C# code. Because of this, it doesn't live in the *global scope*, isn't tracked by Polyglot Notebooks and is not available in other cells. Only variables defined outside of methods, classes, and `{ }` scopes are available in the **Variables** view.

Let's try a few different inputs to `CountOddNumbers` and see if the values you get back seem correct:

- `CountOddNumbers(3)` returns **2** (correct)
- `CountOddNumbers(100)` returns **50** (correct)
- `CountOddNumbers(0)` returns **0** (correct)
- `CountOddNumbers(-1)` returns **0** (incorrect: **1** expected)

Looking at our method, it appears that it doesn't handle negative values correctly. Upon closer inspection, this is because the `Math.Ceiling` call to round up is only the correct operation for positive numbers.

We could fix this by adding an `if` statement to use `Math.Ceiling` when dealing with positive numbers and `Math.Floor` when dealing with negative numbers. Alternatively, we could use `Math.Abs` to ensure that we're always looking at the absolute value when rounding – after all, a number should never have a negative amount of odd numbers composing it!

Since the `Math.Abs` requires the least amount of work, let's change the method to use it as follows:

```
// This method counts all odd numbers leading up to the number
// For example, 5 would return 3 counting the numbers 1, 3, and 5.
int CountOddNumbers(int input)
{
    double divResult = Math.Abs(input / 2.0);
    return (int)Math.Ceiling(divResult);
}
```

If you make this change, do not execute the cell, and then execute the `CountOddNumbers(-1)` cell, you should see the curious result shown in *Figure 2.15*:

```
// This method counts all odd numbers leading up to the number
// For example, 5 would return 3 counting the numbers 1, 3, and 5.
int CountOddNumbers(int input)
{
    double divResult = Math.Abs(input / 2.0);
    return (int)Math.Ceiling(divResult);
}
```

✓ 0.0s csharp - C# Script Code

```
CountOddNumbers(-1)
```

✓ 0.0s csharp - C# Script Code

0

Figure 2.15 – Seeing the result of invoking an out-of-date method

Here, our logic now looks like it should handle this, but we're still seeing the incorrect result of **0** for an input of **-1**. This is because *edits to previously executed cells don't actually matter until you re-execute the cell*. In other words, we're still working with the old version of `CountOddNumbers` until we rerun the `CountOddNumbers` method definition.

We could do this by executing that cell and then executing the `CountOddNumbers` cell, but let's instead take this opportunity to showcase the **Run All** button.

The **Run All** button is located on the Polyglot Notebooks toolbar at the top of your notebook, as shown here:

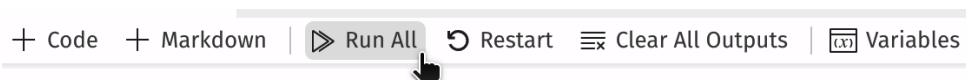


Figure 2.16 – The Run All button

When executed, this will execute every cell in your notebook from top to bottom. This makes it a great way of making sure all cells in your notebook have been executed and executed in the notebook's intended sequence.

When we run all of our cells, we now see the correct version of `CountOddNumbers` executed and an input of **-1** returning the expected output of **1**.

Now that we've seen a simple definition of a method, let's look at what a class definition looks like in Polyglot Notebooks.

Declaring classes

Polyglot Notebooks does not use a simplified version of C# or .NET, and as a result, it can work with full classes and objects. In fact, we'll need to work regularly with pre-defined classes to perform data analytics, visualization, and machine learning tasks, as you'll see in later chapters.

You can define classes, records, and structs in a C# cell just like you can in C# code.

To illustrate this, let's define a `Book` class by adding a new cell with the following code:

```
public class Book
{
    public required string Title {get; init;}
    public required string Author {get; init;}
    public required string ISBN {get; init;}
}
```

This cell defines a new `Book` class with three required `string` properties: `Title`, `Author`, and `ISBN`.

required and init

The class definition uses the `required` and `init` keywords, which are newer C# language features introduced in C# 11 and 9 respectively. The `required` keyword indicates that when an object is created, this property must be non-null. The `init` keyword, on the other hand, indicates the property is settable only in constructors or in object initializers, as we'll see shortly. While you may accuse me of "showing off" by including these language features, I'm trying to illustrate the fact that Polyglot Notebooks has the full power of .NET and C#.

Now that we've defined our class, let's instantiate it and see how Polyglot Notebooks handles it when we leave our object as the last value in a code cell. Add a cell with a definition like the following code:

```
Book thisBook = new() {
    Title = "Data Science with .NET and Polyglot Notebooks",
    Author = "Matt Eland",
    ISBN = " 978-1-83588-296-2"
};

Console.WriteLine($"You are currently reading {thisBook.Title}");
thisBook
```

When you run this code, it will use the **object initializer** for `Book` to instantiate the object and set its properties. We then can access those properties in the standard manner, just like we are with the `Title` property on the `WriteLine` call.

The final line of the cell tells Polyglot Notebooks that we want it to display the value of `thisBook` below the cell.

While you might think that this would be the result of calling `ToString()`, much like `Console.WriteLine(thisBook)` would do, Polyglot Notebooks actually does something way more awesome: it displays all public properties on the object and their values, as shown in *Figure 2.17*:

You are currently reading Data Science with .NET and Polyglot Notebooks

▼ **Submission#9+Book**

Title	Data Science with .NET and Polyglot Notebooks
Author	Matt Eland
ISBN	978-1-83588-296-2

Figure 2.17 – The `thisBook` variable formatted as a table

This is one of my favorite features in Polyglot Notebooks because it makes it trivial to inspect the current public state of any object.

If you wanted to do this manually, you'd need to do a series of `Console.WriteLine` statements for each property you wanted to display. Even then, the results wouldn't be as well formatted following this tabular structure.

Class names in Polyglot Notebooks

You may notice the object is labeled as `Submission#9+Book` in *Figure 2.17*. This is a quirk of how .NET Interactive manages class definitions internally. Here, `Submission#9` indicates the cell is the ninth cell executed. I've never seen this nuance affect the functionality of any notebook code, but if you write code that displays the name of a class using the `nameof` keyword or reflection, this is something to be aware of.

Before we move on, I want to reinforce the point one last time: this is the full power and capability of .NET at our fingertips. If you want to define methods on your classes, you can. If you want to build abstract or sealed classes, inherit from other classes, or implement interfaces, those are all normal things you can do in C# code in Polyglot Notebooks.

On that note, let's see how Polyglot Notebooks handles languages other than C#.

Working with other languages

Polyglot Notebooks wouldn't be appropriately named if all it could do was C#, so let's take a look at support for an additional language: F#.

For those who haven't used it before, F# is a **functional programming language** built on top of .NET. Like C# or VB, you have access to all of the base class library aspects of .NET and anything that compiles down to code executed by the **common language runtime (CLR)**. This means that F# code can work with generic collections, LINQ, SQL databases, and any other aspect of .NET you're used to as a C# developer.

Since F# code is executed by the CLR just like C#'s code is, this means that F# projects can rely on assemblies compiled in C# and C# can rely on assemblies compiled in F#.

Don't know F#? Don't panic!

I don't assume that you've programmed in (or even seen) F# before in order to read this book and I'm choosing examples likely to help you get a sense of F# code without dwelling on functional programming. See the *Further reading* section at the end of the chapter for additional learning resources on F#.

To work with F#, add a new code cell, then click on the **Select cell kernel** area in the lower right of the cell to change the language from C# to F#, as shown in *Figure 2.18*:

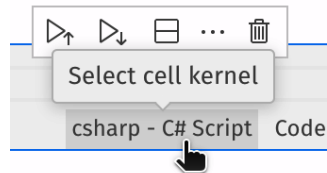


Figure 2.18 – Changing the cell kernel

Next, select **fsharp - F# Script** from the list of supported languages. Now, let's add some sample F# code into the cell:

```
let mutable maxNum = 15

let fizzbuzz n =
    match n with
    | n when n % 15 = 0 -> "FizzBuzz"
    | n when n % 5 = 0 -> "Buzz"
    | n when n % 3 = 0 -> "Fizz"
    | _ -> n.ToString ()

[1..maxNum]
|> Seq.iter (fun x -> printf "%s " (fizzbuzz x))
```

This code tackles a common interview coding task called “FizzBuzz” where the programmer must write a small program to print out numbers from 1 to some larger number.

For numbers that are divisible by 3, the program should print `Fizz`; for numbers divisible by 5, the program should print `Buzz` and the program should print `FizzBuzz` when numbers are divisible by 3 and 5. Numbers that are not divisible by either 3 or 5 are printed out on their own. This code declares a variable named `maxNum` and sets it to 15. We need to mark the variable as `mutable` so it can be changed by a cell later on in this chapter.

Next, the code defines a `fizzbuzz` function that takes in a parameter named `n`. The function uses pattern matching to find cases where `n` is divisible by 3, 5, and 15 (divisible by both 3 and 5) and returns the appropriate value. For all other cases, the `_` case just returns the number in string format.

The final portion of this code takes a sequence of numbers from 1 to `n` (set to 15 earlier) and prints the value returned from the `fizzbuzz` function for that number.

Does this code look familiar?

If this F# code looks a lot like modern C# code, there's a reason for that. As programming evolves over the years, C# continues to be updated with new features such as **switch expressions** and **pattern matching**. However, C# was not intended to be a functional programming language from its inception, while F# was. As a result, you'll find that functional programming in F# sometimes has features that C# doesn't have, such as easy-to-use **discriminated unions**.

When we execute our F# cell, you should see the following output:

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

Additionally, since the `maxNum` variable and `fizzbuzz` function were declared in the global scope, they are available in subsequent cells.

In fact, if we look at the **Variables** view now, you'll see the `maxNum` variable and its value:

Name	Value	Type	Kernel	Actions
<code>i</code>	2	System.Int32	csharp - C# Script	
<code>thisBook</code>	Submission#8+Book	Submission#8+Book	csharp - C# Script	
<code>maxNum</code>	15	System.Int32	fsharp - F# Script	

Figure 2.19 – `maxNum` appearing in the Variables view

Once you have several languages in the same notebook, variable sharing gets a little tricky, so let's talk about that next.

Sharing variables between languages

When I first tried Polyglot Notebooks, I had an immediate thought to try to work directly with an F# variable from C# code.

Let's try that now by adding a new code cell and changing its language to C#. Next, add a line of code attempting to reference the `maxNum` variable:

```
Console.WriteLine($"The value in maxNum is {maxNum}");
```

When you run this, you should see a “red squiggly line” under `maxNum` and get a CS0103 compiler error below your cell, as shown in *Figure 2.20*:

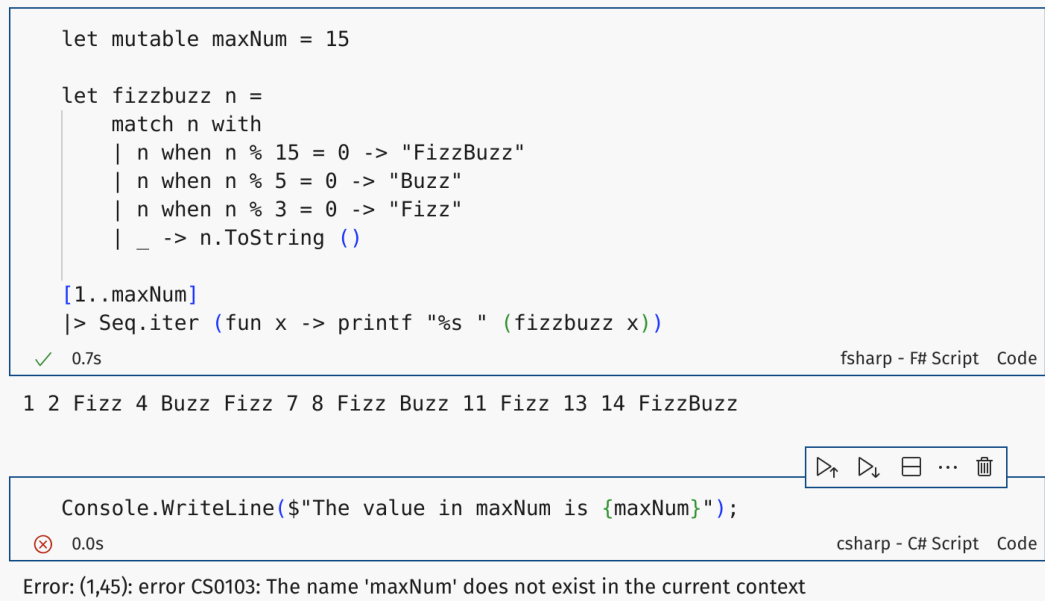


Figure 2.20 – A compiler error shown in the notebook

As we saw earlier, `maxNum` is available in the **Variables** view, so what's going on?

It turns out that each language has its own kernel that is part of .NET Interactive. This means that your C# variables are kept in a separate area from your F# variables, and so on for other languages.

Effectively, your C# code cannot reference F# variables and vice versa unless those variables are explicitly shared across kernels.

Exporting variables

You can export a variable from one language to another in a few different ways. The easiest way is to find the variable you want to share in the **Variables** view and then click on the **Share value** button next to the variable in the **Actions** column, as shown in *Figure 2.21*:

Name	Value	Type	Kernel	Actions
i	2	System.Int32	csharp - C# Script	
thisBook	Submission#8+Book	Submission#8+Book	csharp - C# Script	
maxNum	15	System.Int32	fsharp - F# Script	

Share value maxNum from fsharp - F# Script kernel

Figure 2.21 – The shortcut to share variables to other kernels

Next, you'll be asked to select the language you want to have access to the variable's value, as shown in this dialog:

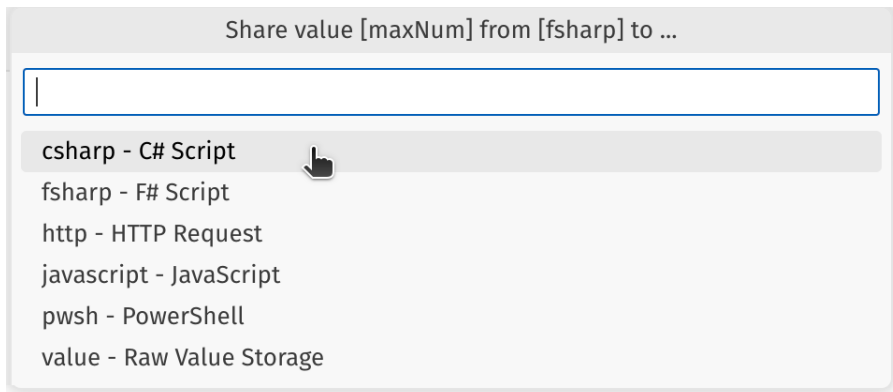


Figure 2.22 – Specifying the destination language kernel

We'll choose **csharp - C# Script** to give C# access to F#'s maxNum variable. This adds a new C# code cell to the bottom of your notebook with the following contents:

```
#!set --value @fsharp:maxNum --name maxNum
```

This **#!set** command is the first of several **magic commands** we'll explore throughout the book.

Magic commands are not secretive techniques only known to wizards. Instead, magic commands are special directives Polyglot Notebooks provides that allow you to control and customize the notebook experience.

In this case, the **#!set** magic command is setting a new C# variable with a *name* of maxNum to the *value* currently in the F# kernel's maxNum variable.

Data type note

Because the C# kernel isn't sure what you'll be using the numeric value for, the variable comes over as a double instead of as an integer, giving you some added flexibility.

When you run this line, you'll see a new variable in the **Variables** view, this time listed under the C# kernel, as shown in *Figure 2.23*:

Name	Value	Type	Kernel	Actions
i	2	System.Int32	csharp - C# Script	
thisBook	Submission#8+Book	Submission#8+Book	csharp - C# Script	
maxNum	15	System.Double	csharp - C# Script	
maxNum	15	System.Int32	fsharp - F# Script	

Figure 2.23 – The F# and C# kernels own different maxNum variables

Now, if you try to run the `Console.WriteLine($"The value in maxNum is {maxNum}");` cell, you'll get a result:

```
The value in maxNum is 15
```

While, at first, you might think that the `#!set` line gave C# access to the F# `maxNum` variable, this is not the case. Instead, the `#!set` command *defined a new variable* in the C# kernel with the same name and value as the F# kernel's `maxNum` variable.

Let's illustrate this by adding a new code cell, setting the language to F#, and adding the following code:

```
maxNum <- 5
maxNum
```

Executing this cell sets `maxNum` to 5 and displays that new value below the cell. Note that this would not work if we did not define the F# variable as `mutable` earlier.

Take a look at the **Variables** view and notice how the C# and F# versions of `maxNum` now have different values, as seen in *Figure 2.24*:

Name	Value	Type	Kernel	Actions
i	2	System.Int32	csharp - C# Script	
thisBook	Submission#8+Book	Submission#8+Book	csharp - C# Script	
maxNum	15	System.Double	csharp - C# Script	
maxNum	5	System.Int32	fsharp - F# Script	

Figure 2.24 – Different values of each kernel's maxNum variable

The different values in different versions of `maxNum` reinforce the concept that the `#!set` magic command only creates a copy of the current value. It does not create any sort of linkage between the two variables. While this may seem like a major limitation of Polyglot Notebooks, in my experience, this isn't a massive obstacle.

While I may switch between languages while teaching Polyglot Notebooks to new learners, in practice, I tend to shift between languages less frequently and at pre-defined handoff points. For example, I may choose to fetch data using C# and then use F# to generate visualizations of that data. In this case, I'm really handing off the data once and don't need to coordinate values between languages frequently.

We'll see this in action in future chapters as we use our new notebook superpowers to accomplish data analytics, machine learning, and artificial intelligence tasks.

For now, let's conclude this chapter with a discussion of what to do when notebooks aren't working properly.

Troubleshooting notebook execution

Polyglot Notebooks is powerful and allows you to do some really interesting things, as we'll see over the remaining chapters of this book.

However, it's important to know how to handle potential issues you encounter when running your notebooks.

Resolving compiler errors

Because the code we send Polyglot Notebooks must be compiled, that means you still need to worry about writing valid code and resolving compiler errors.

VS Code provides reliable code completion and syntax highlighting for the languages supported by Polyglot Notebooks. This means that you get red squiggly lines when VS Code detects compiler issues with your code.

If you try to run code that doesn't compile, you'll see the compiler error detailed below the cell, as shown here:

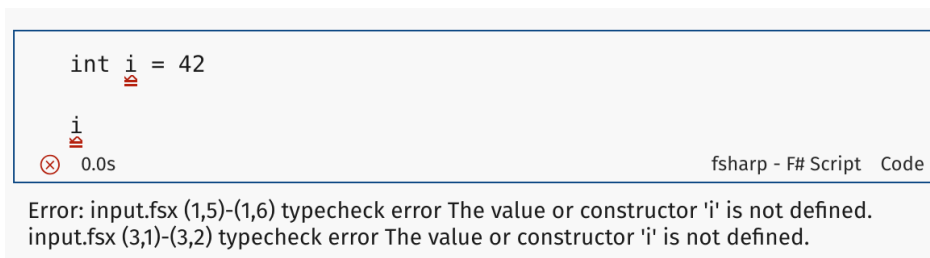


Figure 2.25 – A compiler error below a code cell

The steps for resolving these issues are largely the same for programming in general: read what the message is telling you, search online for the error code and error text, and re-read your code looking for mistakes.

For me, the most common causes of compiler errors when working with Polyglot Notebooks are the following two mistakes:

- Putting a semicolon at the end of the final line in C# when I'm trying to display the results of a variable
- Not correctly setting the cell's language to the language I'm trying to write code in

Otherwise, your existing programming skills will be very relevant when working with Polyglot Notebooks.

Problems with notebook execution

Polyglot Notebooks is still a relatively new technology and you will occasionally still encounter scenarios where a notebook seems to “get stuck.”

In these scenarios, your code compiles but executing even simple lines can take a very long time or not complete at all.

When this occurs, the .NET Interactive kernel can be jogged into behaving by trying a few different things in order:

1. Click on the **Stop Execution** button on the toolbar. This will attempt to stop the execution of the currently running cell, which can restore control of your notebook.
2. If this fails, you can click the **Restart** button on the toolbar. This will restart the .NET Interactive kernel.

Kernel variables and the Restart command

Note that restarting the kernel will discard any variables the kernel is currently tracking. You will very likely need to rerun earlier cells in the notebook to get back to a good state.

3. In the case where restarting the notebook does not succeed, I recommend saving your current file in VS Code, closing VS Code entirely, and then reopening it. This usually causes the kernel to restart and can help restart from a fresh kernel.

These steps are rarely ever necessary in recent versions of Polyglot Notebooks, but I do still sometimes find myself needing them to resolve kernel stability issues.

Diagnostic output for Polyglot Notebooks errors

If you encounter extremely unusual issues running your notebook or installing the Polyglot Notebooks extension, there are two sources of diagnostic output available with Polyglot Notebooks.

Open the **Output** view in the footer of your notebook. If this is not visible, you can open it by pressing *Ctrl + Shift + P* to bring up the **Command Palette** and then search for and execute the **View: Toggle Output** command.

Next, click on the drop-down at the right edge of the view to select the data source you're interested in. Select **Polyglot Notebook : diagnostics**, as shown in *Figure 2.26*:

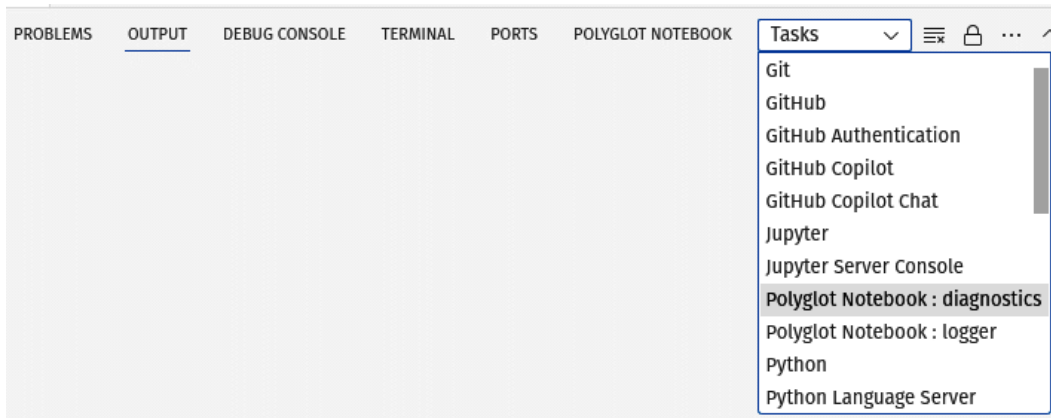


Figure 2.26 – Selecting the output source

The Polyglot Notebooks diagnostics output will display the current state of your notebook and may reveal potential issues. For example, when I was unable to fully install Polyglot Notebooks due to a NuGet feed issue, the issue showed up in this diagnostics output.

The other diagnostic option we have available is the **Polyglot Notebook : logger** output source. This logger produces an additional stream of information about your environment and may help pinpoint issues that are not evident by other means.

If all else fails, you can always search and submit issues to the Polyglot Notebooks team based on issues you encounter.

Issues and the Polyglot Notebooks repository

Like most other aspects of .NET, the Polyglot Notebooks project is open source, with its GitHub repository located at <https://github.com/dotnet/interactive/>.

This repository is a great source of additional information on Polyglot Notebooks, including release plans, frequently asked questions, and open issues.

To see active issues, click on the **Issues** tab or navigate to <https://github.com/dotnet/interactive/issues/>. This will show you a list of current issues.

Next, type in some search terms to help you find any issues similar to your own.

If you don't see anything that looks like the problem you're encountering, click **New Issue**. You may need to log in or sign up for GitHub if you weren't logged in before.

After this, select the type of issue you need to create and follow the instructions in the issue template. I've had the pleasure of interacting with the Polyglot Notebooks team off and on for a few years now and can attest that the team on the project is fantastic and very responsive to issues and feedback.

Hopefully, you won't need their help, but it's nice to have the option available if things go wrong.

Summary

In this chapter, we covered the process of installing Polyglot Notebooks, creating and executing code cells, defining Markdown cells, adding methods and classes to our code, working with variables, and how languages can share data between kernels. We also identified a few steps you can take when your notebook isn't working properly.

Chapter code

The final code for this chapter is available as a .dib notebook in the book's repository (<https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks>) in the Chapter02/Chapter2.dib file. Subsequent chapters in this book will also have their notebooks placed in chapter-specific folders.

This chapter gave you enough to install and get started using Polyglot Notebooks for your own experiments. I'd encourage you to spend some time trying various language features to see how they behave in Polyglot Notebooks as working with notebooks is the best way to gain comfort.

In the next chapter, we'll move beyond these simpler demos and get into the beginnings of data analysis and data science workflows using Polyglot Notebooks.

Further reading

Here are a few extra resources related to things we covered in this chapter:

- *Polyglot Notebooks GitHub repository*: <https://github.com/dotnet/interactive/>
- *Polyglot Notebooks VS Code Extension Page*: <https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.dotnet-interactive-vscode>
- *A discussion of the .dib file format*: <https://github.com/dotnet/interactive/issues/467>
- *VS Code User Interface reference*: <https://code.visualstudio.com/docs/getstarted/userinterface>
- *C# Arithmetic Operators*: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/arithmetic-operators>
- *F# Language Resources*: <https://fsharp.org/>
- *Functional Programming, the .NET Way*: <https://visualstudiomagazine.com/articles/2016/02/10/functional-programming-fsharp.aspx>
- *Functional Programming in C#*: <https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/functional-programming-in-csharp>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/polyglot-notebooks/>

3

Getting Data and Code into Your Notebooks

Now that we've discussed how to install and work with Polyglot Notebooks to perform basic operations, let's investigate loading different data sources into your notebook.

In order to perform data analysis or machine learning at any scale, you'll need to be able to work with the files, formats, and query languages used by different data sources.

You'll also need to be able to pull in other code files and libraries so that you can work with larger objects and previously existing pieces of code.

In this chapter, we'll cover the following:

- Importing code and NuGet packages
- Reading CSV data
- Getting JSON data with PowerShell
- Building DataFrames from objects
- Connecting to databases with SQL
- Querying Kusto clusters with KQL

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter03` folder, along with relevant data sources and scripts.

Importing code and NuGet packages

In the last chapter, we saw how code cells can be used to define variables, classes, and methods. These features are fantastic when you need to write small pieces of code to accomplish some goal.

However, it can be helpful to store code outside of notebooks and import it in when needed. This keeps the details of the code outside of your notebook and allows the notebook to focus on the task that it is trying to accomplish, instead of the supporting code. This is particularly useful when working with code that's been written elsewhere.

In this section, we'll cover importing individual files, projects and assemblies, and NuGet packages.

Importing code files

If you have a C#, F#, HTML, JavaScript, PowerShell, or other type of supported code file, you can import that code into your notebook using the `#!import` **magic command**.

`#!import` works by taking in a relative or absolute path to a supported code file and effectively injecting it into your notebook.

In other words, `#!import` effectively takes the contents of a code file and adds an invisible code cell with the contents of the loaded file. This is handy when the specific code is not the central part of your notebook, when you want to share code between multiple notebooks, or when code is simply too long and involved to be digestible in your notebook.

Let's see how it works. First, let's assume that our notebook works in the same directory as a file named `FootballPlayer.cs`.

International football

We'll be dealing with Football data for much of this book going forward, and the term "football" can be confusing. In this book, when I refer to football, I'm talking about what fellow Americans might refer to normally as "soccer."

`FootballPlayer.cs` contains a C# class with the following code:

```
public class FootballPlayer
{
    public string FirstName {get; set;}
    public string LastName {get; set;}
    public string FullName => $"{FirstName} {LastName}";
    public string Position {get; set;}
    public int Number {get; set;}
    public string CurrentClub {get; set;}
    public string Nationality {get; set;}
```

```
public bool IsRightFooted {get; set;} = true;
public int HeightInCm {get; set;}
public int RookieYear {get; set;}
}
```

In our notebook, we can import this entire C# file with the following C# code cell and magic command:

```
#!import FootballPlayer.cs
```

When you execute this cell, the C# kernel looks for a file named `FootballPlayer.cs` in the same directory as your notebook, and then it executes the code in that file as if the code cell contained that code to begin with.

Using relative and absolute paths

If you wanted to reference a code file that lived in a different directory, you could use either a relative or absolute path. For example, if your `FootballPlayer.cs` file lived in a directory named `CodeFiles` that was in the same directory as your notebook, you could use `#!import CodeFiles/FootballPlayer.cs`.

Alternatively, if you wanted to use an absolute path and your files were in your `C:\Code` directory on Windows, you could use `#!import C:\Code\FootballPlayer.cs`.

When the code executes, it defines the `FootballPlayer` class. This means we should be able to use it in subsequent cells.

Let's try that now with the following object declaration code cell:

```
FootballPlayer player = new()
{
    FirstName = "Matt",
    LastName = "Eland",
    Position = "Keeper",
    Number = 42,
    Nationality = "United States",
    CurrentClub = "Packt Authors",
    IsRightFooted = true,
    RookieYear = 2023,
    HeightInCm = 168 // 5ft, 6in
};
player
```

This code defines a new instance of the `FootballPlayer` class and stores it in the `player` variable. In this case, we're creating a player representing myself and assigning him as a goaltender for the *Packt*

Authors football club – a player assignment that’s certain to be more beneficial to the club’s opponents, given my limited athletic prowess!

Because the cell ends with the `player` variable, with no semicolon, we can see the visualization of the constructed object, as shown in *Figure 3.1*:

Submission#3+FootballPlayer	
FirstName	Matt
LastName	Eland
FullName	Matt Eland
Position	Keeper
Number	42
CurrentClub	Packt Authors
Nationality	United States
IsRightFooted	True
HeightInCm	168
RookieYear	2023

Figure 3.1 – The player’s properties visualized

By keeping the class definition stored in a separate file, we are able to keep the notebook focused on just the task we’re trying to accomplish.

Importing .dib and .ipynb files

The `#!import` magic command is not just limited to importing code. It can also import the contents of other notebooks. By using `#!import` with a `.dib` or `.ipynb` notebook file, you can have one notebook include the cells of another notebook during its execution. This is handy when you have several different notebooks that want to perform identical steps and don’t want to duplicate code or Markdown cells between notebooks.

Let’s see how this works. First, we’ll have a `NestedNotebook.dib` notebook file that contains code and Markdown cells, as shown in *Figure 3.2*:

This is a nested notebook

This is a markdown cell inside of `NestedNotebook.dib`

```
string message = "This variable was defined in NestedNotebook.dib";  
  
message
```

csharp - C# Script Code

Figure 3.2 – A small notebook that we'll import later

Note how the code block defines a message string variable and displays it below the cell. This will be relevant later. Next, back in our `Chapter3.dib` notebook, we'll add three cells – a Markdown cell, a code cell, and an additional Markdown cell. The first markdown cell contains a simple note that indicates we're about to import, as shown here:

```
Below this cell we'll import `NestedNotebook.dib`
```

Next, we'll use our `#!import` magic command to import from the `NestedNotebook.dib` file:

```
#!import NestedNotebook.dib
```

As you can see, this looks just like `#!import` for a code file.

Finally, we'll use another Markdown cell to indicate that we've finished the import:

```
The previous content was from `NestedNotebook.dib`
```

These Markdown cells aren't explicitly necessary, but they should help you see how the imported notebook is integrated into the `Chapter3.dib` notebook.

When we run this notebook, we can see the contents of `NestedNotebook.dib` appear below the `#!import` cell and just before our final Markdown cell, as shown in *Figure 3.3*:

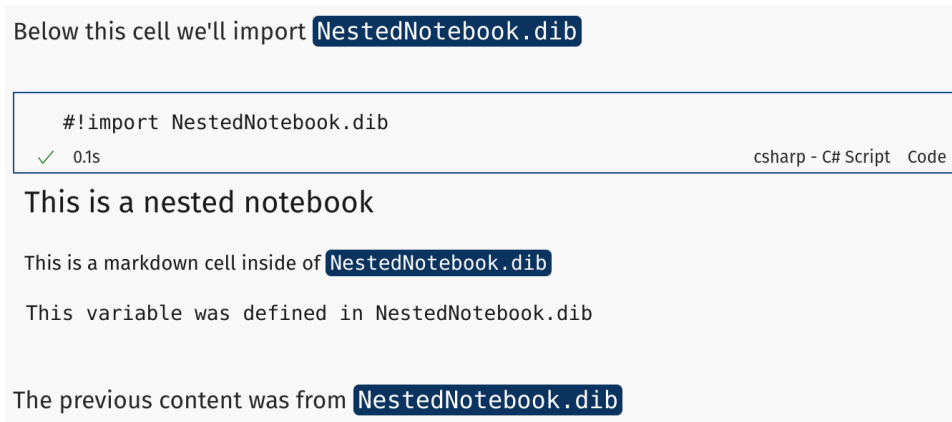


Figure 3.3 – The nested notebook shown below the `#!import` cell

Because the imported notebook is executed by your kernel, any variables, classes, or methods defined in that kernel are accessible to you after `#!import` runs.

We can demonstrate that by adding a C# code cell that displays the value of the message variable:

```
message
```

When executed, we can see that this displays the string defined in `NestedNotebook.dib`:

```
This variable was defined in NestedNotebook.dib
```

The `#!import` capability works with `.dib` and `.ipynb` notebook file formats, and it is exciting for scenarios where you might have many similar notebooks that need to repeat major steps or significant amounts of documentation.

The limits of the `#!import` magic command

While the `#!import` magic command is extremely useful, it has a limitation, which we should discuss before we move on.

As we discussed earlier, `#!import` effectively loads the contents of a file and runs them as if they were code in your notebook. That's actually a bit different from how code files usually work in .NET.

In C#, it's almost universal for code in one file to reference code in another file, which, in turn, may reference code in other files or even back in the original files.

With C# .NET projects, this doesn't matter much, as long as the code is all valid and your code is either all in the same project or you have dependencies set up between projects. This is because C# files are not restricted to running in a specific order during compilation (although, interestingly enough, F# files do have a specific compile order).

Since Polyglot Notebooks execute code sequentially, importing a class file that relies on another class file that hasn't been loaded yet will result in a kernel error.

Let's see a case where this will be a problem.

First, imagine you have a `Game.cs` file with the following C# code:

```
public class Game
{
    public FootballClub Home {get; set;}
    public FootballClub Away {get; set;}
    public DateTime ScheduledDate {get; set;}
    public int HomeScore {get; set;}
    public int AwayScore {get; set;}
}
```

Note how this class references the `FootballClub` class, which is defined in a `FootballClub.cs` file.

The C# compiler knows to look at classes defined in all files within a project during compilation, but Polyglot Notebooks doesn't have this luxury if we import the file on its own with `#!import`, as shown in *Figure 3.4*:

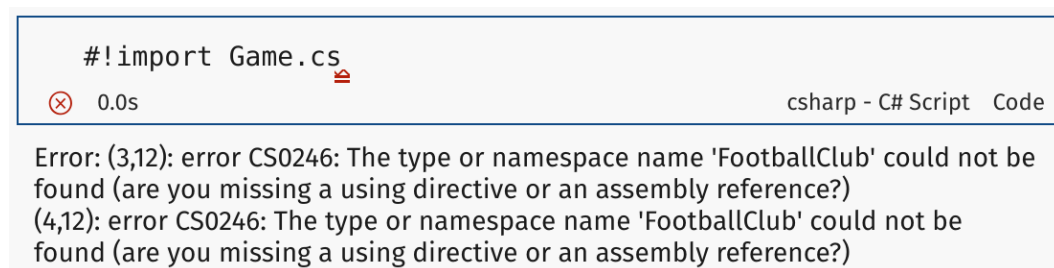


Figure 3.4 – A compiler error appearing in a notebook

You can solve this problem by importing `FootballClub` before `Game`.

Of course, if `FootballClub` depends on `Game`, and `Game` depends on `FootballClub`, you'll have the same problem regardless of the order in which you import these files. You can solve this problem by defining both classes in the same code file. In that case, Polyglot Notebooks will compile both definitions at the same time and be able to make sense of the relationship.

Doubtless, a number of you reading this right now are wondering why we don't just import project files or even solution files. Don't worry; we'll talk about that more at the end of this section.

For now, let's move on to an extremely common need – importing NuGet packages.

Importing NuGet packages

We've now explored how to import code files and even other notebooks, but most of the time, what people really want to import is other **assemblies**.

Microsoft defines an assembly as “*a collection of types and resources that are built to work together and form a logical unit of functionality*” (<https://learn.microsoft.com/en-us/dotnet/standard/assembly/>).

While this is true, I've found better results with new learners by referring to an assembly as *a compiled .NET project*. This helps clarify that larger solutions are composed of multiple assemblies, since solutions typically have many different projects to accomplish various tasks.

Default assemblies

All .NET code will rely on external assemblies defining necessary definitions for classes and other data structures. In Polyglot Notebooks, you'll have access to a few assemblies for every project. This is just like when you create a new C# or F# project in Visual Studio or VS Code.

With C# and F#, you get access to the following assemblies without needing to explicitly install them:

- `System`
- `System.Collections`
- `System.Collections.Generic`
- `System.Linq`
- `System.Text`
- `System.Threading.Tasks`

These common parts of .NET's base classes can be referenced by adding a `using` statement in C# or an `open` statement in F#. However, if you want to reference more specialized assemblies, you'll have to import NuGet packages for them.

NuGet packages

If you're newer to .NET and aren't familiar with **NuGet**, it's a way of managing dependencies for .NET applications. In IDEs such as Visual Studio, VS Code, and Rider, you can use **NuGet Package Manager** to install, remove, and upgrade dependencies for your project. These dependencies are downloaded through **NuGet feeds**, which contain all published versions of the assemblies they track. Most .NET projects use the NuGet feed from `nuget.org`, but there are alternatives, and some organizations host their own internal NuGet feeds for security reasons.

Notebooks are different from C# or F# projects, and they don't have their own list of external dependencies for a tool such as NuGet Package Manager.

The #r magic command

In order to work with existing NuGet packages, you can direct Polyglot Notebooks to download and load requested packages into your notebook by using the **#r magic command**.

For example, if you wanted to install the `Newtonsoft.Json` package using its latest version, you could do so with the following C# or F# code cell:

```
#r "nuget:Newtonsoft.Json"
```

When run, this lists the packages that were installed, along with their versions:

```
Installed Packages
  Newtonsoft.Json, 13.0.3
```

With `Newtonsoft.Json` added, we can now reference it with a C# `using` or F# `open` statement, and then use its classes.

The following C# code cell does just that – it uses our `Newtonsoft.Json` NuGet package to serialize the player object we defined earlier into the JSON format:

```
using Newtonsoft.Json;

Formatting format = Formatting.Indented;
JsonConvert.SerializeObject(player, format)
```

The result of the formatted `SerializeObject` call is displayed below the cell:

```
{
  "FirstName": "Matt",
  "LastName": "Eland",
  "FullName": "Matt Eland",
  "Position": "Keeper",
  "Number": 42,
```

```
"CurrentClub": "Packt Authors",  
"Nationality": "United States",  
"IsRightFooted": true,  
"HeightInCm": 168,  
"RookieYear": 2023  
}
```

As you can see, the `#r` magic command lets us reference .NET assemblies and use their capabilities in our notebooks. This greatly extends what we can do with our notebooks, as you'll see throughout the rest of this book.

Referencing specific versions and pre-release packages

If you want to use `#r` to install a *specific version* of a package, you can specify the desired version after the package name by adding a comma and the package number, as shown here with `Azure.Core`:

```
#r "nuget:Azure.Core,1.38.0"  
Installed Packages  
    Azure.Core, 1.38.0
```

When we ran this cell it downloaded the specific requested version of a package, even if a more recent version was available. We'll do this throughout the book in an attempt to avoid breaking changes from new releases in the future, thus impacting your reading experience.

Caution

One thing to be aware of is that if you install a specific version of a package, and then try to run the `#r` magic command to install a different version of the package, it will not work the way you expect. The kernel does not expect you to try to install different versions within the same execution session, so the first version you install will be used, even if you use `#r` to request a different version.

This isn't as large of a problem as it may seem because you can restart the kernel to have it re-evaluate your `#r` magic command, and situations where you'd legitimately want to use two different versions are extremely rare.

Sometimes, you'll want to reference the latest version of a package including pre-release packages. This is particularly necessary when working with preview SDKs that *only* have pre-release versions available.

Let's see this in action by referencing the `Azure.AI.Vision.ImageAnalysis` package, which I frequently use for computer vision tasks using pre-trained models on Azure.

Looking at this package on nuget.org, you can see that it only has beta versions available, as shown in *Figure 3.5*:

Azure.AI.Vision.ImageAnalysis 1.0.0-beta.2

Prefix Reserved

.NET Standard 2.0

This is a prerelease version of Azure.AI.Vision.ImageAnalysis.

Package Manager PackageReference Paket CLI Script & Interactive Cake

```
> dotnet add package Azure.AI.Vision.ImageAnalysis --version 1.0.0-beta.2
```

README Frameworks Dependencies Used By **Versions** Release Notes

Version	Downloads	Last updated
1.0.0-beta.2	2,538	a month ago
1.0.0-beta.1	6,159	2 months ago
0.15.1-beta.1	61,665	6 months ago
0.13.0-beta.1	9,231	9 months ago
0.11.1-beta.1	7,414	5/2/2023
0.10.0-beta.1	4,866	4/3/2023
0.9.0-beta.1	2,675	3/4/2023
0.8.1-beta.1	386	2/4/2023
0.8.0-beta.0.33537970	104	1/17/2023
0.8.0-beta.0.33395485	129	1/10/2023

Figure 3.5 – NuGet.org’s listing of package versions

While we could reference the `1.0.0-beta.2` package explicitly, we could also use a special `,*-*` syntax to tell the `#r` magic command that we want the latest package available, including pre-release packages.

As usual, we’ll do this in a standard C# code cell:

```
#r "nuget:Azure.AI.Vision.ImageAnalysis,*-*"
```

When this is executed, it pulls down the latest version, which was 1.0.0-beta.2 at the time of writing:

```
Installed Packages
- Azure.AI.Vision.ImageAnalysis, 1.0.0-beta.2
```

If we had failed to include `, **` in the `#r` statement, older versions would have been considered until it found a version that was not marked as pre-release, or threw an error if no public version was found.

Referencing specialized NuGet feeds

If you are trying to install a package from a non-public NuGet feed, you can register an additional NuGet feed using the `#i` magic command, with syntax such as the following line of code:

```
#i "nuget:N:\DevTeam\NuGetFeed"
```

This code adds a NuGet feed from the `DevTeam\NuGetFeed` directory on a network share called drive N.

Creating and managing NuGet feeds is beyond the scope of the book, but if you're interested in more details, see the *Working with NuGet packages* article referenced in the *Further reading* section of this chapter for more information on the `#i` magic command.

Referencing local packages

In some cases, you may want to install assemblies from sources outside of NuGet.

If you had a `DLLs` directory in the same directory as your notebook and a `SomeAssembly.dll` file in that directory, you could import it with the following command:

```
#r "DLLs\SomeAssembly.dll"
```

Note that this use of the `#r` magic command doesn't start with `#r "nuget :` as every other example does. That's because we're directly referencing the `.dll` file by location on disk.

What are DLLs?

DLLs are **Dynamic-Link Library** files that contain code that can be executed or referenced by other code. Most .NET projects compile down to `.dll` files, so think of a `.dll` in the .NET context as a specific form of a compiled assembly.

You may be wondering when you might use local `.dll` files. One scenario where these files may be useful would be if you have a .NET solution somewhere on your computer, and you want to load the output of compiling it directly into Polyglot Notebooks.

Another use case would be if you wanted to be able to load `.dll` files from NuGet packages in places where you had no internet access. I once gave a conference presentation on Polyglot Notebooks from a movie theater with no internet and unreliable cellular coverage.

Because I knew about the lack of internet access ahead of time, I downloaded the .dll files for the assemblies I wanted to demonstrate to the audience and was able to reference them by their local path, giving a successful demonstration.

Importing project files

While my example of referencing a .dll for offline demos is a legitimate one, you're probably more likely to use this to point to the compiled results of projects you have locally on your development machine.

This is a critically important scenario for Polyglot Notebooks because it does not currently allow you to import .csproj or .fsproj files, using either the `#!import` or `#r` magic command.

I understand this decision because allowing importing project files would involve greatly expanding the scope of Polyglot Notebook's usage of the kernel to compile code, and it would exponentially increase the number of edge cases and error scenarios that Polyglot Notebooks has to deal with.

Instead, you should compile your .NET assemblies down to .dll files and then either publish them on a NuGet feed you can reference from Polyglot or reference the files directly with the `#r` magic command.

If you feel strongly about importing projects, there's an ongoing discussion regarding the proposed feature, which I reference in the *Further reading* section of this chapter as "Discussion of support for referencing C# projects". I encourage you to read the discussion and post a comment if you have a use case that this feature would be particularly helpful for.

For now, let's move on to getting data into your notebooks, starting with CSV data.

Reading CSV data

If you've not worked with CSV files before, **CSV** stands for **comma-separated values**. A CSV file, therefore, is a file consisting of individual rows of data, with each field within a row broken up by a comma.

Understanding CSV data

An example of a CSV file for a list of football players is shown here:

```
First,Last,Pos,Num,Club,Nation,RightFoot?,Height,RookieYear
Fidel,Yost,Midfielder,74,Houston Niners,Serbia,True,163,2022
Annetta,Preslo,Midfielder,30,Code Monekys,Guam,True,175,2019
Lance,Casper,Sweeper,49,Purple Cobras,Maldives,True,180,2019
Tom,Wolf,Left Back,48,Houston Niners,Egypt,False,177,2017
Gillian,Pouros,Sweeper,86,Code Monkeys,Canada,False,185,2020
```

Here, the first row acts as a header and identifies what each value represents in the subsequent row. We then have five additional rows representing individual players.

Most spreadsheet software and text editors can work with CSV files. Additionally, the major relational databases support exporting query results to CSV files. This makes CSV files a lightweight way of storing snapshots of small- to medium-sized data.

Because of the simplicity, portability, and overall prevalence of CSV data, it is not uncommon to perform data analysis and even machine learning tasks against CSV data files.

As a result, we need to be able to load CSV data into memory and perform operations on these datasets. Working with the loaded datasets will be the major focus of *Chapter 4, Working with Tabular Data and DataFrames*, but for now, we'll focus simply on how to load data from CSV files.

Reading CSV data into a DataFrame

In order to work effectively with CSV data, we must first load it into an object called a **DataFrame**. DataFrames are tabular data structures that let you analyze, sort, filter, group, and otherwise manipulate tabular data, such as the data loaded from CSV files.

Let's load our sample CSV file into a DataFrame so that we can see how it works.

First, we'll start by importing the `Microsoft.Data.Analysis` NuGet package into a C# code cell:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
```

Note how we're specifying a specific version here to keep the chapter's code more future-proof for you.

This cell results in a pair of messages:

```
Installed Packages
  Microsoft.Data.Analysis, 0.21.1
Loading extensions from ~/home/matteland/.nuget/packages/microsoft.
data.analysis/0.21.1/interactive-extensions/dotnet/Microsoft.Data.
Analysis.Interactive.dll~
```

The first message is the normal message, letting you know which version of the package was installed. The second message is more interesting, telling us that Polyglot Notebooks found and loaded some extensions from the package that was installed. Extensions modify the way that Polyglot Notebooks behaves fundamentally. In this case, the `Microsoft.Data.Analysis.Interactive` extension provides a rich way of visualizing and paging through tabular data presented in a DataFrame, as we'll see shortly.

We'll revisit the concept of extensions in *Chapter 14, Extending Polyglot Notebooks*, but for now, let's focus on how to work with this new package.

Next, let's use our newly imported library to read our CSV file into a `DataFrame` with a C# code cell:

```
using Microsoft.Data.Analysis;
DataFrame df = DataFrame.LoadCsv("Players.csv");
df.Head(5)
```

This code calls the `LoadCsv` static method on `DataFrame` and tells it to load data from the `Players.csv` file. The end result will be a `DataFrame` object that we will store in a variable called `df`.

The final line in the code cell calls the `Head` method on the `DataFrame` to display the first five rows in the `DataFrame`. The `Microsoft.Data.Analysis.Interactive` extension then formats these results into a nicely shaded table, as shown in *Figure 3.6*:

<i>index</i>	<i>FirstName</i>	<i>LastName</i>	<i>Position</i>	<i>Number</i>	<i>CurrentClub</i>	<i>Nationality</i>	<i>IsRightFooted</i>	<i>HeightInCm</i>	<i>RookieYear</i>
0	Fidel	Yost	Midfielder	74	Houston Niners	Serbia	True	163	2022
1	Annetta	Predovic	Midfielder	30	Columbus Coders	Cape Verde	True	175	2019
2	Lance	Casper	Sweeper	49	Globalgym Cobras	Maldives	True	180	2019
3	Conner	Wolf	Left Back	48	Houston Niners	Gibraltar	False	177	2017
4	Gillian	Pouros	Sweeper	86	Columbus Coders	Uruguay	False	185	2020

Figure 3.6 – A `DataFrame` displaying a set of players

The `DataFrame` we've created isn't just a way of getting a nice tabular display of our CSV data; `DataFrames` are capable of so much more, such as data analysis, data cleaning, and feature engineering, as we'll explore in the next chapter.

Before we move on, I want to highlight a few potentially helpful aspects of the `LoadCsv` method.

Specialized CSV loading scenarios

First, you may find yourself working with CSV files that do not contain a header row – that is, CSV data where the very first line is the actual data you want to load.

By default, `LoadCsv` uses the first row as a list of column names and does not load them as rows of actual data. You can correct this by specifying `header: false` when you load your data, as shown here:

```
DataFrame.LoadCsv("Players.csv", header: false)
```


This causes columns to be named sequentially and the first row to be treated as actual data, as shown in *Figure 3.7*:

<i>index</i>	<i>Column0</i>	<i>Column1</i>	<i>Column2</i>	<i>Column3</i>	<i>Column4</i>	<i>Column5</i>	<i>Column6</i>	<i>Column7</i>	<i>Column8</i>
0	FirstName	LastName	Position	Number	CurrentClub	Nationality	IsRightFooted	HeightInCm	RookieYear
1	Fidel	Yost	Midfielder	74	Houston Niners	Serbia	True	163	2022
2	Annetta	Predovic	Midfielder	30	Columbus Coders	Cape Verde	True	175	2019
3	Lance	Casper	Sweeper	49	Globalgym Cobras	Maldives	True	180	2019
4	Conner	Wolf	Left Back	48	Houston Niners	Gibraltar	False	177	2017

Figure 3.7 – A DataFrame with its first row incorrectly treated as data

In the case of our data, this doesn't make much sense, but if your data doesn't start with a header row, this is an important capability that you'll need.

Renaming columns

If you're wondering how to fix the column names in this example, we'll discuss renaming columns in the next chapter.

For now, let's move on to resolving an error I've encountered a few times before.

Troubleshooting CSV loading errors

Depending on your data, you may sometimes encounter errors loading CSV files, particularly when some columns can have `null` values.

When you load data into a `DataFrame`, it has to infer the type of data contained in each column. This lets it handle columns of boolean values differently than string columns, number columns, or date columns.

The `DataFrame` does this by looking at a subset of initial rows in a larger dataset and trying to identify the type of data contained by those, based on the format of individual rows. It doesn't look at all of the data because Microsoft wants to keep the data-loading experience as fast and streamlined as possible.

Most of the time, this is fine. But occasionally, you'll bump into `FormatException` being thrown when you try to load your data. This exception contains a somewhat unhelpful message, `Input string was not in a correct format`.

This message is particularly maddening because you're not sure what string it is referring to, which row, or which field in the data is being referred to.

When you see this message, it usually means one of two things:

- You have a legitimate issue in your CSV file, where a date is incorrectly formatted, a boolean value is misspelled, or something equally corrupt exists and is throwing off the load process
- The `DataFrame` tried to guess the data type for a column but didn't look at enough rows to get an adequate idea of what type of data that column contained

In the latter case, the problem usually arises because the beginning of your data file has many `null` values for a field. In this case, instructing `LoadCsv` to read additional rows when guessing the data type for each column will fix the issue.

You can accomplish this by specifying a value for `guessRows`, indicating how many rows you want to be read.

The following C# code cell demonstrates this by inferring the data type from the first 25 rows, instead of the default 10-row count used:

```
DataFrame.LoadCsv("Players.csv", guessRows: 25)
```

I find using `guessRows` to be frustrating but occasionally necessary, and I hope the `LoadCsv` experience improves in the future to at least offer a better error message on failure. Before we move on to other data sources, let's talk about a data source closely related to CSV files – TSV files.

Loading TSV and other delimited file formats

TSV files are lesser-used cousins of the more popular CSV file format. A TSV file is a **tab-separated value** file.

This means that TSV files are nearly identical to CSV files, except TSV files use tab characters instead of commas to separate fields. This makes TSV files slightly easier to read than CSV files and also reduces the need to escape comma values, by surrounding values in strings in a traditional CSV file.

Because TSV files are so similar to CSV files, we can actually use the same `LoadCsv` method to load TSV data. We just need to tell the `DataFrame` what our separator character is.

We can do this by specifying the `separator` character, as shown here:

```
DataFrame.LoadCsv("Players.tsv", separator: '\t')
```

Note that the separator takes in a character, not a string, and we're using `\t` to indicate a tab character explicitly. This same separator strategy can be applied to other common delimiters, such as pipe-delimited files that use `|` as a separator between values. In such a case, you'd set `separator: ' | '` to have the `DataFrame` split on each pipe character.

We'll revisit `DataFrames` in more depth in the next chapter, but for now, let's move away from tabular data and talk about getting JSON data from external sources.

Getting JSON data with PowerShell

While we often work with tabular data, there are many times we're faced with JSON data.

JSON, short for **JavaScript Object Notation**, is a relatively concise way of representing objects as strings. Over the past few decades, we've seen JSON data become more popular than traditional more structured data structures, such as XML. In fact, JSON is so popular that when I work with external APIs, I find that JSON is the most common format they send back.

If you've not seen JSON before, the following is a sample JSON response provided by the `ICanHazDadJoke.com` website, a meme site designed to provide random "dad jokes" to visitors.

Here's a sample dad joke in the JSON format provided by the website:

```
{
  "id": "D5wAA5o4TCd",
  "joke": "What do you call a careful wolf? Aware wolf.",
  "status": 200
}
```

This JSON object has three properties – `id`, `joke`, and `status`. The `id` and `joke` properties both have string values, while `status` is numeric and appears to represent a 200 OK REST status code. This object represents a single random joke from the website and, sadly, represents my sense of humor somewhat accurately.

While dad jokes aren't a necessity in life (depending on who you ask), working with APIs that return JSON is nearly unavoidable. As you perform your data science and software engineering experiments, it's almost certain that you'll need to call APIs that return JSON data.

Let's take a look at how to fetch JSON data using PowerShell, by making an `HTTP GET` call to `ICanHazDadJoke.com` to get another dad joke.

We'll start with a `PowerShell` code cell to create the headers for our `GET` request:

```
$obj =
  "System.Collections.Generic.Dictionary[[String],[String]]"
$headers = New-Object $obj
```

```
$headers.Add("Accept", "application/json")
$headers
```

Since we end the cell with the `$headers` variable, Polyglot Notebooks will display its value below the cell:

```
Key      Value
---      -
Accept  application/json
```

You don't always need to add custom headers when making an HTTP GET call to an API, but this site requires it in order to get back a JSON result instead of HTML.

Next, we'll make the actual GET request to the API:

```
$url = "https://icanhazdadjoke.com"
$response = Invoke-RestMethod -Uri $url `
    -Method Get -Headers $headers
$response
```

This uses the headers we defined earlier to make the GET request, stores the result of the call in `$response`, and displays the current value of `$response`.

Each time you make a GET request to this API, it returns a different joke. *Figure 3.8* shows a sample of a rather “punny” response about parenthood:

```
id      joke      status
--      -
fiyPR7wPZDd When does a joke become a dad joke? When it becomes apparent.    200
```

Figure 3.8 – A regrettably bad joke shown in the API result

Here, `$response` is a `PSObject`, but we want to convert it to the JSON format so that it's easier to share with other language kernels.

To handle the conversion, we can send the response to the `ConvertTo-Json` cmdlet, converting the `PSObject` to a string of JSON text representing the object.

We do this with the following PowerShell code cell:

```
$json = $response | ConvertTo-Json
$json
```

This converts the value to JSON and stores it in our appropriately named `$json` variable. The value is then displayed as follows:

```
{
  "id": "fiyPR7wPZDd",
  "joke": "When does a joke become a dad joke? When it becomes
apparent.",
  "status": 200
}
```

As you can see, we now have JSON representing our joke (which does not get better with repetition).

We could now work with the JSON data in PowerShell, or we could create a variable in the C# kernel set to the current value of `$json` in the PowerShell kernel, by running the following code in a C# code cell:

```
#!set --value @pwsh:json --name json
```

With the `json` variable in the C# kernel containing the JSON, we can then deserialize it into a .NET object.

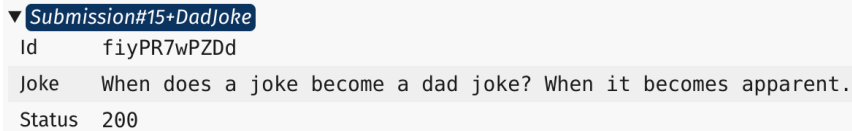
First, let's define a class to represent the object with a C# code cell:

```
public class DadJoke
{
    public string Id {get; set;}
    public string Joke {get; set;}
    public int Status {get; set;}
}
```

Next, we'll perform case-insensitive deserialization from the `json` string using `System.Text.Json`, with another C# code cell:

```
using System.Text.Json;
JsonSerializerOptions options = new();
options.PropertyNameCaseInsensitive = true;
DadJoke joke = JsonSerializer.Deserialize<DadJoke>(json, options);
joke
```

When this cell runs, we can see that the results of our PowerShell API call are now available as an object in the C# kernel, as shown in *Figure 3.9*:

A screenshot of a web interface showing a submission object. At the top, there is a dropdown menu with the text 'Submission#15+DadJoke'. Below it, the object is displayed as a table with three rows: 'Id' with value 'fiyPR7wPZDd', 'Joke' with value 'When does a joke become a dad joke? When it becomes apparent.', and 'Status' with value '200'.

Id	fiyPR7wPZDd
Joke	When does a joke become a dad joke? When it becomes apparent.
Status	200

Figure 3.9 – The joke interpreted as an object

You may have noticed that this chapter discussed both `Newtonsoft.Json` and `System.Text.Json`. Both are valid and powerful ways of serializing and deserializing objects in .NET. While `Newtonsoft.Json` is the most popular .NET package on `NuGet.org`, with 4.6 billion downloads at the time of writing, Microsoft's newer `System.Text.Json` is equally powerful. Pick whichever approach you and your team prefer, and use it in your projects.

Before we move on to other data sources, let's talk briefly about how you can take objects, such as our `joke` variable here, and create a new `DataFrame` out of them.

Building DataFrames from objects

Sometimes, you'll have a list of objects that you wish to get into a `DataFrame` for further analysis. In this case, you can manually add data to an empty or existing `DataFrame`.

While `DataFrame` has a `LoadFrom` method that can be used to load data, in practice, this method is a little cumbersome to work with when you're trying to translate from a series of objects to rows and columns in a `DataFrame`.

Instead, I'll share a small proof of concept using .NET's **reflection** capabilities.

Reflection

.NET's reflection feature allows you to enumerate the methods and properties of different types within your assemblies, as well as perform more advanced operations in defining new types. Reflection is relatively slow and complex compared to other code, but it is a great way of allowing code to enumerate the different capabilities offered by other pieces of code.

In order to add data to a `DataFrame`, we need to know the type of each column we're adding, along with the initial values for rows in that column. While we could do this manually for our small object, it's more interesting to look at an approach that could be modified to handle more complex objects.

We'll start by getting all public instance-level properties on the `DadJoke` class, using reflection in a C# code cell:

```
using System.Reflection;

Type reflectedType = joke.GetType();
```

```

BindingFlags flags = BindingFlags.Instance |
                    BindingFlags.Public |
                    BindingFlags.GetProperty;

var properties = reflectedType.GetProperties(flags);
properties

```

This code pulls in all non-static properties with a getter associated with instances of the DadJoke class.

As expected, it lists our Id, Joke, and Status properties and includes the correct type of each property, as shown in *Figure 3.10*:

<i>index</i>	<i>value</i>
0	▶ <code>System.String Id</code>
1	▶ <code>System.String Joke</code>
2	▶ <code>Int32 Status</code>

Figure 3.10 – The reflected properties of our object

Now that we have the properties, let's loop over them, adding each one as a column to a new DataFrame in a new C# code cell:

```

DadJoke[] jokes = [ joke ];
DataFrame dfObjects = new();
foreach (var info in properties) {
    switch (info.PropertyType.Name) {
        case "String":
            dfObjects.Columns.Add(new StringDataFrameColumn(
                info.Name,
                jokes.Select(j => info.GetValue(j).ToString())
            ));
            break;
        case "Int32":
            dfObjects.Columns.Add(new SingleDataFrameColumn(
                info.Name,
                jokes.Select(j => float.Parse(info.GetValue(j).ToString()))
            ));
            break;
    }
}
dfObjects

```

This code loops over all the properties we detected and adds an appropriate column to a new `DataFrame`. This column retains the name and type of the property on the `DadJoke` object.

We then use a `jokes.Select` call to get the property value for each joke we have and convert it to the right type. This effectively adds new pieces of data to the `DataFrame`, column by column.

Important note

The code listed here only works with string and integer columns, but it serves as a basis for code that you could expand to handle additional types of columns. It is my hope that, at some point in the future, a method or extension method for the `DataFrame` class will handle adding columns automatically from any collection of objects.

Now that we've seen a few ways of getting data into a `DataFrame`, let's move on to adding data through an extremely common data source – SQL databases.

Connecting to databases with SQL

Relational databases such as SQL Server, Postgres, and Maria DB have been the lifeblood of businesses for over 50 years at the time of writing. This is due to their ability to store normalized data about various entities in an easily queryable manner, while ensuring that data follows a specific schema.

Recently, non-relational databases such as MongoDB, Apache HBase, Cassandra, Cosmos DB, and Couchbase have become popular with enterprises. Despite this, the fact remains that, in most organizations, if you need data, you probably need it from a relational database.

Polyglot Notebooks understands this and provides support to query relational databases, through a series of connectors for specific databases and a common connection architecture.

In this section, I'll explore this by connecting to an Azure SQL database that I've set up for this chapter. Azure SQL is a variant of SQL Server, hosted on Microsoft Azure, but the concepts in this section can work with a variety of databases, as long as a connector exists for that database.

Where's the data?

While everything else in this chapter, and almost all of the rest of the book, can be used without additional setup or resources, querying a SQL database or a KQL cluster, unfortunately, requires additional setup, which is beyond the scope of this book. If you're curious about these topics and want to learn more, I encourage you to review the additional resources listed in the *Further reading* section of this chapter.

Let's explore SQL connections in more depth by taking a look at a connection to a sample database.

Connecting to a SQL database

We'll start by registering the SQL Server connector for Polyglot Notebooks, using this C# code cell:

```
#r "nuget:Microsoft.DotNet.Interactive.SqlServer,1.0.0-beta.24229.4"
```

This should install the connector's NuGet package.

Next, we need to connect to a specific SQL Server instance by providing its **connection string**.

Connection strings

Connection strings contain the information needed to locate and connect to a database. These strings contain information such as the network location of the database and how you want to authenticate with the database. Each database provider has a slightly different connection string, and different database tools have different places you need to look to find the connection string for your specific database. Consult your database's documentation on locating the appropriate connection string.

We'll connect to our SQL server instance with the `#!connect` magic command, using the following format:

```
#!connect mssql --kernel-name kernel-name connection-string
```

Here, `mssql` refers to the database technology to use. If you need help finding this identifier, you can run `#!connect -h` after installing your database connector for a list of supported technologies.

The `--kernel-name kernel-name` listed here creates a new SQL kernel with the name of `kernel-name`. Typically, you substitute this with the name of your database.

Finally, the `connection-string` is just your connection string, surrounded by double quotes.

Here's some redacted code to connect to my database:

```
#!connect mssql --kernel-name dad-jokes "Server=tcp:MyServer,1433;
Initial Catalog=MyDatabase;Persist Security Info=False;User
ID=REDACTED;
Password=REDACTED;MultipleActiveResultSets=
False;Encrypt=True;TrustServerCertificate=False;Connection
Timeout=30;"
```

Connection strings are rarely easy to read, but this code connects to the `MyDatabase` database hosted on the `MyServer` database server, which we connect to on port 1433 using a provided user ID and password.

Secrets management and notebooks

Polyglot Notebooks offers a growing number of ways to get secrets, such as connection strings, securely into your notebook. We'll explore some of these options in *Part 3* of this book when working with generative AI.

Once this connection is established, the database will be available as a new SQL kernel named `sql-dad-jokes` in Polyglot Notebooks. This name was chosen by combining the "sql - " string and our provided `kernel-name` earlier.

Using this connection, we can create a new code cell and select the `sql-dad-jokes` kernel, using the selection box in the lower right of the cell.

Executing SQL from SQL kernels

With the kernel selected, we can execute a SQL query against the database.

Figure 3.11 contains an example of querying my sample database of programmer-related dad jokes:

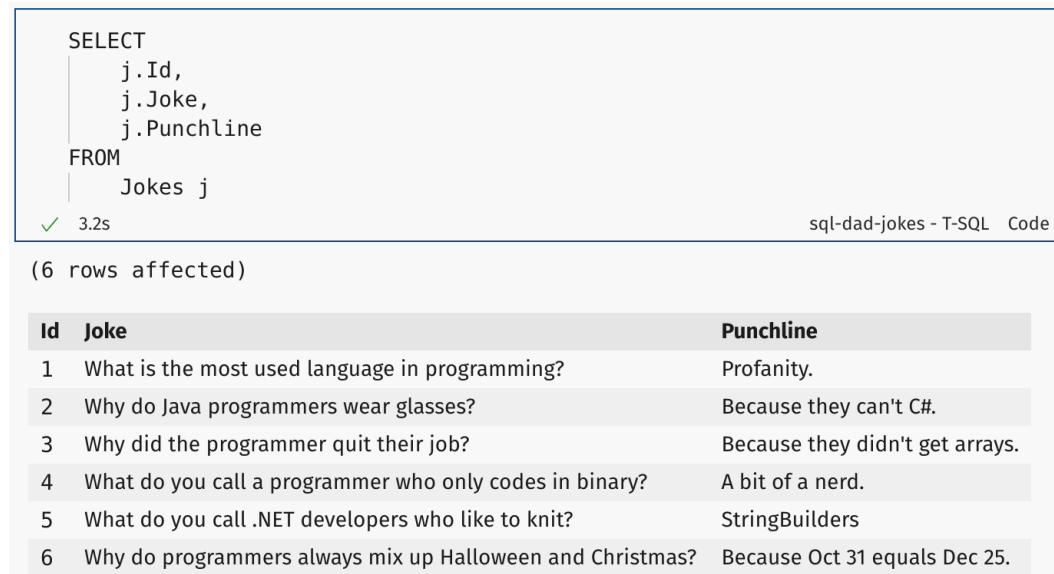


Figure 3.11 – Sample query results rendered below a cell

The results of our query are displayed below the cell as a formatted table. Any SQL execution messages that were encountered, such as the **(6 rows affected)** message, are shown below the cell as well.

Querying databases is important, and I can see a lot of value from this feature on its own. For example, you could mix together Markdown cells and SQL queries to tell a compelling story using a notebook.

I think the real value of database queries comes through the ability to get data out of your SQL kernel and into other code cells. Let's look at how to do that next.

Sharing SQL results with other kernels

You can store the results of a SQL query by modifying your SQL cell with a new magic command, storing the results of the kernel's execution:

```
#!sql-dad-jokes --name JokesTable
SELECT
    j.Id,
    j.Joke,
    j.Punchline
FROM
    Jokes j
```

This connects to our SQL kernel, named `sql-dad-jokes`, and stores the results of the query in a variable named `JokesTable`.

If you look in the Variables view, you should see a new variable named `JokesTable` belonging to the `sql-dad-jokes` kernel, with the value of an array of a single `TabularDataResource` object.

Of course, that's not terribly useful on its own, so let's share this result with the C# kernel by running the following C# code cell:

```
#!set --value @sql-dad-jokes:JokesTable --name JokesTable
```

Running this results in a new `JokesTable` variable getting added to the C# kernel. This C# kernel's `JokesTable` variable contains the results of our query, but what's its type?

According to the Variables view or by running `JokesTable.GetType()`, this C# `JokesTable` variable is a `JsonDocument`.

This `JsonDocument` can be a little daunting to work with, but if you display the value in a Polyglot Notebook and expand the results a few levels, you will see that it contains the joke data in a `data` property, as shown in *Figure 3.12*:

```

JokesTable
✓ 0.0s csharp - C# Script Code

▼ [{"profile":"tabular-data-resource","schema":{"primaryKey":[],"fields":[{"type":"integer","name":"Id"}, {"type":"string","name":"Joke"}, {"type":"string","name":"Punchline"}]}}]
  ▼ [{"profile":"tabular-data-resource","schema":{"primaryKey":[],"fields":[{"type":"integer","name":"Id"}, {"type":"string","name":"Joke"}, {"type":"string","name":"Punchline"}]}}]
    profile "tabular-data-resource"
    schema
      ▼ [{"type":"integer","name":"Id"}, {"type":"string","name":"Joke"}, {"type":"string","name":"Punchline"}]
        primaryKey ▶ []
        fields
          ▼ [{"type":"integer","name":"Id"}, {"type":"string","name":"Joke"}, {"type":"string","name":"Punchline"}]
            ▶ [{"type":"integer","name":"Id"}]
            ▶ [{"type":"string","name":"Joke"}]
            ▶ [{"type":"string","name":"Punchline"}]
    data
      ▼ [{"Id":1,"Joke":"What is the most used language in programming?","Punchline":"Profanity."}, {"Id":2,"Joke":"Why do Java programmers wear glasses?","Punchline":"Because they can't C#."}, {"Id":3,"Joke":"Why did the programmer quit their job?","Punchline":"Because they didn't get arrays."}, {"Id":4,"Joke":"What do you call a programmer who only codes in binary?","Punchline":"A bit of a nerd."}, {"Id":5,"Joke":"What do you call .NET developers who like to knit?","Punchline":"StringBuilder"}, {"Id":6,"Joke":"Why do programmers always mix up Halloween and Christmas?","Punchline":"Because Oct 31 equals Dec 25."}]
        ▶ [{"Id":1,"Joke":"What is the most used language in programming?","Punchline":"Profanity."}]
        ▶ [{"Id":2,"Joke":"Why do Java programmers wear glasses?","Punchline":"Because they can't C#."}]
        ▶ [{"Id":3,"Joke":"Why did the programmer quit their job?","Punchline":"Because they didn't get arrays."}]
        ▶ [{"Id":4,"Joke":"What do you call a programmer who only codes in binary?","Punchline":"A bit of a nerd."}]
        ▶ [{"Id":5,"Joke":"What do you call .NET developers who like to knit?","Punchline":"StringBuilder"}]
        ▶ [{"Id":6,"Joke":"Why do programmers always mix up Halloween and Christmas?","Punchline":"Because Oct 31 equals Dec 25."}]

```

Figure 3.12 – The query results as a JsonDocument

Transforming this `JsonDocument` to a `DataFrame` is a bit of a chore, but we can do it by identifying the `JsonElement` containing the data, converting that to a `TabularDataResource`, and then taking that `TabularDataResource` and making a `DataFrame` out of it, even if it using some extensions.

First, we have to identify the data element in our `JokesTable`. We can do this by using `RootElement[0]` to get the first child of the root and then getting the `data` property of that child.

Here's a C# code cell that does that and stores the result:

```

var data = JokesTable.RootElement[0].GetProperty("data");
data

```

The resulting object displayed below the cell indeed looks like our data, as shown in Figure 3.13:

```

JsonElement data = JokesTable.RootElement[0].GetProperty("data");
data
✓ 0.0s csharp - C# Script Code

▼ [{"Id":1,"Joke":"What is the most used language in programming?","Punchline":"Profanity."}, {"Id":2,"Joke":"Why do Java programmers wear glasses?","Punchline":"Because they can't C#."}, {"Id":3,"Joke":"Why did the programmer quit their job?","Punchline":"Because they didn't get arrays."}, {"Id":4,"Joke":"What do you call a programmer who only codes in binary?","Punchline":"A bit of a nerd."}, {"Id":5,"Joke":"What do you call .NET developers who like to knit?","Punchline":"StringBuilder"}, {"Id":6,"Joke":"Why do programmers always mix up Halloween and Christmas?","Punchline":"Because Oct 31 equals Dec 25."}]
  ▶ [{"Id":1,"Joke":"What is the most used language in programming?","Punchline":"Profanity."}]
  ▶ [{"Id":2,"Joke":"Why do Java programmers wear glasses?","Punchline":"Because they can't C#."}]
  ▶ [{"Id":3,"Joke":"Why did the programmer quit their job?","Punchline":"Because they didn't get arrays."}]
  ▶ [{"Id":4,"Joke":"What do you call a programmer who only codes in binary?","Punchline":"A bit of a nerd."}]
  ▶ [{"Id":5,"Joke":"What do you call .NET developers who like to knit?","Punchline":"StringBuilder"}]
  ▶ [{"Id":6,"Joke":"Why do programmers always mix up Halloween and Christmas?","Punchline":"Because Oct 31 equals Dec 25."}]

```

Figure 3.13 – The query results as a JsonElement

Next, we'll translate this `JsonElement` data to a `TabularDataResource`, which is *almost* like a `DataFrame` but lacks important capabilities.

The code for this involves the `Microsoft.DotNet.Interactive.Formatting` namespace, which contains a `ToTabularDataResource` extension method used in this C# code cell:

```
using Microsoft.DotNet.Interactive.Formatting;
TabularDataResource res = data.ToTabularDataResource();
```

Now that we have our `TabularDataResource`, we're almost there – we just need to perform one final step to convert it to a `DataFrame`.

We'll do this by installing the `Microsoft.DotNet.Interactive.ExtensionLab` extension, which provides a helpful extension method.

As you should be familiar with by now, this is done using a `#r` magic command in a C# code cell:

```
#r "nuget:Microsoft.DotNet.Interactive.ExtensionLab,1.0.0-beta.24229.4"
```

Note a number of extensions registered during this installation process. We'll revisit this package later on in *Chapter 14, Extending Polyglot Notebooks*. For now, let's use it to get our `DataFrame` by adding one final C# code cell:

```
using Microsoft.DotNet.Interactive.ExtensionLab;
using Microsoft.Data.Analysis;
DataFrame dfJokes = res.ToDataFrame();
dfJokes.Head(5)
```

This invokes the `ToDataFrame` extension method to create a `DataFrame` from our `TabularDataResource`, which gives us access to all manner of new capabilities as we'll explore in the next chapter.

For now, let's celebrate the achievement by admiring *Figure 3.14*, which shows the first five rows of our `DataFrame`:

<i>index</i>	<i>Id</i>	<i>Joke</i>	<i>Punchline</i>
0	1	What is the most used language in programming?	Profanity.
1	2	Why do Java programmers wear glasses?	Because they can't C#.
2	3	Why did the programmer quit their job?	Because they didn't get arrays.
3	4	What do you call a programmer who only codes in binary?	A bit of a nerd.
4	5	What do you call .NET developers who like to knit?	StringBuilders

Figure 3.14 – Our joke results in a `DataFrame`

The code in this section is, in this author's estimation, the nastiest code in the book. Thankfully, as this book was nearing publication, I began to hear discussions that much of this code may eventually be unnecessary, given a new proposal to have variable sharing between kernels come across as a `DataFrame` instead of as a `JsonDocument`. Such an improvement couldn't come soon enough, in my opinion!

Before we move on from databases, let's discuss some of the limitations that currently exist with databases.

Alternative ways of connecting to the Database

You may have noticed earlier that the version of `Microsoft.DotNet.Interactive.SqlServer` we're working with is a beta version. This is because the Polyglot Notebooks team is still building out SQL connection support.

This means that not all databases will have a connector available yet, so you may encounter some missing features, such as advanced support for securing connection strings (see the *Further reading* section for more on the work item for this feature).

While the SQL kernel is not fully ready, I still find the topic exciting enough to include in the book.

Working with other databases:

Even if your database isn't explicitly supported by Polyglot Notebooks yet, you can still use libraries such as Dapper, Entity Framework, or even good old ADO.NET to connect to your database and load up data the way you normally would in your .NET applications.

As we saw in the section earlier on JSON data, once you have a list of objects, it becomes relatively trivial to create a `DataFrame` out of those objects. You can use the SQL kernel and variable sharing to retrieve this list of objects, or you can load the data manually and create a `DataFrame` that way as well.

I firmly believe that as Polyglot Notebooks mature, we'll see additional connectors for additional databases, as well as more features and quality-of-life improvements for the ones already present.

For now, let's move on to getting data from Kusto clusters using KQL.

Querying Kusto clusters with KQL

If, like me, you were not familiar with the idea of a Kusto cluster before learning about Polyglot Notebooks, allow me to summarize what they are.

The name **Kusto** came from Microsoft's **Azure Data Explorer** product as a reference to Jacques Cousteau, a famous explorer and oceanographer.

Like Cousteau, Kusto was built to explore large bodies of data through a new query language. This query language was dubbed **Kusto Query Language** or **KQL** and was designed to search through Azure Data Explorer's proprietary **Kusto cluster** format.

This makes KQL a very specialized tool that you may or may not use in your day-to-day development tasks. For those who need to be able to query Azure datastores using KQL, you may be keen to know that Kusto is a supported kernel option in Polyglot Notebooks.

Creating a sample Kusto cluster

Take a look at <https://dataexplorer.azure.com/> if you want to create a Kusto cluster to try working with KQL. You'll need to create a database and then upload at least one table to be able to query it.

The process to work with KQL is very similar to the process to work with SQL:

1. First, install a NuGet package, registering the new kernel.
2. Connect to the KQL cluster.
3. Query the data.
4. Share the results.

We'll start with a code cell to install the KQL extension:

```
#r "nuget: Microsoft.DotNet.Interactive.Kql,1.0.0-beta.24229.4"
```

Next, we'll connect to our Kusto cluster and give that connection a new kernel name:

```
#!connect kql --kernel-name samples-database --cluster "REDACTED.kusto.windows.net/" --database "MyDatabase"
```

This establishes a connection to the MyDatabase database using the cluster connection string, and then it names the resulting kernel `samples_database`.

We can then execute a KQL query and store the results by using the same `#!kernel-name --name ResultingVariable` syntax for our SQL cell. The only thing that really changes is using a KQL code cell and query with KQL, instead of SQL:

```
#!kql-samples-database --name BooksTable  
SampleData  
| where PublicationYear > 2020 and Author == 'Matt Eland'
```

Here, we use `#!kql-samples-database` because we used the `samples-database` name for our kernel earlier.

We then choose to store the result of our query in a `BooksTable` variable, which will be `TabularDataResource`, just like the results of a SQL query.

Even if you're not familiar with KQL syntax, there are some obvious parallels between this query and a SQL equivalent of `SELECT * FROM SampleData WHERE PublicationYear > 2020 AND Author == 'Matt Eland'`.

With a valid connection and data in the Kusto cluster, this query results in the familiar-looking tabular display shown in *Figure 3.15*:

Book	Topic	Author	PublicationYear
Refactoring with C#	Refactoring	Matt Eland	2023
Data Science in .NET with Polyglot Notebooks	Data Science	Matt Eland	2024

Figure 3.15 – KQL results as a DataFrame

The results of this query are now stored in a `BooksTable` variable in the KQL kernel.

Like before, we can share this with the C# kernel (or another language kernel) via the `#!set` command:

```
#!set --value @kql-samples-database:BooksTable --name BooksTable
```

And, just like with SQL, the resulting C# kernel variable stores a `JsonDocument`, and you can follow the same steps to translate from the `JsonDocument` to a `TabularDataResource` and then on to a `DataFrame`.

As you can see, in this chapter, all roads lead to tabular data stored in a `DataFrame`. In the next chapter, we'll see why this is so important.

Summary

In this chapter, we saw how we could get both code and data into our notebooks from external sources.

While you could write all the code you need in one notebook, in reality, most of the time you'll need to reference code either in an external file or code defined in an external assembly, such as a local `.dll` file or a package in a NuGet feed.

We introduced how the `#!import` magic command helps pull in code from files or notebooks and how `#r` allows you to reference external assemblies.

We also discussed how important data is to data analysis and data science tasks, and how it can come from a variety of sources. We saw how to import CSV, TSV, and other delimited files. We also saw how to make HTTP requests to external APIs using PowerShell and how to query data stores, using SQL and KQL. We also saw how you can populate a `DataFrame` from a list of any `.NET` objects that you happen to have available.

All of these different paths of getting data into your notebook ultimately resulted in data loaded into a `DataFrame`, which will be key to future tasks.

In the next chapter, we'll explore the various ways we can manipulate and explore `DataFrame` objects, and how this sets us up well for data analytics and machine learning tasks.

Further reading

- *More about the `#!import` magic command:* <https://github.com/dotnet/interactive/blob/11cc0f1d32862ee6d2620e34ecbe8ad42ac1739f/docs/import-magic-command.md>
- *More about the `#!r` magic command:* <https://github.com/dotnet/interactive/blob/main/docs/nuget-overview.md>
- *Connecting to Data in Polyglot Notebooks:* <https://github.com/dotnet/interactive/blob/main/docs/working-with-data.md>
- *Discussion of support for referencing C# Projects:* <https://github.com/dotnet/interactive/issues/890>
- *What is REST?:* <https://restfulapi.net/>
- *PowerShell Overview:* <https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.4>
- *SQL authentication management in Polyglot Notebooks:* <https://github.com/dotnet/interactive/issues/992>
- *SQL Queries:* <https://learn.microsoft.com/en-us/sql/t-sql/queries/queries?view=sql-server-ver16>
- *Azure SQL Databases:* <https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview?>
- *Azure Data Explorer:* <https://azure.microsoft.com/en-us/products/data-explorer>
- *Kusto Query Language overview:* <https://learn.microsoft.com/en-us/azure/data-explorer/kusto/query/>
- *Additional online resources:* <https://MattOnML.NET/Books/DataScience/Data/>
- *Working with NuGet packages:* <https://learn.microsoft.com/en-us/nuget/consume-packages/overview-and-workflow>

4

Working with Tabular Data and DataFrames

In the last chapter, we saw how `DataFrames` can load and store tabular data sources, such as CSV files, and the results of SQL queries.

`DataFrames` are the foundation of many common tasks in data analysis and machine learning. As a result, it's important to be able to work effectively with `DataFrames` and perform common operations as you explore and transform your data – regardless of the tasks you're trying to accomplish.

In this chapter, we'll build a solid foundation in working with `DataFrames` as we cover the following:

- Understanding data cleaning and data wrangling
- Working with `DataFrames` in C#
- Working with columns
- Handling missing values
- Sorting, filtering, grouping, and merging data
- `DataFrames` in other languages

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter04` folder.

Understanding data cleaning and data wrangling

If you analyze data or train models on datasets, a large chunk of your time will be spent doing **data cleaning**. Data cleaning is the act of resolving inconsistencies and impurities in your data so that you can take subsequent actions with your data source. This is critical because most data you work with will require some degree of cleaning before you can work with it.

Where unclean data comes from

Data sources are rarely ever the way you wish them to be. Data commonly has artifacts such as missing values for required fields or various typos or inconsistencies.

For example, a data source may have a `Country` field that could have many different values that all refer to the same country. To help illustrate this, *Figure 4.1* contains a set of football players all born in the United States, but each one has a different value for their `country_of_birth`:

player_id	name	current_club_id	country_of_birth	country_of_citizenship	date_of_birth	sub_position	position
3333	Abel Sauer	1237	US	England	1986-01-04 00:00:00Z	Central Midfield	Midfield
33027	Dewey Luetngen	1031	United States	Netherlands	1988-04-03 00:00:00Z	Goalkeeper	Goalkeeper
43530	Alonzo Harber	1237	Untied States	England	1988-05-10 00:00:00Z	Attacking Midfield	Midfield
45494	Moses Mitchell	703	USA	Wales	1987-01-24 00:00:00Z	Goalkeeper	Goalkeeper
51321	Gerald Hermann	989	U.S.	Ireland	1987-05-12 00:00:00Z	Goalkeeper	Goalkeeper
52059	Rex Treutel	1148	America	Denmark	1990-04-23 00:00:00Z	Centre-Back	Defender

Figure 4.1: Different US values – note the United States typo

While many software systems have data validation in place to prevent duplicates such as these, not all systems do, and those validation mechanisms may not have always been in place or worked correctly.

Imagine you have a dataset that stores the readings of temperature sensors in an office building over time. From time to time, sensors may become faulty and record temperature readings that deviate from the actual temperature by a significant amount. By the time this flaw is detected and resolved, your dataset has already accumulated some **invalid data**.

Alternatively, imagine that, over time, sensors in certain rooms are added while others fail to connect to the database to store their measurements. In either scenario, you will have time ranges where you have **missing data** for a given room.

Finally, a temperature sensor may be working perfectly but recording very hot readings at certain times of the day. Upon investigation, the team may find that these hot temperature **outliers** represent

times when an employee put a hot cup of tea or coffee next to the temperature sensor, causing it to record a temperature much higher than the room's overall temperature.

All of these are examples of everyday data issues, but each represents a different challenge in data processing when data is inaccurate, missing, or represents unusual and rare situations that could be considered outliers.

We'll talk more in this chapter about handling tricky data, but first, let's talk about the impact of bad data that has not been properly cleaned.

The impact of unclean data

Typically, datasets are used for one of several tasks:

- **Analyzing data** to discover relationships and trends to drive decisions
- **Training models** using machine learning, which can be used to interpret future data points
- **Powering other applications** that rely on data to operate properly

When data cleaning is not performed and data is left in an improper state, this leads to bad results downstream.

When values are inaccurate, we risk drawing the wrong conclusions from the data during data analysis, by relying on data that is factually incorrect.

If this same data is used to train a machine learning model, the model will fit itself to the mistakes of the data source. This will significantly affect the model's accuracy in generating predictions, as the data the model fitted to was incorrect. I've personally seen a single incorrect value in a small dataset cause a model to be 7% less accurate than it otherwise would have been!

Model fitting

We'll talk more about the process of "fitting" models to data in *Part 2* of this book when we discuss training machine learning models with ML.NET. For now, just consider fitting as the process of a model memorizing the trends in your data.

Datasets that are missing important values will result in data visualizations with significant gaps, and they may cause errors when training machine learning models or when relied on as a data source by a running application.

Outliers are also important to consider, but they can sometimes be distracting in data visualizations or throw off data metrics, such as the average values for a column. Machine learning models will often perform better when we can remove extreme outliers from our datasets.

All of these issues can be detected using exploratory data analysis (as we'll see in the next chapter) and resolved using the data-cleaning techniques from this chapter.

Data cleaning and data wrangling

When performing data cleaning, your goal is to remove or resolve the impurities in your data. Sometimes, data cleaning will involve correcting existing values by fixing typos, consolidating and standardizing values, or fixing numerical issues in data. Other times, data cleaning will involve providing values for values that are missing, as we'll discuss in the *Handling missing values* section of this chapter.

Other times, you'll find yourself in a situation where you need to remove data from your dataset. For example, you may choose to remove outliers or data that you have reason to believe may be inaccurate. Alternatively, you may find yourself facing missing values for required fields that you cannot provide a replacement value for.

Both of these flavors of data cleaning involve analyzing data to detect accuracy issues and then determining the best action to take to resolve the issue.

Sometimes, you'll find yourself moving beyond data cleaning and looking to add entirely new fields to your dataset.

The process of adding new fields to an existing dataset is called **feature engineering** since you're taking existing rows in your data and adding new features (represented as columns) to that dataset.

An example of feature engineering would be starting with the date and time of an observation and adding new columns to represent factors such as the following:

- The day of the week
- The year
- The month
- The quarter
- Whether the event occurred in the morning or evening

While these examples are all related to dates, feature engineering can apply to any type of field. I frequently find myself adding features related to the difference between values, relative percentages, and whether or not some condition is true.

What about normalization?

Relational databases often stress normalizing data by not repeating values unnecessarily. However, we clearly have multiple values all driven by the same date value in this example. This helps machine learning find important factors for your data. For example, a day-of-the-week column would be helpful when training a model about driving habits, since the day of the week determines people's commutes. In general, our data is stored in a normalized manner, but you can denormalize it through feature engineering before training a model without modifying the underlying database.

The goal of feature engineering is to provide important additional context to your data visualization and machine learning efforts in order to, hopefully, get additional value.

You'll sometimes hear the overall process of cleaning data and feature engineering referred to by the broader term **data wrangling**.

Data wrangling involves taking a dataset and transforming it into the overall state that you need it to be in for data analysis and machine learning tasks, during the process of data cleaning and feature engineering.

Now that we've covered the nature of data cleaning and the concepts of feature engineering and data wrangling, let's get into the specifics by working with DataFrames.

Working with DataFrames in C#

While Polyglot Notebooks gives us a wide range of capabilities in C#, F#, and other languages, the primary tool we'll use to wrangle our data is the `DataFrame` class, defined in the `Microsoft.Data.Analysis` NuGet package and namespace.

Naming note

The `Microsoft.Data.Analysis.DataFrame` is also referred to as the ML.NET `DataFrame`, due to it being included in the `dotnet/MachineLearning` GitHub repository that also includes ML.NET.

Using the `DataFrame` class, we can perform a variety of data-wrangling steps, as we'll see throughout the chapter. Before we cover those operations, let's explore the fundamentals of working with a `DataFrame` to explore its data.

We'll start by installing our NuGet dependency and then load up a dataset of football players using the `LoadCsv` method, as we've done before.

First, we'll start with a C# code cell to install the dependency:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
```

Next, we'll import our players' dataset into a new `DataFrame`, called `dfPlayers`, in a subsequent C# code cell:

```
using Microsoft.Data.Analysis;
DataFrame dfPlayers = DataFrame.LoadCsv("Players.csv");
```

With our data loaded into a `DataFrame`, let's explore some of the ways we can explore its contents.

Viewing and sampling data

The easiest way of viewing data in a `DataFrame` is to have a C# code cell ending with your `DataFrame` variable, as shown here:

```
dfPlayers
```

This results in the `DataFrame` being rendered below the cell.

The display of data is limited to 25 rows per page and automatically adds interactive paging controls that let you flip through the data using your mouse or keyboard, as shown in *Figure 4.2*:

DataFrame - 221 rows											
index	player_id	name	current_club_id	country_of_birth	country_of_citizenship	date_of_birth	sub_position	position	foot	height_in_cm	market_value_in_eur
0	3333	Abel Sauer	1237	US	England	1986-01-04 00:00:00Z	Central Midfield	Midfield	right	175	1000000
1	33027	Dewey Luetggen	1031	United States	Netherlands	1988-04-03 00:00:00Z	Goalkeeper	Goalkeeper	left	193	500000
2	43530	Alonzo Harber	1237	United States	England	1988-05-10 00:00:00Z	Attacking Midfield	Midfield	both	172	1000000
3	45494	Moses Mitchell	703	USA	Wales	1987-01-24 00:00:00Z	Goalkeeper	Goalkeeper	right	198	250000
4	51321	Gerald Hermann	989	U.S.	Ireland	1987-05-12 00:00:00Z	Goalkeeper	Goalkeeper	right	187	300000
5	52059	Rex Treutel	1148	America	Denmark	1990-04-23 00:00:00Z	Centre-Back	Defender	right	189	1000000
6	52570	Julius Skiles	148	England	England	1988-03-17 00:00:00Z	Goalkeeper	Goalkeeper	right	201	1200000
7	55742	Ernest Lynch	703	England	Ireland	1989-12-28 00:00:00Z	Central Midfield	Midfield	left	177	400000
8	60444	Danny Effertz	31	Italy	Spain	1991-04-11 00:00:00Z	Central Midfield	Midfield	right	174	10000000
9	61841	George Carroll	989	England	England	1991-04-29 00:00:00Z	Right-Back	Defender	right	174	1000000
10	61842	Isaac Jenkins	1031	England	England	1991-07-16 00:00:00Z	Right Winger	Attack	left	181	1800000
11	66934	Gerardo Kutch	703	Senegal	Senegal	1989-12-21 00:00:00Z	Defensive Midfield	Midfield	right	189	1000000
12	67063	Brandon Heller	1237	England	England	1990-11-26 00:00:00Z	Centre-Forward	Attack	right	185	6000000
13	71271	Nelson Bednar	31	Spain	Spain	1987-01-03 00:00:00Z	Goalkeeper	Goalkeeper	right	190	600000
14	73564	Ben Stracke	1237	England	England	1990-08-18 00:00:00Z	Goalkeeper	Goalkeeper	right	188	3500000
15	74810	Corey Kohler	1148	England	England	1989-09-21 00:00:00Z	Centre-Back	Defender	left	180	1000000
16	74960	Wm Kreiger	762	CSSR	Slovakia	1989-01-15 00:00:00Z	Goalkeeper	Goalkeeper	right	191	1000000
17	77757	Darnell Cummings	1031	Belgium	Belgium	1992-10-23 00:00:00Z	Goalkeeper	Goalkeeper	right	190	2800000
18	82105	Cory Wilderman	31	Germany	Cameroon	1991-08-08 00:00:00Z	Centre-Back	Defender	right	195	8000000
19	82873	Omar Gusikowski	1237	Germany	Germany	1991-06-15 00:00:00Z	Central Midfield	Midfield	right	181	8000000
20	85864	Francis Volkman	762	Germany	Germany	1993-06-22 00:00:00Z	Goalkeeper	Goalkeeper	right	191	1000000
21	91340	Jose Pacocha	1031	England	England	1991-06-16 00:00:00Z	Goalkeeper	Goalkeeper	<null>	200000	
22	91845	Caleb Stark	148	Korea, South	Korea, South	1992-07-08 00:00:00Z	Left Winger	Attack	both	184	50000000
23	92469	Josh Welch	762	England	Scotland	1989-09-10 00:00:00Z	Right Midfield	Midfield	left	173	1000000
24	95810	Theodore Gaylord	762	England	England	1990-09-19 00:00:00Z	Right-Back	Defender	right	173	11000000

Figure 4.2 – An overview of the first page of data in a `DataFrame`

In practice, I find that I usually only need to see a few rows to get a sense of the general contents and structure of data, so most of the time when I just want a preview of the basic data format, I use the `Head` method.

The `Head` method gives you the first few rows of your `DataFrame` and requires an integer parameter, letting you specify the number of rows you are interested in.

Here's an example requesting the top five rows:

```
dfPlayers.Head(5)
```

This produces a preview of the data, as shown in *Figure 4.3*:

index	player_id	name	current_club_id	country_of_birth	country_of_citizenship	date_of_birth	sub_position	position
0	3333	Abel Sauer	1237	US	England	1986-01-04 00:00:00Z	Central Midfield	Midfield
1	33027	Dewey Luetgen	1031	United States	Netherlands	1988-04-03 00:00:00Z	Goalkeeper	Goalkeeper
2	43530	Alonzo Harber	1237	United States	England	1988-05-10 00:00:00Z	Attacking Midfield	Midfield
3	45494	Moses Mitchell	703	USA	Wales	1987-01-24 00:00:00Z	Goalkeeper	Goalkeeper
4	51321	Gerald Hermann	989	U.S.	Ireland	1987-05-12 00:00:00Z	Goalkeeper	Goalkeeper

Figure 4.3 – The results of `dfPlayers.Head(5)`

The first five rows represent players on a variety of teams who were all born in the mid-1980s. We can see a few different positions, with a larger-than-average proportion of goalkeepers.

If you, instead, wanted to look at the last few rows of your `DataFrame`, you could instead call the `Tail` method, as shown here:

```
dfPlayers.Tail(5)
```

This results in the final five records from the `DataFrame`:

index	player_id	name	current_club_id	country_of_birth	country_of_citizenship	date_of_birth	sub_position	position
0	922769	Jackie Jones	762	England	England	2006-05-01 00:00:00Z	Central Midfield	Midfield
1	973085	Brendan Bins	1237	Cameroon	Cameroon	2004-01-03 00:00:00Z	Defensive Midfield	Midfield
2	983989	Lance Lang	1237	Argentina	Argentina	2004-12-23 00:00:00Z	Attacking Midfield	Midfield
3	1005649	Rodney Will	703	Brazil	Brazil	2002-07-04 00:00:00Z	Centre-Back	Defender
4	1141628	Carl Reinger	1148	Turkiye	Turkiye	2006-01-10 00:00:00Z	Defensive Midfield	Midfield

Figure 4.4 – The results of `dfPlayers.Tail(5)`

The final five records in the `DataFrame`, as shown in *Figure 4.4*, are all much younger than those at the beginning, with birth years from 2002 to 2007. Additionally, four of the five players are midfielders. This implies that the data may have some form of a sort order based on time, perhaps related to when the player joined the league.

If your `DataFrame` is ordered in a certain way, such as by the time of an event or by other columns, looking at the end of your `DataFrame` may tell you things that the beginning of that `DataFrame` doesn't.

`DataFrame` also gives us a `Sample` method that is like `Head` or `Tail`, but instead, it picks random rows from the `DataFrame`. This is particularly helpful when working with data that has some sequential order and you want to get a sense of the data, at random, throughout the `DataFrame`.

Tip for Python developers

If you have exposure to Python's pandas library and its `DataFrame` implementation, it's important to note that ML.NET's `Head`, `Tail`, and `Sample` all *require* the number of rows parameter and do not have a default value as they do in pandas.

`Sample` takes in a number of rows parameter, just like `Head` and `Tail` do, as shown in this C# code cell:

```
dfPlayers.Sample(5)
```

As before, this provides a preview of your data, as shown in *Figure 4.5*:

index	player_id	name	current_club_id	country_of_birth	country_of_citizenship	date_of_birth	sub_position	position
0	328658	Domingo Dickinson	1148	Norway	Norway	1998-04-17 00:00:00Z	Centre-Back	Defender
1	381967	Timmy Bayer	1031	Belgium	Belgium	1999-10-22 00:00:00Z	Central Midfield	Midfield
2	209212	Roberto Langworth	1237	England	England	1994-07-20 00:00:00Z	Right Winger	Attack
3	134270	Edmund Collier	762	England	England	1992-05-09 00:00:00Z	Left-Back	Defender
4	565822	Darryl Leffler	31	England	England	2003-04-04 00:00:00Z	Attacking Midfield	Midfield

Figure 4.5 – The results of a sample call

Unlike `Head` and `Tail`, `Sample` will produce a different result every time you run it because the rows are chosen at random. Because of this, I will often rerun a cell containing a `Sample` call multiple times to get a sense of how uniform the data is.

Now that we've seen how to sample our data, let's look at how we can work with individual rows, columns, and cells in a `DataFrame`.

Rows

DataFrames allow us access to their row data through a Rows property, which is a DataFrameCollection object. Using the Rows property, you can get an individual row by its index. For example, the following C# code cell gets the fifth row in the DataFrame:

```
DataFrameRow row = dfPlayers.Rows[4];
row
```

Since we end the cell with row, we see its structure rendered below the cell when the cell executes:

▼ [51321, Gerald Hermann, 989, U.S., Ireland, 5/12/1987 12:00:00 AM, Goalkeeper, Goalkeeper, right, 187, 300000]

(values)	index	type	value
	0	System.Single	51321
	1	System.String	Gerald Hermann
	2	System.Single	989
	3	System.String	U.S.
	4	System.String	Ireland
	5	System.DateTime	1987-05-12 00:00:00Z
	6	System.String	Goalkeeper
	7	System.String	Goalkeeper
	8	System.String	right
	9	System.Single	187
	10	System.Single	300000

Figure 4.6 – The data in the fifth row of the DataFrame

Looking at *Figure 4.6*, it's clear that the DataFrameRow class actually consists of a list of values of different types. These values are linked to the column values for the row in the DataFrame.

Getting and setting cell values

Sometimes, you'll want to read the value of a particular cell in a DataFrame. If you want to get a value for the row for a particular column, such as the height in centimeters of a given player, you can pull that value out of DataFrameRow by its index, as shown with this C# code cell:

```
row[10]
```

Alternatively, you can use the name of the column, as shown here:

```
row["height_in_cm"]
```

Using the column index or the column name gives you the same result. I generally prefer to use the name of the column so that my code is easier to read and doesn't break if columns are added, removed, or reordered.

You don't need to store `DataFrameRow` in a separate variable to get a cell. You can work directly with the `DataFrame` by using a row indexer and then a column indexer, as shown in this C# code cell:

```
dfPlayers.Rows[4]["height_in_cm"]
```

Being able to access an individual cell for any given row is helpful, as we'll see later when we talk about iterating over rows. Indexers not only allow you to get values out of cells but also allow you to set values in cells. For example, if you wanted to correct the spelling of "*United States*" in your dataset, you could use the following C# code cell to set it on a single row:

```
dfPlayers.Rows[0]["country_of_birth"] = "United States"
```

This updates the value of a single cell in the `DataFrame`, which is sometimes what you want to do. More often, you'd want to check a value for a specific other value before changing it or use `DataFrame` level statements to modify multiple cells at once, as we'll explore later with filtering.

We'll explore using conditional statements when updating individual rows in a moment when we discuss iterating over rows.

Iterating over rows

Occasionally, you may find a need to loop over every row in your `DataFrame`. This isn't as common of a need as you might think, since the `DataFrame` class gives us a lot of advanced capabilities, but sometimes you'll still want to iterate over each row.

Thankfully, you can enumerate each row in a `foreach` statement. We'll use that here to loop over and standardize the spelling of the United States in the `country_of_birth` field, as shown here in this C# code cell:

```
HashSet<string> usAnalogues = new()
{ "US", "USA", "America", "U.S.", "Untied States" }

foreach (var row in dfPlayers.Rows)
{
    if (usAnalogues.Contains(row["country_of_birth"])) {
        row["country_of_birth"] = "United States";
        Console.WriteLine($"Fixed country for {row["name"]}");
    }
}
```

This code defines common misspellings of the United States and corrects rows that have inconsistent values from our standardized *"United States"* string.

There's almost always a better way of working with data than looping over the `Rows` property, but it's nice to know that this is possible when you need it.

To explore this, let's look at the surprising range of things you can do with columns.

Working with columns

`DataFrame`s are capable of some advanced operations relating to column manipulation. So far in the chapter, what we've explored has utilized normal methods, properties, and indexers and simply feels like a very standard .NET approach (for lack of another description).

When we involve columns, we can do more advanced operations, including filtering, looking at distributions of data, and even mathematical operations that affect the entire column of data.

Let's see some of the things that columns allow us to do.

Columns

The `DataFrame` class has a `Columns` property to match the `Rows` property we discussed earlier. The `Columns` property returns a `DataFrameColumnCollection`, which consists of the columns in the `DataFrame`.

When you load a CSV file using the `LoadCsv` method, the columns it loads will be one of the following column types:

- **`StringDataFrameColumn`**: A column storing `string` values
- **`BooleanDataFrameColumn`**: A column storing boolean values (`bool`s)
- **`SingleDataFrameColumn`**: A column storing numeric values as singles (`float`s)
- **`DateTimeDataFrameColumn`**: A column storing `DateTime` values

This is interesting because all numerical values will be loaded into a `SingleDataFrameColumn`. `DataFrame`s do support built-in columns for various types of integers, as well as doubles, bytes, and decimals; however, these types are not used when loading files from a CSV file, as of ML.NET 4.0.

If you want to have exact control of the types loaded into your `DataFrame`, I recommend loading data into an object and then using `DataFrame`'s `LoadFrom` static method to provide both the values and the types of each column, as we did in *Chapter 3, Getting Data into your Notebook* when working with lists of objects.

One common task I want to do when working with DataFrames is to list the names of each column. This can be done by using a LINQ `Select` expression on the `Columns` property, as demonstrated in this C# code cell:

```
dfPlayers.Columns.Select(c => c.Name)
```

This outputs the list of columns in the dataset:

```
[ player_id, name, current_club_id, country_of_birth, country_of_citizenship, date_of_birth, sub_position, position, foot, height_in_cm, market_value_in_eur ]
```

If we wanted to see the column types, we could do a slightly different `Select` operation:

```
dfPlayers.Columns.Select(c => c.GetType().Name)
```

However, I find that simply having Polyglot Notebooks render the `Columns` property is more readable. We do this with a C# code cell:

```
dfPlayers.Columns
```

This produces a formatted table that shows the column type and some sample values from the first few rows of each column, as shown in *Figure 4.7*:

index	type	value
0	Microsoft.Data.Analysis.SingleDataFrameColumn	▶ [3333, 33027, 43530, 45494, 51321, 52059, 52570, 55742, 60444, 61841, 61842, 66934, 6706
1	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [Abel Sauer, Dewey Luetggen, Alonzo Harber, Moses Mitchell, Gerald Hermann, Rex T
2	Microsoft.Data.Analysis.SingleDataFrameColumn	▶ [1237, 1031, 1237, 703, 989, 1148, 148, 703, 31, 989, 1031, 703, 1237, 31, 1237, 1148, 762, 103
3	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [United States, United States, United States, United States, United States, United St
4	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [England, Netherlands, England, Wales, Ireland, Denmark, England, Ireland, Spain,
5	Microsoft.Data.Analysis.DateTimeDataFrameColumn	▶ [1/4/1986 12:00:00 AM, 4/3/1988 12:00:00 AM, 5/10/1988 12:00:00 AM, 1/24/1987 12:00
6	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [Central Midfield, Goalkeeper, Attacking Midfield, Goalkeeper, Goalkeeper, Centre-B
7	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [Midfield, Goalkeeper, Midfield, Goalkeeper, Goalkeeper, Defender, Goalkeeper, Mid
8	Microsoft.Data.Analysis.StringDataFrameColumn	▶ [right, left, both, right, right, right, right, left, right, right, left, right, right, rig
9	Microsoft.Data.Analysis.SingleDataFrameColumn	▶ [175, 193, 172, 198, 187, 189, 201, 177, 174, 174, 181, 189, 185, 190, 188, 180, 191, 190, 195, 1
10	Microsoft.Data.Analysis.SingleDataFrameColumn	▶ [1000000, 500000, 1000000, 250000, 300000, 1000000, 1200000, 400000, 10000000,

Figure 4.7: The different column types and their values

Drilling into a specific column can be very helpful to gain additional insights about your data. Let's demonstrate this by looking at the `position` column, which indicates a footballer's position on the pitch:

```
dfPlayers["position"]
```

This outputs an object containing the number of values in that column, the number of `null` values, the name of the column, the data type stored in that column, and the sequential values stored in the column, as shown in *Figure 4.8*:

```

▼ [ Midfield, Goalkeeper, Midfield, Goalkeeper, Goalkeeper, Defender, Goalkeeper, Midfield, Midfield, Defender, Attack, Midfield, Attack, Goalkeeper, ...
NullCount  0
Length     221
Name       position
DataType   System.String
(values)   [ Midfield, Goalkeeper, Midfield, Goalkeeper, Goalkeeper, Defender, Goalkeeper, Midfield, Midfield,

```

Figure 4.8: Column summary info for the position column

You may have noticed that while we used the `Rows` property earlier to get an individual row, here we used an indexer on the `DataFrame`. This is because getting a column is such a common operation that the `DataFrame` supports an indexer for it.

If you'd prefer, you can also use the equivalent indexer notation on the `Columns` property, as shown here:

```
dfPlayers.Columns["position"]
```

Once you have a column, you can gain a wide range of information from it. Let's look at some methods that help analyze your columns.

Analyzing columns

`DataFrame` columns have a trio of methods that help us understand the characteristics of the data they contain.

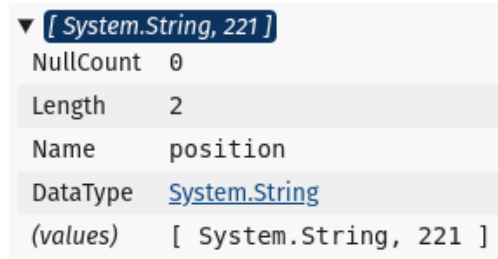
Displaying column information with `Info`

The first method is the `Info` method, and it's nearly identical to the view of the column we saw earlier in *Figure 4.8*.

Let's call the `Info` method from a C# code cell and see its results:

```
dfPlayers["position"].Info()
```

This outputs a series of values similar to what we saw earlier:



The screenshot shows a Jupyter Notebook cell with a dropdown menu set to '[System.String, 221]'. Below the dropdown, the following information is displayed:

NullCount	0
Length	2
Name	position
DataType	System.String
(values)	[System.String, 221]

Figure 4.9 – Displaying information from a column

The primary difference here is the (values) collection, which contains the type of data value that is stored in the column and then the number of non-null values in the column.

In this case, the position column contains strings and has 221 non-null values in its collection.

Tabulating value occurrences with ValueCounts

While Info has its uses, I find myself relying more on the ValueCounts method as an easy way of analyzing **categorical data**. Categorical data simply refers to something that is going to be one of several possible values.

The ValueCounts method generates a grouped display of unique values in a column and gives you a count of how many times each value occurs in that column. This makes ValueCounts a great way of understanding what's in your column when there are only a few possibilities.

Let's see ValueCounts in action, using a C# code cell and our position column:

```
dfPlayers["position"].ValueCounts()
```

This generates and displays a small table of counts, as shown in Figure 4.10:

index	Values	Counts
0	Midfield	66
1	Goalkeeper	29
2	Defender	73
3	Attack	53

Figure 4.10: A distribution of position values among players

The `ValueCounts` method tells us that most of the players are defenders, but we have a similar number of midfielders and a lower number of attacking players. As you would expect, there are much fewer goalkeepers relative to other types of football players.

While `ValueCounts` is great for categorical data values with few overall possibilities, it doesn't tend to be very helpful for **quantitative data** that focuses on numeric measurements.

Next, let's see how to look at all the values in a particular column.

Getting values from a column

You may find yourself wanting to get an array of values representing the contents of a column. This need is particularly important for providing values to various charting libraries, as we'll see in *Chapter 5, Visualizing Data*, and *Chapter 6, Visualizing Variable Relationships*.

When I first tried to connect `DataFrames` to charts, I was stumped at how to turn a column into an array of values, since `dfPlayers["height_in_cm"].ToArray()` doesn't compile, as the default type of any column is just `DataFrameColumn`, which does not support LINQ extension methods.

In order to get an array of values out of a column, you first need to cast the column to its appropriate type. Once that is done, you'll be able to extract its values into an array.

Let's see this in action with a C# code cell:

```
var colHeight = (SingleDataFrameColumn)dfPlayers["height_in_cm"];
float?[] values = colHeight.ToArray();
```

Here, we cast the column to `SingleDataFrameColumn`, since we know the column is that type from earlier. `SingleDataFrameColumn` implements `IEnumerable<float?>` so that LINQ methods work on it, and we're able to call `ToArray()` to generate an array of nullable float values.

If you're not afraid of readability, you can condense this down into a single line of code:

```
float?[] values = ((SingleDataFrameColumn)dfPlayers["height_in_cm"]).
    ToArray();
```

It is my hope that future versions of ML.NET will give us convenient methods to extract values out of a column without needing this verbose casting code.

Removing columns

Sometimes, you'll load up a set of data and realize that you don't need all the data represented in its columns.

In these cases, removing unneeded columns can help you focus on other data while also ensuring that you don't provide unnecessary data to the model training process with machine learning.

Removing a column is easy – just call the `Remove` method on the `Columns` collection, as shown here:

```
dfPlayers.Columns.Remove("country_of_citizenship")
```

This code removes the `country_of_citizenship` column from the `DataFrame`.

`DataFrames` do not offer a way of removing multiple columns at once, but you can do this easily enough with a `foreach` loop and a set of columns to remove, as shown by this code cell:

```
string[] columnsToRemove = {"ColA", "ColB", "ColC"};
foreach (var col in columnsToRemove)
{
    dfPlayers.Columns.Remove(col);
}
```

This code removes the `ColA`, `ColB`, and `ColC` columns if they exist in our `DataFrame`. Note that the `Remove` method will silently ignore instructions to remove columns that are not present to begin with.

Renaming columns

There will be times when you will need to work with datasets that have column names that you find confusing or misleading. In these cases, you can and should rename the column.

To rename a column, access the column with its indexer and then call the `SetName` method, providing a new name for the column.

The following C# code cell renames the `player_id` column to simply `Id`.

```
dfPlayers["player_id"].SetName("Id")
```

Like removing a column, renaming a column is immediate and modifies the `DataFrame`.

Immutability and DataFrames

Some methods executed on `DataFrames` will produce a different `DataFrame` instead of modifying the original one. This behavior of returning a new object that represents a modified version of the original is part of the practice of designing **immutable** objects.

Now that we've seen how to remove and rename columns, let's look at how we can add new columns.

Adding a new column

When you perform feature engineering on your data, you will need to add a new column of data to your `DataFrame`. To do so, you'll need to instantiate a new `DataFrameColumn` of the appropriate type and provide it with an array of values. These values represent the values of the column for the existing rows in a dataset.

We saw one approach to this in *Chapter 3, Getting Data into Your Notebooks*, when we discussed creating `DataFrames` from lists of objects, but we'll see another approach here.

Let's demonstrate feature engineering by adding a new column, containing the year each player was born. We have this information already through the `date_of_birth` column, but having the year as a separate column will make determining the age of each player easier later.

We'll start by getting the year values from our `date_of_birth` column into a variable, using a C# code cell:

```
DateTimeDataFrameColumn colDob = (DateTimeDataFrameColumn)
dfPlayers["date_of_birth"];
IEnumerable<int?> years = colDob.Select<DateTime?, int?>(d =>
d.HasValue ? d.Value.Year : null);
```

This code takes our `date_of_birth` column and selects the year from each row into an `IEnumerable<int?>` value, called `years`. The code is a little tricky to read because `DateTimeDataFrame` stores its values as nullable dates, so you need to worry about null values with each date.

Once we have our `years` variable, we can add a new column to the `DataFrame` by using an indexer for our column name and setting a new column in the `DataFrame`, containing the name and values to use.

We have this information already, so the code to add the new column is a minimal C# code cell:

```
dfPlayers["BirthYear"] = new Int32DataFrameColumn("BirthYear", years)
```

Since we ended the cell with a variable, we can see a preview of the data in the new column, as shown in *Figure 4.11*:

▼	[1986, 1988, 1988, 1987, 1987, 1990, 1988, 1989, 1991, 1991, 1991, 1989, 1990, 1987, 1990, 1989, 1
NullCount	0
Length	221
Name	BirthYear
DataType	System.Int32
(values)	[1986, 1988, 1988, 1987, 1987, 1990, 1988, 1989, 1991, 19

Figure 4.11 – The new column containing player birth years

Now that we've seen how to add a new column by specifying the values, let's see some even more powerful capabilities by adding a new column to track the approximate age of the player.

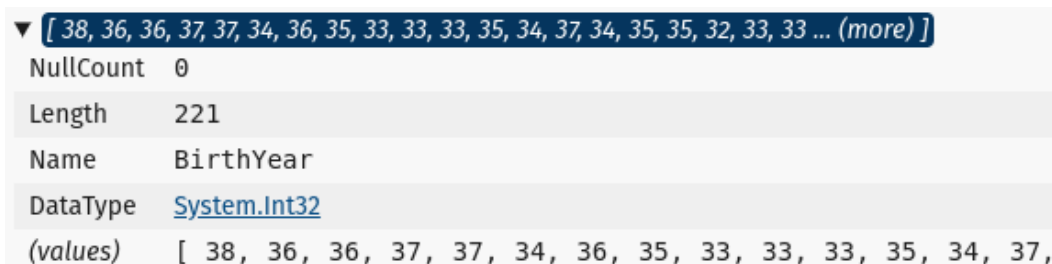
We'll do this by subtracting the entire `BirthYear` column from the current year, as shown with the following C# code cell:

```
DateTime.Now.Year - dfPlayers["BirthYear"]
```

While it may seem nonsensical to subtract `DataFrameColumn` from an integer, this code compiles and works perfectly.

What this does is create a new column, containing the result of subtracting each row's birth year from the current year. The resulting calculations are then stored in a new `Int32DataFrameColumn`, containing the result of this calculation *for every value* in the `BirthYear` column.

This can be demonstrated by looking at the results of the previous cell, as shown in *Figure 4.12*:



▼	[38, 36, 36, 37, 37, 34, 36, 35, 33, 33, 33, 35, 34, 37, 34, 35, 35, 32, 33, 33 ... (more)]
NullCount	0
Length	221
Name	BirthYear
DataType	System.Int32
(values)	[38, 36, 36, 37, 37, 34, 36, 35, 33, 33, 33, 35, 34, 37,

Figure 4.12 – The approximate age of each player

I find it almost magical that we can add, subtract, multiply, and divide columns by values and `DataFrameColumn` then generates an entirely new column.

It's important to note that just performing mathematical operations on a column doesn't cause a new column to be added to the `DataFrame`. To do that, we'll need to use an indexer to set the column in the `DataFrame` as shown here:

```
dfPlayers["Age"] = DateTime.Now.Year - dfPlayers["BirthYear"]
```

With this logic in place, we now have our new column added to the `DataFrame`.

Now that we've covered feature engineering, let's look at another aspect of data wrangling – handling missing values.

Handling missing values

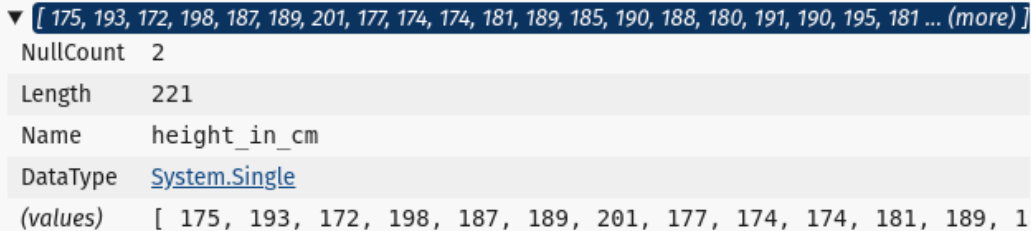
Data is messy and often includes incomplete values for data points. Failing to fill in missing values or remove rows containing them can cause errors downstream when training machine learning models. Because of this, you sometimes will need to replace the missing value with an actual value – or choose to remove rows with missing values entirely.

Using a `DataFrame`, you can detect missing values in any column by displaying the column in your notebook and looking at the `NullValues` property in the result.

The following C# code cell displays the `height_in_cm` column:

```
dfPlayers["height_in_cm"]
```

This outputs the information about the column, including a `NullCount` value of 2, as shown in *Figure 4.13*:



```
▼ [ 175, 193, 172, 198, 187, 189, 201, 177, 174, 174, 181, 189, 185, 190, 188, 180, 191, 190, 195, 181 ... (more) ]
NullCount  2
Length     221
Name       height_in_cm
DataType   System.Single
(values)   [ 175, 193, 172, 198, 187, 189, 201, 177, 174, 174, 181, 189, 1
```

Figure 4.13 – Two null values in the `height_in_cm` column

When examining this data, we know that there can't really be any players who have a null height. These null values represent incomplete records of two players in the league. In a real-world scenario, we'd probably reach out to the teams and ask them to provide the heights of the two players in question. However, since we're doing a data science exercise with hypothetical teams and players, we don't have that option.

Instead, we need to choose whether we want to remove those rows entirely or whether we should replace those values with an arbitrary value, such as 0.

A height of 0 doesn't make much sense, so let's replace the height of these players with a value that represents the average height of players in the league.

There are two measurements we might consider when looking at average heights:

- **Mean height**, which is simply the total height of all players divided by the number of players
- **Median height**, which is the height in the middle of the list of values if we were to sort all heights from shortest to tallest

We can determine these values by using the `Mean` and `Median` methods on the column, as shown by this C# code cell:

```
int medianHeight = (int)dfPlayers["height_in_cm"].Median();
int meanHeight = (int)Math.Round(dfPlayers["height_in_cm"].Mean());
new {median=medianHeight, mean=meanHeight}
```

This code uses some of the mathematical functions built into each column to get the median and mean heights and store them in variables, and then it displays the results at the bottom of a cell using an anonymous object.

This outputs the following values:

median	184
mean	183

While these values are close, I generally prefer to use the median instead of the mean, as the median is less susceptible to influence from extremely high or low outliers in the dataset.

To fill in height values using the median for rows missing a height, we use the `FillNulls` method on the column, as shown here:

```
dfPlayers["height_in_cm"] =  
    dfPlayers["height_in_cm"].FillNulls(medianHeight)
```

The `FillNulls` method considers the column to be **immutable** and returns a new column with values replaced, instead of modifying the original column. That's why this code takes the resulting column and puts it back into the `DataFrame`, using the column indexer to update the version of the `height_in_cm` column stored by the `DataFrame`.

The `DataFrame` object contains a `FillNulls` method not associated with any specific column, but this method will only work if all of your missing values are in the same type of column. Because of this, I find myself cleaning data at the individual column level instead of using the `DataFrame`-level method.

Let's close our discussion on `DataFrames` by exploring how to sort, filter, group, and merge data.

Sorting, filtering, grouping, and merging data

We've covered a lot of capabilities with `DataFrames`, but there are still a few advanced operations that are important to be able to perform, in order to successfully carry out advanced data wrangling scenarios.

We'll start with the simplest one of these – sorting data.

Sorting DataFrames

Occasionally, you will want to make sure your data follows a specific order in order to make your data analysis tasks easier. The `DataFrame` class allows you to order your data at the `DataFrame` level through the `OrderBy` and `OrderByDescending` methods. We'll try this out by sorting by the `name` column and then showing the first three rows, via the `Head` method:

```
dfPlayers.OrderBy("name").Head(3)
```

As expected, this results in names starting with A at the top of the `DataFrame`, as shown in *Figure 4.14*:

<i>index</i>	<i>Id</i>	<i>name</i>	<i>current_club_id</i>	<i>country_of_birth</i>	<i>date_of_birth</i>	<i>sub_position</i>
0	3333	Abel Sauer	1237	United States	1986-01-04 00:00:00Z	Central Midfield
1	591949	Al Padberg	1148	Scotland	2002-06-10 00:00:00Z	Right-Back
2	192080	Alan O'Reilly	762	England	1992-04-19 00:00:00Z	Goalkeeper

Figure 4.14 – The first three players in alphabetical order

As you might expect, the `OrderByDescending` method sorts by the column specified but in descending order, as illustrated by the following code:

```
dfPlayers.OrderByDescending("name").Head(3)
```

This results in several players named Zachary at the top of the `DataFrame`.

Like some other operations, the `OrderBy` and `OrderByDescending` methods follow immutability by creating a new `DataFrame` instead of modifying the original. This means that in order to actually permanently sort the data, we'd need to store the resulting `DataFrame` back in the `dfPlayers` variable.

The following code orders the players by their date of birth and stores the results in the `dfPlayers` variable:

```
dfPlayers = dfPlayers.OrderBy("date_of_birth")
```

I don't frequently find myself needing to sort data, but it's helpful to have the capability if you need it.

Grouping and aggregating DataFrames

Let's step back a moment and consider our dataset of player information. We have a number of high-level pieces of information about a player, such as where they are from, how old they are, what position they play, and how tall they are.

What we don't have at the moment is any sort of indication of how well they've performed in the league up to this point. To get that information, let's load up a second `DataFrame` that contains player stats by individual games, using the following C# code cell:

```
DataFrame dfStats = DataFrame.LoadCsv("PlayerGameStats.csv");
dfStats.Head(5)
```

This loads up our DataFrame and displays the top five rows:

appearance_id	game_id	player_id	player_club_id	player_current_club_id	yellow_cards	red_cards	goals	assists	minutes_played
2230141_33027	2230141	33027	762	1031	0	0	0	0	90
2225449_33027	2225449	33027	762	1031	0	0	0	0	90
2225453_33027	2225453	33027	762	1031	0	0	0	0	90
2225466_61842	2225466	61842	148	1031	0	0	0	0	3
2225483_61842	2225483	61842	148	1031	0	0	0	0	1

Figure 4.15 – Player game statistics

This information is more helpful in determining how well a player performed since it tells us how many goals and assists the player had, as well as how much of the game the player played. It also indicates red and yellow cards, which might give us some clues into the player's behavior on the pitch.

You may note that each player has multiple rows in this dataset. Every game that a player played in will be a single row in the dataset, with the player statistics for each game.

If we wanted to get more information on a player, we would need to group the DataFrame by `player_id` and roll up the statistics for the player at the grouping level. This concept is similar to how we can use `GROUP BY` in SQL queries to aggregate data.

To help us with grouping later, it'd be helpful to add a new column that represents the number of games each row represents. While this may seem odd, by adding a column with 1 in it for every row, we can summarize this later when grouping.

The code for this should be familiar:

```
dfStats["GamesPlayed"] = new Int32DataFrameColumn("GamesPlayed",
    Enumerable.Repeat(1, (int)dfStats.Rows.Count))
```

This code creates a column of integer values and then adds an enumerable of 1 values as long as the number of rows.

To group data, we call `GroupBy` on the DataFrame and provide the column name we wish to group by.

Next, we call an aggregation function, which will be one of the following:

- **Count:** Gets the number of non-null values
- **Min:** Gets the smallest value
- **Mean:** Gets the average value
- **Max:** Gets the largest value
- **Sum:** Adds values together
- **Product:** Multiplies values

When you call one of these aggregation functions, you can specify no parameters, indicating that the operation should be performed on each column, or you can specify a list of columns to perform the action on.

Typically, I provide a list of columns because aggregating string columns will result in exceptions being thrown, and it also makes it explicitly clear what I'm trying to aggregate.

Let's see this in action by grouping our stats by the `player_id` column and then getting the average value for playing time, goals, assists, and red and yellow cards:

```
var dfStatsByPlayer = dfStats.GroupBy("player_id")
    .Mean(new[] { "minutes_played", "goals", "assists", "yellow_cards",
        "red_cards" });
dfStatsByPlayer.Head(5)
```

This first statement groups rows by the player and then aggregates values, using `Mean` to provide averages of the specified columns. The result of this is then stored in the `dfStatsByPlayer` variable.

This results in a per-player set of averages, as shown in *Figure 4.16*:

index	player_id	minutes_played	goals	assists	yellow_cards	red_cards
0	33027	89.66326530612245	0	0.02040816326530612	0.01020408163265306	0
1	61842	50.532467532467535	0.1038961038961039	0.1038961038961039	0.05194805194805195	0
2	170321	77.6938775510204	0.02040816326530612	0.006802721088435374	0.10884353741496598	0.006802721
3	192765	74.38116591928251	0.03139013452914798	0.07623318385650224	0.13452914798206278	0
4	43530	54.7906976744186	0.09767441860465116	0.09767441860465116	0.07906976744186046	0

Figure 4.16 – Average per-game stats per player

`DataFrames` do not currently allow you to mix and match different aggregation methods, such as `Sum` and `Mean`, but you could potentially combine two separate aggregation `DataFrames` through merging.

Let's explore merging next.

Merging DataFrames

If you've ever written a `JOIN` statement in SQL, you should be familiar with the idea of merging two `DataFrames`.

Merging `DataFrames` involves taking two separate `DataFrames` and joining them together as a new `DataFrame`, combining the columns of both `DataFrames`.

In our case, we have `dfPlayers`, which is a `DataFrame` containing high-level player details, and our new `dfStatsByPlayer` `DataFrame`, which contains average game statistics for each player but lacks high-level context on the players themselves.

The structure of these two DataFrames is illustrated in *Figure 4.17*:

dfPlayers	
int	Id
string	name
int	current_club_id
string	country_of_birth
Date Time	date_of_birth
string	sub_position
string	position
bool	foot
int	height_in_cm
int	market_value_in_eur
int	BirthYear
int	Age

dfStatsByPlayer	
int	player_id
float	minutes_played
float	goals
float	assists
float	yellow_cards
float	red_cards

Figure 4.17 – Our two DataFrames

By merging these two DataFrames into a new DataFrame, we'll combine these two sources of information so that we get high-level and game statistics in a single row for each player.

When merging two DataFrames, you determine which DataFrames you want to merge, what columns to use to join between the two DataFrames, and what type of join to perform.

In our case, we want to join between `dfPlayers` and `dfStatsByPlayer`, with `Id` being the key column in the `dfPlayers` DataFrame and `player_id` being the corresponding column in the `dfStatsByPlayer` DataFrame.

We also need to determine which `JoinAlgorithm` to use. The options to consider should be very familiar to anyone with a SQL background:

- **Inner:** Only keep rows that have an equivalent row in both the left and right DataFrames
- **Left:** Keep all rows from the left DataFrame, but use null values for columns from the right DataFrame if no row in the right DataFrame matches the left DataFrame
- **Right:** Keep all rows from the right DataFrame, but use null values for columns from the left table if no row in the left DataFrame matches the right DataFrame

- `FullOuter`: Keep all rows from both the left and right `DataFrame`, but use null values where joins were not possible

Most of the time, you'll find yourself wanting either `Inner` or `Left`, depending on whether you want to keep rows that don't have matches on the right.

In our case, we want to get game summary data for players, and we don't want to deal with null values, so we'll choose `Inner`.

The syntax for this join looks like this:

```
DataFrame dfJoined = dfPlayers.Merge<int>(dfStatsByPlayer,
    leftJoinColumn: "Id",
    rightJoinColumn: "player_id",
    joinAlgorithm: JoinAlgorithm.Inner);
```

This uses `dfPlayers` as the left `DataFrame` and `dfStatsByPlayer` as the right one, and it specifies the appropriate columns and join algorithm.

You'll note here that we had to specify a generic type parameter for the `Merge` method. This parameter indicates the type used in the join columns, which is an integer in our dataset.

The resulting joined `DataFrame` is stored in `dfJoined`. Let's look at the columns it contains by transforming our `Columns` property, using the `Select` extension method:

```
dfJoined.Columns.Select(c => c.Name)
```

This outputs the following columns:

```
[ Id, name, current_club_id, country_of_birth, date_of_birth,
sub_position, position, foot, height_in_cm, market_value_in_eur,
BirthYear, Age, player_id, minutes_played, goals, assists, yellow_
cards, red_cards ]
```

As you can see, we now have all of the columns from both `DataFrames`, although we do have both `Id` and `player_id` columns representing duplicate data.

Column name collisions

It's possible that you will encounter cases where both `DataFrames` declare columns with the same name. By default, `Merge` will add a suffix of `_right` and `_left` to both columns in a new table when this occurs. You can customize these suffixes by specifying values for the `leftSuffix` and `rightSuffix` optional parameters.

Let's get rid of the duplicated `player_id` column by using the technique we covered earlier in this chapter:

```
dfJoined.Columns.Remove("player_id")
```

Before we move on, I want to point out one final thing by checking the count of rows in our joined `DataFrame`:

```
dfJoined.Rows.Count
```

This produces a count of 204 rows, which is lower than the 221 rows found in `dfPlayers`. When we did our `Inner` join, we lost rows for players who did not have a corresponding summary record in `dfStatsByPlayer`. In other words, if a player had never played a game, we dropped them from our results.

If, instead, we wanted to keep those rows but use null values for their game stats, we could do this by specifying a `Left` join.

Speaking of removing rows from a `DataFrame`, let's take a look at filtering next.

Filtering DataFrames

The `DataFrame` class has a `Filter` method that produces another `DataFrame`, following the same immutable pattern we saw earlier in this chapter with sorting, grouping, and merging.

The `Filter` method has several overloads, but the most common one takes in a boolean column as a parameter.

What `Filter` does with this column is look at each row in the column for a `true` or `false` value, using that value to determine which rows should be kept. If a row has a `true` value, it will be included in the resulting `DataFrame`, and if a row has a `false` value, it won't be.

You can think of this column as a boolean “mask” that identifies the rows that should be included and excluded.

Although you can manually create a new boolean column with `true` and `false` values of your choosing, it's a lot easier to use the built-in equality methods that are present on `DataFrameColumns` to generate this mask.

Let's say that we wanted to get a new `DataFrame` that only contains players who are attackers primarily trying to score goals. In this case, we'd want to filter down to just rows that have a position value of `Attack` in our dataset.

We can do this by using the `ElementwiseEquals` method on the `position` column to generate a new boolean column, as shown here:

```
PrimitiveDataFrameColumn<bool> attackFilter =
    dfJoined["position"].ElementwiseEquals("Attack");
attackFilter.ValueCounts()
```

This creates a boolean column for the length of the `DataFrame` and uses values of `true` and `false` for each row, depending on whether or not that player is in an attacking position.

The `ValueCounts()` call here tells us that 151 rows are set to `false` and 53 rows are set to `true`. Adding this up, we get the 204 rows total in our joined `DataFrame`.

Now that we have our row mask, we can pass it to the `Filter` method to create a new `DataFrame` that contains only the attacking rows, using the following C# code cell:

```
DataFrame dfAttackers = dfJoined.Filter(attackFilter);
dfAttackers.Head(5)
```

The resulting `dfAttackers` `DataFrame` consists entirely of attacking forwards, as shown in *Figure 4.18*:

index	Id	name	current_club_id	country_of_birth	date_of_birth	sub_position	position	foot	height_in_cm	market_value_in_
0	67063	Brandon Heller	1237	England	1990-11-26 00:00:00Z	Centre-Forward	Attack	right	185	6000000
1	61842	Isaac Jenkins	1031	England	1991-07-16 00:00:00Z	Right Winger	Attack	left	181	1800000
2	108725	Milton Roob	703	New Zealand	1991-12-07 00:00:00Z	Centre-Forward	Attack	right	191	7000000
3	123682	Floyd Johnston	762	England	1992-02-27 00:00:00Z	Centre-Forward	Attack	right	180	13000000
4	148455	Hector Gottlieb	31	Egypt	1992-06-15 00:00:00Z	Right Winger	Attack	left	175	65000000

Figure 4.18 – The new filtered `DataFrame` consists only of attackers

There are a number of different methods, such as `ElementwiseEquals` on `DataFrameColumn`, that help check for ranges of values, equality, inequality, and other boolean operations. All of these methods can be used to generate a mask that can then be used to filter your `DataFrame`.

With that, we've covered the breadth of ways that the .NET `DataFrame` helps us wrangle our data.

Let's conclude the chapter by looking at some other data-wrangling options.

DataFrames in other languages

Much of what is written in this chapter was not possible just a short time ago before ML.NET 3.0 was released. The `.NET DataFrame` is a capable tool, but it is one that is still growing and improving its usability and feature set.

If you prefer, you can perform your data wrangling tasks using another language or `DataFrame` implementation.

First, you may want to investigate the `Deedle Frame` object, which has been the F# `DataFrame` of choice since its inception. The `Deedle Frame` object was built with functional programming in mind and may be preferable to `Microsoft.Data.Analysis DataFrame` if F# is your primary programming language.

Second, the gold standard in the data analysis world is the `pandas` library in Python. The `pandas` library is efficient, full-featured, and intuitive. If you find that the `DataFrame` class doesn't fully meet your needs, I encourage you to investigate `pandas` to fulfill them.

Python and R in Polyglot Notebooks

It is possible to connect a Python or R kernel to Polyglot Notebooks with some features in early support. Using these kernels, you can mix and match C#, F#, Python, and R code in the same notebook. See the *Further reading* section for more details on this process.

If you prefer to work with `pandas` and Python, `Deedle Frames`, or just do your data cleaning in Excel or some other spreadsheet software, nothing is stopping you from doing that.

What this chapter points out is that with ML.NET 3.0's enhancements to the `Microsoft.Data.Analysis DataFrame`, it is now possible to do these data wrangling tasks in .NET if your preference is to keep as much of your code in .NET as possible.

If you'd prefer, you can always save your raw data to a CSV file, open that file up in another tool or library, process it, and then save the results back in a different CSV file. You can then load up this later CSV file in .NET or some other tool and use it for data visualization, machine learning, or whatever else you want to do with the data.

The thinking behind Polyglot Notebooks is that you should be able to use whatever languages and libraries you feel are most relevant to the work you're trying to do, and the right combination of languages may be different for every person and every team.

Summary

In this chapter, we covered data cleaning, feature engineering, and data wrangling, as well as strategies to deal with missing values.

We saw how `Microsoft.Analysis.DataFrame`, provided by ML.NET, is capable of working with rows and columns and provides advanced filtering, grouping, and analysis capabilities that are necessary to get our data sources into a clean state.

In the next chapter, we'll move on from data wrangling to data analysis and see some options to explore the trends and nature of our data sources, through data visualization and some additional `DataFrame` features.

Further reading

See the following resources for more information on the topics covered in this chapter:

- *The ML.NET GitHub Repository*: <https://github.com/dotnet/machinelearning>
- *Getting started with DataFrame*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/getting-started-dataframe>
- *Deedle Frames*: <https://fslab.org/Deedle/reference/deedle-frame.html>
- *Using Python and R in Polyglot Notebooks*: <https://github.com/dotnet/interactive/blob/main/docs/jupyter-in-polyglot-notebooks.md>
- *Pandas Python Data Analysis Library*: <https://pandas.pydata.org/>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/DataFrame/>

5

Visualizing Data

We've now seen how powerful `DataFrames` can be for manipulating data and some of what they can do to help you understand the data they contain. In this chapter, we'll delve into several ways of understanding your data through the process of exploratory data analysis. We'll do this through the use of descriptive statistics and interactive data visualizations. Along the way, we'll cover the following topics:

- Understanding exploratory data analysis
- Extracting insights with descriptive statistics
- Creating a box plot with `ScottPlot`
- Performing univariate analysis with `Plotly.NET`

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2, Exploring Polyglot Notebooks*, for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter05` folder.

Understanding exploratory data analysis

When you first work with a dataset, you don't usually start with much knowledge of the data it contains. In order to understand data, data analysts and data scientists perform **exploratory data analysis (EDA)**.

EDA is the process of using descriptive statistics and data visualization tools to understand the characteristics of your dataset.

The numbers and visuals behind EDA will help you understand things such as the following:

- Does the data have any outliers?
- Are there any missing values anywhere?
- Does my data feature clusters of similar values or a broad spectrum of values spread throughout the landscape?
- Do values in one column imply anything about probable values in another column?

All of these questions can help identify missing pieces of knowledge that can help with the tasks you're trying to accomplish. Additionally, what you discover during EDA can guide your data wrangling by identifying outliers to potentially remove and missing values to potentially replace, and even help you discover columns to add to your dataset through feature engineering.

In this chapter, we'll continue looking through the football dataset that we've been covering thus far. This time around we'll be applying EDA through the eyes of a football club trying to identify trends in the market value of various players through descriptive statistics and data visualization.

Data visualization's role in exploratory data analysis

Data visualization involves plotting one or more variables in some form of a chart in an attempt to gain or communicate insight from that data. There are a wide variety of charts that you can use in data visualization, but the most useful ones for us will be the box plot, histogram, violin plot, scatter plot, and heatmap. To help illustrate this concept, let's look at an example scatter plot in *Figure 5.1*:

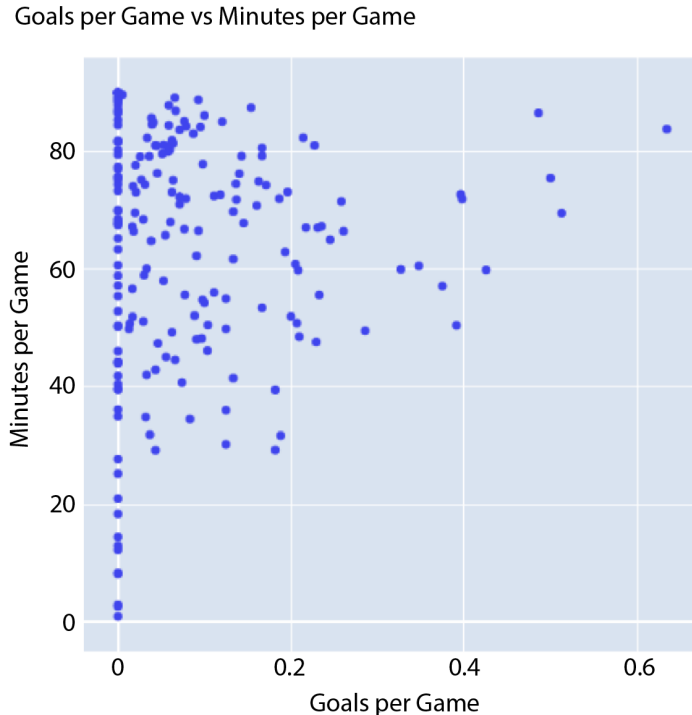


Figure 5.1 – A scatter plot of average minutes and goals per game

Each dot on this scatter plot represents the performance of a single player over a season in the dataset. This chart shows us two variables along its x (horizontal) and y (vertical) axes. Specifically, it shows the relationship between the average number of minutes a player plays each game and the average number of goals that the players score over the same season.

We can see that as a player's goals per game increases, the number of minutes per game also increases, implying a **positive correlation** between the two variables. *In other words, the more goals a player scores, the more likely they are to also play more of each game on average.* This relationship makes sense because players who play more of a game will have more opportunities to score and because players who score tend to stay in games in the hope that they'll score again.

The other particularly interesting thing about this chart is that it shows us that there are plenty of players who never scored over the entire season. We can see that some of them played only a few minutes each game, while many others played the majority of each game.

While this may seem counter-intuitive, it makes sense when you remember this dataset includes players who rarely score, such as goalkeepers and defenders.

In the next chapter, we'll see how to create scatter plots like this and explore the topic of correlation in more depth. For now, let's turn our attention back to the other way we can perform EDA: descriptive statistics.

Descriptive statistics for EDA

Descriptive statistics is a phrase that's used to describe measurements that summarize a group of data points.

Don't panic!

If you have an aversion to math and statistics, your brain might freeze up at terms such as descriptive statistics, quartiles, and standard deviation. Don't worry; we won't be focusing on the mathematical basis of these terms, and you won't see any equations here. Instead, we'll talk about what these things represent at a high level and how you can use them.

Focus on the concepts and how they can help you. Don't worry about memorizing terminology right now; take deep breaths, and we'll be building beautiful data visualizations soon.

Let's explore descriptive statistics by seeing how `DataFrames` and the `MathNet.Numerics` library can help us get a deeper understanding of our data.

Extracting insights with descriptive statistics

In this section, we'll use two tools to generate descriptive statistics for us based on our data and see what insights they can provide.

Using `DataFrame.Description` to generate descriptive statistics

Let's start by using the `DataFrame` class to see how its `Description` method can be applied to an entire dataset. First, we need to load up the final dataset from the last chapter into a new `DataFrame`. This time around, we're going to do this in F# instead of C#. This is because many of the data visualization libraries in .NET were designed specifically for F# due to F#'s popularity in the scientific community as a functional programming language.

Want to work with C#?

While most of what we're covering in this chapter is possible using C#, it typically isn't quite as easy to do as it is in F# or requires an additional wrapper library that doesn't have all of the features of the F# version. Because of this, we'll use F# in this chapter and explain the F# syntax for those unfamiliar with F#.

In my experience, you'll have a better time performing data visualization work using F# instead of C#, so I encourage you to try it, but the point of Polyglot Notebooks is *you should be free to pick the language you feel is best for your team and the task you're trying to accomplish*.

As always, we'll reference the `Microsoft.Data.Analysis` NuGet package and then use `DataFrame.LoadCsv` to load our data, as shown in this F# code cell:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
open Microsoft.Data.Analysis

let df = DataFrame.LoadCsv("JoinedStats.csv")
df.Head(5)
```

If you're not used to F# code, you may notice that this looks nearly identical to the C# version of the same code you saw in the last chapter, with a few different keywords.

The main differences here are that we use `open` instead of `using` to import a namespace, and we use `let` to declare a variable in a similar manner to the C# `var` keyword.

Because we end our cell with a call to `Head`, we see the results of that call below the cell, as shown in *Figure 5.2*:

index	Id	name	country_of_birth	date_of_birth	sub_position	position	foot	height_in_cm	market_value_in_eur
0	3333	Abel Sauer	United States	1986-01-04 00:00:00Z	Central Midfield	Midfield	right	175	1000000
1	71271	Nelson Bednar	Spain	1987-01-03 00:00:00Z	Goalkeeper	Goalkeeper	right	190	600000
2	45494	Moses Mitchell	United States	1987-01-24 00:00:00Z	Goalkeeper	Goalkeeper	right	198	250000
3	52570	Julius Skiles	England	1988-03-17 00:00:00Z	Goalkeeper	Goalkeeper	right	201	1200000
4	33027	Dewey Luetngen	United States	1988-04-03 00:00:00Z	Goalkeeper	Goalkeeper	left	193	500000

Figure 5.2 – The first five rows of the `DataFrame`

With our data loaded, we'll call the `Description` method on the `DataFrame` object, as shown by this F# code cell:

```
df.Description()
```

As shown in *Figure 5.3*, `Description` analyzes all numeric and date columns to generate a table of descriptive statistics.

Description	date_of_birth	height_in_cm	market_value_in_eur	BirthYear	Age	minutes_played
Length (excluding null values)	204	204	204	204	204	204
Max	<null>	201	85000000	2006	38	90
Min	<null>	163	250000	1986	18	1
Mean	<null>	183.47058	19237496	1996.9166	27.083334	64.336006

Figure 5.3 – Some of the descriptive statistics shown by `DataFrame`

The `DataFrame` presents four metrics per column:

- **Length** – The number of non-null values in the column.
- **Max** – The largest value in the column.
- **Min** – The smallest value in the column.
- **Mean** – The average value for the column, also referred to as the mean value. The mean should be the sum of all values divided by the column's length.

One thing you may note here is the presence of `<null>` for `Min`, `Max`, and `Mean` in the `date_of_birth` column. This is because that column is a `DateTime` column and its implementation as of ML.NET 3.0 does not include calculations of minimum and maximum date or a way of averaging together all dates in the column. This is something I would expect to be supported in future releases.

For the other columns, take a moment and see what insights we can extract using just these four values and then read my interpretation of this data.

I noticed a few interesting things:

- All of these columns appear to have the same length, which suggests there are no missing values.
- Every player averages at least 1 minute of play time per game.
- Some players played every minute of every game for the entire season.
- On average, players are playing 2/3 of the games in a season.
- Most players are in their late 20s and earn just under 20 million euros each year.
- There are some players with very young ages and very old ages. I suspect these are unusually talented players who were able to reach the league early and stay longer than average.

- The lowest pay is a quarter of a million euros per season, while the highest is 85 million.
- There is some height differential in players, but the difference between the minimum and maximum heights is less than 20 centimeters.

The point I'm trying to make is not that being a footballer is a brief but rewarding career, but rather that a few numbers can tell you a lot about the distribution of data. I will often use even basic measurements such as these to understand the scope and scale of data as I'm designing systems to process and display data. While these basic measures can do a lot for you, there are a few more specialized statistical measurements that you may sometimes benefit from. When you need them, you can either calculate them by doing the math or pull in a library built to calculate them for you.

In this next section, we'll use the `MathNet.Numerics` library to calculate and display additional descriptive statistics for us and see what value we can gain from their use.

Descriptive statistics with `MathNet.Numerics`

`MathNet.Numerics` is an open source library built to perform specialized mathematical tasks in a variety of areas, including regression analysis, probability, and statistics.

You can check out the full breadth of capabilities at <https://numerics.mathdotnet.com/> and find a link to the source repository there if you're curious, but in this chapter, we'll look at a few uses of the statistical capabilities `MathNet.Numerics` introduces.

As we do with everything else, we'll need to import a NuGet package for `MathNet.Numerics` into our notebook with an F# code cell:

```
#r "nuget:MathNet.Numerics,5.0.0"
open MathNet.Numerics.Statistics
```

This will install the prescribed version of `MathNet.Numerics` into the notebook and make classes defined in its `Statistics` namespace available to us for use. `MathNet.Numerics` has several different ways of calculating descriptive statistics, but the most effective ones for us with our `DataFrame` will be some of the array-based methods. Specifically, `MathNet.Numerics` does very well if we can provide it a *pre-sorted* array of values to analyze for a given column.

Since we're focusing on player market value in this chapter, let's write an F# code cell to create a sorted array of values from the `market_value_in_eur` column:

```
let sortedValues = df["market_value_in_eur"]
                    |> Seq.cast<single>
                    |> Seq.map float
                    |> Array.ofSeq
                    |> Array.sort
```

This code probably looks a little unusual to you if you're not used to F#, so let's walk through it step by step:

1. The code grabs the `market_value_in_eur` column out of the `DataFrame` using the indexer. This is a `SingleDataFrameColumn`, but the compiler assumes it is simply a `DataFrameColumn`.
2. We take that column from step one and “pipe” it into the `Seq.cast<single>` function using the `|>` **pipng operator**. This operator takes the results of the prior step and passes them in as an argument to `Seq.cast<single>`.
3. `Seq.cast<single>` takes our column and casts it to a sequence of single values. In C# terminology, this is roughly equivalent to casting it as an `IEnumerable<float>` that `SingleDataFrameColumn` implements.
4. We then take the sequence of F# singles and pipe it into the `Seq.map` function, which has been instructed to convert them to F# floats (which are roughly equivalent to C# doubles, to the frequent confusion of this author). This conversion is necessary because `MathNet.Numerics` prefers to work with the additional precision these values support.
5. Once we have our sequence of F# floats, we convert that sequence to an array by piping it to the `Array.ofSeq` function. This is roughly the equivalent of a C# `.ToArray()` call using LINQ.
6. Finally, we send our array to the `Array.sort` method, which orders the values in ascending order.
7. The end result is then stored in `sortedValues`, where we can use it in subsequent cells.

Some of you will find yourself gaping in horror at that chain of processing tasks, but I promise you that you will get used to this style of pipeline-based programming the more you play with F#.

Additionally, I should note that `MathNet.Numerics` does have some optional F# extensions that may make processing tasks such as this slightly easier. I chose to keep us focused on the bare minimum so you could see some F# in action.

Now that we have our sorted values for our market values, we can generate descriptive statistics with them using the `DescriptiveStatistics` class, as shown in this F# code cell:

```
DescriptiveStatistics(sortedValues)
```

Because this object is the last item in our cell, Polyglot Notebooks displays its properties to us, showing us the additional statistical measurements it provides:

▼ <i>MathNet.Numerics.Statistics.DescriptiveStatistics</i>	
Count	204
Mean	19237500
Variance	315500963669950.8
StandardDeviation	17762346.795115527
Skewness	1.4088863400583054
Kurtosis	1.788555733184937
Maximum	85000000
Minimum	250000

Figure 5.4 – Descriptive statistics using MathNet.Numerics

As you can see, we have our count, minimum, maximum, and mean as we did before, but we also have a few new specialized metrics: standard deviation, variance, skewness, and kurtosis.

This book is thankfully not a statistics textbook, so I'll not try to teach these more advanced statistical concepts, other than to say that they collectively describe the shape of the distribution of the data. When we assume the data follows a typical “bell curve” representing a normal distribution, these values talk about how wide and tall that curve is as well as how much of the data remains near the center versus at the two extremes.

Standard deviation

Of all the statistical measurements covered, if you have to learn one you don't already understand, I strongly recommend looking into standard deviation and its relationship with bell curves. Put simply, the standard deviation describes aspects of the overall width of the bell curve by describing how close to the mean value most values are likely to occur. See the *Further reading* section at the end of the chapter for more resources on standard deviation.

I usually don't find myself using these additional statistics on a daily basis, but it is nice to know that they're present when I need them.

You'll notice that a few statistics of interest are not included in this list, such as the median value. `MathNet.Numerics` provides even more statistical measurements available using methods in the `SortedArrayStatistics` static class.

The following F# code cell uses this class to calculate the median from our sorted array of values:

```
SortedArrayStatistics.Median(sortedValues)
```


This displays 15,000,000 euros as the median valuation. The median is the midpoint or 50% marker for your data. One thing that you'd need to create a visual such as a box plot would be markers for where the first quartile of data lies (the 25% marker) and where the third quartile of data is (the 75% marker).

We can calculate these quartiles with `SortedArrayStatistics.LowerQuartile` and `SortedArrayStatistics.UpperQuartile` respectively.

With these values, we have everything we need to construct a box plot, so let's take our first steps into data visualization by doing that now.

Creating a box plot with ScottPlot

The first data visualization we'll create will be a box plot. Box plots show you the same information we just saw with descriptive statistics, but in a graphical way. This helps you look at a single chart to understand how values are distributed in your dataset.

There are several different options for charting in .NET that support Polyglot Notebooks, but we'll start with the `ScottPlot` library because it's going to make it most apparent how our descriptive statistics translate into a visual display.

We'll start by installing `ScottPlot` and `Plotly.NET`, which we'll use later this chapter:

```
#r "nuget:ScottPlot,5.0.37"
#r "nuget:Plotly.NET,5.0.0"
#r "nuget:Plotly.NET.Interactive,5.0.0"
```

Next, we'll open the `ScottPlot` namespace and register a custom formatter that renders `ScottPlot` plots graphically in our notebook. We'll talk more about these formatters near the end of the book, but for now, run the following F# code cell:

```
open ScottPlot
open System.IO
open Microsoft.DotNet.Interactive.Formatting

Formatter.Register<ScottPlot.Plot>(
    (fun (plot: ScottPlot.Plot) (writer: TextWriter) ->
        writer.Write(plot.GetImageHtml(400, 300))),
    mimeType = "text/html"
)
```

```
open ScottPlot
open System.IO
open Microsoft.DotNet.Interactive.Formatting

Formatter.Register<ScottPlot.Plot>(
    (fun (plot: ScottPlot.Plot) (writer: TextWriter) ->
        writer.Write(plot.GetImageHtml(400, 300))),
    mimeType = "text/html"
)
```

This sets us up nicely to create and display our first chart using ScottPlot.

Additional steps for non-Windows systems

ScottPlot may require additional dependencies to be installed in order for plots to render correctly on Mac or Linux. See <https://scottplot.net/faq/dependencies/> for more steps for your platform.

With everything in place, it's now time to create our box plot in an F# code cell:

```
let plot = new Plot();
plot.Title "Player Market Value Distribution"
plot.Add.Box(new Box(
    WhiskerMax = (sortedValues |> Array.max),
    BoxMax = (sortedValues |> SortedArrayStatistics.UpperQuartile),
    BoxMiddle = (sortedValues |> SortedArrayStatistics.Median),
    BoxMin = (sortedValues |> SortedArrayStatistics.LowerQuartile),
    WhiskerMin = (sortedValues |> Array.min)
))
plot
```

This code creates a ScottPlot Plot object, gives it a title, and then adds a Box object to it. The Box object has BoxMax, BoxMiddle, and BoxMin properties that relate to the Q3, median, and Q4 markers from our descriptive statistics. The WhiskerMax and WhiskerMin properties refer to the lines coming out of the box, which correspond to our max and min values.

When this code runs, it renders a box plot, as shown in *Figure 5.5*:

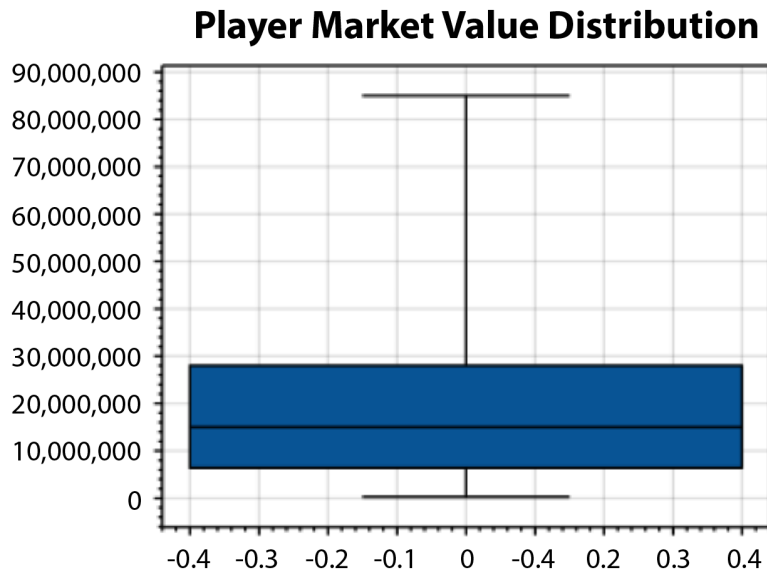


Figure 5.5 – A box plot showing our data distribution

Up until this point in the chapter, we've focused a lot on numbers. This chart contains those same numbers, but tells a story of where the majority of the data is distributed.

We can see clearly that the average player makes a little less than 20 million euros a season, while a chunk of players are compensated below this or above this – up until a little before 30 million euros. The two whisker lines representing the top and bottom 25% of our data show us that there are some extreme cases where players are paid 2 or 3 times as much as their peers based on the perceived value of these top-tier players.

At this point in the chapter, we're going to move away from ScottPlot and move towards Plotly.NET as we move on to exploring additional ways of analyzing data visually. This has more to do with Plotly.NET's wide range of supported visuals than any limitations of ScottPlot. In fact, I strongly recommend you look into ScottPlot on your own, particularly if you want to do a lot of data visualization work using C# as Plotly.NET is currently much better for F# development than it is for C# development.

With that said, let's see what Plotly.NET has to offer us for analyzing variable distributions.

Performing univariate analysis with Plotly.NET

One of our jobs when performing data analysis work is to understand how data is distributed. We explored this earlier using descriptive statistics and our first box plot in ScottPlot.

In this section, we're going to take our exploration of our market value up a notch by using Plotly.NET and its specialized charts. Before we do that, I want to put a label on the task we're working on. When we're exploring how data for a single column of values is distributed, that analysis process is called **univariate analysis**.

The goal of univariate analysis is to discover everything we can from just that one series of values – in this case, our market values. If this term doesn't feel natural, that's okay; just remember that univariate is a fancy word that just means *one type of variable*. Now that we've ironed out that bit of terminology, let's get started with Plotly.NET.

Plotly and Plotly.NET

Plotly offers a set of standard charting visuals using a common API that is available for a wide range of languages and platforms, including .NET, Python, R, and JavaScript.

What I like about Plotly is that it takes only a little bit of code to get a very polished and interactive data visualization. Plotly also offers comprehensive customization options and documentation. I find that Plotly gets me close to where I want to be by default and provides the additional resources needed to get the visuals I want in those other cases.

Earlier, we installed the `Plotly.NET` and `Plotly.NET.Interactive` NuGet packages, which help us see the charts in our notebook. With those libraries installed, let's declare a series of variables that we'll want to chart over the remainder of this chapter. For convenience, we'll do that in a single F# code cell:

```
let marketValues = df["market_value_in_eur"] |> Seq.cast<single>
let positions = df["position"] |> Seq.cast<string>
let clubs = df["club_name"] |> Seq.cast<string>
let goals = df["goals"] |> Seq.cast<single>
let assists = df["assists"] |> Seq.cast<single>
let minutesPerGame = df["minutes_played"] |> Seq.cast<single>
let redCards = df["red_cards"] |> Seq.cast<single>
let playerNames = df["name"] |> Seq.cast<string>
```

Note that we're not sorting any of these variables because Plotly.NET does not need them to be sorted and, in fact, sorting them will cause some issues later when we plot multiple variables on the same graph.

Let's revisit box plots by seeing how they work in Plotly.NET.

Box plots in Plotly.NET

We'll declare an F# code cell to render a box plot charting our `marketValues` variable:

```
Chart.BoxPlot(Y = marketValues)
  |> Chart.withTitle "Player Market Value Distribution"
  |> Chart.withTraceInfo "Market Value"
  |> Chart.withSize(300, 400)
```

This cell creates a Plotly.NET `BoxPlot` object and specifies that the `Y` property for this object points at our `marketValues` array. Believe it or not, this one line is actually all we need for a functioning box plot, but the following three lines customize the appearance of the chart.

We use the F# `|>` piping operator to take our `BoxPlot` and customize it by giving it a title, customizing the trace info displayed in the margin of our data, and giving it a custom pixel size that works well for displaying the results in this book.

The result is a minimal and elegant box plot showing our data:

Player Market Value Distribution

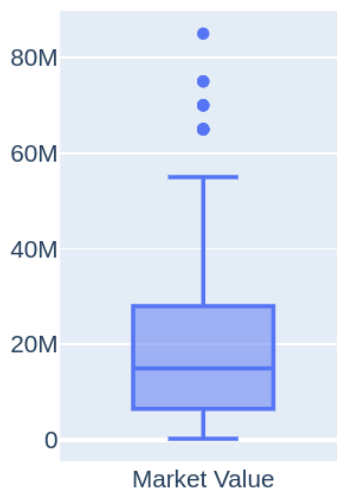


Figure 5.6 – A Plotly.NET box plot of player market value

If we compare *Figure 5.5* and *Figure 5.6*, you'll notice a key difference beyond normal styling differences.

With the ScottPlot box plot, the maximum value whisker was at the largest 85 million euros – the largest data point in the series. With Plotly.NET, the whisker representing the “upper fence” of the data ends at 55 million euros, and we see 4 dots representing extreme outliers above the average range of data.

In other words, not only did we not need to calculate the statistical boundaries ourselves, but Plotly.NET helped flag a few key outliers by adjusting the visual it produced for us!

Let's explore Plotly.NET's customization options by changing that Y to an X and adding another property containing player names, as shown in the following F# code cell:

```
Chart.BoxPlot(X = marketValues, MultiText=playerNames)
|> Chart.withTitle "Player Market Value Distribution"
|> Chart.withTraceInfo "Market Value"
|> Chart.withSize(400, 300)
```

The change from Y to X causes the box plot to be rendered horizontally instead of vertically. Additionally, by providing `playerNames` to the `MultiText` property, we now see the names of our outlier players when you hover over them with a mouse, as shown in *Figure 5.7*:

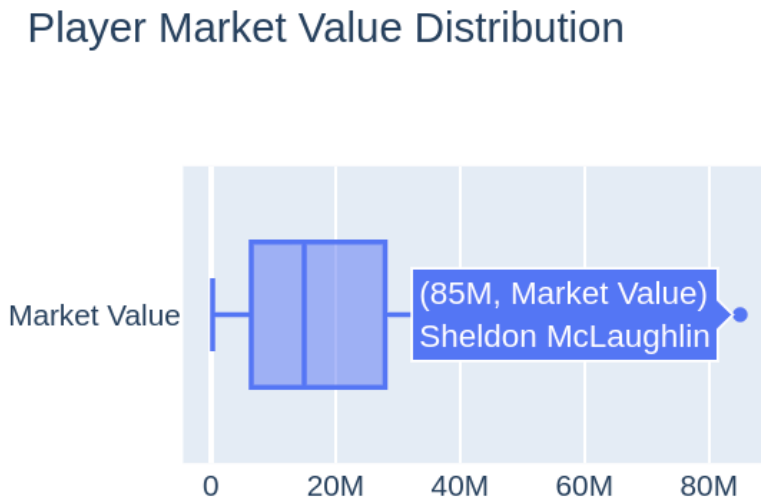


Figure 5.7 – Interactive data for our outliers

While we're talking about outliers, I should also note that Plotly.NET offers a way of customizing which points are shown as outliers. You can show definite outliers, suspected outliers, no outliers at all, or show all points on the chart.

The final option of that is particularly interesting. Take a look at this chart, which renders all points in the data below the box plot:

```
Chart.BoxPlot(X = marketValues,  
              BoxPoints = StyleParam.BoxPoints.All,  
              MultiText=playerNames,  
              Jitter=0.35)  
|> Chart.withTitle "Player Market Value Distribution"  
|> Chart.withTraceInfo "Market Value"  
|> Chart.withSize(500, 350)
```

This sets `BoxPoints` to show all points and then specifies a `Jitter` that spaces out the different dots so you can see their volume and density, as shown in *Figure 5.8*:

Player Market Value Distribution

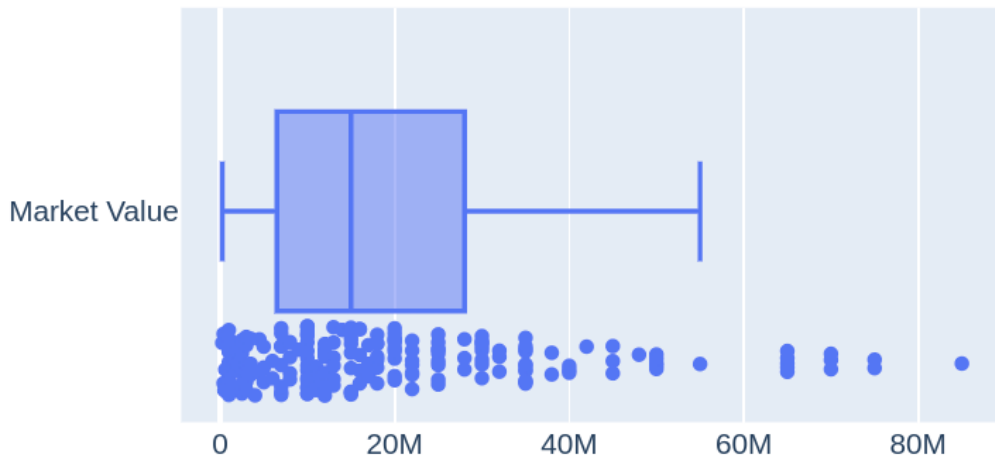


Figure 5.8 – A box plot showing all points with jitter applied

This chart feels less clean and elegant than the previous ones, but it does help us understand more of the individual data points. Plus, because we're still using the `MultiText` property, you can hover over any point to see the player's name and market value.

Now that we've seen how effectively Plotly.NET handles box plots, let's look at a closely related plot called the violin plot.

Violin plots with Plotly.NET

If box plots got their name because they're shaped like a box, violin plots get their names for a similar reason. Violin plots are nearly identical to box plots except they feature curves along the boxes that show you the rough distribution of the data.

Let's see this in action through an F# code cell that's very similar to a box plot call:

```
Chart.Violin(X = marketValues)
|> Chart.withTitle "Player Market Value Distribution"
|> Chart.withTraceInfo "Market Value"
|> Chart.withSize(600, 350)
```

Here, the only major difference between creating a violin plot and a box plot is that we're using `Violin` instead of `BoxPlot`. With that minor difference, we get a significantly different visual:

Player Market Value Distribution

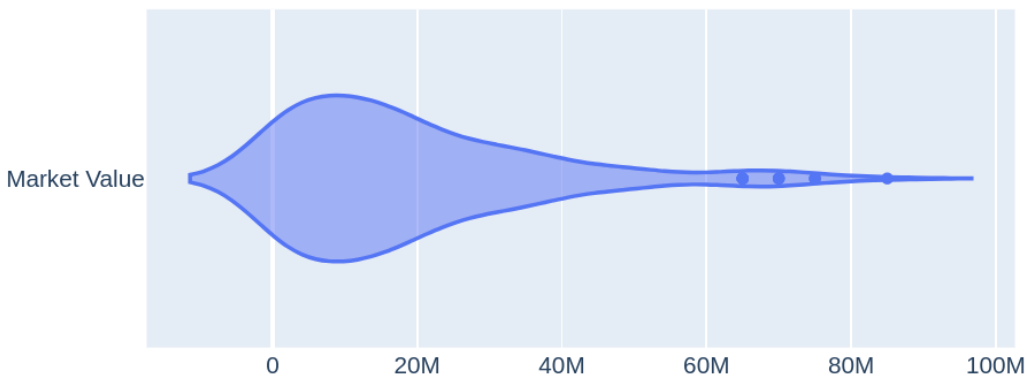


Figure 5.9 – A violin plot showing the distribution of data

Instead of highlighting the Q2 and Q4 markers like a box plot, the violin plot helps you get a rough idea of how many data points are at each part of the plot.

It's important to note that a violin plot is not a perfect representation of where data is. For example, none of our players are paid 0 or fewer euros each year, and yet the chart extrapolates that density from the curves of data it does have.

To illustrate this, let's add our points back in and see their density mirror the shape of the violin plot:

```
Chart.Violin(X = marketValues, Points = StyleParam.JitterPoints.All)  
|> Chart.withTitle "Player Market Value Distribution"  
|> Chart.withTraceInfo "Market Value"  
|> Chart.withSize(600, 350)
```

This gives us a clearer picture of how our violin mirrors our data, as shown in *Figure 5.10*:

Player Market Value Distribution

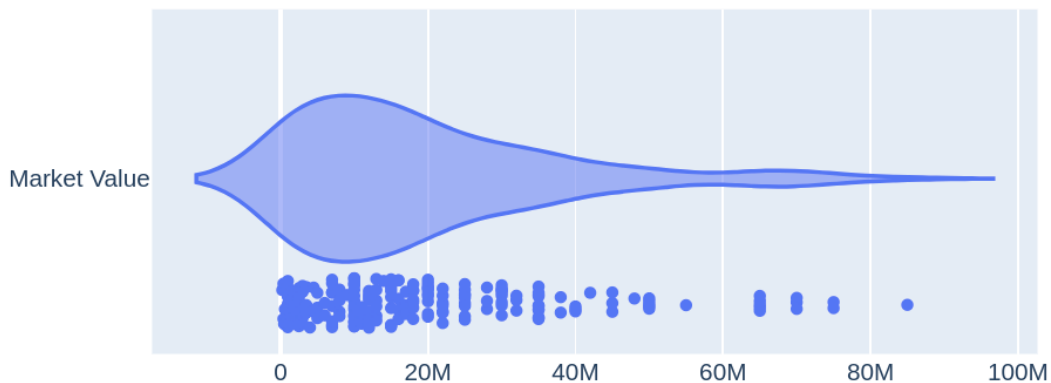


Figure 5.10 – Point densities mirroring the shape of the violin plot

In general, I prefer to use violin plots when understanding how data is distributed, but I usually switch to box plots or histograms when communicating data distributions to people not used to data analysis.

On that note, let's see how histograms help us understand variable distributions.

Histograms with Plotly.NET

Histograms are another way we can see how values are distributed among a single variable type. Unlike box or violin plots, histograms use simple bar charts to show us where our data is.

Let's see one in action by adding an F# code cell:

```
Chart.Histogram(X = marketValues)
|> Chart.withTitle "Player Market Value Distribution"
|> Chart.withSize(500, 300)
```

This time, we're calling `Histogram` instead of `BoxPlot` or `Violin` and continuing to customize our chart's title and sizes.

The resulting chart is shown in *Figure 5.11*:

Player Market Value Distribution

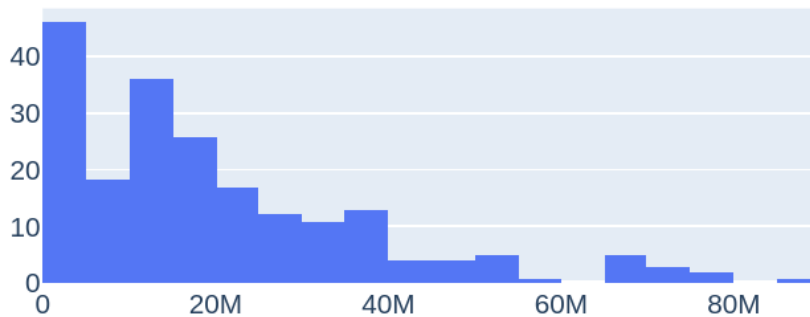


Figure 5.11 – A histogram showing player market values

What this chart shows is the frequency of values between specific ranges. When Plotly.NET builds this histogram, it picks a number of **bins** that data can fall into.

A bin is going to have a range of values, such as 20 million to 24.9 million euros. Plotly.NET counts the number of players who fall into that range, determines that 17 players do, and then represents that 20 – 24.9 million range with a single bar going to 17 on the Y axis.

In other words, each one of these little bars we see is a count of how many players are in that band of salary, with many near the left and very few outliers past the 60 million euros marker.

Market value

When I first saw this histogram of market value, I was very surprised since my natural assumption was that because we had so many players at the extreme low end, that must mean that their salaries are 0 euros. I had assumed that perhaps some of our players were missing valuation information and were valued at 0 by accident, but upon further investigation, this turned out to be legitimate.

I share that anecdote of my exploration process into the dataset in writing the book just to show you the power these visuals have. Despite looking at descriptive statistics and box and violin plots, the histogram still surprised me and showed me a new aspect of the data that I hadn't appreciated without this view.

You may be wondering about the binning process. By default, Plotly.NET picks the number of bins for you, but you can pick one yourself by specifying the `NBinsX` property on the `Histogram`. The default in our dataset here appears to be 20, but *Figure 5.12* shows a histogram with 50 bins by setting `NBinsX=50`:

Player Market Value Distribution

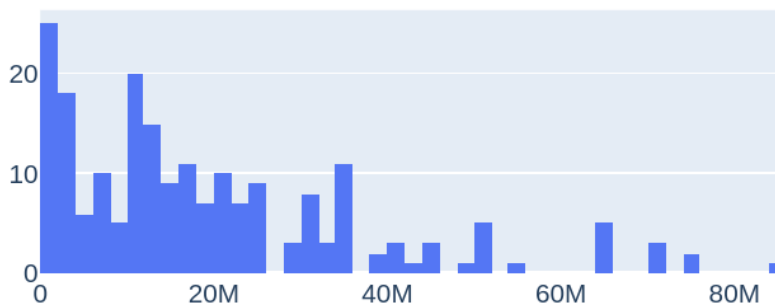


Figure 5.12 – A histogram with 50 bins

Notice how each bar is less wide and the data appears more jagged. This effect increases if we increase the bin size to 150, as shown in *Figure 5.13*:

Player Market Value Distribution

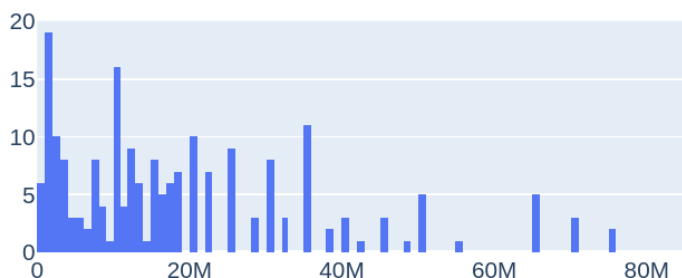


Figure 5.13 – A histogram with 150 bins

With larger bin values, the gaps between data points get clearer, but the overall data feels noisier. It's up to you to determine the best number of bins to use for your datasets. I will usually leave it to the default value and then customize it if I'm curious about specific cases. With that, we've explored univariate analysis quite well with descriptive statistics, ScottPlot, and Plotly.NET.

It's time to up our game a bit and bring in multiple variables into our charts with multivariate analysis as we'll see in the next chapter.

Summary

In this chapter, we went back and forth between statistical measurements and graphical representations of them and showed how EDA lets you use descriptive statistics and data visualizations to make sense of your data.

We saw that box plots give you a graphical way of seeing your minimum, maximum, median, and quartile values of data to help you perform univariate analysis by seeing how data is distributed over a single column of data. We also explored alternative ways of visualizing single-variable distributions, such as the violin plot and the histogram.

In the next chapter, we'll take our data visualization efforts up a notch by adding additional dimensions and seeing how different columns interact with each other.

Further reading

See the following resources for more information on the topics covered in this chapter:

- *MathNet.Numerics Descriptive Statistics*: <https://numerics.mathdotnet.com/DescriptiveStatistics>
- *ScottPlot 5.0 Cookbook*: <https://scottplot.net/cookbook/5.0/>
- *Plotly.NET*: <https://plotly.net/>
- *Plotly.NET C# Examples*: <https://plotly.com/csharp/creating-and-updating-figures/>
- *F# Piping*: <https://theburningmonk.com/2011/09/fsharp-pipe-forward-and-pipe-backward/>
- *Standard Deviation and Variance*: <https://www.mathsisfun.com/data/standard-deviation.html>
- *Understanding skewness and kurtosis*: <https://www.datacamp.com/tutorial/understanding-skewness-and-kurtosis>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/DataVis/>

6

Variable Correlations

In the previous chapter, we saw how data visualizations and applications of descriptive statistics can help you understand how your data is distributed. This information was helpful in identifying extreme outliers and prompting decisions about how to handle those large values.

In this chapter, we'll continue our data analysis work by visualizing multiple data points together and seeing how different features of our dataset correlate with each other.

This technique is helpful for understanding the relationships between different features in your dataset and determining what information is likely to help a machine learning model.

In this chapter, we'll cover the following topics:

- Performing multivariate analysis with Plotly.NET
- Identifying variable correlations

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2, Exploring Polyglot Notebooks*, for additional details on this process.

You can find the notebook and data for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter06` folder.

Performing multivariate analysis with Plotly.NET

In the last chapter, we covered univariate analysis, which focused on analyzing a single dimension of data, such as one column in a `DataFrame`. By contrast, **multivariate analysis** studies how two or more variables interact with each other.

While we'll look at some charts built specifically for multivariate analysis, such as the scatter plot, let's start by seeing how some of the charts we've already introduced can illustrate multiple variables at the same time.

Loading data and dependencies

In this chapter, we'll be working with the same data as *Chapter 5, Visualizing Data*, and using the same steps to get set up.

We'll start with our dependencies:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
#r "nuget:Plotly.NET,5.0.0"
#r "nuget:Plotly.NET.Interactive,5.0.0"
#r "nuget:MathNet.Numerics,5.0.0"
open MathNet.Numerics.Statistics
open Plotly.NET
open Microsoft.Data.Analysis
```

As before, this sets us up to load data into a `DataFrame` and then analyze it with Plotly.NET. With our dependencies in place, we can load the data into a `DataFrame` and extract sequences of values for relevant columns, using F# as we did before:

```
let df = DataFrame.LoadCsv("JoinedStats.csv")
df.Columns.Remove("Id")
df.Columns.Remove("club_id")

let marketValues = df["market_value_in_eur"] |> Seq.cast<single>
let positions = df["position"] |> Seq.cast<string>
let clubs = df["club_name"] |> Seq.cast<string>
let goals = df["goals"] |> Seq.cast<single>
let assists = df["assists"] |> Seq.cast<single>
let minutesPerGame = df["minutes_played"] |> Seq.cast<single>
let playerNames = df["name"] |> Seq.cast<string>
let redCards = df["red_cards"] |> Seq.cast<single>
```

With our data in place, we're now ready to start the practice of analyzing how multiple variables interact through multivariate analysis.

Multivariate analysis with box and violin plots

In the last chapter, we used the box plot to visualize how players are valued. This showed us the common valuation levels players received and highlighted a few outlying values.

But what if we wanted to see how players were valued based on the positions they played in, the teams they were on, whether or not they kicked with their right foot, or even what age bracket they were in?

One approach might be to use `DataFrame.Filter` to create a smaller `DataFrame` with just attacking players, and then extract the valuation data from that `DataFrame`, build a box plot around it, and then repeat the process for other positions.

It turns out that box and violin plots offer a better alternative to this manual process: you can set both the X and Y properties of your data to visualize how different variables interact.

We'll try this now by using the Y property to store our market values and put our position variables in the X property, as shown in this F# code cell:

```
Chart.BoxPlot(Y = marketValues, X = positions)
|> Chart.withTitle "Player Market Value by Position"
|> Chart.withSize(500, 350)
```

This causes Plotly.NET to segment your data into distinct groups by each position and render a box plot for all supported positions, as shown in *Figure 6.1*:

Player Market Value Distribution by Position

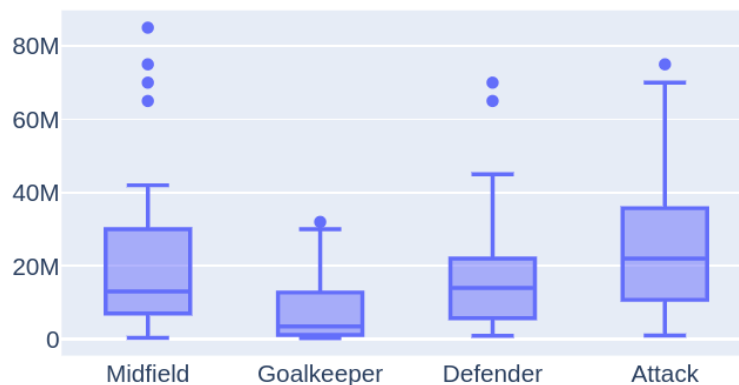


Figure 6.1 – A box plot of valuations segmented by position

This box plot lets us see that the highest-paid positions are not goalkeepers as I might have suspected coming into this project without much football knowledge, but rather elite attackers, as well as a few outstanding midfielders and defenders.

Note

It's important to note that when you have a numerical and categorical set of data points as we do here with market value and positions, the categorical data points need to go on the *x* axis. If you try to swap the two, Plotly.NET will render a nonsensical graph.

We can specify both *X* and *Y* for a violin plot as well:

```
Chart.Violin(Y = marketValues, X = positions)
|> Chart.withTitle "Player Market Value by Position"
|> Chart.withSize(500, 350)
```

This produces *Figure 6.2*, which shows the different distributions among players for the pay amounts:

Player Market Value Distribution by Position

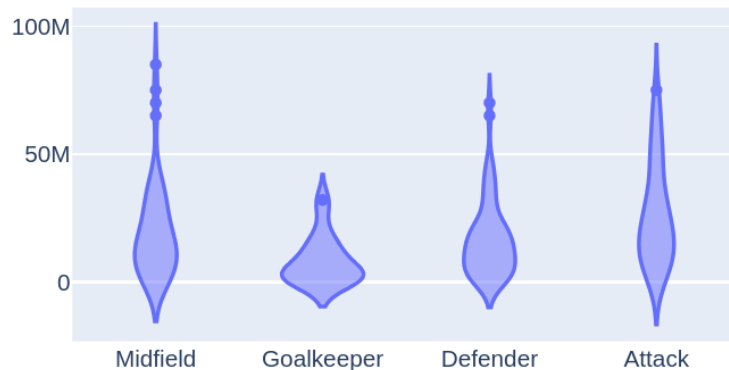


Figure 6.2 – A violin plot of market value by position

This particular violin plot makes it clear that so many of those players at the extreme low end of the market value range appear to be goalkeepers.

My assumption would be that these are backup goalkeepers who don't see much playtime.

Let's validate this assumption by charting how much playtime each player received based on their position:

```
Chart.BoxPlot(Y = minutesPerGame, X = positions)
|> Chart.withTitle "Playtime by Position"
|> Chart.withSize(500, 350)
```

The result clearly rejects my theory that these are backup goalkeepers:

Playtime by Position

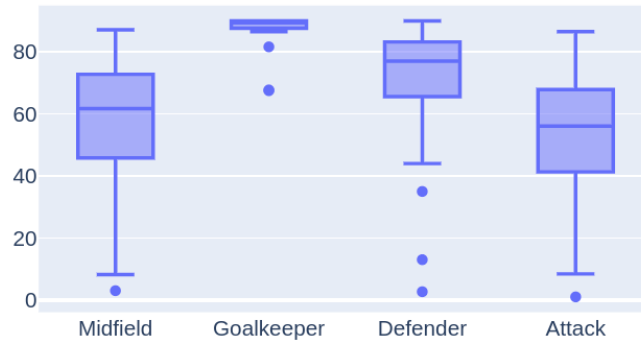


Figure 6.3 – Box plots for each position's playing time

It appears that most goalkeepers in this dataset play the majority of their games on average.

It simply seems to be the case that the goalkeeper position is not nearly as prestigious as your elite midfielder or attacking positions or does not require the same degree of skill level and therefore the pay is significantly lower.

Biases and assumptions

It's important to remember that you carry your own assumptions, biases, and knowledge into any data analysis tasks. In my case, I've been a lifelong fan of ice hockey, where goalies are extremely skilled and sought-after individuals who can lead a team to success in the postseason. When I analyzed a dataset for football, which I know much less about, I incorrectly applied assumptions that information I knew in one sport applied to another sport. It sounds silly in retrospect, but as a data analyst, you have to watch the assumptions you make because they can cause you to miss interesting facts and relationships.

To more thoroughly explore the relationship between different variables, let's move on to a chart intended for that purpose: the scatter plot.

Plotting multiple values with scatter plots

Earlier in this chapter, we saw a scatter plot that compared a player's average goals per game with their playtime. It did this by scattering points on the x and y axes based on their values for those two variables

That chart was built using Plotly.NET. Let's see how it was done by building a similar chart to show the relationship between market valuation and minutes per game.

We'll do this in an F# code cell as always:

```
Chart.Point(x = minutesPerGame,
            y = marketValues,
            MultiText = playerNames)
|> Chart.withTitle "Market Valuation vs Playtime"
|> Chart.withSize(600, 400)
|> Chart.withXAxisStyle(TitleText = "Minutes per Game")
|> Chart.withYAxisStyle(TitleText = "Market Valuation")
```

This uses `Chart.Point` to create and display a `Point` object. In this case, we're using `minutesPerGame` as our series of *x* values and `marketValues` as our *y* values (note that these properties are lowercase here instead of capitalized as they were with other charts).

We then customize the title and size of the chart, as we've done before, and we also include player names for tooltips on individual data points.

The final two lines add text to the *x* and *y* axes, which help the reader understand the data they're looking at.

The end result of this code is *Figure 6.4*:

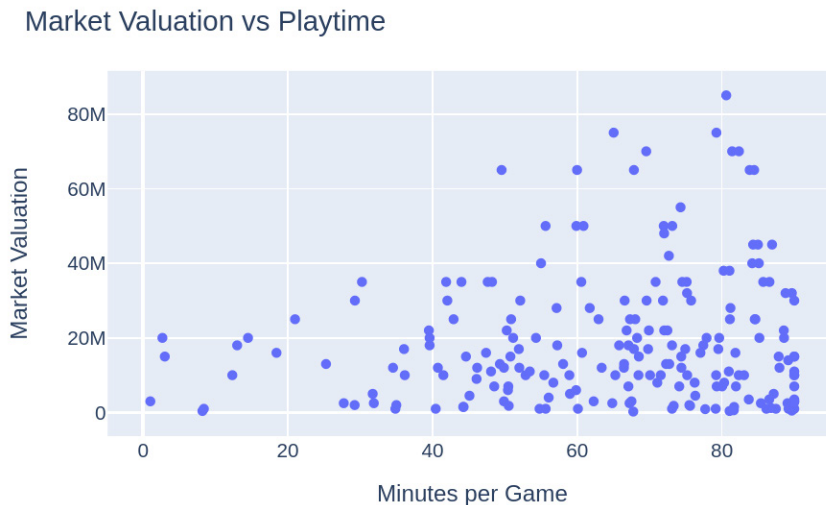


Figure 6.4 – Minutes per game versus market valuation

Here, each dot is a player, and you can see that players who play less than 40 minutes each game on average are never your higher-paid players.

What's interesting is that the highest-paid players tend to play between 60 and 85 minutes per game, but appear to take at least one 5-minute break each game. Players who play more than this tend to make significantly less money.

As someone without a lot of football knowledge, I might guess that either your elite players work so hard that they need a brief break to recharge and be effective in the closing bits of the game, or that when teams are ahead by several goals, they might intentionally rest their elite players to allow others to get more play time and practice.

Let's apply scatter plots to a different problem: determining the relationship between goals and assists.

We'll do this in a similar way to our prior chart, but use goals and assists as our variables, as shown in this F# code cell:

```
Chart.Point(x = goals,
            y = assists,
            MultiText = playerName)
|> Chart.withTitle "Goals vs Assists Per Game"
|> Chart.withSize(600, 400)
|> Chart.withXAxisStyle(TitleText = "Goals")
|> Chart.withYAxisStyle(TitleText = "Assists")
```

This produces an interesting scatter plot in *Figure 6.5*:

Goals vs Assists Per Game

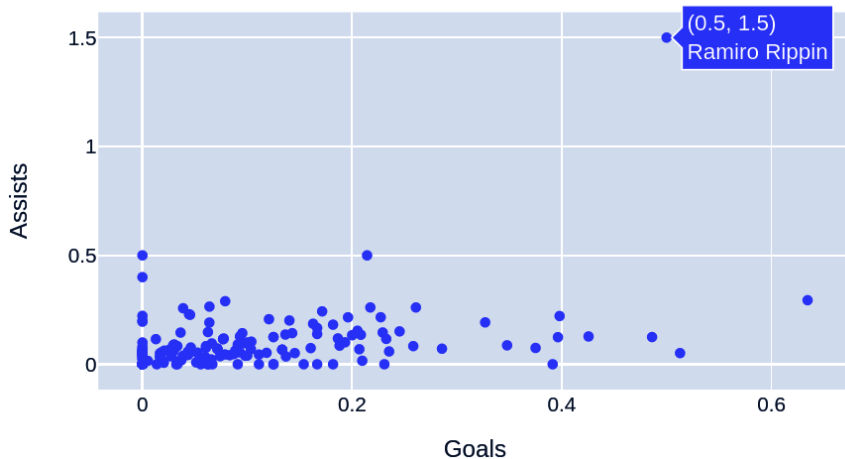


Figure 6.5 – A scatter plot of goals versus assists per game

The scatter plot shows that players tend to both score goals and gather assists and that players who score a lot of goals tend to score more than they assist – with the notable exclusion of Ramiro Rippin (highlighted in *Figure 6.5*), who has 1.5 assists per game and scores a goal every other game.

Athlete names

The data in this book uses randomized athletes with fake names, which is probably why you've never heard of Ramiro Rippin!

This chart also features different ranges on its x and y axes, which make it harder to compare values equally.

We can fix this by using a square size and specifying a MinMax range in our axis styling lines, as shown here:

```
Chart.Point(x = goals,
            y = assists,
            MultiText = playerNames)
|> Chart.withTitle("Goals vs Assists Per Game")
|> Chart.withSize(400, 400)
|> Chart.withXAxisStyle(TitleText = "Goals", MinMax=(0,2))
|> Chart.withYAxisStyle(TitleText = "Assists", MinMax=(0,2))
```

This results in a much easier-to-interpret graph in *Figure 6.6* that uses equal ranges on the x and y axes:

Goals vs Assists Per Game

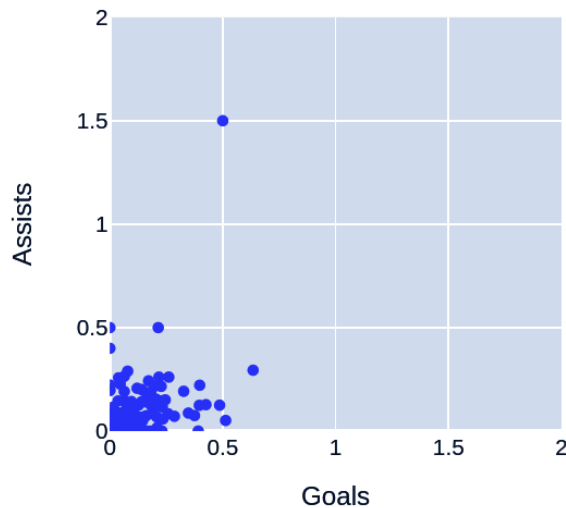


Figure 6.6 – The same scatter plot but with different ranges

Having the same scale when comparing similar things makes it easier to avoid mistakes due to unequal axis ranges or intervals.

Let's move on to cover adding additional information to a scatter plot.

Adding color to a scatter plot

Often, I'll find myself analyzing two pieces of information in a plot and wondering how they relate to a third piece of information.

There are several different ways of adding additional dimensions to a chart, but my favorite by far is to introduce color to the plot.

By adding color to a plot, you keep your x and y positions meaning what they previously did, but you add colors to the individual markers on the plot, which allows you to highlight an additional variable.

With Plotly.NET, we do this by specifying `MarkerColor` and then specifying that the color values should come from a color scale based on another series of data.

Here's a sample F# code cell comparing goals and assists but coloring based on the minutes played per game:

```
Chart.Point(x = goals,
            y = assists,
            MultiText = playerNames,
            MarkerColor = (Color.fromColorScaleValues minutesPerGame))
|> Chart.withTitle "Goals vs Assists Colorized by Play Time"
|> Chart.withSize(400, 400)
|> Chart.withXAxisStyle(TitleText = "Goals", MinMax=(-0.1,1.6))
|> Chart.withYAxisStyle(TitleText = "Assists", MinMax=(-0.1,1.6))
|> Chart.withMarkerStyle(ShowScale = true)
```

This produces this very colorful *Figure 6.7*:

Goals vs Assists Colorized by Play Time

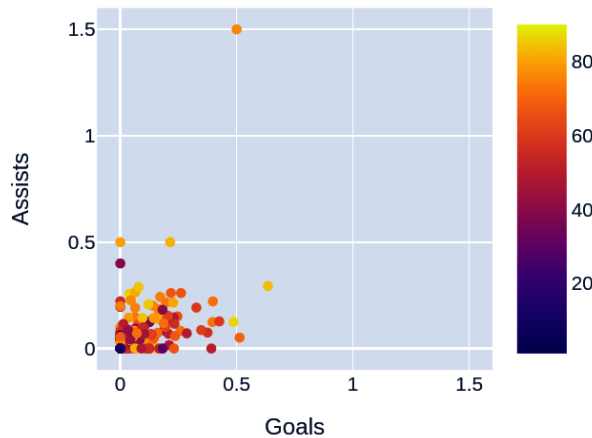


Figure 6.7 – Average goals versus assists colorized by average playtime

Here, we see the same marker points as before, but those points are now colored by the respective player's average minutes played per game.

Players who have fewer minutes appear in darker purples while those who played close to the full playtime appear in yellow and orange. By adding this third dimension, we're able to spot some unusual outliers, such as a player who plays 40 minutes on average.

Note that we did add a new `Chart.withMarkerStyle` line to this chart. This call adds the color bar to the right of the scatter plot, which helps viewers make sense of the color dimension.

Color can also be used to customize charts to match your organization's branding needs, and Plotly.NET has many additional styling options you can read about on their website.

For all the styling capabilities offered in Plotly.NET, I still love color for its ability to quickly add another dimension to data in a chart.

Let's close this section out with a discussion of adding additional dimensionality to your visuals.

3D scatter plots with Plotly.NET

Sometimes, representing that third dimension in a scatter plot in color is not enough and you'll want to see it spatially as well. In that case, a 3D scatter plot may meet your needs.

Switching from a scatter plot to a 3D scatter plot is relatively simple and requires a few simple changes:

1. Call `Point3D` instead of `Point`.
2. Provide a `z` value to the `Point3D`.

3. Add a `withZAxisStyle` call to annotate your new axis.
4. Replace the existing `withXAxisStyle` and `withYAxisStyle` to specify `Id`, indicating a scene. This is needed for these labels to work in 3D space.

The final piece of code for adding marketValues to our goals and assist information is as follows:

```
Chart.Point3D(x = goals,
              y = assists,
              z = marketValues,
              MultiText = playerNames,
              MarkerColor = (Color.fromColorScaleValues marketValues))
|> Chart.withTitle "Goals vs Assists vs Valuation"
|> Chart.withXAxisStyle(
  TitleText = "Goals", \
  Id = StyleParam.SubPlotId.Scene 1)
|> Chart.withYAxisStyle(
  TitleText = "Assists", \
  Id = StyleParam.SubPlotId.Scene 1)
|> Chart.withZAxisStyle(TitleText = "MarketValue")
|> Chart.withSize (800, 600)
```

This produces an interactive, colored 3D scatter plot, as represented by *Figure 6.8*:

Goals vs Assists vs Valuation

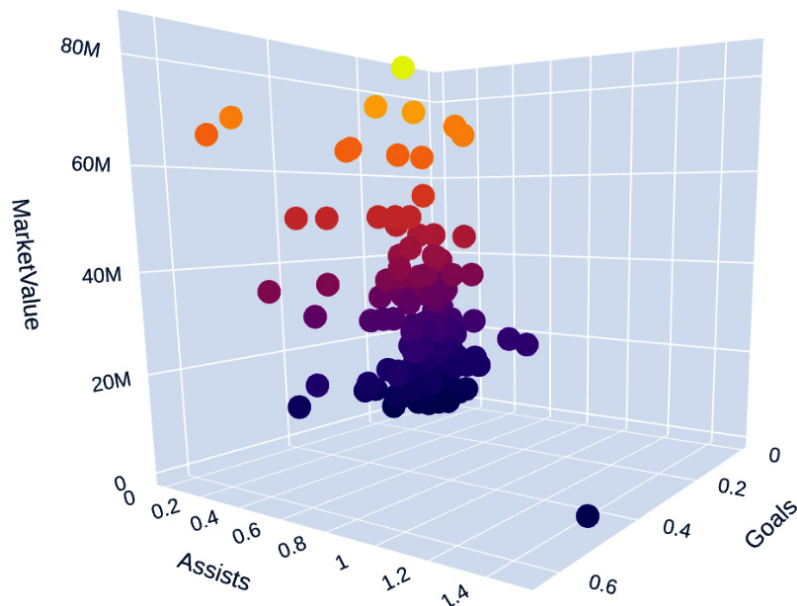


Figure 6.8 – A 3D scatter plot with colors representing market value

All Plotly.NET charts are interactive to some degree, but you can hold your left and right mouse buttons down over 3D scatter plots to rotate and pan the chart to see data from different perspectives.

Note here that this chart includes goals, assists, and market values on its x , y , and z axes, along with colors being determined by market value.

If we wanted to, we could instead change marker colors to be colored by a fourth piece of information, such as the player's average playtime per game, as shown in *Figure 6.9*:

Goals vs Assists vs Valuation

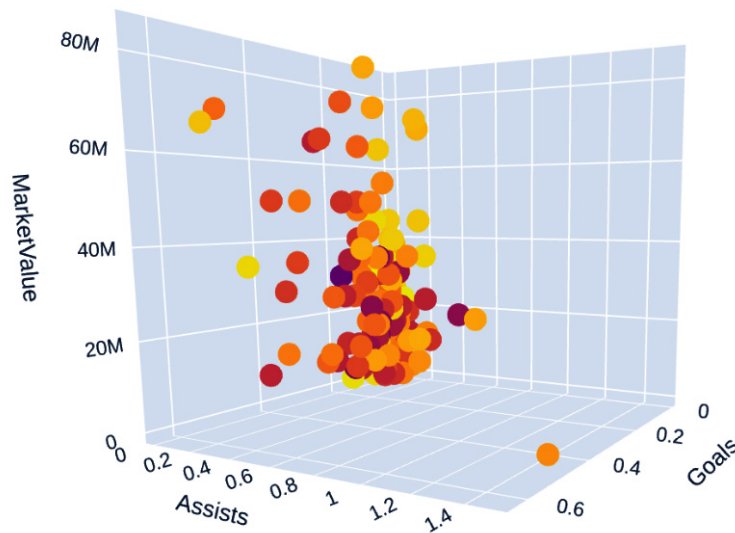


Figure 6.9 – A 3D scatter plot with the color showing a fourth dimension

This effectively creates a 4D chart, which is harder to understand, but can help spot interesting correlations.

Most of the time, I try to keep my charts to 2D charts with the optional use of color as an additional dimension, but I will experiment with 3D charts to see if I can spot any interesting trends in data.

In the next section, as we close the chapter, we'll explore a 2D way of visualizing the different relationships between variables by discussing variable correlations.

Identifying variable correlations

When performing data analysis or considering which variables you might want to use in a machine learning model, you will often want to look at variable **correlation**.

Correlation describes how frequently changing one variable has an impact on a specific second variable – and whether that impact is positive or negative.

Let's consider an example. If we were to look at all of our players' birth years and all of their ages, we'd notice a correlation: the earlier the birth year is, the higher the player's age will be.

While this particular relationship shouldn't shock anyone, it is a good example of a pair of strongly correlated variables. We can say that age and birth year share a *strong negative correlation*; meaning the more one goes up, the more the other one goes down.

Calculating variable correlations

Correlations can be represented numerically with values ranging from -1 to $+1$, with correlation values near 0 indicating no correlation between two data points.

One of the most popular ways of calculating this correlation is by using the **Pearson correlation method**. The `MathNet.Numerics.Statistics` package has an implementation of this, using `Correlation.Pearson`.

Let's try this out in a code cell using our goals and assists values:

```
Correlation.Pearson(goals |> Seq.map float,  
                    assists |> Seq.map float)
```

Here, we take our two arrays of F# singles and map them into arrays of F# floats (C# doubles) because Pearson requires it.

This method then calculates and returns the correlation between player goals and player assists:

```
0.4374489344525743
```

Here, the correlation is roughly 0.44 , which is a moderate positive correlation. In other words, when a player scores more goals, generally speaking, they'll also score more assists.

Let's do another calculation of values, this time looking at assists and red cards received:

```
Correlation.Pearson(assists |> Seq.map float,  
                    redCards |> Seq.map float)
```

This time, the result comes out slightly negative:

```
-0.04253652627440338
```

Because the value is negative, you could infer that players who make more assists tend to commit fewer red card infractions, but in this case, the value is so close to 0 that I'd probably simply say that assists and red cards are not correlated or perhaps weakly negatively correlated.

You won't look at correlation scores every day, but they can be helpful in identifying relationships in your data.

Guess what? Correlation scores can be visualized, so we'll close our chapter out with a demonstration of a **correlation matrix**.

Building feature correlation matrixes

If we can manually calculate the correlation between two specific numeric values, then we have enough to be able to use loops to calculate the correlations of all combinations of numeric variables.

What you'd need to do in order to do this would be to get a list of numeric columns, and then loop over that list of columns. Inside that loop, you'd need to loop over all the other columns and then calculate and store the correlation between those two variables.

When you've finished this nested loop, you'll have a matrix of values from -1 to $+1$, indicating how correlated each variable would be. You could then graphically map this matrix using a heatmap to produce a visual resembling *Figure 6.10*:

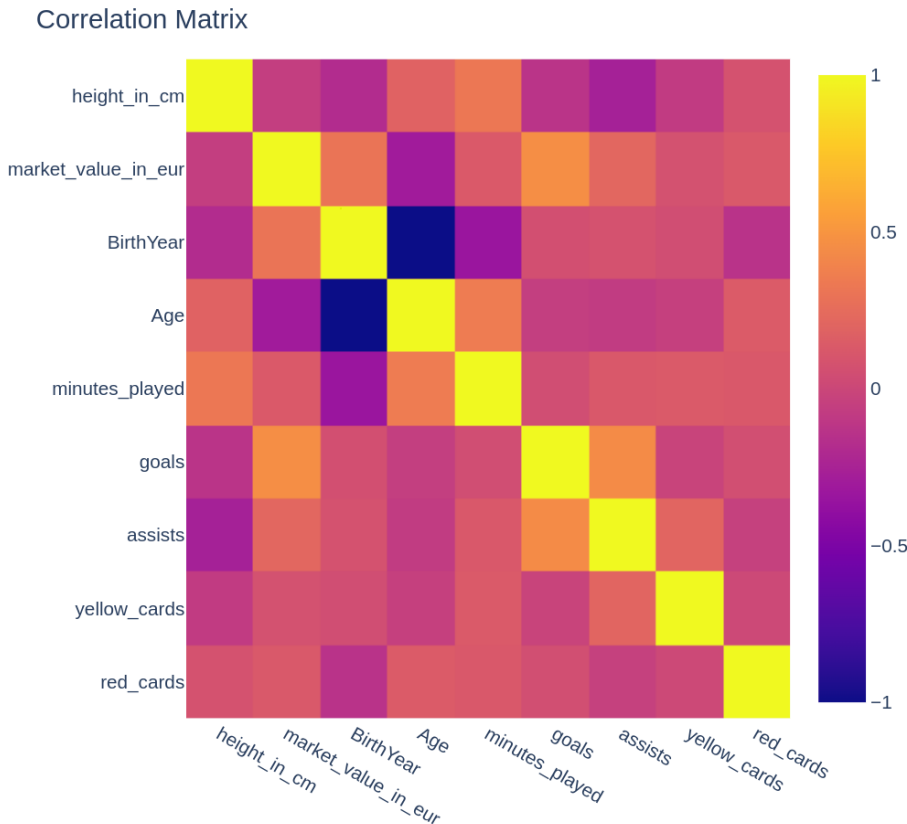


Figure 6.10 – A correlation matrix for our numeric columns

That's a lot of information, and we'll talk about what it means, but let's start by seeing how we can get there.

First, we'll start by identifying the columns in our dataset containing numbers:

```
let columnsToAnalyze =  
    df.Columns  
    |> Seq.filter (fun c -> c.GetType() = typeof<SingleDataFrameColumn>)  
    |> Seq.map (fun col -> col.Name)  
    |> Array.ofSeq  
let numColumns = Array.length columnsToAnalyze
```

This gives us a list of numeric columns and their lengths.

What about boolean columns?

You can represent boolean columns as numbers by using 1 for `true` and either 0 or -1 for `false`. In this code sample, I'm keeping things simple and focusing just on our numeric columns but look at the book's repository for an example that handles boolean columns as well.

Next, we'll loop over these columns in a pair of nested `for` loops to build a jagged array of F# floats since that's the data structure `Plotly.NET` likes for its heatmaps:

```
let results : float[] [] = Array.zeroCreate numColumns  
for y = 0 to numColumns - 1 do  
    let colY = df[columnsToAnalyze[y]]  
        |> Seq.cast<single>  
        |> Seq.map float  
    let rowResults : float[] = Array.create numColumns nan  
    results.[y] <- rowResults  
  
    for x = 0 to numColumns - 1 do  
        let colX = df[columnsToAnalyze[x]]  
            |> Seq.cast<single>  
            |> Seq.map float  
        rowResults.[x] <- Correlation.Pearson(colX, colY)
```

This is a lot of code and it's difficult code if you're not used to F#, but we're effectively creating an array of arrays with each array containing correlations for one variable in the list of columns.

Inside the loops, we get the values from the two columns of interest, convert them to sequences of F# floats, get a Pearson correlation score for those two variables, and then store that in the `results` array.

Once we have the `results` array, we can visualize it using the heatmap chart as described here:

```
Chart.Heatmap(zData=results,  
              ReverseYAxis=true,  
              X=columnsToAnalyze,  
              Y=columnsToAnalyze)  
|> Chart.withTitle "Correlation Matrix"
```

This generates a heatmap of the different correlations, with yellow values being strong positive correlations and purple values being strong negative correlations.

You may have noticed that this matrix has a strong correlation along its top-left to bottom-right diagonal or that the upper-right portion of the diagram is the same as the lower-left portion of the diagram.

These two anomalies are normal in correlation matrixes since features are always strongly correlated with themselves and if you swap the order of the two variables, you get the same result.

In fact, most of the time, correlation matrixes will not render the top-right portion of the correlation matrix to help keep the focus on the lower-left portion.

Let's do that now by making a small modification to our nested loop. We'll have the `x` values range from 0 to `y` instead of from 0 to `numColumns - 1`.

The corrected code is shown here:

```
for x = 0 to y do  
  let colX = df[columnsToAnalyze[x]]  
  |> Seq.cast<single>  
  |> Seq.map float  
  rowResults.[x] <- Correlation.Pearson(colX, colY)
```

By skipping over values beyond the `y` value, we leave those cells as `nan`, which causes the correlation matrix not to render them, as shown here:

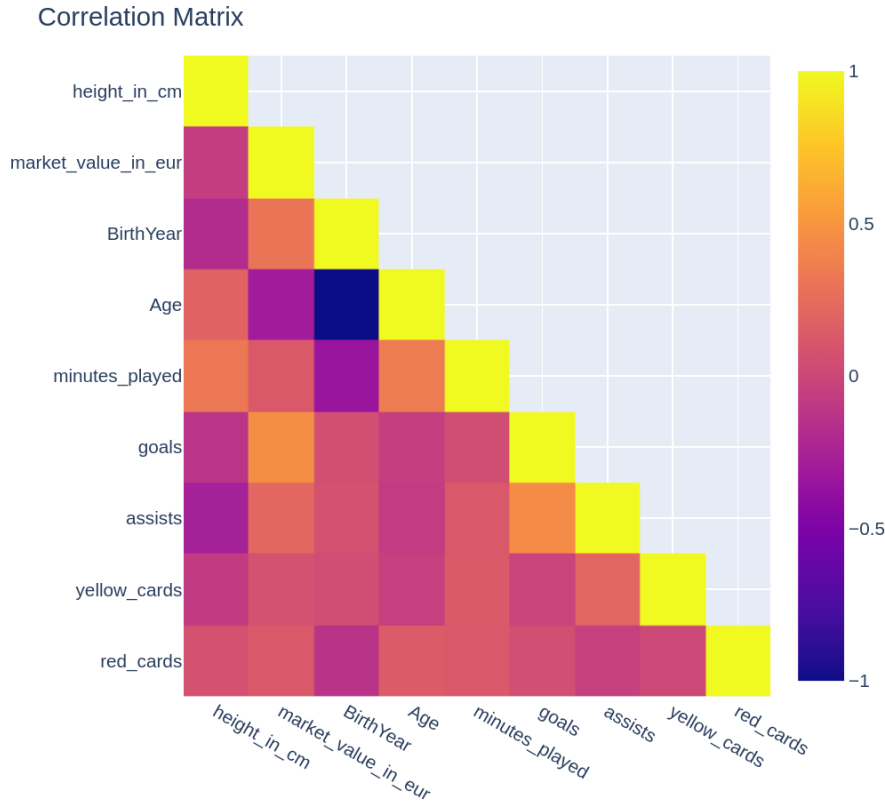


Figure 6.11 – A masked correlation matrix

With this view more focused, it's now easier to read and interpret its results, so let's try that now.

The first thing that jumps out to me on this graph is the large purple cell where Age meets BirthYear. This purple value represents -1 and indicates Age has a strong negative correlation with BirthYear. This makes sense and is something we discussed earlier in this section.

What's more interesting about age is that it *also* has a negative correlation with market value, meaning as players get older, they generally earn less.

The chart also highlights that players who score more goals are often valued more and players who have more assists tend to have more goals.

What's interesting to me is that there is a negative correlation between height and assists. In other words, the taller a player is, the less likely they are to get assists – or conversely, the shorter a player is, the more assists they make!

Not only can correlation matrixes help you identify unexpected associations in your data like this, but they can help you simplify your dataset as well.

Why remove columns?

In machine learning, we don't like to have two columns in a dataset that mean close to the same thing as it may bias the training process or slow our ability to find a good result. Columns that seem to have a very similar impact on other columns can be identified using correlation matrixes and help you make the decision to drop one of them from your dataset before conducting model training.

On that note, let's conclude the chapter and move on to our next adventure: machine learning.

Summary

In this chapter, we saw how multivariate analysis allows you to visualize how two or more variables interact with each other. We explored this through scatter plots using both simple x and y dimensions but later adding in color and an optional z dimension for 3D scatter plots.

We finished the chapter by talking about the Pearson correlation and showing how you can calculate a correlation score between two columns of numeric data. Finally, we saw how many such correlations can be visualized in a correlation matrix to help you spot unusual associations in your data.

This concludes the first part of this book, where we've explored the basics of Polyglot Notebooks and applied them to data wrangling and data analysis workloads.

In the next part of the book, we'll move on to new territories as we discuss machine learning and the process of training, evaluating, and deploying machine learning models.

Further reading

To learn more about the topics covered in this chapter, you can go through the following:

- *How to Read a Correlation Matrix*: <https://www.statology.org/how-to-read-a-correlation-matrix/>
- *Pearson correlation coefficient*: <https://libguides.library.kent.edu/spss/pearsoncorr>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/correlations/>

Part 2:

Machine Learning with Polyglot Notebooks and ML.NET

With the basics of Polyglot Notebook development and data analysis behind us, we now move on to explore the field of machine learning.

In this part, we'll discuss the overall flow of machine learning and the machine learning tasks of regression, binary classification, and multi-class classification using ML.NET, Microsoft's open-source machine learning framework.

We'll start with the basics and let ML.NET AutoML automatically select the optimal model trainer and hyperparameters for our machine learning tasks.

Once we get the basics down, we'll explore what AutoML is doing under the hood and study building our own machine learning pipelines and gaining additional control over hyperparameter tuning.

We'll end this part with an exploration of deploying ML.NET models to production and how to monitor and maintain them after they're out in the wild.

This part has the following chapters:

- *Chapter 7, Classification Experiments with ML.NET AutoML*
- *Chapter 8, Regression Experiments with ML.NET AutoML*
- *Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms*
- *Chapter 10, Deploying Machine Learning Models*

Classification Experiments with ML.NET AutoML

We've seen how useful data analysis can be for gaining insights from data and communicating those insights to others. In this chapter, we'll enter the realm of **machine learning** and show how **machine learning models** can be fitted to our datasets, as well as how these fitted models can generate predictions from new data points.

In this chapter, we'll cover the following topics:

- Understanding machine learning
- Introducing ML.NET and AutoML
- Creating training and testing datasets
- Training a classification model with ML.NET AutoML
- Evaluating binary classification models
- Generating predictions from binary classification models

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter07` folder, along with the chapter's dataset.

Understanding machine learning

Machine learning uses mathematics to help understand patterns in data and make decisions based on those patterns.

There are several different branches of machine learning, including the following:

- **Supervised learning**, which involves using historical data to predict new values
- **Unsupervised learning**, which is typically used to cluster data points together based on similarity or to spot anomalies
- **Reinforcement learning** and **semi-supervised learning**, which involve systems that iteratively experiment and adapt based on the results of their experiments.

Typically, when people mention machine learning, they are referring to supervised learning, as this is the most common application of machine learning and more specific terms are usually used for the other forms of machine learning.

We'll focus on supervised learning in this book and its two most common tasks.

Supervised learning

Supervised learning is the art of training a machine learning model to the trends and patterns present in a collection of historical data, in order to make predictions about previously unknown data points.

In financial industries, this could involve looking at a potential loan and predicting whether the loan will be repaid. In healthcare, it might be determining whether a sample is benign or cancerous. In politics, it might be predicting which candidate a voter will vote for – or whether they'll vote at all!

In all of these examples, we rely on patterns present in our dataset from past data to identify relationships that can predict future values with some degree of accuracy.

To train a model, you typically select a **machine learning algorithm** through a specific **model trainer** and configure the **hyperparameters** on that algorithm. You then send it a set of training data, and the algorithm spends its time memorizing the trends in that training data to “fit” a model to that data.

Hyperparameters versus parameters

The first time I encountered the term **hyperparameter**, my initial reaction was, “*Someone is just trying to be clever.*” However, there is a distinction between **parameters**, which are properties of the trained model, and hyperparameters, which are properties of the model training algorithm. In other words, we tweak hyperparameters to adjust how a model is trained, and the model training process outputs the parameters of our model.

The model training process is depicted visually in *Figure 7.1*:

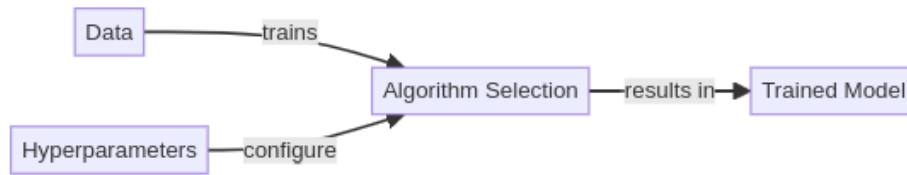


Figure 7.1 – The model training process

Training a model takes a relatively high amount of time. This can be anywhere from seconds to hours, or even days. However, once a model is trained, you can use it to get predictions on new data within moments.

The exact time taken to generate predictions depends on the size of your machine learning model and the performance of the specific algorithm used to train the model, but predictions are typically made within milliseconds.

Additionally, once a model is trained, it can be saved to disk or the cloud to be loaded later on. This means that models can be considered investments of upfront training time, serving up predictions later on in real-time.

This way, you can think of machine learning projects as similar to software development projects – we spend a lot of time creating software systems, only to deploy them where people can use our software in real time.

However, unlike software systems, updating models is not as easy. If you find that a model isn't as accurate as you thought it was or if you find its accuracy gets worse over time as industry trends change, you may need to replace a model with a new one trained on more recent data, or with different hyperparameters.

Before we get into the specifics of machine learning in .NET, let's talk about the two most popular types of machine learning – classification and regression.

Classification and regression

Classification is a form of supervised machine learning that tries to predict a categorical variable, such as a loan being repaid or defaulting, a mole being cancerous or benign, or the classic example from data science of studying whether someone would have survived the *Titanic*, based on their ticket information.

The Titanic as the “Hello World” of data science

Believe it or not, training a model around historical data from the *Titanic* is somewhat akin to a “Hello World” program in the world of data science. Many tutorials exist to guide you through training a model based on the *Titanic* dataset and classifying whether someone would have lived or died, based on their ticket information. If you’re curious and want to try out the techniques from this chapter on a classic experiment, you can find the dataset and fantastic tutorials on Kaggle at <https://www.kaggle.com/c/titanic>.

While classification focuses on predicting categorical values, **regression** is oriented toward predicting numerical values such as a stock price, temperature, the sale price for a used vehicle, or the number of goals a football player will get in a season.

Classification and regression are very similar but involve different metrics and classes, so we’ll focus on classification in this chapter and regression in the following chapter.

In this chapter, we’ll apply classification to predict whether a football player is going to retire at the end of a season, based on that player’s stats during the season, how those stats compared to their prior season stats, the age and position of the player, and historical trends in player retirement.

Such a trained model could be used to help a football team forecast which players it will need to replace next season. Having this information could help teams with salary negotiation, trades, and drafting.

Let’s talk about the technologies we’ll use to do this.

Introducing ML.NET and AutoML

There are many ways of training machine learning models, including **Python**, **R**, and cloud tools such as **Azure Machine Learning Studio** or **Amazon SageMaker**, but the best way of using .NET to train or deploy a machine learning model is with **ML.NET**.

ML.NET is an open-source library designed to train, save, and load machine learning models to accomplish a variety of tasks. We actually worked with ML.NET earlier in this book through the `DataFrame` class, which is maintained in the same GitHub repository as ML.NET (<https://github.com/dotnet/machinelearning>).

You can use ML.NET to train models, evaluate their performance, and use them to make predictions. Because ML.NET is a .NET library, it can be integrated directly into other .NET applications, including web applications, Azure functions, and cross-platform desktop and mobile applications.

In fact, because ML.NET supports **Open Neural Network Exchange (ONNX)** file formats, you can train a model in Python, save it as an ONNX model, and then deploy it to ML.NET (or vice versa). You can even find some advanced models, such as **large language models (LLMs)** (e.g., Llama2), that support ONNX (<https://github.com/microsoft/Llama-2-Onnx>).

ML.NET has a wide range of features, including a large number of supported algorithms and the ability to create model pipelines (which we'll explore in *Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms*).

Because ML.NET is so capable, it can be somewhat intimidating to get started. This is because you need to learn a number of things, including the following:

- How to work with ML.NET pipelines
- How and why to one-hot-encode categorical variables
- Scaling and transforming numerical data sources
- Different machine learning algorithms of relevance for the task you're trying to accomplish
- Hyperparameters for the relevant machine learning algorithms

When you're just starting out your machine learning journey, this can be a lot of information and a high cognitive workload.

I'm a firm believer that when a burgeoning data scientist performs their first few experiments, they shouldn't focus on the specific machine learning algorithm being used to train the model. Instead, their focus should be on the data coming into the model, how the model's results are evaluated, and how effective the resulting model is.

Terminology note

ML.NET makes a distinction between machine learning algorithms and specific model trainers. This is a slight distinction that we'll cover in more depth in *Chapter 8, Regression Experiments with ML.NET AutoML*.

Details of algorithm and hyperparameter selection are important, particularly for models that will be deployed to production, but it's important to keep the learning curve manageable. Just as most developers don't start programming by building a full web application from scratch, data scientists shouldn't be expected to manage the full cognitive load of their first few models.

Thankfully, ML.NET has an optional helper library called AutoML that can help with the cognitive complexity of learning machine learning.

Understanding AutoML

AutoML, short for **automated machine learning**, is a library that automates parts of the machine learning process.

Specifically, AutoML automates selecting machine learning algorithms and hyperparameters for those algorithms.

What AutoML does is look at the task you're trying to accomplish and the machine learning metric that's most important to you. AutoML will then train a series of machine learning models, using different model trainers and sets of hyperparameters, in an attempt to find the combination that performs the best.

I've seen AutoML described as “*a shotgun approach to data science*” or “*throwing science at the wall to see what sticks*.” While these metaphors are rather colorful, they are not completely inaccurate.

AutoML's key value is that it can help experienced data scientists find algorithm/hyperparameter combinations that they might not have considered. I also find that AutoML helps beginner data scientists focus on the core task that they're trying to accomplish, deferring learning about algorithms and hyperparameters until slightly later in their learning process.

AutoML and data pre-processing

Algorithm selection isn't the only thing that AutoML can do. AutoML can also pre-process data in various ways, including the following:

- Replacing missing values with average column values or simply with zero.
- Transforming categorical columns to a series of numerical columns through the process of one-hot encoding. We'll explore one-hot encoding later in this section.
- Scaling numerical values to a standardized range.

These steps are important before a model can be trained.

Missing values can cause errors in some model training processes and must either be removed or replaced with average values, zeros, or some other numerical value. This process is sometimes called **imputing missing values**. We discussed the process of doing this manually in *Chapter 4, Working with Tabular Data and DataFrames*.

Categorical columns such as a player's position in a football club are important pieces of information, but strings are hard for models to understand.

Instead, with **one-hot encoding**, we take a categorical column and replace it with a series of numerical columns, with either a 1 or a 0.

For example, our `Position` column in our football club dataset has values of *Goalkeeper*, *Defense*, *Midfield*, and *Attack*. We can one-hot-encode this by dropping the `Position` column and replacing it with the `Position.Goalkeeper`, `Position.Defense`, `Position.Midfield`, and `Position.Attack` columns.

Each of these columns will have either a 0 or a 1, based on the player's original position. This is where the term “one-hot” comes from, where a 1 in the column indicates that a value is `true` while 0 indicates its absence. When there are many different possibilities, there will be far more 0s than 1s for any given row.

By encoding a categorical column into multiple numerical columns, we make it easier for the model training process to understand the nature of our data, which makes it more likely that the information will be helpful to it in making accurate predictions.

Finally, the process of **scaling** columns is about making each column of data follow a similar scale. Many machine learning algorithms are sensitive to numerical values.

Without scaling, a large value in one column might be interpreted as much more important than a smaller value in another column. This may sound reasonable but consider the data. If a player was 32 and had 25 red cards in a season, the model might gravitate toward the player's age as the most important factor, instead of noticing the excessive amount of red cards that player has accumulated (if your football club has a player whose number of red cards approaches their age, you have a problem).

We solve this problem by scaling each numerical column to have a value between either -1 and 1 or 0 and 1, depending on what makes sense for the column. Under this scaling, your largest values for a column would be near 1.

This means that a value of 1 in the Age column would mean a very old player, while a 0 would indicate one of the youngest players in the league.

Terminology note

The process of scaling values is sometimes referred to as **normalizing** values. ML.NET uses the word *normalize* in its class names as a result.

While steps such as scaling, one-hot encoding, and replacing missing values are important and ones that I'd encourage you to tackle yourself when decisions truly matter, ML.NET AutoML is also capable of making its own decisions on these matters by pre-processing your data for you.

We'll revisit data pre-processing in *Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms*, but for now, let's see what an ML.NET AutoML classification experiment looks like.

Creating training and testing datasets

We'll start our classification experiment with the familiar – adding our dependencies and loading a CSV file into a `DataFrame`:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
using Microsoft.Data.Analysis;
DataFrame df = DataFrame.LoadCsv("PlayerRetirement.csv");
```

This chapter's dataset contains rows indicating a player's stats for an entire season and an `IsLastSeason` column, indicating whether that player did not return for the following season.

A sampling of a few rows and columns is shown in *Figure 7.2*:

YellowCards	RedCards	Goals	Assists	Games	PlayerId	Season	Position	Age	IsLastSeason
4	0	19	2	32	626724	2023	Attack	23	True
0	0	0	0	3	407021	2016	Defender	24	False
12	0	6	2	33	333241	2022	Midfield	27	False
0	0	0	0	1	71271	2022	Goalkeeper	37	True
2	0	0	4	13	504148	2023	Defender	24	True

Figure 7.2 – A sample of some of the rows and columns in the dataset

The idea of this dataset is that players will play for multiple seasons, and their stats will change from season to season until they ultimately decide it is time to move on or their team does not renew their contract.

Each row also contains the age of the player that season and the way that their numerical stats have changed from the previous season. These two factors are intended to help the model we train understand relative differences from season to season.

The dataset also includes a `PlayerId` column as well as a `Season` column. Columns with row identifiers should always be removed before training a machine learning model, as you do not want a machine learning model to focus on an arbitrarily assigned ID as an important factor. Additionally, we want our model to work regardless of the season, so we should remove the `season` identifier from the dataset.

Removing columns before training

When training a machine learning model, it is important to consider the data that you give the model training process. If a column is something arbitrary or meaningless, you should remove it before training a model. Additionally, if a column indicates gender, ethnicity, race, sexual orientation, religion, or anything else that may result in a model that learns to discriminate against groups of people, those columns should be removed as well.

The code for removing these columns should look familiar:

```
df.Columns.Remove("PlayerId");
df.Columns.Remove("Season");
```

With the basic data loading and column removal accomplished, let's start setting up for machine learning by importing ML.NET:

```
#r "nuget:Microsoft.ML,3.0.1"
using Microsoft.ML;
using Microsoft.ML.Data;
```

Here, we install the `Microsoft.ML` NuGet package and then import two of its namespaces, using commands. The `Microsoft.ML` and `Microsoft.ML.Data` namespaces give us everything we need to start preparing our experiment.

Everything in ML.NET flows through a `MLContext` instance to orchestrate your experiments. You'll typically create one `MLContext` and then rely on operations on that instance.

ML.NET also relies heavily on data. Datasets can come in a variety of sources and can be imported via methods on `MLContext`'s `Data` property, but it turns out that the `DataFrame` object we've been working with this whole time implements `IDataView`, so we'll cast it to that interface with the following setup code:

```
MLContext context = new();
IDataView data = df;
```

Now that we have our `MLContext` and our `IDataView`, the next thing we need to do is split our data into a set of data to train our model and a smaller set of data to test the accuracy of our model.

Most beginner data scientists assume that we would simply throw all of our data at the model during training. The problem with that approach is that we would have a trained model but no idea of how accurate it is!

By reserving a certain percentage of rows (typically, anywhere from 10% to 30% of our data) where we have both the columns associated with that row and the label we're trying to predict, we can ask our model to predict values for these rows it never saw during training. We can then take the prediction results and compare them against what actually happened. This way, we can build metrics around our model's strengths and weaknesses.

We can revise the simplified diagram of model training shown in *Figure 7.1* with *Figure 7.3*, which makes the train/test split explicit:

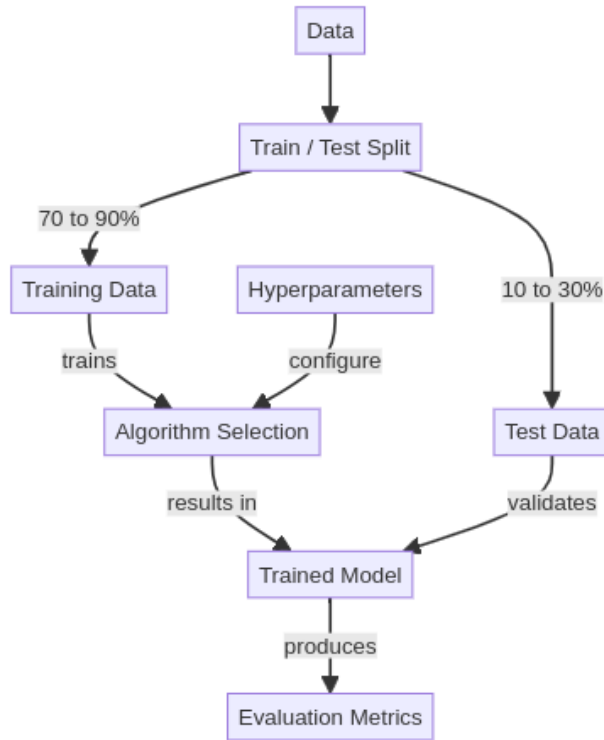


Figure 7.3 – Model training with the Test/Train split present

Let's take a look at a C# code cell that splits our data into a training set and a test set:

```

var dataSplit = context.Data.TrainTestSplit(data,
    testFraction: 0.3,
    seed: 123);

IDataView trainData = dataSplit.TrainSet;
IDataView testData = dataSplit.TestSet;

```

This code calls to the `TestTrainSplit` method on the `Data` property of our `MLContext`.

The code supplies the `DataFrame` (an `IDataView` implementation), and it instructs the split process to retain 30% of the data for testing and use 70% of it for training.

This code uses a random number seed of 123 to determine which rows make it into the test dataset and which ones make it into the training dataset. Using a random seed can help ensure that your experiments don't deviate significantly on repeat attempts, and it will be helpful when sharing code with many of you who want to follow along with the code in this chapter.

The `TestTrainSplit` method returns a `DataOperationsCatalog.TrainTestData` instance that has `TestData` and `TrainData` properties – both of which are `IDataView` instances and represent a portion of our data.

Splitting the data into a dataset for testing and a dataset for training is highly recommended, but you don't need to use `TrainTestSplit` to do so. If you wanted to, you could have separate CSV files for your training data and your test data. We'll also look at some advanced partitioning features in `TrainTestSplit` in the next chapter that give you additional control and why you might want this.

Now that we've separated our training and test datasets, let's set up and run our AutoML experiment.

Training a classification model with ML.NET AutoML

AutoML is maintained in a separate NuGet package from the main ML.NET package, so we'll need to install that with the `#r` magic command, as shown in this C# code cell:

```
#r "nuget:Microsoft.ML.AutoML,0.21.1"
using Microsoft.ML.AutoML;
```

With ML.NET AutoML installed and imported, we can set up our classification experiment. In this chapter, we'll be classifying football players as likely to retire next season or not.

Because there are only two categories something could be in, this is what's formally called a **binary classification** experiment, since each prediction is either going to be in one category or the other.

If there are three or more categories something might be categorized in, this is called a **multi-class classification** experiment. An example from our football dataset might be looking at a prospective player and predicting which position in your football club they'd perform best at – attack, midfield, defense, or goalkeeper.

Binary classification and multi-class classification have different code and metrics associated with them. In this chapter, we'll perform a binary classification experiment, but we'll revisit multi-class classification in *Chapter 10, Deploying Machine Learning Models*.

To set up our experiment, we need to configure a few settings for it. When performing an experiment with AutoML, you need to supply the following information:

- The data to experiment on
- The metric you care about measuring when comparing models
- A limit on how many models to train

The data will be specified later, but you can specify the experiment's optimizing metric and maximum time through code, like the following:

```
uint maxSeconds = 10;
BinaryExperimentSettings settings = new() {
    MaxExperimentTimeInSeconds = maxSeconds,
    OptimizingMetric = BinaryClassificationMetric.F1Score,
};
```

These settings tell AutoML that the most important metric we care about is the resulting model's F1 Score, and it should not spend more than 10 seconds training different models.

For now, let's not worry about what the F1 Score is, as we'll describe it in more detail when we discuss evaluating our model later on in this chapter.

Before we move on, I want to point out that instead of specifying `MaxExperimentTimeInSeconds`, you can specify `MaxModels` instead to restrict the training process, stopping new models from being added after a certain count is reached.

The exact parameters you specify will depend on the power of your computing platform, the size of your dataset, the difficulty of your problem, and how important finding the best model is to you.

Now that we have our settings, let's create our experiment by using the `Auto` method on our `MLContext` and then execute that experiment:

```
string label = "IsLastSeason";
BinaryClassificationExperiment experiment = context
    .Auto()
    .CreateBinaryClassificationExperiment(settings);

ExperimentResult<BinaryClassificationMetrics> result =
    experiment.Execute(trainData,
        testData,
        labelColumnName: label);
```

Here, we create `BinaryClassificationExperiment` using the `CreateBinaryClassificationExperiment` method on our `MLContext`'s `AutoML` object.

We then execute this experiment by providing it with the training data, the test data to grade trained models on, and the name of our **label column**. In machine learning terminology, we call the value that we're trying to predict the **label** and the characteristics that influence which label will be predicted the **features**. In other words, every column in our dataset consists of one label that represents the value we're trying to predict, and every other column is a feature that has some bearing on whether or not that is the player in question's last season. Since we're trying to predict whether players are about to retire, `IsLastSeason` is our label and the remaining columns are our features.

Now that we've covered terminology, let's get back to that call to `experiment.Execute`. This tells AutoML to spend its time training as many models as it can and then identify the best model it found, based on the model's F1 Score.

Since we stored the results of our experiment in the `result` variable, we can run the following code cell to see how many different models it evaluated:

```
result.RunDetails.Count()
```

For me, this results in a value of 68, but your results will vary, based on your operating environment and any random number generation carried out by AutoML.

Thankfully, we don't necessarily have to go through all 68 of these models. The `ExperimentResult` object has a `BestRun` property that contains the best model it found, according to our preferred metric. The following code gets access to the `BestRun` property for the result and displays its basic data:

```
RunDetail<BinaryClassificationMetrics> best=result.BestRun;
best
```

For me, this displays the model details shown in *Figure 7.4*:

```
▼ Microsoft.ML.AutoML.RunDetail`1[Microsoft.ML.Data.BinaryClassificationMetrics]
  ValidationMetrics    ► Microsoft.ML.Data.BinaryClassificationMetrics
  Model                ► [ Microsoft.ML.Transforms.MissingValueReplacingTransformer, Microsoft.ML.Transforms.
  Exception            <null>
  TrainerName          ReplaceMissingValues=>OneHotEncoding=>Concatenate=>FastTreeBinary
  RuntimeInSeconds    0.077
  Estimator            ► Microsoft.ML.Data.EstimatorChain`1[Microsoft.ML.ITransformer]
```

Figure 7.4 – A binary classification model result

We'll explore the `ValidationMetrics` portion in the next section, but we can see from these results that the model was trained without an exception arising, and it took 0.077 seconds to train this model on my laptop.

Additionally, we can see from the `TrainerName` property that this model involved replacing missing values, then one-hot-encoding a column, followed by concatenating data together, and finally, performing a `FastTreeBinary` model fitting.

While this doesn't tell me everything that was done for this model, it does help me understand that missing values were replaced and a categorical column was one-hot-encoded to have columns for discrete values.

By examining our data, we can see that the missing values that needed replacing were almost certainly the columns referring to differences between a player's last two seasons. When players enter the league, initially there are no prior values for them to compare against, so those columns result in null values.

The one-hot encoding process relates to our only categorical column of `Position` and involves creating `Position.Attack`, `Position.Defense`, `Position.Goalkeeper`, and `Position.Midfield` binary columns to help our model training process.

The concatenate step is typically done to join all features together to support model training and is not particularly relevant; however, the final step with the `FastTreeBinary` step is very relevant, as this is the machine learning algorithm applied to our data.

Tip

You can also find the specific algorithm used via the `LastTransformer` property on the `RunDetail` object.

In this chapter, we're focusing on the overall training process and not on a specific algorithm implementation. If you're curious about the specifics of `FastTreeBinary`, you can read more from Microsoft's documentation at <https://packt.link/zVxRf>, or check out the *Further reading* section for links to learn more about AutoML and the wide variety of models it supports. We'll also compare and contrast common regression and classification model training algorithms in *Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms*.

Now that we've run our experiment and identified the best model, let's see how we can interpret its performance and what factors are important to it.

Evaluating binary classification models

Model metrics tell you how accurate your models are and some of the areas they struggle in. In classification problems, many of these metrics can be embodied into a single graphic – the **confusion matrix**.

Confusion matrixes take the form of a table that tells you how your model classifies test data and what data points its getting confused by and incorrectly classifying.

In machine learning, we refer to our predictions as either *true* or *false*, based on their correctness, and *positive* or *negative*, based on our prediction. This is shown in the following table:

	Predicted true	Predicted false
Actually true	True positive	False negative
Actually false	False positive	True negative

Table 7.1

ML.NET can create a confusion matrix for us via its `GetFormattedConfusionTable` method:

```
BinaryClassificationMetrics runMetrics = best.ValidationMetrics;
runMetrics.ConfusionMatrix.GetFormattedConfusionTable()
```

This produces a formatted output, as shown in *Figure 7.5*:

```

TEST POSITIVE RATIO:    0.1768 (64.0/(64.0+298.0))
Confusion table
PREDICTED ||=====
TRUTH     || positive | negative | Recall
positive  ||         20 |         44 | 0.3125
negative  ||         32 |        266 | 0.8926
Precision ||    0.3846 |    0.8581 |

```

Figure 7.5 – A confusion matrix with predictions and actual values

The horizontal axis of the confusion matrix describes what the model is predicting. Because this is a binary classification problem, we're either predicting a true or false value for our `IsLastSeason` column.

The `positive` column indicates cases where our model predicted it was the player's last season, and the `negative` column indicates where it predicted the player would have at least one more season.

Conversely, the vertical axis shows us what those values actually were. Here, the `positive` row indicates that a player did retire that season, while the `negative` row tells us that the player didn't retire.

In this example, our models predicted 20 players would retire, and those players did. This correct prediction is called a **true positive**, since we correctly predicted a positive result. However, our model also predicted 32 players would retire when they, in fact, did not retire that season. These 32 cases are all **false positives**. Our model predicted they'd do something and they didn't.

On the other side of the grid, our model predicted that players wouldn't retire 44 times when they actually did. This is a **false negative** because our model predicted that they wouldn't retire (a negative result), but it got it wrong because the players actually did.

Finally, our model correctly predicted 266 **true negatives**, where it predicted players would not retire and those players remained in the league.

Tip

When thinking about true positives, true negatives, false positives, and false negatives, the second word (positive or negative) tells you what the model predicted and the first word (true or false) tells you whether your model got it right or wrong

Note that the model in the preceding figure has a test positive ratio displayed above it. This is calculated by taking the number of positive cases and dividing it by the total number of rows in the test dataset. This metric helps you identify whether there is a **class imbalance**, where you have significantly more positive values than negatives or vice versa.

In our case, we do have a modest difference between the two classes, where players only retire 17.68 % of the time in the test dataset, as shown in the test positive ratio in *Figure 7.5*.

Because of this class imbalance, we might benefit from removing some of the non-retiring rows from our dataset so that the model can focus more on the cases where players did retire.

Before we move on to other metrics, let's talk about the precision and recall metrics visible in the margins of the previous figure's table. Precision and recall are both metrics that help us understand our model's accuracy at different things. Recall is your model's ability to recognize a positive case. In other words, if a player was going to retire, what percentage of time did your model spend saying that the player was going to retire? Models with high recall tend to avoid false negatives. Precision is the trustworthiness of your model's positive predictions. In other words, if a model predicts a player is going to retire, what percentage of the time is your model correct? Models with high precision tend to avoid false positives.

Both precision and recall are important, but depending on the machine learning experiment you're trying to run, you may value one over another if a false positive is worse than a false negative, or vice versa. If our football club wanted to be absolutely certain about whether or not a player would retire, we'd value a high recall metric. If being surprised by a player retiring was acceptable, but we wanted a greater degree of confidence in trusting the model's predictions on when players were going to retire, we'd focus on recall instead.

ML.NET gives us recall and precision on a per-class basis, so you can see the model's recall for positive and negative cases as well as its precision for positive and negative predictions. These aren't the only metrics we get. If you have a C# code cell to display the validation metrics, you'll see even more classification metrics.

Let's try that with this code cell:

```
best.ValidationMetrics
```

This results in *Figure 7.6*, which shows a table of the remaining metrics:

▼ Microsoft.ML.Data.BinaryClassificationMetrics	
AreaUnderRocCurve	0.7915268456375839
Accuracy	0.7900552486187845
PositivePrecision	0.38461538461538464
PositiveRecall	0.3125
NegativePrecision	0.8580645161290322
NegativeRecall	0.8926174496644296
F1Score	0.3448275862068966
AreaUnderPrecisionRecallCurve	0.35344442808482074
ConfusionMatrix	► Microsoft.ML.Data.ConfusionMatrix

Figure 7.6 – Additional binary classification metrics

We've covered most of these when we discussed the confusion matrix, but let's talk briefly about the remaining ones:

- **Accuracy** is the percentage of the time your model predicted the correct result. Accuracy can be biased if you have a significant class imbalance with one category that's far more common than others. For example, I once trained a classification model to predict whether a movie should be considered a Christmas movie or not. The majority of movies in our dataset were not Christmas movies, so my earliest models predicted that nothing was a Christmas movie and still had accuracies of over 99%, despite getting it wrong for every positive case. An accuracy of 100% is perfect.
- **F1 Score** is a balanced measure or harmonic mean of your precision and recall scores. In cases where you don't value precision or recall as more important than the other, the F1 Score gives you a balanced way of comparing models without preferring high recall to high precision, or vice versa. An F1 Score of 1 is perfect.
- **Area under the ROC curve** refers to the shaded portion of a graphical plot of your model's true positive rate and false positive rate for various thresholds of predicting a value as positive or negative. See the further reading section for more details, but values range from 0 to 100%, with 100% being perfect.
- **Area under the precision recall curve** refers to the shaded portion of a graphical plot of precision and recall values and is a more complex measurement, dealing with how correct your model is for different thresholds that classify something as positive or negative, based on precision and recall scores. Values range from 0 to 100%, with 100% being perfect.

When you start an AutoML run, you must provide one of these metrics as the most important one for your experiment. If you don't have a strong preference, I find that the F1 Score is usually a good default metric to go with.

Evaluating our model

Now that we've covered the various metrics associated with binary classification models, let's talk about how good our trained model is.

Frankly, our model is not a terribly good model. While it correctly predicts staying over 89% of the time, it predicts retiring players will retire only 31% of the time. In fact, when it does predict a player will retire, there's a 62% chance that the prediction is wrong ($1.0 - \text{the positive precision of } 0.38$)! While an accuracy of 79% may look good on a report, the F1 Score of 0.34 shows us that our model has major problems accurately predicting retiring players.

So, what went wrong here?

First of all, we limited our training process to only 10 seconds, which limited the number of models that could be evaluated. Secondly, our dataset is rather small, which may lead to missing nuances or details that the model might have noticed.

Additionally, because we have a class imbalance with far more non-retiring players than retiring players, it's quite possible there aren't enough rows in our dataset for retiring players or that those rows aren't getting split evenly between the test and training datasets. We'll explore some potential solutions to this problem in *Chapter 8, Regression Experiments with ML.NET AutoML*.

The final thing to consider is that a reliable prediction of whether or not a player will retire may not be achievable with the data we're training our model on. There could be critical factors, such as yearly pay or other contract details, that are not captured in the dataset.

Calculating feature importance

Even if our model performed perfectly, it's important to understand what factors of the dataset matter the most to our model. We can determine what columns are most important to the model overall by looking at the **permutation feature importance (PFI)**. PFI is calculated by making small modifications to the training data that a model uses one column at a time, and then observing how this impacts the overall performance of the trained model.

If changing a column has a significant impact on the model's metrics, we know that it's more important to the overall model than other columns. This way, we can use PFI to calculate how much each column contributes to a metric of interest. The columns that cause the metric to increase or decrease the most drastically are the most important features of your trained model and what it typically looks at.

ML.NET has a verbosely named `PermutationFeatureImportanceNonCalibrated` method that will let us build a dictionary of column names and their impact on various metrics.

The code to calculate overall importance is a single method call:

```
var importance = context.BinaryClassification
    .PermutationFeatureImportanceNonCalibrated(
        model,
        model.Transform(testData),
        labelColumnName: label,
        permutationCount: 10);
```

Here, we use our best model and our training data, and we specify that we want to perform 10 different permutations per column. Note that we do provide the name of the label column, and we use our model's `Transform` method to ensure that the test data is in an expected state.

Why do we need to transform?

Remember that AutoML added some pre-processing steps to manipulate our data by replacing missing values, one-hot encode columns, and a few other tasks. These decisions are baked into the columns that the model expects. As a result, we need to transform our data so that it has the same pre-processing steps applied to it, in order to match the input format that our model expects.

After calling our PFI method, we'll need to display the resulting features, ordered by their impact on the F1 Score. This can be done by ordering the importance dictionary by the absolute value of the way it modified the metric we care about:

```
var features = importance.OrderByDescending(k =>
    Math.Abs(k.Value.F1Score.Mean)
);
```

Once that's accomplished, we loop over the ordered features and variables and display each feature's name and average impact on the F1 Score:

```
foreach (var kvp in features.Take(10)) {
    string column = kvp.Key;
    MetricStatistics impact = kvp.Value.F1Score;
    Console.WriteLine($"{column} {impact.Mean}");
}
```

This produces a helpful list of the top 10 features and how they impacted the overall F1 Score of the trained model:

```
MinutesPlayedDiff 0.085
      Games -0.071
    GamesDiff -0.065
      Age -0.062
    YellowCards -0.025
    GoalsDiff -0.014
Position.Defender -0.014
    AssistsDiff 0.012
    MinutesPlayed 0.009
Position.Goalkeeper 0.009
      Assists -0.008
    YellowDiff -0.007
```

Here, we can see the largest impact on the F1 Score was the difference between each season's minutes played over the course of a season, followed by the games played, the difference in games played, and age.

Why absolute values?

Note that we don't care whether a feature made the F1 Score increase or decrease; we just want the features that impacted the F1 Score the most.

Most of these top columns seem to relate to playing time per season in terms of minutes and games played. It's reasonable to think that if a player's playing time was decreasing, they'd be more likely to consider retirement. Additionally, age is a column we'd expect to be relevant for a retirement decision.

The remaining columns look to measure player behavior or specific positions. We can see that yellow cards, assists, and goals are all important factors for players potentially retiring. We also see that defenders may retire at different points than other types of players, based on the presence of this one-hot encoded column.

PFI is not a tool you necessarily need to use for every model you create, but I find it to be a helpful way of determining what is important to a model overall at a high level.

With a greater understanding of our model's strengths, weaknesses, and internal reasoning, let's look at how we can use our trained model to make predictions.

Predicting values with binary classification models

Compared to evaluating model performance or behavior, generating predictions for new rows is relatively straightforward. We'll start by defining a class to represent each row of data in our training data, including `IsLastSeason`, our label column.

This is a plain old C# class:

```
public class PlayerSeason {
    public float YellowCards {get; set;}
    public float RedCards {get; set;}
    public float Goals {get; set;}
    public float Assists {get; set;}
    public float MinutesPlayed {get; set;}
    public float Games {get; set;}
    public string Position {get; set;}
    public float Age {get; set;}
    public float YellowDiff {get; set;}
    public float RedDiff {get; set;}
    public float GoalsDiff {get; set;}
    public float AssistsDiff {get; set;}
    public float MinutesPlayedDiff {get; set;}
    public float GamesDiff {get; set;}
    public bool IsLastSeason {get; set;}
}
```

Note that every property is public, is named after the column header, and has a public getter and setter.

Also, note that we only use `float`, `bool`, and `string` values. This is because ML.NET is very specific in the types of data it works with and prefers `float` instead of `int`, `double`, or `decimal`.

Caution

If you try to use a column with a data type that ML.NET does not support, you'll get an `ArgumentOutOfRangeException` later on.

Next, we need to create a class representing the prediction that the model will generate. This should have the `PredictedLabel` property of the same type as our label column:

```
public class RetirePrediction {  
    public bool PredictedLabel {get; set;}  
}
```

Since `IsLastSeason` is a `bool`, we'll use `bool` for `PredictedLabel`.

With these two classes defined, we'll create `PredictionEngine`, which will allow us to generate predictions on new rows:

```
var engine = context.Model.  
    CreatePredictionEngine<PlayerSeason, RetirePrediction>(  
        model, trainData.Schema);
```

Note that we provide the prediction engine with the model we want to use to generate predictions and the `Schema` property of either our training or test dataset.

With `PredictionEngine` ready, we can now instantiate a new `PlayerSeason` object that represents the player we'd like to predict will retire or not:

```
PlayerSeason season = new() {  
    YellowCards = 5,  
    RedCards = 2,  
    Goals = 1,  
    Assists = 3,  
    MinutesPlayed = 1814,  
    Games = 21,  
    Position = "Midfield",  
    Age = 40,  
    YellowDiff = 3,  
    RedDiff = 1,  
    GoalsDiff = -6,  
    AssistsDiff = -12,  
    MinutesPlayedDiff = -450,  
    GamesDiff = -6  
};
```

The `PlayerSeason` instance we defined represents a 40-year-old midfielder who is seeing decreased goals, assists, and playing time. We can call the `Predict` method on our `PredictionEngine` to get an instance of our `RetirePrediction` result class, as shown here:

```
RetirePrediction prediction = engine.Predict(season);  
prediction
```

When I run this on my own best model, I see the following output:

PredictedLabel	True
----------------	------

In other words, our model predicts that this player will retire. If we were to tweak some of the values of the hypothetical player's season stats, it's likely we could get a different result with the same trained model.

In my observations, increasing `MinutesPlayed`, `MinutesPlayedDiff`, `GamesDiff`, and `GoalsDiff` was enough to get `PredictionEngine` to change its prediction from `True` to `False`, but the exact prediction you get will depend on the nature of your model and what it views is important.

At this point, if we are satisfied with our model's performance, we could begin the process of saving it to a file, modifying another application to load the trained model, and creating `PredictionEngine` to make predictions on new data points.

How good is enough?

A common question I encounter is, “*How good does my model need to be?*” The answer depends on how reliable and accurate you need it to be for the purpose it was built for. If I'm building a model to detect cancer, I'm probably going to want to make sure the model is as accurate as I can possibly make it, and a 70% accurate model isn't good enough. Conversely, if my business needs me to build a model to predict the result of coin flips and my model has 70% accuracy, that's phenomenal because it's better than the 50% accuracy an average human has.

We'll explore the model deployment process in more detail in *Chapter 10, Deploying Machine Learning Models to Production*.

Before we get to deploying our models, we'll need to spend some time talking about regression models and more advanced forms of ML.NET over the next few chapters.

Summary

In this chapter, we covered high-level concepts in supervised machine learning in general and, more specifically, in classification.

Classification involves predicting a categorical variable, such as the presence or absence of cancer, predicting which football position a player might perform best in, predicting whether a customer will unsubscribe/churn in the next six months, or determining whether a social media post is likely to “go viral.”

In machine learning, we train models by providing training data to the model training process and selecting the machine learning algorithm and its hyperparameters to use. Pre-processing data may also be necessary to get data into a standardized form.

Once a model is trained, we can evaluate its performance by getting predictions for data we’ve reserved for testing purposes. This test data gives us an idea of whether our model accurately predicts values for data it hasn’t seen before.

Models are evaluated using a variety of metrics, each geared toward one specific aspect of a model’s performance. Some of the most important metrics for classification are the model’s overall accuracy, its recall (its ability to recognize data of specific types), and precision (its reliability when it does recognize specific types). The recall and precision are both important, and they are bound together as an F1 Score that helps summarize the model’s performance in a single number.

ML.NET is one of many different machine learning frameworks out there, but it is the best one currently for .NET developers. ML.NET revolves around an `MLContext` object and `IDataView` instances, such as `DataFrame` objects, containing the testing and training data.

AutoML is an add-on library for ML.NET that automates some of the decisions in data science, such as algorithm selection, hyperparameter tuning, and even aspects of pre-processing data.

As we’ve seen, there are a lot of new concepts and terms to cover with machine learning, but the code isn’t very extensive.

If this chapter was overwhelming, don’t worry because you’ll get a second chance at the material in the next chapter, as we explore using AutoML and ML.NET to train regression models capable of making numerical predictions.

Further reading

To know more about the topics discussed in this chapter, you can visit the following links:

- *ML.NET Repository*: <https://github.com/dotnet/machinelearning>
- *ML.NET AutoML*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/automated-machine-learning-mlnet>
- *ML.NET Model Algorithms (Trainers)*: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.ml.trainers?view=ml-dotnet-preview>
- *Interpreting Confusion Matrixes*: https://accessibleai.dev/post/interpreting_confusion_matrixes/

- *Class imbalances*: <https://learn.microsoft.com/en-us/training/modules/machine-learning-confusion-matrix/>
- *Evaluation Metrics for Binary Classification*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/metrics#evaluation-metrics-for-binary-classification>
- *Permutation Feature Importance*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/explain-machine-learning-model-permutation-feature-importance-ml-net>
- *Open Neural Network Exchange (ONNX)*: <https://onnx.ai/>
- *Automated Machine Learning on Azure*: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-automated-ml>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/Classification/>

8

Regression Experiments with ML.NET AutoML

In the last chapter, we saw how machine learning classification can predict a categorical value and how that's helpful for determining what category a new piece of data fits into.

In this chapter, we'll see how machine learning can be used in **regression experiments** to predict numerical values and how to accomplish this in **ML.NET** using **AutoML**.

Since regression experiments are so similar to classification experiments, we'll also be exploring more advanced ways of splitting data for training and testing by talking about **cross-validation**.

In this chapter, we'll explore the following topics:

- Understanding regression
- Performing a regression experiment
- Interpreting cross-validation results
- Evaluating regression metrics
- Applying a regression model

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter08` folder along with the chapter's dataset.

Understanding regression

Regression is a form of supervised learning that involves training a model to predict a numerical value for a single label column for new rows the model hasn't seen before.

Using a **trained regression model**, you can predict numerical values such as the following:

- Prices of services and commodities
- Durations of commutes
- Sports scores or point differentials
- Volumes of customers or sales
- Number of readers for a prospective blog post or book

Since being able to predict a number has so many uses, regression models can serve almost any industry – provided you have an adequate dataset to train your regression model.

Our regression task

In this chapter, we'll be training and evaluating a regression model related to our football dataset. Specifically, we'll attempt to *calculate an expected salary for a footballer* based on their current season and last season's performance, their age, their position, and other factors. A trained model that could accurately predict the expected pay for a football player could be used in contract negotiations to ensure the organization is not overpaying for a player. More critically, a football club with this model could identify players in other clubs that are undervalued and attempt to trade for them by trading players the model believes are overvalued.

In such a manner, an accurate regression model would create a competitive advantage for the football club that has it since it would allow them to optimize their roster and overall spending.

Regression as a numerical formula

To illustrate regression, let's consider a simple formula that suggests an attacker player's pay, which can be approximated at a base pay, plus additional pay based on the goals they scored last season.

While such a function would doubtlessly be very inaccurate, it could be implemented as a specific function rather easily in C#:

```
float CalculateAttackerPay(float goals) {  
    const float basePay = 25000;  
    float payPerGoal = 250;  
    return basePay + (payPerGoal * goals);  
}
```

As per this function, attackers get a base pay of 25 thousand euros plus an additional pay of 250 euros for every goal they scored last season. If we were to plot the expected pay for attackers scoring between 0 goals in a season and players scoring 50 goals in a season, you'd see something like *Figure 8.1*:

Pay for players by goal count

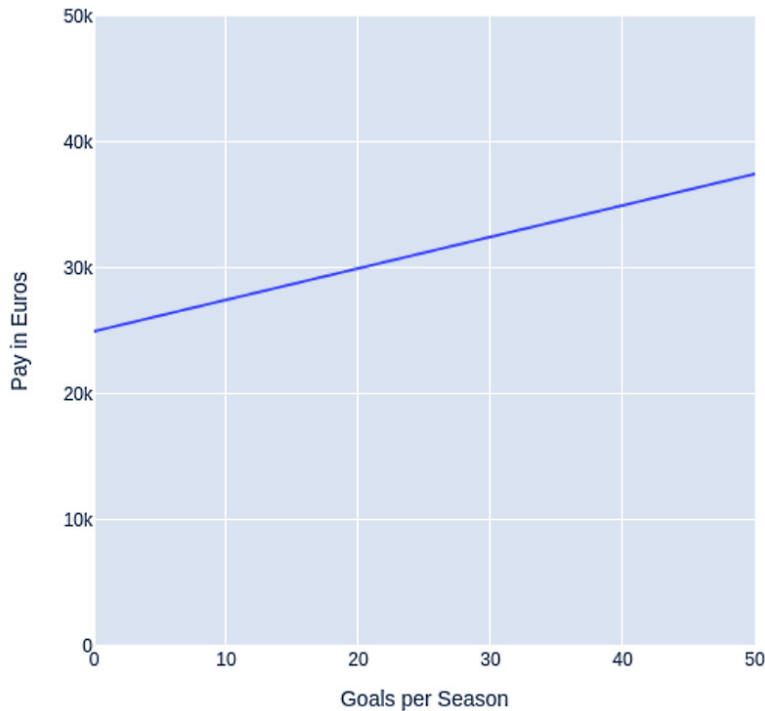


Figure 8.1 – A chart showing a simple pay model for attackers

While this mathematical model is a horrible predictor for players, it should hopefully illustrate the process of regression. In this example, our function's parameters are just the number of goals the attacker scored last season. We could write this function mathematically as follows:

```
expectedPay = 25000 + (goals*250)
```

If this style of equation looks slightly familiar, it's likely you encountered it in school in the form of linear algebra. More specifically, this simple problem could be looked at as the **slope-intercept form of a line**, commonly referred to as $y = mx + b$.

In our example, y is the value we're trying to predict, or our `expectedPay` variable. 25000 represents the y value for 0 goals, which is represented as b in the equation. Meanwhile, mx is some input value for goals (x) multiplied by some weight (m) of the importance of each goal.

Given this, we could rewrite our equation in pseudocode as follows:

```
b = 25000; // Base pay
m = 250; // Bonus per goal
x = goals;
y = (m * x) + b
```

Most machine learning models will use far more than a single variable, but linear equations can handle additional variables fairly easily. For example, if we wanted to expand our `expectedPay` equation to also reward players 100 euros per assist, we could rewrite the function as follows:

```
expectedPay = 25000 + (goals*250) + (assists*100)
```

We could also represent this with `x1` referring to goals, `x2` referring to assists, `m1` referring to the value per goal, and `m2` referring to the value per assist, as shown here:

```
y = (m1*x1) + (m2*x2) + b
```

In such a way, our equation is expandable every time we add a new feature to our model, and each new feature can make the model that much more accurate.

This is not a book on linear algebra or mathematics and machine learning. You don't need to be able to perform linear algebra to train a model. However, my technical reviewers and I thought it was important to show you the relationship between the numbers we're predicting in this chapter and the underlying mathematics.

Let's move on from this mathematical exploration and get into our dataset for the chapter.

Our regression dataset

In this chapter, we'll use a dataset of 5,776 football players, their specific playing position, their height, their current and highest pay, and the total number of their yellow and red cards, goals, assists, playing time, and shootout performance for the past two seasons.

Let's take a look at our data now by loading this chapter's `training.csv` file into a `DataFrame` using C#:

```
#r "nuget:Microsoft.Data.Analysis,0.21.1"
using Microsoft.Data.Analysis;

DataFrame df = DataFrame.LoadCsv("training.csv");
df.Head(5)
```

As usual, this gives us a scrollable preview of our data, as shown in *Figure 8.2*:

BirthCountry	SubPosition	Position	Height	CurrentPay	HighestPay	YellowCards	RedCards	Goals	Assists	Minutes	Games
Brazil	Defensive Midfield	Midfield	178	15000000	65000000	3	0	1	1	2169	39
Cote d'Ivoire	Right-Back	Defender	176	7000000	20000000	5	0	1	2	2357	30
Jamaica	Left Winger	Attack	170	45000000	160000000	7	0	10	6	2645	39
France	Centre-Back	Defender	195	2500000	18000000	4	0	2	1	1594	25
Switzerland	Central Midfield	Midfield	180	6500000	25000000	6	0	0	0	2439	32

Figure 8.2 – The first five rows and some of the columns of our dataset

This dataset is nearly in a perfect state for us, except it does contain the name of each player in its `Name` column.

We don't want to accidentally get a model that rewards players with a first name of **Rodolfo** or penalizes players named **Alan** based on observations from its dataset. Due to this, we'll want to remove the `Name` column prior to training.

However, we'll have a use for a `DataFrame` with the `Name` column still included later on in this chapter. For that, we'll make a copy of our `DataFrame` while it still has the `Name` column, then drop the `Name` column from the original using the following code:

```
DataFrame dfWithName = df.Clone();
df.Columns.Remove("Name");
```

Once this completes, `df` no longer has the `Name` column, while `dfWithName` does. In this way, the `Clone` method gives us the ability to create an independent copy of a `DataFrame` prior to any modifications.

An alternative to `Clone()`

It would likely be more efficient to simply load up the `DataFrame` from the file later on so we wouldn't need to store both `DataFrame` objects in memory at the same time. However, `Clone` is still a useful method if you've already made many modifications to a `DataFrame` through filtering, grouping, merging, and feature engineering, or if you otherwise can't easily recreate a `DataFrame` by loading it from a file.

Now that we have our data in place, let's proceed with setting up an ML.NET regression experiment.

Performing a regression experiment

Our experiment in this chapter will rely both on ML.NET and AutoML once more, so we'll start by importing those NuGet packages and creating an `MLContext`, as we did last chapter:

```
#r "nuget:Microsoft.ML,3.0.1"
#r "nuget:Microsoft.ML.AutoML,0.21.1"
using Microsoft.ML;
using Microsoft.ML.Data;
using Microsoft.ML.AutoML;
MLContext context = new();
```

Ordinarily, our next step would be to split our `DataFrame` (which implements `IDataView`) into a training and test set using `context.Data.TrainTestSplit` as we did in the last chapter. This process is still an option for us in regression and with this dataset, but in this chapter, I want to show you an alternative way of splitting our data later on.

Since we're performing a regression experiment, we'll instantiate a `RegressionExperimentSettings` object:

```
RegressionExperimentSettings settings = new() {
    OptimizingMetric = RegressionMetric.MeanAbsoluteError,
    MaxModels = 10
};
```

This should look similar to how we set up `BinaryClassificationExperimentSettings` in the previous chapter, with a few notable differences:

- Since we're performing regression instead of classification, the type of the `settings` object is different.
- We're now selecting a `RegressionMetric` instead of a `BinaryClassificationMetric`. Regression metrics are fundamentally different from classification metrics and we'll explore them in more detail later in this chapter.
- Instead of setting `MaxExperimentTimeInSeconds` to limit our experiment's time, we instead set `MaxModels` to restrict the maximum number of models that are trained and evaluated.

Max models or max time?

It's up to you to decide whether you want to limit your experiment to a fixed number of seconds or cap its processing at a maximum number of models. By specifying the number of seconds, you know how long your model training process will run, but the number of models evaluated may vary based on the machine the models are trained on or their active processor load. In practice, you could use either approach based on your needs.

Let's also inspect the `settings` object by including it as the last value in a code cell:

```
settings
```

This produces a tree of the object, as shown here in *Figure 8.3*:

▼ <i>Microsoft.ML.AutoML.RegistrationExperimentSettings</i>		
OptimizingMetric	MeanAbsoluteError	
Trainers	<i>index</i>	<i>value</i>
	0	FastForest
	1	FastTree
	2	FastTreeTweedie
	3	LightGbm
	4	LbfgsPoissonRegression
	5	StochasticDualCoordinateAscent
MaxExperimentTimeInSeconds	86400	
CancellationToken	► <i>System.Threading.CancellationToken</i>	
CacheDirectoryName	Microsoft.ML.AutoML	
CacheBeforeTrainer	Auto	
MaximumMemoryUsageInMegaByte	<null>	
MaxModels	10	

Figure 8.3 – Our regression settings

The reason I want to highlight this is that the `Trainers` property on the settings object lists the different **model trainers** that AutoML will evaluate against our data when we run our regression experiment. In other words, the model it recommends will involve one of these six trainers. Each model trainer represents a specific machine learning algorithm applied to solve a machine learning task.

Model trainers versus algorithms

Machine learning involves applying specific algorithms to generate predictions. We illustrated the basics of linear regression at the beginning of this chapter to explore the potential relationship between goals and pay for attackers. However, that same algorithm could be used to classify players if it were slightly modified such that instead of predicting a pay amount, it might be predicting the probability that the player is an attacker versus not an attacker.

In such a way, the linear algebra algorithm could fulfill regression tasks and classification tasks. This task difference is the difference between the **linear regression trainer** and the **logistic regression trainer**.

ML.NET uses the term **model trainer** to refer to an implementation of a machine learning algorithm for a specific task.

If you're curious about the classification options available, *Figure 8.4* shows what our `BinaryExperimentSettings` looked like for the experiments we did in *Chapter 7, Classification Experiments with ML.NET AutoML*:

▼ `Microsoft.ML.AutoML.BinaryExperimentSettings`

OptimizingMetric	F1Score												
Trainers	<table><tr><th>index</th><th>value</th></tr><tr><td>0</td><td>FastForest</td></tr><tr><td>1</td><td>FastTree</td></tr><tr><td>2</td><td>LightGbm</td></tr><tr><td>3</td><td>LbfgsLogisticRegression</td></tr><tr><td>4</td><td>SdcaLogisticRegression</td></tr></table>	index	value	0	FastForest	1	FastTree	2	LightGbm	3	LbfgsLogisticRegression	4	SdcaLogisticRegression
index	value												
0	FastForest												
1	FastTree												
2	LightGbm												
3	LbfgsLogisticRegression												
4	SdcaLogisticRegression												
UseAutoZeroTuner	False												
MaxExperimentTimeInSeconds	10												
CancellationToken	► <code>System.Threading.CancellationToken</code>												
CacheDirectoryName	Microsoft.ML.AutoML												
CacheBeforeTrainer	Auto												
MaximumMemoryUsageInMegaByte	<null>												
MaxModels	2147483647												

Figure 8.4 – `BinaryExperimentSettings` with different trainers

As you can see, there are some differences in available trainers between the binary classification and regression, though both problems support the `FastForest`, `FastTree`, and `LightGbm` algorithms.

We'll talk more about these common algorithms in *Chapter 9, Beyond AutoML: Pipelines, Trainers, and Transforms*.

Ignoring specific trainers

If you don't want a trainer to be considered, you can find it and remove it from the `Trainer` property on your `settings` object prior to executing your experiment by calling the `Remove` method. For example, `settings.Trainers.Remove(RegressionTrainer.FastForest)` ; would remove the `FastForest` model trainer from consideration.

With our settings prepared, it's time to create our `RegressionExperiment` and execute it. The code for this will be familiar aside from a change in how we're separating our training and test data:

```
string label = "CurrentPay";
RegressionExperiment experiment =
    context.Auto().CreateRegressionExperiment(settings);

CrossValidationExperimentResult<RegressionMetrics> result =
    experiment.Execute(df,
        numberOfCVFolds: 3,
        labelColumnName: label);
```

Here, we set the label variable to `CurrentPay` because we'll want our trained model to predict the `CurrentPay` column for a row of data representing a player in our league. This will help us identify overpaid and underpaid players to make intelligent roster moves. Like we did with classification, we can create an experiment through `context.Auto()`, but this time around, it's a `RegressionExperiment`.

The final chunk of code is the most interesting and most different. Here, we're calling the `Execute` method, but instead of providing a training `IDataView` and a testing `IDataView`, we instead provide the entire data as an `IDataView` through our `df` variable.

Next, we specify `numberOfCVFolds` to be 3. This stands for the number of cross-validation folds because in this specific example, we're performing cross-validation.

Understanding cross-validation

Before, when we split our data into a training set and a testing split, we effectively "folded" our data into two unequal groups that could be used to train and validate our data.

If we imagine nine rows of our dataset with six going to the training set and three going to the testing set, this could be visualized as shown in *Figure 8.5*:

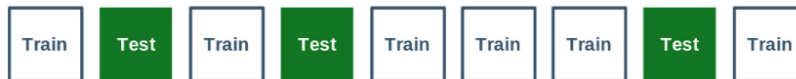


Figure 8.5 – A typical test/train split dividing rows into two groups

In a traditional test/train split, we split our data and evaluate it once. With cross-validation, we can split our data and evaluate it multiple times. Since we specified 3 as our `numberOfCVFolds`, ML.NET is actually going to take our data and "fold" it into three different distributions of training data and test data, as illustrated in *Figure 8.6*:

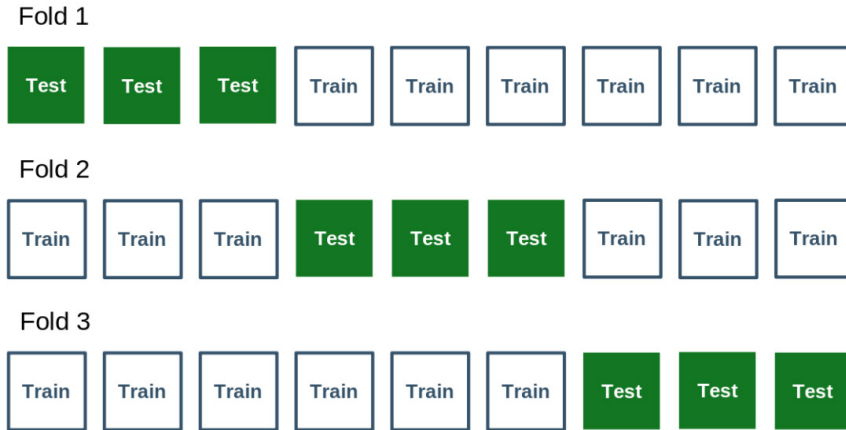


Figure 8.6 – The same data split into three different cross-validation folds

In this way, we can effectively perform three experiments (the number of folds we provided) instead of only a single experiment. Each of these experiments will be run against a different set of training data and test data and, as a result, will produce different metrics.

K-Fold cross-validation

You'll sometimes hear cross-validation referred to as **K-Fold cross-validation**. The **K** in K-Fold cross-validation refers to the number of unique times the data was folded into different combinations. In our case, the data was folded three times, so K would be 3 and we could refer to the splitting process as a 3-fold cross-validation approach.

This complicates the results, as we'll see in the next section, but is why we're storing the results of our `experiment.Execute` call into a `CrossValidationExperimentResult` object instead of an `ExperimentResult` object as we did in the previous chapter.

Don't want cross-validation?

If you don't want to use cross-validation for regression, you can still call the `experiment.Execute(dfTrain, dfTest, labelColumnName: label)` overload as we did with our classification experiment in *Chapter 7, Regression Experiments with ML.Net AutoML*.

One of the advantages of cross-validation is that it helps protect us against sensitivity issues that can arise from splitting your data into testing and training sets. If you have certain scenarios that are important to your model and all of the rows that represent that scenario get put into the testing set, your model will not encounter these factors in the training data.

Using cross-validation guarantees that important data makes it into the training data set in at least one fold. The downside of cross-validation is that training a model on multiple folds of data multiplies the amount of time it takes to train a model.

Additionally, the resulting objects are more complex because they store information from all the folds, not just a simple train/test split.

Let's explore our cross-validation results now.

Interpreting cross-validation results

Recall that the `result` variable we stored after the `experiment.Execute` call is a `CrossValidationExperimentResult<RegressionMetrics>` instance.

If we end a cell with that object in Polyglot Notebooks, we see a graphical representation of the results, as shown in *Figure 8.7*:

▼ <code>Microsoft.ML.AutoML.CrossValidationExperimentResult`1[Microsoft.ML.Data.ReggressionMetrics]</code>		
RunDetails	index	value
	0	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	1	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	2	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	3	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	4	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	5	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	6	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	7	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	8	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	9	▶ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>
BestRun ▼ <code>Microsoft.ML.AutoML.CrossValidationRunDetail`1[Microsoft.ML.Data.ReggressionMetrics]</code>		
Results	index	value
	0	▶ <code>Microsoft.ML.AutoML.TrainResult`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	1	▶ <code>Microsoft.ML.AutoML.TrainResult`1[Microsoft.ML.Data.ReggressionMetrics]</code>
	2	▶ <code>Microsoft.ML.AutoML.TrainResult`1[Microsoft.ML.Data.ReggressionMetrics]</code>
TrainerName	ReplaceMissingValues=>OneHotEncoding=>FeaturizeText=>Concatenate=>FastForestRegression	
RuntimeInSeconds	0.402	
Estimator	▶ <code>Microsoft.ML.Data.EstimatorChain`1[Microsoft.ML.ITransformer]</code>	

Figure 8.7 – Our experiment results, including RunDetails and BestRun

Much of this is similar to what we saw with a standard train/test split in the previous chapter: we still have a `RunDetails` property listing all models evaluated and we still have a `BestRun` property containing the most effective model according to the evaluation metric we specified in our settings object.

What's different here is that each run details contain not just a single model and set of metrics, but one model and associated metrics for every cross-validation fold that was evaluated.

Let's illustrate this by displaying the `MeanAbsoluteError` of each of the three folds evaluated with our best run:

```
CrossValidationRunDetail<RegressionMetrics> best = result.BestRun;
int index = 1;
double totalMAE = 0;
foreach (var result in best.Results) {
    double mae = result.ValidationMetrics.MeanAbsoluteError;
    Console.WriteLine($" Fold {index++} Mean Absolute Error: €
{mae:N0}");
    totalMAE += mae;
}

double averageMAE = totalMAE / best.Results.Count();
Console.WriteLine($"Average Mean Absolute Error: € {averageMAE:N0}");
```

This loops through each fold, displays its error amount, and then calculates an overall average error across all folds.

When I run this, I see the following metrics:

```
Fold 1 Mean Absolute Error: € 2,183,861
Fold 2 Mean Absolute Error: € 2,372,676
Fold 3 Mean Absolute Error: € 2,370,424
Average Mean Absolute Error: € 2,308,987
```

Looking at this, we see that all three folds have similar values for their **Mean Absolute Error (MAE)** metrics. This is a good indicator as large fluctuations between folds could tell us that sensitive data is moving between training and testing folds and impacting the overall training process. In such a case, you likely need additional data in your dataset to reduce the likelihood that important data isn't represented in either the training or test set.

In this case, we see that the first fold has the best MAE and is likely our best model, but we haven't yet discussed what this metric represents. We'll do that shortly, but let's briefly cover identifying the best fold automatically.

You can get your best fold by ordering by the metric you care about and then getting the first fold in those results:

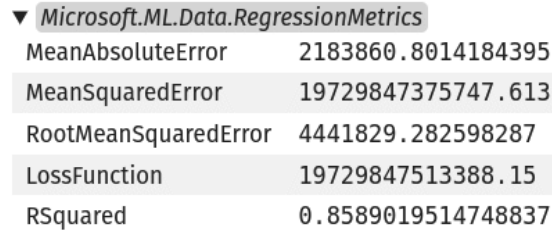
```
TrainResult<RegressionMetrics> bestFold = best.Results
    .OrderBy(f => f.ValidationMetrics.MeanAbsoluteError)
    .First();
```

Note that depending on which metric you're evaluating, a low score may be better than a high score, or vice versa. For regression, we prefer low values for all metrics aside from **R Squared**.

Now that we have our best fold, let's see its metrics by ending a code cell with those metrics:

```
RegressionMetrics metrics = bestFold.ValidationMetrics;  
metrics
```

This produces five metrics, as shown in *Figure 8.8*:



▼ Microsoft.ML.Data.RegressionMetrics	
MeanAbsoluteError	2183860.8014184395
MeanSquaredError	19729847375747.613
RootMeanSquaredError	4441829.282598287
LossFunction	19729847513388.15
RSquared	0.8589019514748837

Figure 8.8 – Our regression metrics for the best fold of the best run

Let's explore what these metrics say about our model next.

Evaluating regression metrics

Metrics in classification experiments, such as precision and recall, were oriented around sorting data into the right categories. Regression metrics are different and focus on how close our predicted values are to the correct value.

The difference between our model's prediction and the actual value that occurred is referred to as the model's **error**.

Understanding our model's error helps us know both how accurate it generally is and how sensitive it can be to outliers.

The error can be positive or negative based on whether the model guessed a number greater than or less than the actual value. This is why metrics for regression involve getting the absolute value of error or squaring that error for each row so positive or negative values don't matter.

Let's cover the three simplest regression metrics first, as these directly measure error:

- **Mean Absolute Error (MAE)** is the simplest metric. If you add up the absolute value of each row's error and then divide by the number of rows in your dataset, you get the average overall error for your model. This is easier to understand but may be influenced by a few wildly incorrect guesses.
- **Mean Squared Error (MSE)** is similar to MAE, but instead of getting the absolute error value for each row, we square the error by multiplying it by itself. This results in high error amounts being exponentially larger and can help us spot models that generate wildly incorrect guesses for outlying rows. MSE is a helpful value, but because it is squared, it can be hard to interpret.

- **Root Mean Squared Error (RMSE)** is the square root of MSE. This takes our MSE and returns it back to the same scale as our original error values, or our MAE. The result of this is an error value that indicates sensitivity to outliers.

Our football model has an MAE of 2.1 million euros, an MSE of over 19 trillion, and an RMSE of 4.4 million euros.

This indicates that most of the time, the model will be within 2.1 million euros of the correct answer, but due to the distance between the MAE and RMSE, we know the model has a few outlying cases where the error is significantly higher

This helps us know that our model will be most effective with players who resemble your average footballer and less effective for superstars or players who barely play.

If 2.1 million euros still sounds like a high error when we're talking about money, we should keep in mind the scale of our dataset which deals with large player salaries.

We can use `MathNet.Numerics` to get descriptive statistics for our current pay column much as we did in *Chapter 5, Visualizing Data*:

```
#r "nuget:MathNet.Numerics,5.0.0"
using MathNet.Numerics.Statistics;

SingleDataFrameColumn targetColumn = (SingleDataFrameColumn)df[label];
double[] values = targetColumn.Where(v => v != null)
    .Select(v => (double)v)
    .OrderBy(v => v)
    .ToArray();

DescriptiveStatistics stats = new(values);
stats.StandardDeviation
```

This calculates and displays our standard deviation:

```
12621285.730493644
```

Due to extreme pay disparities in football, our standard deviation is 12.6 million euros. With this in mind, our error of 2.1 million euros is relatively accurate given the wide distribution of our dataset.

Improving the accuracy further

If we wanted to improve the accuracy of our model further, we would likely need to filter out outlying players who are superstars or bench warmers in order to reduce our model's error metrics to more accurate and focused levels. Additional training time could also help.

The `RegressionMetrics` class also provides two additional metrics:

- **R Squared** is a value ranging from negative infinity to 1 that indicates how much of the model explains the variations in the data. A value of 1 indicates a perfect model that can accurately predict all data. A value near 0 indicates a model guessing at random. Negative values are possible and indicate that the model is worse than simply making up numbers with a random number generator.
- **Loss function** is a relatively undocumented regression metric offered by ML.NET. However, inspecting its code reveals that it appears to use a combination of our other metrics. Lower values are better than higher values. In our model, the loss function closely resembles the MSE. See the *Further reading* section for links to the current implementation of this feature.

Our model's R-squared value is 0.85, indicating that it explains around 85% of the variance in the model, making it a good but not perfect model.

Loss

The loss function column makes more sense if you understand that error in a regression model is sometimes referred to as a loss.

Given our model's accuracy and the outlier-heavy nature of our dataset, I would be comfortable using our model as a tool for identifying players who are likely undervalued or overvalued. However, the still-large MAE and RMSE values would prevent me from using this model as an aid when actively negotiating contracts with players, or when evaluating superstars or bench warmers.

Let me illustrate what I mean by predicting how much I would be paid as a footballer.

Predicting values for outliers

The code to generate a prediction for regression is identical to the code for classification. We can generate a prediction by following the following process:

1. Identify the model we want to use.
2. Create a class to represent a row of data.
3. Create a class to represent the results of a prediction.
4. Create a `PredictionEngine` using these classes.
5. Instantiate an object for the row we want to evaluate.
6. Call the `Predict` method on our `PredictionEngine` to get a prediction on our object.

We'll start by getting a reference to our model:

```
ITransformer model = bestFold.Model;
model
```

This shows us some high-level information of our best model in *Figure 8.9*:

▼ [Microsoft.ML.Transforms.MissingValueReplacingTransformer, Microsoft.ML.Transforms.OneHotHashEncodingTransformer, Microsoft.ML.Data.ColumnCon

LastTransformer ▶ Microsoft.ML.Data.ReggressionPredictionTransformer<Microsoft.ML.Trainers.FastTree.FastForestRegressionModelParameters>

(values)

index	type
0	Microsoft.ML.Transforms.MissingValueReplacingTransformer
1	Microsoft.ML.Transforms.OneHotHashEncodingTransformer
2	Microsoft.ML.Data.ColumnConcatenatingTransformer
3	Microsoft.ML.Data.ReggressionPredictionTransformer<Microsoft.ML.Trainers.FastTree.FastForestRegressionModelParameters>

Figure 8.9 – The sequence of transformations and our model

It looks like our model works by replacing missing values, one-hot encoding columns, concatenating feature columns together, and then using a *FastForest* regression model – a form of random forest which we'll discuss in the next chapter.

Let's move on and define a class to represent a player:

```
public class Player {
    public string Position {get; set;}
    public string BirthCountry {get; set;}
    public string SubPosition {get; set;}
    public float Height {get; set;}
    public float CurrentPay {get; set;}
    public float HighestPay {get; set;}
    public float BirthYear {get; set;}
    public float Minutes {get; set;}
    // Many columns omitted for brevity...
}
```

Next, we create a class for the predicted value. In this case, we're predicting a numerical value representing the expected *CurrentPay* of this player, which we'll refer to as their market value:

```
public class Prediction {
    [ColumnName("Score")]
    public float MarketValue {get; set;}
}
```

Note that we use `ColumnName` here to map from ML.NET's default output property, called `Score`, to a more understandable `MarketValue` property. After this, we create our `PredictionEngine`:

```
var engine = context.Model.CreatePredictionEngine<Player,
Prediction>(model);
```

With our classes and `PredictionEngine` defined, let's instantiate our `Player` object:

```
Player matt = new() {
    Position = "Attack",
    SubPosition = "Centre-Back",
    Height = 168,
    BirthYear = 1980,
    Goals = 0,
    Assists = 0,
    YellowCards = 1,
    RedCards = 0,
    ShootoutGoals = 0,
    ShootoutAppearances = 0,
    Minutes = 2,
    Games = 21
};
```

This player represents my fictitious contributions to a football club. In this scenario, they drafted me to their club, and I sat on the bench for the whole season.

The only actual contributions I made during this season (besides sitting on the bench) were being on the pitch for two minutes before the coaching staff realized I was out there and got me back on the bench, along with a yellow card I received for telling dad jokes to the officials in a moment of heightened emotions.

Now, I certainly wouldn't want a player like this in my football club, but we'll see what our model says this player should be paid:

```
Prediction salary = engine.Predict(matt);
salary.MarketValue
```

This results in the following lofty prediction:

```
998974.3
```

In other words, this model suggests that were I to step down from being a tech author to become a footballer, I could expect to make just under a million euros per season!

By the way, if you manage a football club or other form of sports team and would like to pay me a comparable amount of money to sit on the bench and read books, we should talk; please get in touch with my agent.

Obviously, I shouldn't make nearly a million euros for sitting on a bench. I use this example because it's an amusing way of illustrating that models are less accurate at predicting values for unusual data points.

While our model may be very accurate for most actual football players, it's going to be less accurate for players that are not adequately represented in the training data – whether that's your bench warmers (such as myself) or the greatest footballers of all time.

So, what's important to this model? Let's find out.

Applying PFI to regression models

A simple way of determining what's important to our model would be to change properties on our player instance and generate a new prediction to see how the values changed.

A more scientific approach would be to apply **Permutation Feature Importance (PFI)** to our regression model, just like we did in the previous chapter with our classification model.

To calculate regression feature importance, we use the `Regression.FeatureImportance` method on our `MLContext`, passing it our model, a transformed `IDataView`, the label column we're predicting, and a count of permutations to apply:

```
var importance = context.Regression
    .PermutationFeatureImportance(
        model,
        model.Transform(df),
        labelColumnName: label,
        permutationCount: 3);
```

Just like training a model, this operation will take some time. The amount of time depends on the volume of data in your dataset and the number of permutations you've chosen.

Once the feature importances have been calculated, you can display the most important ones by ordering by a specific metric and then writing out the extent to which the feature impacts that metric, as shown here:

```
var orderedFeatures = importance.OrderByDescending(k =>
    Math.Abs(k.Value.MeanAbsoluteError.Mean));

foreach (var kvp in orderedFeatures.Take(5)) {
    double avgImpact = kvp.Value.MeanAbsoluteError.Mean;
    Console.WriteLine($"{kvp.Key} {avgImpact}");
}
```

This outputs the following list of features, ordered from most to least important:

```
HighestPay 5439919.579568448
BirthYear 1183248.9380708751
Games 252602.4532026824
Minutes 93727.03556769558
MinutesLastSeason 74741.90876112848
```

Our model's most important factors are the highest amount of money a player has been paid, the birth year (and derived age) of a player, and playing time stats such as the number of games and amount of minutes the player has played.

Should we include HighestPay?

If your initial reaction to the HighestPay for a player being the most important factor for determining their current market value, was skepticism, good job. For most players, this will be the same as or near their current salary and, as a result, this metric lets the label column “bleed into” our set of features. A way of storing information about relative pay without “leaking” the actual salary information would be to store the percentage of their highest salary a player is currently making.

If we were to remove the HighestPay column from the dataset and retrain, the resulting model's MAE would drop from 2.1 million euros to around 4.4 million euros. As a result, my hypothetical player's projected salary would increase by 1.5 million euros!

While our model clearly has some shortcomings in its lowered effectiveness for outliers and its dependency on a player's highest salary thus far, the model should still be generally useful for detecting players who are significantly underpaid or overpaid.

Let's try that out now as we close our chapter.

Applying a regression model

Remember how we called Clone on our DataFrame at the beginning of the chapter to store a copy of our original DataFrame before we dropped the Name column?

In this section, we'll take that variable, dfWithName, and we'll use our trained model to add new columns to the DataFrame. This will let us store the predicted market value and the percentage of that value they're currently being paid. We can then order the DataFrame and look at the highest and lowest rows to identify players to potentially acquire and players to avoid or trade away.

We'll start by defining a method to help us get a value out of a row or default to 0 if the value for that row is null:

```
float GetValueOrDefault(DataFrameRow row, string columnName) {
    object value = row[columnName];
```

```

    return value == null ? 0 : (float)value;
}

```

This method will be helpful when looking at players in their first season who have null values for columns representing their previous seasons.

Next, we'll create a pair of collections to store our predicted values and calculated percentages and filter our `dfWithName` down to just players who are in the `Attack` role in a football club:

```

List<float> predictions = new();
List<float> percents = new();
DataFrame dfAttack = dfWithName.Filter(
    dfWithName["Position"].ElementwiseEquals("Attack")
);

```

The decision to limit to attackers was made because the data our model relies on is mostly related to offensive talents such as goals and assists. As a result, the model will be best able to evaluate attackers.

Next, we'll loop over each row in `dfAttack` and perform the following operations:

1. Create a `Player` object out of data for that row.
2. Use our `PredictionEngine` to predict their market value.
3. Store the market value and the calculated percentage.

The code for this is long given the object's many properties, but is otherwise straightforward:

```

foreach (var row in dfAttack.Rows) {
    Player player = new() {
        Position = row["Position"].ToString(),
        SubPosition = row["SubPosition"].ToString(),
        BirthCountry = row["BirthCountry"].ToString(),
        Height = GetValueOrDefault(row, "Height"),
        // Other properties omitted for brevity...
        Games = (float)row["Games"],
    };

    Prediction prediction = engine.Predict(player);
    float current = GetValueOrDefault(row, "CurrentPay");

    predictions.Add(prediction.MarketValue);
    percents.Add((current / prediction.MarketValue) * 100);
}

```

With these lists of values for our new columns, we can define the new columns and get a preview of the data with just a few lines of code:

```
dfAttack.Columns.Add(new
    PrimitiveDataFrameColumn<float>("MarketValue", predictions));
dfAttack.Columns.Add(new
    PrimitiveDataFrameColumn<float>("PercentOfMarket", percents));
dfAttack.Head(5)
```

This results in *Figure 8.10*, showing a portion of the available columns:

ShootoutGoals	ShootoutAppearances	BirthYear	Name	MarketValue	PercentOfMarket
0	0	1994	Christopher Daugherty	37124360	121.21421
0	0	1993	Allan Rosenbaum	9087014	132.05658
1	1	1997	Lucas Kihn	18931210	147.9039
0	0	1998	Jim Smitham	48619836	65.81676
0	0	2001	Gilberto Murray	41125020	116.71727

Figure 8.10 – Our new columns in the DataFrame

Now that we have our calculations in the DataFrame, let's drop our columns down to a more focused set in a new DataFrame and then show the top 10 underpaid players according to our model:

```
string[] columnsToKeep = new string[] {
    "Name", "SubPosition", "CurrentPay", "MarketValue",
    "PercentOfMarket", "Goals", "Assists", "Games",
    "Minutes", "GoalsLastSeason", "AssistsLastSeason",
    "BirthYear"
};
IEnumerable<DataFrameColumn> columns =
    columnsToKeep.Select(c => dfAttack[c]);

DataFrame dfNew = new DataFrame(columns);
dfNew = dfNew.Filter(dfNew["CurrentPay"]
    .ElementwiseGreaterThanOrEqual(100000));

dfNew.OrderBy("PercentOfMarket").Head(10)
```

Here, we select the DataFrameColumn objects we want in our focused view, then create a new DataFrame by providing those columns.

From there, we filter out players below a certain pay threshold in order to eliminate extreme outliers. Following that, we sort by the percentage of market pay in ascending order. This results in *Figure 8.11*:

Name	SubPosition	CurrentPay	MarketValue	PercentOfMarket	Goals	Assists	Games	Minutes	GoalsLastSeason	AssistsLastSeason	BirthYear
Ronald Gorczany	Right Winger	100000	5407217.5	1.8493799	1	0	9	178	0	0	1992
Karl Russel	Left Winger	300000	7170060	4.1840653	2	2	19	633	0	1	1986
Samuel Kreiger	Right Winger	150000	3471900.8	4.3203998	0	1	15	499	1	2	1987
Rolando Wyman	Centre-Forward	150000	2815946.5	5.3268056	0	2	21	1435	1	1	1986
Dustin Lynch	Centre-Forward	300000	3707930.2	8.090767	6	6	26	1545	1	2	1989
Doyle Greenfelder	Centre-Forward	300000	3471900.8	8.6407995	1	0	12	391	2	1	1991
Ignacio Glover	Left Winger	100000	1148442.1	8.707448	1	3	18	1320	<null>	<null>	1989
Melvin White	Right Winger	100000	1148442.1	8.707448	0	1	12	296	0	1	1988
Eugene Torphy	Centre-Forward	100000	1111025.1	9.000696	0	1	9	61	0	0	2004
Roland Steuber	Centre-Forward	100000	1111025.1	9.000696	3	0	20	926	1	1	1990

Figure 8.11 – Attackers who are currently drastically underpaid

While our model's accuracy isn't perfect, being able to focus on extreme mismatches helps us find players of interest such as Dustin Lynch here, who is paid 300,000 euros despite making six goals and six assists in 26 games last season.

If we were to instead sort in descending order via `OrderByDescending`, we'd see something like *Figure 8.12*:

Name	SubPosition	CurrentPay	MarketValue	PercentOfMarket	Goals	Assists	Games	Minutes	GoalsLastSeason	AssistsLastSeason	BirthYear
Earl Ritchie	Left Winger	3500000	1111025.1	315.02438	4	5	37	2391	1	2	1998
Otis Becker	Left Winger	3500000	1111025.1	315.02438	4	3	29	1574	10	4	1997
Nelson Schoen	Right Winger	3500000	1111025.1	315.02438	4	1	25	1754	2	4	2001
Randolph Ebert	Centre-Forward	3500000	1111025.1	315.02438	9	2	23	1753	<null>	<null>	1998
Francis Wuckert	Right Winger	22000000	7721855	284.90564	10	6	31	2510	1	4	1994
Carl Wilderman	Right Winger	3000000	1111025.1	270.0209	1	1	20	1105	<null>	<null>	1997
Gerard Gulgowski	Centre-Forward	3000000	1111025.1	270.0209	6	0	32	1500	1	0	2001
Karl Bartell	Centre-Forward	3000000	1111025.1	270.0209	4	0	10	729	<null>	<null>	1996
Douglas Ward	Centre-Forward	3000000	1111025.1	270.0209	0	0	12	548	<null>	<null>	2003
Harry Witting	Centre-Forward	3000000	1111025.1	270.0209	6	0	20	1301	<null>	<null>	1999

Figure 8.12 – Attackers our model predicts are significantly overpaid

While most of these footballers are scoring regularly, the model questions exactly how much the players are being paid.

Common values

You may notice some common repeating values in `MarketValue` and `PercentOfMarket` in *Figures 8.11* and *8.12*. This can be a characteristic of tree-based models such as a decision tree or random forest, as we'll see in the next chapter.

The pay for many of these players does look legitimate, but Francis Wuckert jumps out as unusual at a pay of 22 million euros for an output of 10 goals after a previous season that only saw a single goal. Clearly, 10 goals in a season is remarkable and far more than I could achieve, but Francis may be overvalued and his team may want to consider trading him for a more economical player or set of players.

We've now successfully trained, evaluated, and applied a regression model using ML.NET AutoML and cross-validation.

Summary

In this chapter and the previous one, we explored the basics of ML.NET using AutoML to perform classification and regression problems. We saw how AutoML lets us define the problem we want to solve and the data to solve it, then goes to work evaluating different candidate models until the best model is found within the time given to evaluate models.

While classification focuses on sorting data into different categories, regression's focus is on predicting a single numerical value – in the case of this chapter, that was the market value of a player in euros.

Like classification, regression models involve metrics. These metrics focus on the average amount of error (sometimes referred to as loss) that is present in the model's predictions.

We saw how MAE is a simple measurement used to show how close our model usually gets, while RMSE can give us a similar measurement that is more profoundly affected by large errors in the model's predictions for a few rows. R squared is also a very helpful measurement for determining how much of the variance in the data is explained by the model.

In this chapter, we introduced cross-validation, whereby AutoML split our data into multiple folds automatically and generate metrics and models for each fold. This approach helps protect against scenarios where important aspects of the training data get sorted into the test set and are not represented in the training set.

In the next chapter, we'll take more control of ML.NET as we move beyond AutoML and enter the world of defining our own pipelines and selecting our own training algorithms.

Further reading

Here are a few extra resources related to things we covered in this chapter:

- *ML.NET tasks*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/tasks#regression>
- *Regression metrics*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/metrics#evaluation-metrics-for-regression-and-recommendation>

- *Loss function calculation in ML.NET*: <https://github.com/dotnet/machinelearning/blob/feadc4362f648dc9e2f919b3dee447b91a670f3d/src/Microsoft.ML.Data/Evaluators/RegressionEvaluatorBase.cs>
- *The importance of the permutation feature*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/explain-machine-learning-model-permutation-feature-importance-ml-net>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/Regression/>

Beyond AutoML: Pipelines, Trainers, and Transforms

We've now covered regression and classification using ML.NET's AutoML experiments API. In the previous two chapters, we focused more on the task that we wanted to accomplish than the actual process of the machine learning logic, and with good reason – there's a lot to learn when learning about machine learning.

In this chapter, we'll lift the veil on what AutoML does by recreating our regression experiment from *Chapter 8* without using AutoML at all. This will show us the important role of pipelines, trainers, and transforms in the model training process.

Once we have a firm understanding of ML.NET without AutoML, we'll see how advanced AutoML experiments can be configured by using AutoML pipelines. We'll see how this approach gives you more fine-grained control of the training process while still taking advantage of AutoML's hyperparameter tuning abilities.

Along the way, we'll talk through common transformations as well as common classification and regression algorithms. This chapter will help you increase your comfort and competency in ML.NET and learn more about what's going on under the hood when we use AutoML.

In this chapter, we'll cover the following topics:

- Performing regression without AutoML
- Creating an AutoML pipeline
- Controlling AutoML pipelines
- Customizing hyperparameter tuning
- Scaling numeric columns
- Selecting regression algorithms
- Selecting classification algorithms

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter09` folder, along with the chapter's dataset.

Performing regression without AutoML

This chapter will continue the same regression experiment from *Chapter 8* that focuses on predicting market values for footballers based on their stats.

We'll start by bringing in ML.NET, AutoML, and our `DataFrame`, as well as a number of imports we'll need in this chapter. We'll also declare our `MLContext` and give it a random seed for repeatability:

```
#r "nuget:Microsoft.ML,3.0.1"
#r "nuget:Microsoft.ML.AutoML,0.21.1"
#r "nuget:Microsoft.Data.Analysis,0.21.1"
using Microsoft.Data.Analysis;
using Microsoft.ML;
using Microsoft.ML.Data;
using Microsoft.ML.AutoML;
using Microsoft.ML.AutoML.CodeGen;
using Microsoft.ML.Trainers;
using Microsoft.ML.Trainers.FastTree;
using Microsoft.ML.Transforms;
using Microsoft.ML.SearchSpace;
using Microsoft.ML.SearchSpace.Option;
using Microsoft.ML.Runtime;
MLContext context = new(seed: 123);
```

Next, we'll load up our data into a `DataFrame` and split it into training and test sets:

```
DataFrame df = DataFrame.LoadCsv("training.csv");
DataOperationsCatalog.TrainTestData split =
    context.Data.TrainTestSplit(df,
        testFraction: 0.2,
        seed: 123);
```

Finally, we'll perform a simple experiment using ML.NET AutoML to train a model, with the bare minimum of code:

```
string label = "CurrentPay";
uint maxSeconds = 10;
context.Auto().CreateRegressionExperiment(maxSeconds)
    .Execute(split.TrainSet,
            split.TestSet,
            labelColumnName: label)
```

This C# code cell creates and executes the experiment and then displays its result below the cell.

I'll not bore you with full result details; suffice to say that this evaluated 10 models when I ran it, where the best run used a `FastForestRegression` trainer and had a **mean absolute error (MAE)** of 2.38 million euros and an R Squared value of 0.83. In other words, this is very similar to the model we saw at the end of the previous chapter. Now, let's see how we could train a similar model, but without using AutoML, to identify the best model trainer and hyperparameters.

To start with, we'll need to identify the features used in our model.

Features and pipelines

ML.NET relies on a concept called **pipelines** where you connect together a series of transform operations (called **transforms**) to manipulate data in a repeatable way.

For example, you could create a pipeline that takes its input data, replaces missing values, one-hot-encodes a categorical column, extracts features from text, concatenates all feature columns together, and then applies a **Fast Forest regression trainer** to generate a fitted model. This process is illustrated in *Figure 9.1*:

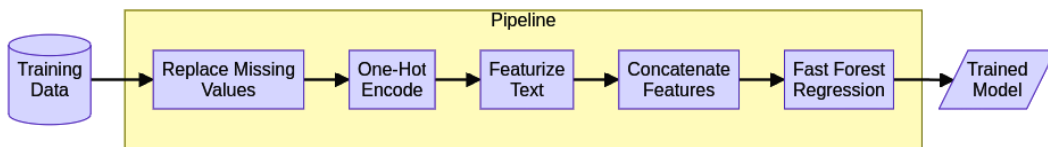


Figure 9.1 – A model training pipeline example

This fitted model could then be used to generate predictions, as we saw in previous chapters. The pipeline that was used to train the model is reusable and expandable. This lets us so potentially run the same pipeline with a different dataset, resulting in a similar but slightly different model that flows through the same training steps.

Most pipelines will have at least two discrete steps:

- A **concatenate step** that merges all features of the model (columns that are not labels) into a single “*Features*” column.
- A **model trainer step** that tries to identify patterns in the “*Features*” column, used to predict the label column. In *Figure 9.1*, this is the **Fast Forest Regression** step.

Concatenating together our feature columns into a single “*Features*” column puts our data into a format that is needed by the model trainer we work with.

In order to concatenate together features, we need an array of names of the columns we’re concerned with. If this is a small list, that’s easy to generate, but for us, this is going to be around 20 columns.

Because we’re dealing with a non-trivial number of columns, let’s use code to identify them by filtering down to only numeric columns in our `DataFrame`:

```
string[] numericColumns = df.Columns
    .OfType<SingleDataFrameColumn>()
    .Where(c => c.Name != label)
    .Select(col => col.Name)
    .ToArray();
```

Here, we use LINQ to find all of our `float` columns (recall that floats are also referred to as *singles*), select their names, and exclude the name of our label column. Removing the label from the set of feature columns is important because, otherwise, models would be able to “cheat” during the training process. This would result in a worthless model, since it would require knowing the value it was trying to predict ahead of time.

Finally, we put the entire set of strings into an array, which the `Concatenate` transform will require.

Next, we can create a new pipeline, starting with our `Concatenate` pipeline:

```
var pipeline = context.Transforms.Concatenate(
    outputColumnName: "Features",
    numericColumns);
```

This pipeline will take whatever data you give it and concatenate columns mentioned in `numericColumns` into a single “*Features*” column. After this, we’ll update our pipeline by appending our model trainer to the end of it:

```
pipeline = pipeline.Append(
    context.Regression.Trainers.FastTree(
        labelColumnName: label,
        numberOfLeaves: 20,
        numberOfTrees: 100,
        minimumExampleCountPerLeaf: 10,
```

```
learningRate: 0.2)  
);
```

The `Append` method always creates and returns a new pipeline instead of modifying the original pipeline. As a result, it's important to store the produced pipeline back in the pipeline variable.

Hyperparameters

Remember in *Chapter 7, Classification experiments with ML.NET AutoML*, how we talked about how each model training algorithm has hyperparameters? This is where those come in. `numberOfLeaves`, `numberOfTrees`, `minimumExampleCountPerLeaf`, and `learningRate` are all **hyperparameters** to the `FastTree` regression model trainer. Different values for these hyperparameters will result in different models with different performance characteristics. Finding the optimal set of values for these hyperparameters is known as **hyperparameter tuning**, which we'll explore later in this chapter.

With the model trainer included, our pipeline now resembles the one in *Figure 9.2*:

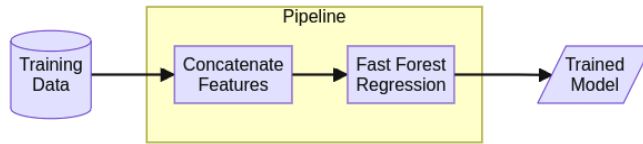


Figure 9.2 – Our simple pipeline

We can train our model by fitting our pipeline to our training data, by calling its `Fit` method:

```
ITransformer model = pipeline.Fit(split.TrainSet);
```

This takes a half second or so to train with this volume of data, but it produces our trained model using the fast tree algorithm.

We can evaluate how good our model is by having that same pipeline transform our testing data, and then using the transformed test data to evaluate the model and produce metrics.

Why transform the test data?

A pipeline expects data to be in a certain shape and format as it flows through to the trainer. If your test data skipped this flow, it wouldn't be in the expected format, so the model couldn't generate predictions for it. If we can't generate predictions, then we can't grade how effective the models are.

Thankfully, transforming and then evaluating our test data only involves a few lines of code:

```
IDataView transformed = model.Transform(split.TestSet);
context.Regression.Evaluate(transformed,
                             labelColumnName: label)
```

When this cell is run in our notebook, it completes with an MAE of 3.1 million euros and an R Squared value of 0.79. In other words, our manually trained model generally performs worse than the one AutoML found for us.

This makes sense when you think about it – AutoML evaluated 10 different models using different sets of hyperparameters and identified the best model to keep after those models were trained.

Predicting values

If you're curious how `PredictionEngine` from the last two chapters worked, it takes in data and calls `model.Transform` to generate the resulting data, including a `Score` column that represents the prediction.

For example, the following code generates predictions for every row in the test set – `model.Transform(split.TestSet).ToDataFrame()["Score"]`. Most of the time, we prefer to work with `PredictionEngine` for convenience, but it's helpful to understand that `PredictionEngine` just transforms your data through the trained model to get its predictions.

It's possible that our manually trained model might perform better with different hyperparameters and, potentially, even outperform what the AutoML solution was able to find in 10 tries. However, we'd have to perform a number of experiments to figure out the ideal combination of hyperparameters.

Keep in mind that this regression model only has access to numeric features, while the AutoML model also has access to text and categorical columns, such as a player's position and sub-position. This is important information that likely can help improve the performance of our model.

We could manually add one-hot encoding and text featurization transformers to our categorical and text columns, but AutoML pipelines have something that will do this for us.

Let's take a look at AutoML pipelines next.

Creating an AutoML pipeline

We've now seen the two extremes of setting up a model training run – on the one hand, we could rely on AutoML experiments to manage the details of training for us and make the best guesses about our data and intentions. Doing this gives up control and may result in needing more time to find an optimal model.

At the other extreme, we can train a model manually, where we must select the exact model trainer and optimal hyperparameters – plus set up any needed transforms to get our data ready for that trainer.

Between these two options, we have another form of AutoML : AutoML with pipelines. By using pipelines like a manual model training process, we retain control over how our data is transformed and interpreted, but we still get the use of AutoML's power for finding the optimal model trainer and hyperparameters.

AutoML pipelines are more code than the AutoML experiments we've seen previously, but they also give you more insight and control over what happens during the model training process.

Let's see how they work:

```
var pipeline = context.Auto()  
    .Featurizer(df, excludeColumns: [label, "Name"])  
    .Append(context.Auto().Regression(labelColumnName: label));
```

We start by declaring a pipeline as we did before, but this time, we use `Auto().Featurizer` instead of `Transforms.Concatenate`, and `Auto().Regression` instead of a specific regression model trainer.

`Featurizer` is effectively a combination of one-hot encoding, missing value replacement, text featurization, and column concatenation. We explicitly tell it that we don't want to include the name of the player as a feature (for fear our model will be biased toward players with similar names) or the current pay of the player that we want to predict.

Text featurization

`Featurizer` relies on **Natural language processing (NLP)** which is a field of machine learning that extracts additional information from text columns by looking for relationships between different sequences of words or word parts. NLP is an advanced topic beyond the scope of this book, although there are a number of books dedicated to the topic from various publishers.

`Auto().Regression` is an interesting step in the pipeline that effectively means, *"This is where I want you to put a regression model trainer and its hyperparameters."* In other words, it's a placeholder for one of the different candidate regression trainers that AutoML might try.

What about classification?

As you might imagine, `Auto().Regression` isn't the only special transformer. We also have `Auto().BinaryClassification` and `Auto().MultiClassification` for different forms of classification pipelines.

There are many ways of customizing both the `Featurizer` and `Regression` pipeline steps, but let's keep it simple as we build our first AutoML pipeline.

Now that we have the pipeline ready to go, let's use it to run an AutoML experiment:

```
AutoMLExperiment experiment = context.Auto()
    .CreateExperiment()
    .SetPipeline(pipeline)
    .SetDataset(split)
    .SetRegressionMetric(RegressionMetric.MeanAbsoluteError,
        labelColumn: label)
    .SetTrainingTimeInSeconds(10);

TrialResult result = await experiment.RunAsync();
ITransformer model = result.Model;
```

This code creates `AutoMLExperiment` and then sets it to use our pipeline and dataset. We then tell it we want to judge models by their best MAE and what column to use as the label. Finally, we specify the standard 10-second training time limit chosen for expediency in this book's demos.

Next, we take our experiment and run it to get `TrialResult`. The result has information on the different models evaluated as well as our best resulting model, which we store in the `model`.

Caution

At the moment, trying to display a `TrialResult` object in Polyglot Notebooks by ending a cell with it will produce `NullReferenceExceptions` deep within the Polyglot Notebook formatting code. This is an implementation detail of properties on this object and not a sign that you have done something wrong.

When I execute this code, I see the results shown in *Figure 9.3*:



▼ [Microsoft.ML.Transforms.MissingValueReplacingTransformer, Microsoft.ML.Transforms.OneHotEncodingTransformer, Microsoft.ML.Data.ColumnConcatenatingTransformer, Microsoft.ML.Data.ReggressionPredictionTransformer<Microsoft.ML.Trainers.FastTree.FastForestRegressionModelParameters>]	
LastTransformer	► Microsoft.ML.Data.ReggressionPredictionTransformer<Microsoft.ML.Trainers.FastTree.FastForestRegressionModelParameters>
(values)	
index	type
0	Microsoft.ML.Transforms.MissingValueReplacingTransformer
1	Microsoft.ML.Transforms.OneHotEncodingTransformer
2	Microsoft.ML.Data.ColumnConcatenatingTransformer
3	Microsoft.ML.Data.ReggressionPredictionTransformer<Microsoft.ML.Trainers.FastTree.FastForestRegressionModelParameters>

Figure 9.3 – Our trained AutoML model details

It looks like `Featurizer` added `MissingValueReplacingTransformer`, `OneHotEncodingTransformer`, and the standard `ColumnConcatenatingTransformer` when featurizing the data. Then, it looks like it settled on a Fast Forest regression model as our model trainer.

Here, our experiment substituted our “placeholder” `Featurizer` and `Regression` pipeline steps with actual concrete steps that it detected were best for our data. Let’s see how good this model was by using the standard `Transform` and `Evaluate` approach:

```
IDataView transformed = model.Transform(split.TestSet);
context.Regression.Evaluate(transformed,
                             labelColumnName: label)
```

This takes us back down to 2.3 million euros for MAE and 0.839 for R Squared.

While we likely could do better if we gave our experiment more time, let’s explore how customizing our pipeline can help make AutoML more efficient.

Controlling AutoML pipelines

In this section, we’ll explore a few ways we can improve our `Featurizer` and `Regression` calls in our pipeline by telling AutoML more about our data and what we want it to try.

We’ll start with `Featurizer`.

Customizing the Featurizer

Right now, we’re getting the default behavior, where `Featurizer` will use all of our columns except for the name and label column. In this setup, the `Featurizer` has to guess what each column means.

However, `Featurizer` also lets us tell it more about our data. It does this by giving us parameters that let us explicitly tell it which columns are numeric, text, or categorical. Actually, we can figure these values out from most `DataFrame` objects by looking at the column types.

Let’s start by declaring a small `ColumnData` class to contain our column names:

```
public class ColumnData {
    public List<string> Text {get; set;} = new();
    public List<string> Numeric {get; set;} = new();
    public List<string> Categorical {get; set;} = new();
    public List<string> Excluded {get; set;} = new();
}
```

Then, we’ll declare an instance of `ColumnData` and inform it of our excluded columns:

```
ColumnData info = new();
info.Excluded.Add(label);
info.Excluded.Add("Name");
```

Next, we'll loop over every column in our `DataFrame` and add it to the appropriate property in our object:

```
foreach (var col in df.Columns) {
    if (info.Excluded.Contains(col.Name)) continue;
    Type t = col.GetType();
    if (t == typeof(StringDataFrameColumn)) {
        if (col.ValueCounts().Rows.Count < 15) {
            info.Categorical.Add(col.Name);
        } else {
            info.Text.Add(col.Name);
        }
    } else if (t == typeof(BooleanDataFrameColumn) ||
               t == typeof(PrimitiveDataFrameColumn<bool>)) {
        info.Categorical.Add(col.Name);
    } else {
        info.Numeric.Add(col.Name);
    }
}
```

This code loops over each column and adds it to the appropriate property, depending on the specific column type. For string columns, we check the number of unique values for the column and treat the column as categorical if there are less than 15 distinct values for the column; otherwise, we treat it as a text column.

When executed against our `DataFrame`, this code sorts the columns into the appropriate properties, as shown in *Figure 9.4*:

▼ **Submission#19+ColumnData**

Text [BirthCountry]

Numeric [Height, HighestPay, YellowCards, RedCards, Goals, Assists, Minutes, Games,

Categorical [SubPosition, Position]

Excluded [CurrentPay, Name]

Figure 9.4 – Our columns by type. Some numeric columns are not shown

Now that we have four lists of string values, we can use these to start our pipeline with `Featurizer`:

```
SweepablePipeline basePipeline = context.Auto()
    .Featurizer(df,
        numericColumns: info.Numeric.ToArray(),
        textColumns: info.Text.ToArray(),
        catelogicalColumns: info.Categorical.ToArray(),
        excludeColumns: info.Excluded.ToArray());
```

As before, we call `context.Auto().Featurizer`, and we provide it our `DataFrame`. However, this time, we explicitly tell it how it should treat each column. This helps our `Featurizer` choose the optimal transforms, given the nature of our data.

This time, the pipeline is stored in a new variable, named `basePipeline`. This will be helpful for us later if we want to rely on this variable as the basis for a pipeline without recreating the beginning of one.

Note that `context.Auto().Featurizer` returns `SweepablePipeline`, which is a special form of pipeline unique to AutoML tasks. When you have a `SweepablePipeline`, you are able to perform hyperparameter tuning and use AutoML experiments. With our `Featurizer` configured, let's move on to customizing the model trainer.

Customizing the model trainer selector

Next, let's take a look at how we can enable and disable specific trainers from consideration when performing our regression experiment.

Let's start by expanding `basePipeline`, which we created in the last step, with a call to `context.Auto().Regression`:

```
SweepablePipeline pipeline = basePipeline.Append(
    context.Auto().Regression(
        labelColumnName: label,
        useFastTree: true,
        useFastForest: true,
        useLgbm: false,
        useLbfgsPoissonRegression: false,
        useSdca: false)
);
```

This time, we explicitly tell the `Regression` step which trainers it should consider. Since the model has consistently selected either `FastTree` or `FastForest` trainers (both forms of decision trees, as we'll discuss later), let's encourage that by disabling other options so that AutoML tries more experiments that focus on these areas.

Now, if we repeat our experiment with our new pipeline, we can see that AutoML produces the following training pipeline:

1. Replace missing values with `MissingValueReplacingTransformer`.
2. One-hot-encode with `OneHotEncodingTransformer` for `Position` and `SubPosition`.
3. Text featurization for `BirthCountry` with `TextFeaturizingEstimator`.
4. Build a "Features" column with `ColumnConcatenatingTransformer`.
5. Train a regression model with a `FastForest` trainer.

The resulting metrics have also improved, decreasing from 2.3 to 2.2 million euros for MAE and increasing to 0.86 for R Squared – an increase of 0.03!

Customizing hyperparameter tuning

In this section, we'll see how AutoML can be used to tune a specific algorithm's hyperparameters by following a search algorithm, to iteratively try different values for those hyperparameters.

If we consider each hyperparameter that a model trainer has, we can see that a hyperparameter typically has a range of valid values from some minimum amount to some maximum amount. You can visualize this as a slider on a user interface where hyperparameter tuning involves trying to find the best value for a setting.

Most model trainers have multiple hyperparameters, so you're not just searching one hyperparameter for the best value; instead, you're searching for an optimal set of values between your different hyperparameters.

Understanding the search space

This set of possible combinations of values is referred to as a search space, where your job as a data scientist is to find the right combinations of hyperparameter values to generate the optimal metrics.

It may be helpful to visualize a topographical map, such as the one depicted in *Figure 9.5*:

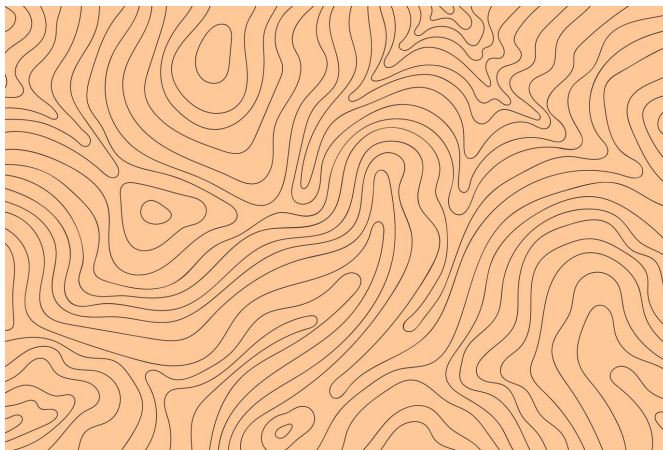


Figure 9.5 – A topographical map (picture from Freepik)

In a topographical map, different points on it have different elevations. Your job is to tweak your position on the map to find the peak of the highest hill. Finding the highest point may take some time and a coordinated search strategy, as topographies have many hills with local maxima (i.e., points that are higher than the things around them) that are not the global maxima – the most efficient point overall.

Thankfully, AutoML can help us with our search.

Since we discovered that a `FastForest` model trainer worked well in the last section, we'll spend this section trying to find the best set of hyperparameters we can find for our `FastForest` model trainer.

We'll start by declaring our search space for a `FastForest` model trainer:

```
SearchSpace<FastForestOption> searchSpace = new();
searchSpace
```

This helpfully displays the hyperparameters present in that search space, as shown in *Figure 9.6*:

key	type	value
NumberOfTrees	Microsoft.ML.SearchSpace.Option.UniformIntOption	▼ Microsoft.ML.SearchSpace.Option.UniformIntOption Min4 Max32768 LogBaseTrue FeatureSpaceDim1 Step[<null>] Default[0]
NumberOfLeaves	Microsoft.ML.SearchSpace.Option.UniformIntOption	▼ Microsoft.ML.SearchSpace.Option.UniformIntOption Min4 Max32768 LogBaseTrue FeatureSpaceDim1 Step[<null>] Default[0]

Figure 9.6 – The search space for our `FastForest` trainer

As you can see, there's a `NumberOfTrees` hyperparameter and a `NumberOfLeaves` hyperparameter. Both of these customize how a random forest operates.

Although we'll talk more about `FastForest` in our next section, it'd be helpful to pause for a moment and talk briefly about what `FastTree` and `FastForest` are.

A `FastTree` model is a form of **decision tree**.

Decision trees are effectively a mess of `if/else` statements, based on ways of dividing your data into halves to find the optimal value. For example, if players who score over five goals in a season get significantly different pay than other players, the decision tree will add that as a root `if` statement and have a branch for whether that's true or whether it's false.

Let's say, in a simplified example, that players who get over 5 goals in a season earn 12 million euros if they play over 20 games in that season; otherwise, they earn 8 million euros. Conversely, if they score 5 goals or less, attackers get 4 million euros and all other players get 2 million euros.

A depiction of such a decision tree is shown in *Figure 9.7*:

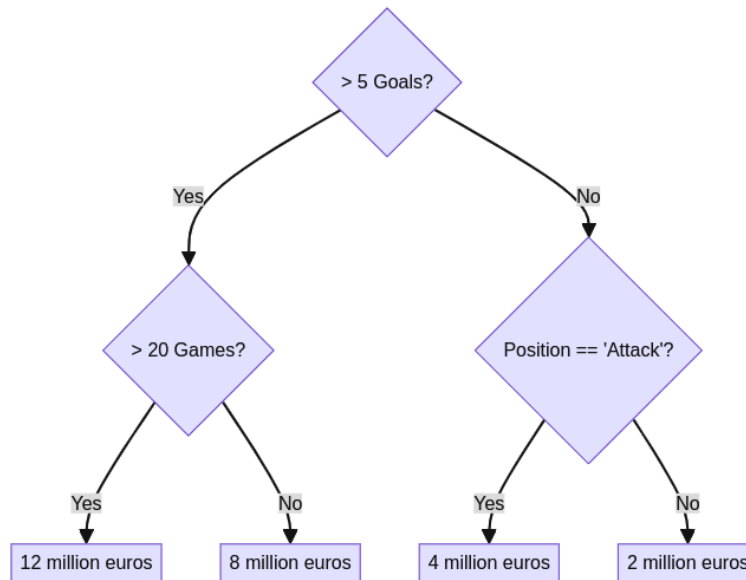


Figure 9.7 – A simple decision tree

If a decision tree is like a tree of auto-generated `if/else` statements, a random forest is a collection of different decision trees that have been grown independently. In a random forest, each decision tree is asked for its estimate, and the overall estimates are averaged together, based on the relative weight of the tree in terms of importance.

Random forests are an example of an ensemble form of machine learning where there are multiple mini-models, and they are asked for their opinions, which are collected together and averaged into a final estimation.

In our search space, the `NumberOfTrees` hyperparameter refers to the number of decision trees to include in our random forest, while the `NumberOfLeaves` parameter talks about the complexity of each decision tree.

A warning on decision trees

Decision trees are easy to understand and tend to be fast, but if you allow them to be complex enough, they tend to overfit their data by memorizing all of the aspects of their training data, and they are less equipped to evaluate new data points, such as their test data. When training a decision tree, be wary of performance that is too good, as it often is a sign that you've allowed a tree to get too complex and overfit its training data.

With this understanding of decision trees and random forests in hand, let's look at customizing the default search space to make it compatible with our scenario.

Customizing the search space

The default values for our search space allow for several trees and several leaves, each between 4 and 32,768. Having 32,768 nodes on an individual decision tree seems like a recipe for inviting our model to overfit, and our data isn't complex enough that we need many decision trees to get a high degree of accuracy.

Let's customize our `SearchSpace` by swapping out the defaults with values that make more sense for our model:

```
searchSpace["NumberOfTrees"] =  
    new UniformIntOption(1, 12, defaultValue: 4);  
  
searchSpace["NumberOfLeaves"] =  
    new UniformIntOption(10, 100, defaultValue: 30);
```

Here, we customize our two hyperparameters by stating that there should be between 1 and 12 trees in our forest, and each tree should have between 10 and 100 nodes. We also provide default values for each hyperparameter that seem like good starting points.

A hint on setting search spaces

Using Polyglot Notebooks, you can view the transformers for any model AutoML comes up with. Often, you can expand the specific model trainer and get a glimpse at some of the hyperparameters it settled on. This is often helpful for determining appropriate scales for values that you're not familiar with.

Next, let's incorporate our search space into our pipeline by manually creating a `FastForest` trainer and appending it to our base pipeline (the starting point we created in our `Featurizer` step):

```
var estimator = context.Auto().CreateSweepableEstimator(  
    (context, options) =>  
        context.Regression.Trainers.FastForest(  
            labelColumnName: label,  
            numberOfLeaves: options.NumberOfLeaves,  
            numberOfTrees: options.NumberOfTrees,  
            minimumExampleCountPerLeaf: 11  
        ), searchSpace);  
SweepablePipeline pipeline = basePipeline.Append(estimator);
```

Here, we create a `SweepableEstimator` by providing an anonymous function that takes in `MLContext` and an `options` object. Because our `searchSpace` variable is `SearchSpace<FastForestOption>`, our `options` variable will also be `FastForestOption`. We then take the values for our two hyperparameters out of the options and provide them to our `FastForest` model trainer.

Finally, we append our new estimator to our `basePipeline` to create a new pipeline, with the estimator at the end of it.

We then train and evaluate as we have before and come up with an MAE of 2.25 million euros and an R Squared value of 0.865.

If we were to visualize the resulting model and expand its transformer several times, we'd see that it resulted in 7 trees with 27 leaves per tree, as shown in *Figure 9.8*:



Figure 9.8 – Our hyperparameters applied to the trained model

Does this mean that 27 is the ideal number of leaves and 7 is the ideal number of trees? Maybe. It could also mean that we ran out of time before finding the best option.

While we can always change our search space and try again or give the training process more time, another approach is to customize the search process by specifying a hyperparameter tuner.

Customizing the hyperparameter tuner

The search process involves **hyperparameter tuning** by selecting a set of values within the search space, observing the resulting metrics, and then adapting our next set of hyperparameters based on this approach.

The hyperparameter tuning process is carried out by a specific tuning algorithm, which implements ML.NET's `ITuner` interface.

There are currently five supported tuner methods in ML.NET's default set of packages, although you can also provide your own implementation or potentially find more through other NuGet packages.

The simplest tuners involve picking values at random with `RandomSearchTuner` or the more structured `GridSearchTuner`, which tries values at regularly distributed intervals but doesn't adjust its plans to observed metrics.

More advanced tuners get more complex, such as the **sequential model-based algorithm configuration (SMAC)** tuner, which uses Bayesian mathematics to adjust its hyperparameter selections, based on observed metrics from previous guesses.

Additionally, ML.NET offers the Cost Frugal and **expected conditional improvement (ECI) Cost Frugal** tuners, which both respond to observed data while being conservative with their use of processing power.

The default tuner for AutoML is `EciCostFrugalTuner`, but you can customize the tuner in use by calling the various set tuner methods on the experiment, such as the `SetSmacTuner` call shown here:

```
AutoMLExperiment experiment = context.Auto()
    .CreateExperiment()
    .SetPipeline(pipeline)
    .SetDataset(split)
    .SetRegressionMetric(RegressionMetric.MeanAbsoluteError,
                        labelColumn: label)
    .SetTrainingTimeInSeconds(10)
    .SetSmacTuner();
```

You can also call `SetTuner<YourTuner>()` if you have a custom implementation of `ITuner` to provide.

To help illustrate the differences in these Tuners, I ran the pipeline with various tuners on their default settings. I did this in training time intervals of 10 seconds, 30 seconds, and 2 minutes and recorded the following MAE values in terms of millions of euros:

Tuner method	10-second MAE	30-second MAE	120-second MAE
SetRandomSearchTuner	1.94	1.88	1.87
SetGridSearchTuner	2.05	1.92	1.92
SetSmacTuner	1.88	1.87	1.87
SetCostFrugalTuner	1.75	1.71	1.70
SetEciCostFrugalTuner	2.27	2.16	1.64

Table 9.1

As you can see, the choice of a search algorithm made a significant difference, both in terms of how well the algorithm started out with this dataset and how much improvement the tuner was able to make. These trends are graphically illustrated in *Figure 9.9*:

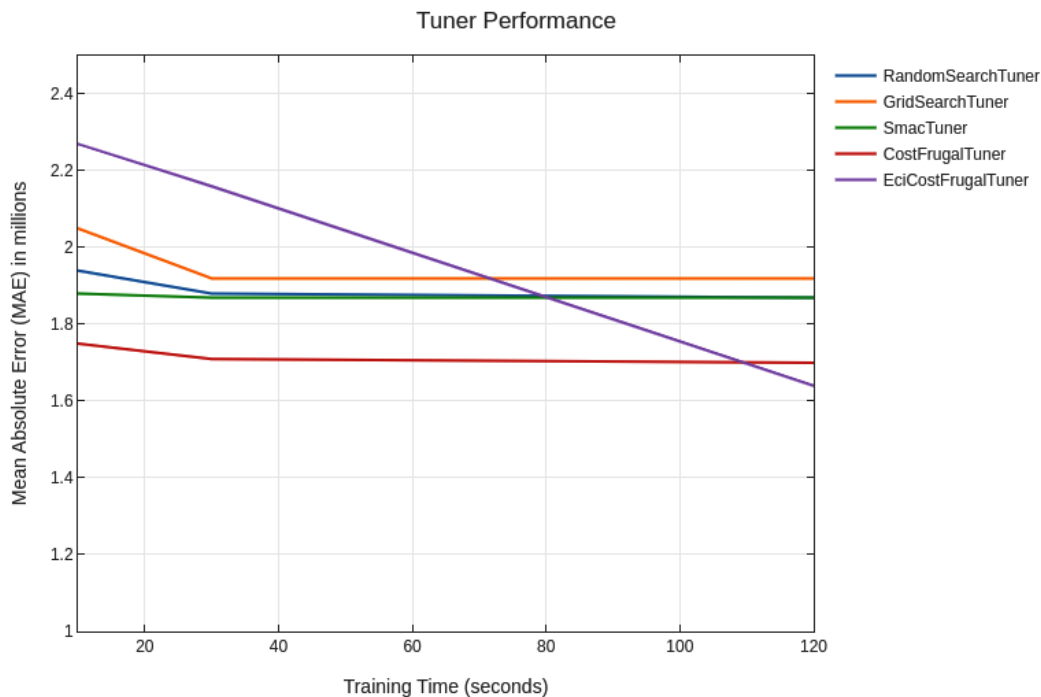


Figure 9.9 – Tuner performance over time

While `CostFrugalTuner` started out the best on this dataset, it had a hard time finding subsequent improvements, while `EciCostFrugalTuner` and other tuners were able to improve over time.

Notably, `GridSearchTuner` and `SmacTuner` seem to have hit barriers to continual performance improvements. This is likely due to their parameter configuration.

For example, `GridSearchTuner` defaults to setting a grid of 10 unit increments when evaluating different options. In our somewhat narrow search space, a 10-minute increment is too large, and this setting prevented the tuner from finding more optimal hyperparameter combinations, whereas a setting such as `.SetGridSearchTuner(step: 2)` could have improved its performance by expanding the range of options it considered.

It's important to note that some tuners will perform better in the short term versus the long term. For example, given enough time, `RandomSearchTuner` will inevitably find the optimal set of hyperparameters, but it is hardly an optimal algorithm!

Additionally, some algorithms will perform better over different values for your search spaces. It's up to you to tweak the settings to determine what your best options are for the hyperparameters of the model you're training, as well as what level of good is "good enough" to stop hyperparameter tuning.

Before we move on to discussing the different model trainers, let's talk about one last important aspect of model training – scaling numerical values.

Scaling numeric columns

Recall in this chapter that we saw initial successes with the `FastTree` decision tree and `FastForest` random forest model trainers. There are a few reasons for that.

First, decision trees and random forests are flexible and can generate a good set of rules for different datasets. Secondly, these algorithms are not adversely affected by unscaled data.

Our dataset is not currently scaled – that is, goals per player might range from 0 to 20 over the course of a season, while minutes may range from 0 to thousands of minutes in a single season.

Some machine learning algorithms have trouble distinguishing between columns when different columns use different scales. As a result, these algorithms tend to focus on very large positive or negative values in one column, focusing less on other columns that don't see as many large values.

Let's see the problem in action by creating a pipeline from our `basePipeline` (`Featurizer`), mixed with a `Regression` trainer set to use the `LBFGS Poisson Regression` model:

```
SweepablePipeline pipeline = basePipeline.Append(  
    context.Auto().Regression(  
        labelColumnName: label,  
        useFastTree: false,  
        useFastForest: false,  
        useLgbm: false,  
        useLbfgsPoissonRegression: true,  
        useSdca: false)  
);
```

If we run an experiment using this pipeline and get its metrics, we can see that the MAE is a whopping 6.03 million euros and the R Squared value is a shocking -0.18! In other words, such a model is somehow *worse* than guessing at random!

This is because the `LBFGS Poisson Regression` is sensitive to scaling issues, and our data is unscaled. To fix this, we need to make sure each numerical column is scaled appropriately. There are many different ways of scaling values, but we'll keep it simple and stick with min/max scaling, which ensures each numerical column is represented as a value between 0 and 1. Let's look at our `Minutes` column as an example of how simple min/max scaling works.

The `Minutes` column has values ranging from 1 minute in a season to 5,174 minutes. Without scaling, those numbers would go directly to our model trainer. But with scaling, we determine the range of values by subtracting the minimum value from the maximum value. We can then take any data point and figure out where it falls on the scale.

Let's say a player played an even 2,000 minutes in a season. We'd take that 2,000 and divide it by the range of our values. That range is 5,173 values (our maximum value of 5,174 – our minimum value of 1), so that'd be $2,000 / 5,173$. This calculates to about 0.39, so the `Minutes` value that went to the model trainer would be 0.39 for that row.

Because all of our numerical columns would be scaled by a similar transformation, we'd know that values near 0 are values near the minimum value, while values near 1 are near the maximum for that column.

Let's see this in action by starting with our min/max transformer:

```
InputOutputColumnPair[] pairs =
    info.Numeric.Select(c => new InputOutputColumnPair(c, c))
                .ToArray();
var minMax = context.Transforms.NormalizeMinMax(pairs);
```

When you call `NormalizeMinMax`, you can either give it the name of a single input column and a single output column, or an array of `InputOutputColumnPair` objects containing multiple input and output column mappings.

Because we have many numeric columns, I elected to go with the array approach and tell the min/max transformer to take the value of each numeric column, transform it, and then store it back in that column.

With our transform created, let's add it to the beginning of a new pipeline and place the existing pipeline with `Featurizer` at the end of that pipeline:

```
SweepablePipeline scaledPipeline = minMax.Append(pipeline);
```

This way, numerical values will be scaled before the `Featurizer` concatenates columns together into a single “*Features*” column.

When I run an experiment involving this new pipeline using the same training parameters, I see an MAE of 3.65 million euros and an R Squared value of 0.13. While these values are still quite bad, we can see that the simple act of scaling our numerical data allowed the LBFGS Poisson Regression algorithm to nearly halve its error, moving from a negative R Squared value to a positive value.

While this model training algorithm isn't ideal for this particular dataset, the point here is that scaling your data appropriately can remove obstacles to success for your trainers and is frequently a good practice – so good of a practice that I'm surprised AutoML doesn't automatically try scaling data in its `Featurizer` in the current implementation.

Speaking of machine learning algorithms, let's close this chapter with a discussion of the various algorithms available to us in ML.NET.

Selecting regression algorithms

By default, ML.NET supports the following regression model trainers:

- **FastTree**: A decision tree algorithm, which we discussed earlier in this chapter. Decision trees are fast, simple, and easy to understand, and they are not sensitive to scaling. However, if you allow them a high number of leaves, they may overfit the training data at the expense of accuracy on data they've not seen.
- **FastTreeTweedie**: Almost identical to **FastTree** but built for a specific distribution of data, referred to as a **Tweedie distribution**. See the *Further reading* section for more details.
- **LightGbm**: Uses **gradient boosting** and Microsoft's **LightGbm** framework for high-performance decision trees.
- **FastForest**: A random forest algorithm. This approach uses a **voting ensemble** of decision trees, with each tree's prediction constituting part of the overall "vote" for the final prediction.
- **LbfgsPoissonRegression**: A linear algebra approach to predicting values by creating a linear model of the relationship between data points. Poisson regression works well for count-based predictions, such as the number of occurrences or events.
- **Ols**: Another linear algebra approach, this time involving **ordinary least squares (OLS)**. This works well with smaller datasets but requires scaled data.
- **OnlineGradientDescent**: A high-performance linear algebra approach, where the model attempts to minimize error by gradually adjusting values, almost like following the curvature of the peaks and valleys of hyperparameter tuning discussed previously in this chapter. This works well with larger or more complex datasets.
- **Sdca**: **Stochastic dual coordinate ascent (SDCA)** is a more complex machine learning approach, involving a quadratic function that may better fit non-linear data or large complex datasets.

This list is daunting, and I don't expect you to know it all to perform data science. In fact, I had to look into the details of a number of these trainers while writing this chapter.

My goal in sharing this list isn't for you to memorize the characteristics of each model trainer and to immediately know which ones to consider.

Instead, I want you to have familiarity with the high-level characteristics of each of the regression model trainers built into ML.NET, giving you some things to keep in mind if you choose a particular trainer over another or if ML.NET recommends a model involving that trainer.

Usually, your choice in picking a model trainer boils down to a few factors:

- Do you want to use a tree-based approach or a mathematics-based approach? If your data is non-linear, you should use a tree or forest. If your data can be represented with mathematical equations or you don't want to risk trees overfitting your training data, use a linear or quadratic approach.
- What scale and complexity of data do you have? If you have many rows in your dataset or a large number of columns (as you might when analyzing text or doing a lot of one-hot encoding), you should try one of the more complex algorithms. Otherwise, start with the simplest fastest trainers.

This section is not intended to teach you the mathematics of these algorithms but, rather, to help you understand *some* of your options. ML.NET is built in a flexible way that allows you to include additional model trainers via NuGet packages.

For more in-depth explorations of these trainers, I recommend reading Microsoft's *How to Choose an ML.NET Algorithm* page or additional online resources linked to in the *Further reading* section.

Algorithm choice and scaling

If you choose to use a regression trainer based on linear algebra, you should make sure that your data is appropriately scaled. Decision trees do not require scaling but may be easier to understand without scaling present.

With regression covered, let's close the chapter by talking about classification model trainers.

Selecting binary classification algorithms

This section will be similar to the last; we'll cover the high-level machine learning algorithms supported by ML.NET.

Our focus is not on the mathematics underlying these trainers but on their key differences and distinctions, and also on when you ought to consider using one versus another.

When I first learned the details of machine learning, one of the things that surprised me the most was how similar classification and regression tasks are.

In regression, you're often trying to build a linear function that can calculate a value by multiplying your data columns by different weights, as we saw at the beginning of *Chapter 8, Regression Experiments with ML.NET AutoML*.

Classification works in very similar ways most of the time, where we determine a linear relationship between our data points and then calculate a value.

The difference with classification is that we usually don't care about the exact value we predict but, rather, where that value lives, relative to our known classes.

Let's use an analogy to explain this. In regression, your model is like spinning a globe and stopping it with your finger. Where your finger lands can be represented with an exact value – the latitude and longitude of the point. In classification, you care more about boundaries and belonging to a specific group. This way, you care about whether your finger is over Europe or Asia instead of being interested in an exact point on the globe.

It turns out that machine learning *doesn't* involve spinning a globe, but the metaphor might be helpful – in classification, we typically determine what point we are at in mathematical space, and then we determine what group (also called a **class**) that point belongs to.

With that orientation established, let's get into the details.

ML.NET provides the following classification model trainers:

- `FastTree`, `LightGbm`, and `FastForest`: The same decision tree and random forest algorithms for regression can also work with classification!
- `LinearSvm` and `LdSvm` are two trainers oriented around **support vector machines (SVMs)**, which use linear algebra to try to find a single dividing line in the various features in data to separate one class from another. `LinearSvm` focuses on maximizing the distance between the two classifications to ensure its predictions are accurate, while `LdSvm` tries to reduce the number of dimensions in the data first in order to simplify the problem. Both are simple and effective but may not work well with complex data.
- `SymbolicSgdLogisticRegression`: This is a linear model that uses **stochastic gradient descent (SGD)**, where a mathematical function is used to separate the two types of objects that the binary classification model tries to classify. Think of it as drawing a line in the sand with mathematics, where data on one side belongs to one class and data on the other side belongs to the other.
- `LbfgsLogisticRegression` is similar in nature to `SymbolicSgdLogisticRegression`, but it uses different mathematics to accomplish its task. `LbfgsLogisticRegression` is *calibrated* to generate probabilities of its classification, while that is not the intention of `SymbolicSgdLogisticRegression`.
- `SgdCalibrated` and `SgdNonCalibrated` use the mathematical approach of SGD, which works similarly in concept to the regression `OnlineGradientDescent` trainer but is applied to classification. This is good for linear relationships. The calibrated version is used to generate reliable probabilities for its predictions, while the non-calibrated version does not and is, therefore, faster and less memory-intensive.
- `SdcaLogisticRegression` and `SdcaNonCalibrated` are similar to the `Sdca` regression trainer that uses SDCA, but these trainers apply the algorithm for classification. These trainers are best used for complex non-linear datasets. The `SdcaNonCalibrated` trainer does not rely on calibrated probabilities and may be faster and have less of a memory footprint than `SdcaLogisticRegression`.

- `FieldAwareFactorizationMachine (FFM)`: This is a linear model, useful for detecting interactions between different features. FFM performs well in recommendation scenarios and scenarios where you have many one-hot encoded columns without many data points for each of them.
- `AveragePerceptron`: An older form of a neural network. This is largely used for teaching purposes.
- `Gam`: A **generalized additive model (GAM)** is a linear model that is able to learn non-linear features. This makes it useful for complex datasets.

When choosing a classification model trainer, you need to think about whether or not your data follows linear relationships and how simple your data is.

Most of the time, a simple support vector machine or logistic regression implementation is sufficient for classification use cases, but you may need to try some of the more advanced and esoteric trainers if you deal with complex data, data that doesn't follow linear relationships, or if you have specific performance or memory concerns that may be better suited for specific models.

Keep in mind that the more complex trainers often take longer to converge on an effective solution and can be significantly more difficult to explain. In data science, just like in programming, sometimes simple, explainable, and “good enough for now” is better than a complex solution designed for much more complex use cases.

Summary

In this chapter, we saw how ML.NET gives us different complexity options depending on our needs.

In its simplest form, you can run an AutoML experiment in a single line of code and accept the default pipeline and model trainer options. You can also get more granular control by building your own pipeline but still rely on ML.NET AutoML to tune hyperparameters for you, or select the appropriate model trainer. Finally, you can get complete control by working directly with ML.NET model trainers and not using AutoML at all.

We saw how different model trainers have different hyperparameters, and each hyperparameter has different ranges of values that make sense for it. These combinations of hyperparameter values constitute the search space that you must navigate to find optimal values for your hyperparameters, through hyperparameter tuning.

We discussed a wide range of machine learning algorithms for regression and classification. We also saw how some linear models need their data to be normalized into values from 0 to 1, in order to take full advantage of their column data.

One thing I hope you come away with is that models can almost always be slightly better than they currently are. You can almost always tweak some hyperparameter value somewhere, give a trainer a bit more time, refine the search space, or change the model trainer altogether to get just a bit of extra accuracy.

At some point, you need to decide that what you have is good enough for the task you're trying to accomplish. While choosing the right model trainer, pipeline, and hyperparameter values makes a difference, the biggest improvement you can make is often improving the quality of your training data.

In the next chapter, we'll conclude our discussion of ML.NET by talking about deploying, optimizing, monitoring, and updating ML.NET models in production. Along the way, we'll also discuss multi-class classification, the final type of model we'll be training.

Further reading

Here are a few extra resources related to things we covered in this chapter:

- *How to Choose an ML.NET Algorithm*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-choose-an-ml-net-algorithm>
- *How to use the ML.NET Automated Machine Learning (AutoML) API*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/how-to-use-the-automl-api>
- *Decision Trees vs Random Forests*: <https://www.statology.org/decision-tree-vs-random-forest/>
- *Classification algorithms in ML .NET*: <https://rubikscore.net/2021/02/01/machine-learning-with-ml-net-ultimate-guide-to-classification/>
- *Online Gradient Descent*: <https://deepgram.com/ai-glossary/online-gradient-descent>
- *LightGBM*: <https://github.com/microsoft/LightGBM>
- *Tweedie Distributions*: https://en.wikipedia.org/wiki/Tweedie_distribution
- *Ordinary Least Squares Regression*: <https://builtin.com/data-science/ols-regression>
- *Poisson Regression*: https://en.wikipedia.org/wiki/Poisson_regression
- *Field Factorization Machines*: <https://towardsdatascience.com/an-intuitive-explanation-of-field-aware-factorization-machines-a8fee92ce29f>
- *SMAC: Sequential Model-based Algorithm Configuration*: <https://www.cs.ubc.ca/labs/algorithms/Projects/SMAC/>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/Beyond-AutoML/>

10

Deploying Machine Learning Models

Now that we've explored the various ways of training and evaluating regression and classification models, let's end our discussion of ML.NET by showing how models can be deployed.

In this chapter, we'll cover our final type of machine learning task: multi-class classification as we train and evaluate a simple model. We'll then take that model and export it to a file on disk. Next, we'll import it into an ASP.NET Web API application and use the model to evaluate incoming requests.

We'll also close the chapter with discussions on monitoring and maintaining machine learning models via MLOps and discuss additional topics of interest in ML.NET.

Along the way, we'll cover the following topics:

- Introducing our multi-class classification model
- Exporting ML.NET models
- Hosting ML.NET models in ASP.NET web applications
- Understanding model performance, data drift, and MLOps
- Surveying additional ML.NET capabilities

Technical requirements

This chapter assumes that you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

Additionally, this chapter will involve hosting an ML.NET model in an ASP.NET Web API project. This can be done in VS Code with the web development workflow installed, or in Visual Studio.

For consistency and cross-platform support, this chapter will feature examples in VS Code when working with our web application. See the *Further reading* section for resources on working with Visual Studio and ASP.NET Web API projects.

You can follow along with the code in this chapter by opening the `Chapter10.dib` notebook located in the `Chapter10` folder of the repository, which is located at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks>

The second half of this chapter will work with the `Chapter10/WinLossPredictorApi/WinLossPredictorApi.sln` solution file from the same repository in either Visual Studio or VS Code.

Introducing our multi-class classification model

Previously, we covered **binary classification models**, which are named that way because they are predicting a categorical variable that's going to be either one of two possible options.

A **multi-class classification model** is similar to a binary classification model except that there are three or more classes that something could fall into. For example, predicting which season something will occur in has four classes: spring, summer, fall, and winter.

In this chapter, we'll be training a multi-class classification model that takes in information about a football club and its opponent in an upcoming match and predicts the winner of the game.

Since football matches can end in draws, this is a multi-class classification problem with 3 distinct classes: *Win*, *Loss*, or *Draw*.

We'll start in our usual manner by importing ML.NET, AutoML, and the `DataFrame` with this hopefully familiar C# code block:

```
#r "nuget:Microsoft.ML,3.0.1"
#r "nuget:Microsoft.ML.AutoML,0.21.1"
#r "nuget:Microsoft.Data.Analysis,0.21.1"
using Microsoft.Data.Analysis;
using Microsoft.ML;
using Microsoft.ML.Data;
using Microsoft.ML.AutoML;
using Microsoft.ML.Trainers;
using Microsoft.ML.Trainers.FastTree;
```

With our dependencies installed and the appropriate using statements in place, we can now load up our `DataFrame` and look at our sample data:

```
DataFrame df = DataFrame.LoadCsv("../data/winloss.csv");
df.Sample(5)
```

This results in a preview of our data, shown in *Figure 10.1*:

<i>index</i>	<i>OurPosition</i>	<i>OpponentPosition</i>	<i>IsHome</i>	<i>Label</i>	<i>OurGoalsLast3Games</i>	<i>OpponentGoalsLast3Games</i>
0	7	18	False	Win	4	0
1	9	8	True	Win	8	3
2	3	14	False	Win	6	6
3	11	8	True	Draw	4	3
4	6	5	False	Loss	0	4

Figure 10.1 – Five rows from our dataset in a DataFrame

Let's take a look and discuss each of our columns:

- *OurPosition* indicates the rank in the league of the team we're evaluating. A position of 3 indicates 3rd place in the league.
- *OpponentPosition* indicates the league rank of our opponent at the time of the game.
- *IsHome* indicates whether or not this game is a home game for our team.
- *Label* is the value we're trying to predict. It will either be *Win* for our team winning, *Loss* for our team losing, or *Draw* for a tie.
- *OurGoalsLast3Games* is the sum of the goals our team has scored in its last three matches. During the first few matches of a season, this will include goals from the end of the prior season.
- *OpponentGoalsLast3Games* is our opponent's total goals in their last three games.

As you can see, this is a fairly simple data set, but not all problems require complex inputs in order to get reliable predictions.

Training our model

With our data loaded, let's use AutoML's `MulticlassClassificationExperiment` class to carry out model training using the simplest variant of AutoML.

We'll start by creating our `MLContext` and splitting our data into training and testing splits, as usual:

```
MLContext context = new(seed: 123);
var split = context.Data.TrainTestSplit(
    data: df,
    testFraction: 0.2,
    seed: 42);
```

Next, we'll create our experiment using AutoML:

```
var metric = MulticlassClassificationMetric.MacroAccuracy;
MulticlassExperimentSettings settings = new() {
```

```
MaxModels = 10,  
    OptimizingMetric = metric  
};  
MulticlassClassificationExperiment exp = context.Auto()  
    .CreateMulticlassClassificationExperiment(settings);
```

This code should be somewhat familiar from the past few chapters. We're defining our experiment using AutoML and specifying that we want AutoML to evaluate 10 total models.

We also tell AutoML to use the macro accuracy metric to determine which model is the best. We'll discuss multi-class classification metrics shortly.

Defining the experiment doesn't actually execute it, so let's run the experiment now:

```
ExperimentResult<MulticlassClassificationMetrics> result =  
    exp.Execute(split.TrainSet, split.TestSet);  
ITransformer model = result.BestRun.Model;
```

Executing our experiment takes a number of seconds. During that time, AutoML directs ML.NET to evaluate 10 different combinations of model trainers and hyperparameters for each trainer, then gathers metrics about the performance of each of them.

When the experiment is finished executing, we can grab our best model, an `ITransformer`, from the result's best run via `result.BestRun.Model`.

Now that we've trained our model, let's explore how accurate it is.

Evaluating multi-class classification models

Like binary classification, multi-class classification models have performance metrics associated with them. They also have a graphical confusion matrix that can help identify problem areas in the model.

Let's generate both from a single C# code cell by taking advantage of the `.Display()` method:

```
var metrics = result.BestRun.ValidationMetrics;  
metrics.Display();  
metrics.ConfusionMatrix.GetFormattedConfusionTable();
```

We've previously covered how the last line in a cell generates a visual below the cell automatically. However, you can also do this manually for any object in a cell by calling the `.Display()` extension method registered by Polyglot Notebooks.

Tip

The `.Display()` method is particularly helpful for situations where you have loops and want to display objects during loop execution.

When you run the code cell, you should see metrics and a confusion matrix similar to *Figure 10.2*:

```
▼ Microsoft.ML.Data.MulticlassClassificationMetrics
LogLoss          0.21632703819235985
LogLossReduction 0.8001346432120061
MacroAccuracy    0.9263405541836539
MicroAccuracy    0.9302250631143578
TopKAccuracy     0
TopKPredictionCount 0
TopKAccuracyForAllK <null>
PerClassLogLoss  [ 0.16973586974220253, 0.1691382020187725, 0.3566851856458876 ]
ConfusionMatrix  ► Microsoft.ML.Data.ConfusionMatrix
```

```
Confusion table
||=====
PREDICTED ||  Win |  Loss |  Draw | Recall
TRUTH     ||=====
Win       ||  6,588 |    12 |   378 | 0.9441
Loss      ||     1 |  6,557 |   419 | 0.9398
Draw      ||   238 |   251 |  4,173 | 0.8951
||=====
Precision || 0.9650 | 0.9614 | 0.8396 |
```

Figure 10.2 – Multi-class classification metrics and a confusion matrix

There are a few metrics here, and all of them are new to us, so let's start with what's familiar by looking over the confusion matrix (labeled in *Figure 10.2* as a **confusion table**).

If you recall from *Chapter 7, Classification Experiments with ML.NET AutoML*, a confusion matrix displays columns for your model's predictions and rows for the actual labels that were encountered.

Our confusion matrix here does the same thing, except it has three rows and three columns instead of two because our model has three classes: *Win*, *Loss*, and *Draw*.

With any confusion matrix, you want to see most of your values on the diagonal from the upper left corner to the lower right. These are cases where the model predicted the correct value.

Here, we can see our model's strengths and a few weak areas. We can see that the model is getting things correct the majority of the time, with the win and loss predictions having a precision of over 96% each. This means that we can trust predictions of wins and losses to be accurate.

Conversely, we can see the recall column at the end and see from that that our model is able to recognize the correct class between 89.5% and 94.4% of the time.

We do see that the model has some issues predicting draws, with draw predictions only being 83.9% reliable and the model appropriately recognizing draws before they happen slightly less than it does for wins and losses.

Overall, this model seems to be surprisingly reliable for such a simple dataset.

With our confusion matrix covered, let's talk briefly about some of our metrics:

- **MicroAccuracy** is the overall accuracy of the model in terms of the percentage of its predictions that are correct. This is roughly equivalent to **Accuracy** in binary classification.
- **MacroAccuracy** is the overall accuracy of the model, taken by averaging the accuracy of each class within the model. This is done without weighting based on how many examples are in each class, so even though there are fewer draws than wins or losses, draws are still responsible for 1/3 of the macro accuracy. This makes macro accuracy somewhat similar to **F1 score** in binary classification.

I am intentionally omitting the **Log Loss** and **Top K** accuracy metrics from the scope of this book as they can be more specialized and harder to understand, but see the *Further reading* section for more information on those metrics.

All told, our model is remarkably accurate for predicting the results of football matches for this league. The model has a roughly 8% chance of getting the wrong answer, and for a model that's not likely to be used in life-or-death decision-making, that performance is significantly better than a coin flip.

Generating test predictions

With our model in place, let's see how we would use it to make a set of test predictions.

We can start by defining a class to represent a row of training data:

```
public class PredictionRequest
{
    public float OurPosition { get; set; }
    public float OpponentPosition { get; set; }
    public float OurGoalsLast3Games {get; set;}
    public float OpponentGoalsLast3Games {get; set;}
    public bool IsHome { get; set; }
    public string Label {get; set;}
}
```

Note how this class uses `float` types for numeric values. This is because ML.NET prefers to work with values of the `float`, `bool`, or `string` type and will error for unsupported data types.

Next, we define a class to represent the result of our predictions:

```
public class PredictionResult
{
    public string PredictedLabel {get; set;}
    public float[] Score {get; set;}
}
```

This class has a `PredictedLabel` property that represents the class the model predicts: *Win*, *Loss*, or *Draw*. The model also has an array of floats called `Score`. This represents the score each individual class received when determining how likely it was to be the correct class. This means we can use the `Score` property to see the relative probability of each class. We'll see this in action shortly.

Next, we'll create a `PredictionEngine` and use it to predict the result of a hypothetical game:

```
var engine = context.Model.CreatePredictionEngine<PredictionRequest,
PredictionResult>(model);
engine.Predict(new PredictionRequest() {
    OurPosition = 1,
    OpponentPosition = 3,
    OurGoalsLast3Games = 5,
    OpponentGoalsLast3Games = 2,
    IsHome = true,
})
```

Calling `Predict` evaluates our data and returns a `PredictionResult` as shown in *Figure 10.3*:

▼ **Submission#12+PredictionResult**

PredictedLabel	Win
Score	[0.5558339, 0.22095284, 0.2232133]

Figure 10.3 – The result of predicting a hypothetical football match

In this case, the model thought it was 69% likely that our team would win, 1.9% likely that our team would lose, and 28.5% likely that the match would end in a draw.

Since the most likely class was our team winning, the `PredictedLabel` property stored the result of that class name.

Note

Classes in ML.NET are ordered based on the order they occur in the training data. In this case, the first value ML.NET saw was a win. The next new value it saw in the training data was a loss, followed by a draw, so our classes are ordered in that sequence.

Now that we've seen how our model behaves and how we can use it to generate a prediction, let's see how we can export our models to use later.

Exporting ML.NET models

Recall that machine learning models take time to train, even if you know the exact model trainer and hyperparameters you intend to use.

Once a model is trained, the model takes only a small fraction of the training time to generate its predictions. Thankfully, we can take our trained models in ML.NET and export them to a file. This is done via the `Save` method on the context's model catalog, as shown by the following code:

```
context.Model.Save(model: model,
    inputSchema: split.TrainSet.Schema,
    filePath: "MyModel.zip");
```

This code takes an `ITransformer` (here, that's the `model` variable) and saves it to the disk. ML.NET also needs to know the format or schema of the data the model takes in. Thankfully, we have this schema in our `DataFrame`, as well as both partitions of our train/test split. Here, we're using the `Schema` from the `TrainSet` for convenience, but all three should be equal.

When this completes, your model is saved to disk. You can load your model later on by calling the `Load` method on the model catalog as shown here:

```
string modelPath = "MyModel.zip";
ITransformer model = context.Model.Load(modelPath, out _);
```

This code loads the model from the zip file and stores it in a new `ITransformer` variable named `loadedModel`.

Note

The second parameter to `Load` is an `out` parameter that returns the schema of the loaded model. Since we don't need the schema, the code discards it via the use of the `_` variable placeholder.

With the model loaded you can then create a `PredictionEngine` as usual and use the loaded model to generate predictions.

Using `PredictionEngine` is straightforward, but there are a few issues with using `PredictionEngine` in multi-threaded environments, such as those found in many enterprise applications. Due to this, we'll explore a more scalable way of doing this in the next section as we show how models can be embedded into ASP.NET applications.

Hosting ML.NET models in ASP.NET web applications

ML.NET models can be embedded into a wide range of applications, including Azure functions, console, desktop, and mobile applications. However, the most commonplace ML.NET models are deployed is to ASP.NET applications.

ASP.NET is a web application framework that allows developers to build web applications and web APIs. At the time of writing this chapter, `builtwith.com` estimated that there were over five million active websites running ASP.NET worldwide.

While covering the way ASP.NET works is beyond the scope of this book, there are a large number of resources available that cover ASP.NET in more detail, including a number of books from Packt.

You can embed an ML.NET model into an ASP.NET application by following these steps:

1. Copy the model's `.zip` file into either the ASP.NET application's root directory or to an online location that can be referenced via a URL.
2. Add classes to your project representing the input to the model and the model's prediction. For us, that will be the `PredictionRequest` and `PredictionResult` classes we created earlier in the chapter.
3. Add a NuGet reference to `Microsoft.ML`, `Microsoft.Extensions.ML`, and any specific packages related to your model. For example, this chapter's model relies on the `Microsoft.ML.FastTree` package.
4. In your `Program.cs` or `Startup.cs` (whichever you use to configure your services), call `builder.Services.AddPredictionEnginePool` to register a pool of `PredictionEngine` instances centered around the model you added in *step 1*.
5. Wherever you need to make predictions, use dependency injection to inject your `PredictionEnginePool` and then use the pool's `Predict` method to generate predictions.

At a high level, these steps are all you need to do in order to work with your model in an ASP.NET application.

Let's drill into the details and see how this works.

Configuring a `PredictionEnginePool`

As mentioned earlier in this chapter, `PredictionEngine` instances are not thread-safe and cannot be safely shared between different threads. This means that for an ASP.NET web application, we'll want each incoming request to have its own `PredictionEngine` resource.

The `Microsoft.Extensions.ML` NuGet package provides a `PredictionEnginePool` class that allows you to safely issue `PredictionEngine` instances to individual requests in an efficient and scalable manner.

When you define a `PredictionEnginePool`, you must tell it the type of input it will receive, the type of output the `PredictionEngine` will generate, and how to retrieve the model.

At the top of `Program.cs`, we take care of this with the following code:

```
using WinLossPredictorApi;
using Microsoft.Extensions.ML;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddPredictionEnginePool<PredictionRequest,
PredictionResult>()
    .FromFile(modelName: "WinLossPredictor",
        filePath: "WinLossPredictor.zip",
        watchForChanges: true);
```

This code configures our pool, telling it that we'll take in `PredictionRequest` objects and the `PredictionEngine` will generate `PredictionResult` objects.

We use the `FromFile` method to specify that the ML.NET model is loaded from disk on the web server and specify `filePath` of that model. We also specify a `modelName` to use when referencing the model later as well as instructing the pool to watch for updates to the file.

If you choose to watch for updates, ML.NET will set up a `FileSystemWatcher` internally that watches for updates to the file and reloads it for future requests. This allows you to update your model without redeploying your application.

Want to host a model elsewhere?

There's also a `FromUri` method that allows you to tell ASP.NET to get the model file from a URL instead of from a local file. In this scenario, a `FileSystemWatcher` isn't possible, so ASP.NET will check to see whether the file has changed every five minutes by default when watching for changes.

This is less efficient than storing the file locally, but many web applications use horizontal scaling where there are many copies of the web server running at once. In these types of deployments, updating a model on each instance can be challenging, while hosting the file in a single location is easier for maintenance.

With the `PredictionEnginePool` registered, you can now reference it elsewhere in your application through dependency injection.

Using the PredictionEnginePool

Typically, you'll use your `PredictionEnginePool` in one of two places:

- Inside of a controller or minimal API endpoint to directly handle an incoming request
- In a class in a shared service layer that is referenced by other classes

In either case, the `PredictionEnginePool` is referenced via dependency injection.

Our sample project for this chapter uses a minimal API method, as shown here:

```
app.MapPost("/winlosspredictor",
    (PredictionEnginePool<PredictionRequest, PredictionResult> p,
    PredictionRequest request) =>
    {
        PredictionResult result =
            p.Predict("WinLossPredictor", request);

        return Results.Ok(result.PredictedLabel);
    })
    .WithName("WinLossPredictor")
    .WithOpenApi();
```

If you're not used to the newer minimal API syntax, this method handles an HTTP POST request to the `winlosspredictor` REST endpoint on the web server. Here, the body of the incoming REST request automatically comes in as the variable named `request`.

This endpoint is provided a `PredictionEnginePool` variable named `p` through dependency injection. ASP.NET is able to do this because we configured this service earlier.

From here, we use the pool's `Predict` method and specify the name of the model to use (you may configure and use multiple ML.NET models in the same application if they have different names).

We also provide the `Predict` method with the `PredictionRequest` object in the request variable.

The `PredictionEnginePool` then takes care of finding or instantiating a `PredictionEngine` for us to work with, has that `PredictionEngine` generate its output, and returns the resulting `PredictionResult` object.

The final line of our POST method handler returns a 200 OK result with the `PredictedLabel` from our model. This will be either *Win*, *Loss*, or *Draw*, reflecting the three classes of our multi-class classification model.

This chapter's code has a `WinLossPredictorApi.http` file that you can use to try out the model. First, run the ASP.NET application via your editor or the `dotnet run` command in your terminal. Next, use the `.http` file to make requests to its endpoint, as shown in *Figure 10.4*:

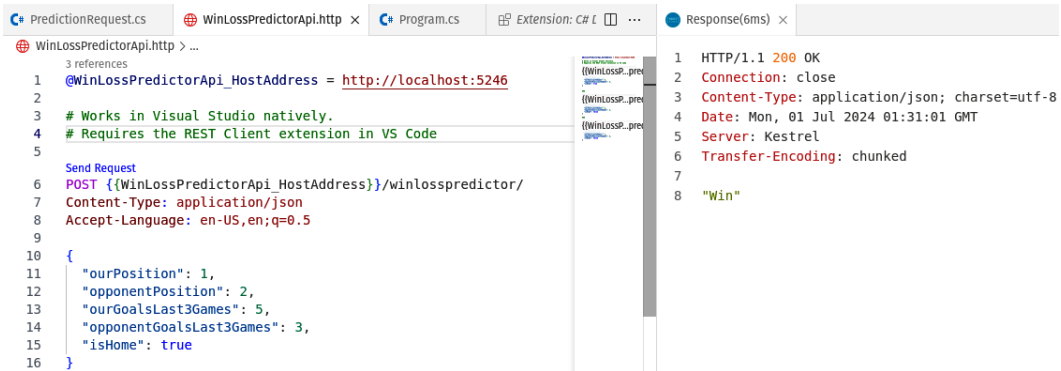


Figure 10.4 – Making an HTTP call to the hosted model

I'd encourage you to experiment with changing the different parameters for your hypothetical matchup to see how it impacts the responses from the server.

As you can see, there aren't many steps involved in connecting ML.NET models to an ASP.NET application.

Before we move on, let's point out what's not part of this process. When you host a ML.NET model in an ASP.NET application, you don't need to do any of the following things:

- Load your training data
- Split your data into training and test splits
- Choose a model trainer or hyperparameters
- Spend time fitting your model to those hyperparameters

The model file saves the different weights and internal states of the trained model. This means that no additional setup or training is required when we load our model into a web application, which is why using a pre-trained model to generate predictions is so much faster than training the model to begin with.

In the next section of this chapter, we'll explore tasks related to monitoring and maintaining machine learning models through the process of MLOps.

Understanding model performance, data drift, and MLOps

In machine learning, you can train a model once and use it forever, just like how in software engineering, you can usually write a program once and use it forever – provided the underlying operating system remains operational and supported. However, models are built from data. That data represents observations from a certain time period.

As time moves forward, the data you provided the model during training may start to resemble the real world less and less accurately as time goes on. For example, if I trained a regression model to predict what price a used car would sell for based on its make, model, and mileage, those assumptions would initially be quite valid. However, as time moved forward, new vehicles entered the market, and overall market trends evolved, the prices my model predicted would get further and further from realistic prices.

A somewhat recent example of this was the worldwide shortage of semiconductor chips that affected car sales by limiting the supply of new cars between the years 2020 and 2023. If a model was trained on pre-2020 data, it would not account for the impact of this shortage on vehicle prices.

Consumer preferences change over time and it's easy to imagine how a shift in preferences from pure gasoline vehicles to hybrid or electric vehicles could also impact sales.

This tendency for our models to lose accuracy as time goes on is known as **data drift** or **model drift** and occurs because the model's internal factors no longer strongly resemble the environment in which it operates.

While some industries are going to be more sensitive to model drift than others, even our football models are likely to be sensitive to model drift to some degree.

Professional sports are not static and defined. The rules of the game can be tweaked in subtle ways between seasons. For example, in ice hockey, there have been important minor changes in rules over the years, including the following:

- Allowing video review and coaches' challenges of key plays
- Introducing new penalties to penalize behaviors that slow down the pace of the game
- Adjusting restrictions on player equipment, particularly goalie pad sizes
- Removing old rules such as the two-line pass, or relaxing rule enforcement such as delayed offsides

These changes alter how the game is played in sometimes subtle and sometimes overt ways. The changing nature of the rules can lead to old strategies proving less effective and new strategies emerging.

Additionally, the rules themselves don't always need to change to result in a change in global behavior. Sometimes, coaches or players come along and try radically new strategies that alter how the game is played without requiring any changes in rules or how a game is officiated.

My point with the examples from sports is that even something you think is going to be relatively static can sometimes change in significant ways and model drift will usually occur over time.

Let's talk about how we can detect and resolve model drift when it does happen.

Detecting model drift

Let's say that our **win/loss/draw** classification model was deployed and remained active for five years. How could we know whether the model had lost accuracy? This case is actually fairly easy: we could use our deployed model to generate predictions about games that have occurred since the model was initially trained.

Since we know the actual result of these games, we can know the correct predictions the model should have made.

When we compare the model's predictions to the actual results, we have a set of guesses that could be plotted in a confusion matrix or used to calculate the model's macro accuracy, micro accuracy, or per-class precision or recall.

We can then take these metrics and compare the metrics we observed to the model's test data. If the metrics are significantly off, we know that we're encountering model drift and can start thinking about replacing our model. It is important to monitor any active model in production to ensure that its accuracy matches your expectations.

As soon as you deploy a model, you should begin monitoring its accuracy and comparing your observed metrics with the metrics recorded during training. If your initial metrics don't closely resemble the training metrics, it's likely that your training data doesn't accurately match real-world data. If this happens, you should revisit your model training process.

On the other hand, if your metrics initially match your expectations but accuracy decays over time, it's time to think about updating your model.

MLOps and updating models

A common misconception people have with machine learning is that models improve over time with more data.

While this can be true for some specialized scenarios, such as reinforcement learning solutions, model drift makes this generally untrue for models trained using supervised learning.

When people imagine their machine learning improving over time, what's usually happening is that their models are periodically being replaced with new models trained on more recent data.

The process of monitoring and periodically replacing machine learning models with better-performing models is referred to as **Machine Learning Operations (MLOps)**. This is similar to other terms such as the software engineering practice of **DevOps**.

Under MLOps, an organization will collect and store new training data based on recent observations.

Using this new data, the organization will train a new model using the same pipeline and training process that the previous model was trained on.

Sometimes, this training will include all data, including the newly collected data. Other times, the training will only use a rolling window of the most recent observations of data.

For example, it may make sense for a model predicting rainfall based on weather conditions to use as much weather data as is available to refine its predictions, despite global shifts in temperature and other factors. This is because observations from one climate at one point in the year may be relevant for predicting behavior in other regions.

On the other hand, it probably makes sense for a used car sales price prediction model to only use the most recent data available since market prices fluctuate and exhibit seasonality.

Under MLOps, an organization has a repeatable way of training its models using automatically collected data.

In ML.NET, this would involve running the same experiment on new data, saving the model, and then deploying the saved model to the web application or a shared location where it can be accessed.

As we saw earlier, ML.NET's `PredictionEnginePool` can be configured to watch for updated models. This way, when a new model is detected, the application won't even need to restart to incorporate the updated model.

It's up to you to determine how frequently your models should be updated and whether this process is done manually (using an established process of training and evaluating a model) or automatically (via a regularly scheduled task).

If you do choose to use an automated task to train and update your models, I strongly recommend enforcing a minimum performance threshold that must be met before a model is deployed. For example, if you have a model with a macro accuracy of 90% and the overnight model training run generates a model with 60% macro accuracy, your system should fail with a failure message and leave the existing model in production.

By failing to deploy a degraded model and flagging the issue to your organization's data scientists, you can investigate the training pipeline and identify the cause of the degraded model.

The cause of model degradation often has to do with data cleanliness issues. Therefore, it's important to investigate and resolve these issues as soon as they come up so data collection issues can be resolved before they pollute your future training data.

While there are entire books about MLOps and how it works and can fit into your team, this section should hopefully serve to establish a baseline level of knowledge.

Let's close this chapter and this part of the book with an examination of some of ML.NET's other capabilities.

Surveying additional ML.NET capabilities

In this book, we covered using ML.NET for regression, binary classification, and multi-class classification. We also covered using AutoML to perform end-to-end experiments for us or perform hyperparameter tuning on our behalf.

ML.NET supports many different tasks, including the following:

- Anomaly detection
- Ranking choices
- Recommending choices based on prior choices
- Time series forecasting
- Image classification and object detection with deep learning
- Text classification with deep learning
- Sentence similarity scores
- Probability analysis
- String tokenization

If you want a more comprehensive list with the latest examples, I recommend you take a look at <https://packt.link/1fjPR> for a full list of currently supported ML.NET tasks.

Additionally, ML.NET includes an increasing number of built-in transformation, conversion, and scaling operations through its various catalogs attached to the `MLContext` object.

If you'd like to use ML.NET inside Visual Studio for a traditional .NET application, you should be aware that ML.NET offers a graphical **model builder** that can help you build and train models.

I'd strongly encourage you to explore the properties and methods provided by the `MLContext` object because these can help you identify new ways of transforming, cleaning, and improving your model training pipelines as well as helping spot new model trainers.

I've also had great luck reading the fantastic documentation available on MS Learn at <https://learn.microsoft.com/en-us/dotnet/machine-learning/>.

Finally, don't forget that ML.NET is open source. I've been curious about how certain aspects of ML.NET work or the options available on various transformers or estimators and <https://github.com/dotnet/machinelearning> will often help you answer questions that the documentation doesn't address.

In general, I find that ML.NET's internals are often hidden and protected, making it difficult to observe the properties of my model and training pipeline. However, ML.NET has an amazing set of features that continues to grow as the ties between .NET and machine learning workloads grow stronger over time.

Before we conclude this chapter and this part of the book, I want to highlight one final capability of ML.NET: its ability to work with models trained in other formats.

ONNX and TensorFlow models in ML.NET

Using ML.NET, you can work with models trained in two other technologies:

- **Open Neural Network Exchange (ONNX)**, an open format for machine learning models
- **TensorFlow** models built for deep learning applications

These capabilities let you use specialized models trained on other platforms in your ML.NET applications.

Using ONNX, you can train a machine learning model using another technology, such as Python and SciKit-Learn, export the model to ONNX, and then load the model via ML.NET. Alternatively, you could use ML.NET to train your model, then save the model to ONNX, and load the model in another language.

Additionally, sites such as <https://github.com/onnx/models> and <https://www.tensorflow.org/resources/models-datasets> allow you to find pre-trained models in ONNX or TensorFlow format that you can then incorporate into your applications after proper vetting.

Both of these capabilities are beyond the scope of this book as they require their own dependencies and advanced troubleshooting steps to get started with, but see the *Further reading* section of the chapter for more learning resources.

Summary

In this chapter, we covered training, evaluating, and exporting a multi-class classification model designed to classify data into one of three or more different categories.

We saw how ML.NET supports saving and loading its models to .zip files and touched on how ML.NET also supports ONNX files to save some of its models and import pre-trained models from other technologies. Further, we were able to take our multi-class classification model and add it to an ASP.NET Web API. We did this with only a few steps by relying on the thread-safe `PredictionEnginePool` class from the `Microsoft.Extensions.ML` NuGet package. Finally, we touched on the many other machine learning tasks that ML.NET is capable of handling.

In the first part of this book, we saw how Polyglot Notebooks can serve as an interactive tool for analyzing, visualizing, and manipulating data.

In this part of the book, we discovered how Polyglot Notebooks pairs well with ML.NET and its machine learning capabilities (particularly with AutoML). We saw how you can train and evaluate models in whatever degree of control you require, as well as how models can easily be deployed to applications where they can be used to make predictions about new data.

We'll leave ML.NET for now and head off into new frontiers in the next part of the book, where we will see how Polyglot Notebooks helps you prototype and debug generative AI applications.

Further reading

The following resources will help deepen your understanding of the topics covered in this chapter:

- *ML.NET documentation*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/>
- *ML.NET metrics*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/metrics>
- *Builtwith.com metrics on ASP.NET*: <https://trends.builtwith.com/framework/ASP.NET>
- *Creating an ASP.NET web application in Visual Studio*: <https://learn.microsoft.com/en-us/visualstudio/get-started/csharp/tutorial-aspnet-core>
- *Hosting ML.NET models in ASP.NET applications*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/serve-model-web-api-ml-net>
- *ML.NET workflows*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/resources/tasks>
- *Save and load trained models*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/how-to-guides/save-load-machine-learning-models-ml-net>
- *Introduction to machine learning operations (MLOps)*: <https://learn.microsoft.com/en-us/training/paths/introduction-machine-learn-operations/>
- *Open Neural Network Exchange*: <https://onnx.ai/>
- *Using an ONNX model in ML.NET*: <https://learn.microsoft.com/en-us/azure/machine-learning/how-to-use-automl-onnx-model-dotnet>

-
- *Analyze review sentiment using a TensorFlow model in ML.NET*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/tutorials/text-classification-tf>
 - *Image classification in ML.NET with TensorFlow*: <https://learn.microsoft.com/en-us/dotnet/machine-learning/tutorials/image-classification>
 - *Additional online resources*: <https://mattonml.net/books/datascience/model-deploy/>

Part 3:

Exploring Generative AI with Polyglot Notebooks

Now that we've seen how Polyglot Notebooks and ML.NET can work together for interactive machine learning model training, let's move beyond machine learning and into the territory of generative artificial intelligence (AI).

In this chapter we'll see how you can interact with external large language models (LLMs) to generate text, images, and text embeddings from textual prompts. We'll also explore retrieval-augmented generation (RAG) and the limitations of generative AI, before introducing the concept of AI orchestration. We'll see these technologies in action by prototyping AI applications using prompt engineering and building a small AI application using Semantic Kernel, Microsoft's open-source AI orchestration framework.

This part has the following chapters:

- *Chapter 11, Generative AI in Polyglot Notebooks*
- *Chapter 12, AI Orchestration with Semantic Kernel*

Generative AI in Polyglot Notebooks

Over Part 1 and Part 2 of this book, we covered data analytics and data science workflows and showed how data can produce insight, guide decision-making, and even help you train predictive models.

This chapter switches direction and covers a specific branch of **artificial intelligence (AI)** that has erupted in popularity and capabilities in recent years – generative AI.

While generative AI is not unique to Polyglot Notebooks, the notebook environment can make inspecting complex result objects easier.

In this chapter, we'll use our prowess with Polyglot Notebooks to connect to and interact with generative AI models, see how Polyglot Notebooks provides an ideal sandbox to experiment with generative AI, and explore complex response objects returned by various APIs.

In this chapter, we'll cover the following topics:

- Understanding Generative AI
- Deploying generative AI models on Azure
- Connecting to an Azure OpenAI service
- Chatting with a deployed model
- Customizing model behavior with prompt engineering
- Using text embeddings
- Generating images with DALL-E

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter11` folder.

Additionally, in order to follow along with some of the examples in this chapter, you will need an Azure account with the ability to create an Azure OpenAI resource and deploy models from Azure OpenAI Studio.

If you don't have an Azure account, you can create a free one at <https://azure.microsoft.com/free/>. This account starts with a \$200 credit and provides many perpetual free services and some limited services for free for the first 12 months.

Understanding Generative AI

While AI is a broad topic, when I hear the community talking about it, most of the time they're referring to a very specific form of AI – generative AI.

Generative AI refers to using AI to generate new content in various formats. Chances are you've interacted with an AI G AI can be used to generate many different types of content, including the following:

- Text content such as summarizations, email drafts, or chat responses
- Synthetic images and videos, based on a description and possibly other parameters
- Sounds, including sound effects, speech synthesis, and even music
- Complex content such as 3D models or architectural diagrams
- Data such as rows in a CSV file or JSON objects

It's important to note that while the output of generative AI systems may seem novel, these systems have no understanding or creativity of their own.

Instead, most generative AI systems are based on **generative pre-trained transformers (GPTs)**. These transformers are trained on historical data, much like a regression or classification model is in typical machine learning.

During the training process, transformer models are structured around the idea of self-attention, where associations are built based on where various words are in sentences relative to each other.

This concept of self-attention was introduced in a 2017 paper from Google entitled *Attention is All You Need*, and it is what allows these large language models to make better associations between different words and phrases, based on their training data. Additionally, these models scale to larger datasets better than anything else did before them.

As a result, researchers are now able to work with a massive corpus of training data and use it to train new large language models. This process is lengthy and requires an astonishing amount of processing power from arrays of GPUs, but it results in trained models that can then take in an input and generate a prediction for the next part of the response.

This prediction comes in the form of a **token**. Tokens will vary by each large language model, based on the model's configuration, but tokens typically represent a portion of a request or response to a large language model. Shorter tokens might be a few characters of text, while longer tokens may be several words or even a phrase.

When you use a large language model, it iteratively generates the next token from the model until an arbitrary condition is reached, such as the model outputting a special pre-configured token indicating the end of a response, or the model reaching a maximum token count in what it is allowed to generate.

Note that there is no hidden intelligence in this model or even any creativity. Instead, the model focuses on common associations between tokens that were observed during training, using these observations to generate content that appears convincingly similar to the source materials.

Now that we know roughly what generative AI is, let's see how you can find some of these pre-trained models to work with yourself.

Deploying generative AI models on Azure

The two best ways of interacting with readily available large language models are currently either through OpenAI directly or the Azure OpenAI Service.

Both services require you to provide an API key to identify yourself and receive charges from requests to the LLM that you wish to interact with.

Both services also offer a variety of different models to interact with. These models are periodically updated with minor changes. Additionally, the model catalog regularly expands as new models are trained, evaluated, and made available to the public.

While both of these services are viable ways of interacting with LLMs, this book will focus on Azure OpenAI Service. This is because Azure OpenAI Service tends to be a better option for enterprise users, due to its privacy, security, and content safety features.

Additionally, many organizations use Azure for other services, and there can be strong advantages to having your LLMs hosted in the same data center as your other applications.

The steps to interact with an Azure OpenAI Service model is as follows:

1. Create an Azure OpenAI Service if you don't already have one.
2. Create a new deployment of the model you want to interact with.
3. Gather the URL and a key for your OpenAI service.
4. Make a request to your model using a REST request or an SDK.

We'll start by creating an Azure OpenAI Service.

Creating an Azure OpenAI Service

The simplest way of creating an Azure OpenAI Service is by logging into the Azure Portal at `portal.azure.com` and then clicking **Create a resource**.

Next, click on **AI + Machine Learning** under **Categories** on the sidebar, and then check the **Azure services only** checkbox.

From there, you should see an **Azure OpenAI** entry, as shown in *Figure 11.1*:

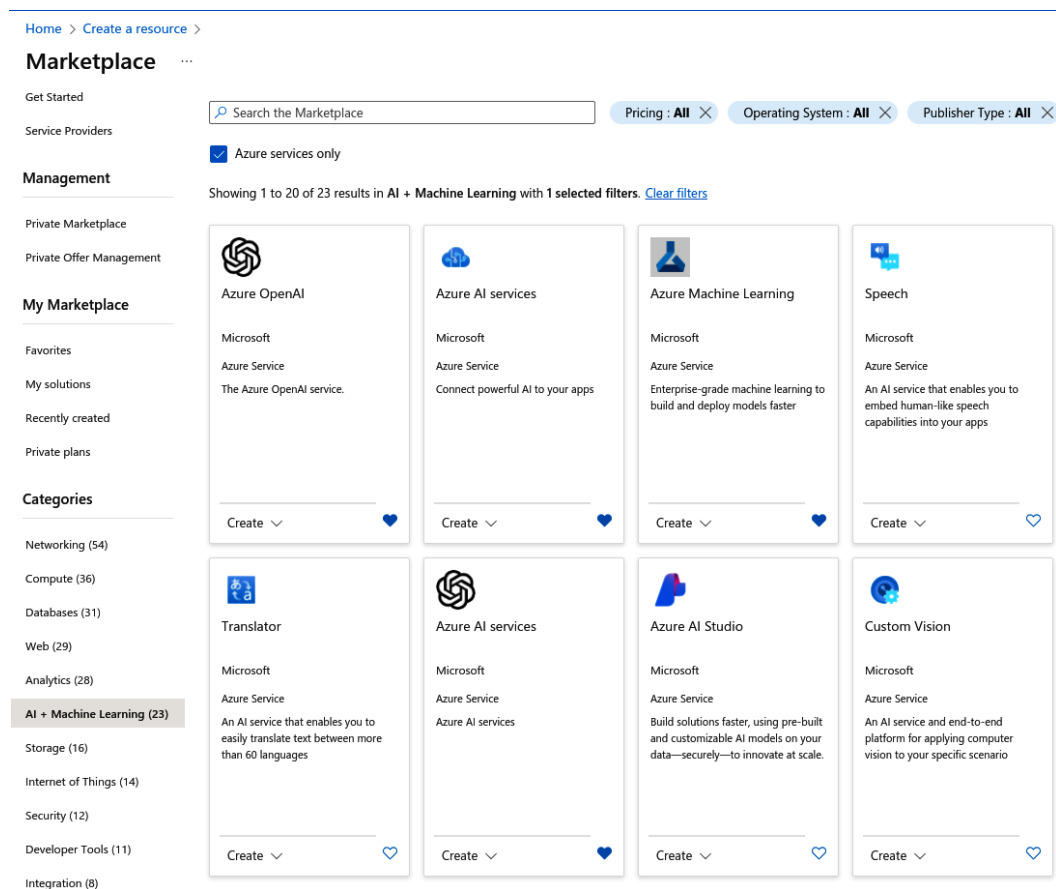


Figure 11.1 – Azure OpenAI in the Azure portal Marketplace

Click **Create** to begin configuring your resource.

Limited access

Azure OpenAI is currently under limited access, requiring individuals and organizations to submit a form before having full access to Azure OpenAI. This is primarily motivated by Microsoft's commitment to ensure that its AI services are used responsibly. See <https://learn.microsoft.com/en-us/legal/cognitive-services/openai/limited-access> for more information and to apply for access.

From the creation screen, you'll need to specify the resource group, region, resource name, and pricing tier for your Azure OpenAI Service resource, as shown in *Figure 11.2*:

[Home](#) > [Azure AI services](#) | [Azure OpenAI](#) >

Create Azure OpenAI ...

1 Basics 2 Network 3 Tags 4 Review + submit

Azure OpenAI Service provides access to OpenAI's powerful language models including the GPT-4, GPT-4 Turbo with Vision, GPT-3.5-Turbo, and Embeddings model series. These models can be easily adapted to your specific task including but not limited to content generation, summarization, image understanding, semantic search, and natural language to code translation.

[Learn more](#)

Project Details

Subscription * ⓘ

Resource group * ⓘ [Create new](#)

Instance Details

Region ⓘ

Name * ⓘ ✓

Pricing tier * ⓘ

[View full pricing details](#)

Figure 11.2 – Creating an Azure OpenAI Service

If you are using OpenAI as part of a larger application, you may want to create the Azure OpenAI Service as part of that application's resource group. Otherwise, I recommend you create a new resource group just for this resource.

Next, you'll need to specify the region. Not all models are available in all regions on Azure, so it's best to check model availability before selecting a region if you're looking for specific models.

Generally, the most popular models will be available in a majority of Azure regions, but some regions will get preview access to various models earlier than others. See <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models#provisioned-deployment-model-availability> for the most up-to-date information.

Provide a name for your resource and then select your pricing tier. Pricing details and tiers will change over time, so I recommend clicking **View full pricing details** for the most current information.

Review the remaining details and pages, and then click **Create** to create your service.

Once your Azure OpenAI Service is created, it's time to deploy your models.

Deploying models on Azure OpenAI Service

Once your Azure OpenAI Service is created, navigate into the resource, and then click the **Go to Azure OpenAI Studio** button or navigate directly to `oai.azure.com`.

Azure OpenAI Studio contains many different capabilities for working with OpenAI models, but in order to get started, we'll need to deploy a model.

We'll start by navigating to **Model catalog** on the sidebar.

The model catalog lists the available models in your resource's Azure region. This list is undergoing revisions and will likely continue to vary over time, but it should resemble something like *Figure 11.3*:

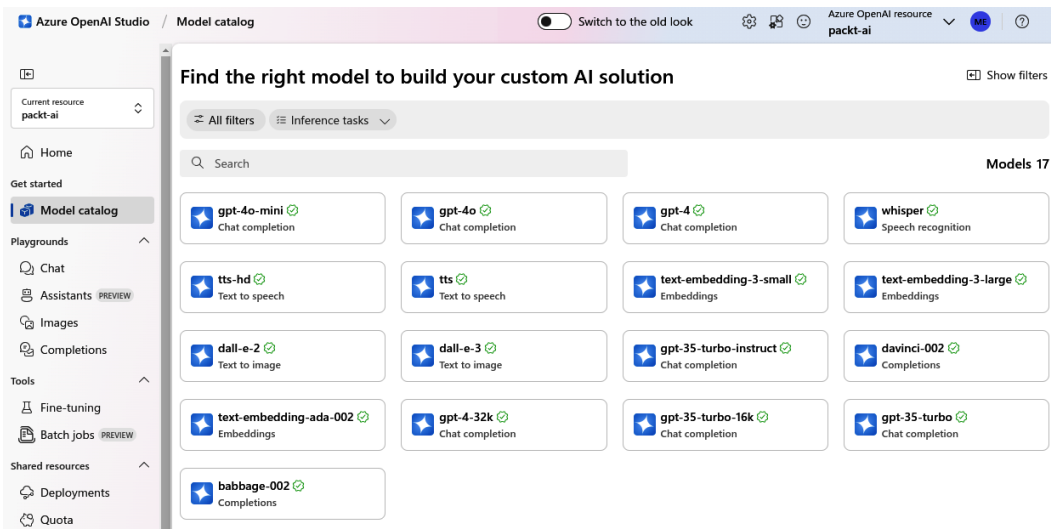


Figure 11.3 – A partial list of models available

From here, you can click on a model you're interested in to see its details page. From there, if you want to deploy the model, you can click **Deploy**.

Alternatively, you can select the **Deployments** blade on the sidebar and then click **Deploy model**.

Next, you'll need to give your deployment a name and configure other options, as shown in *Figure 11.4*:

Deploy model ✕

Set up a deployment to make API calls against a provided base model or a custom model. Finished deployments are available for use. Your deployment status will move to succeeded when the deployment is complete and ready for use.

Deployment name ⓘ *

Select a model ⓘ

gpt-4o

▼

Model version ⓘ *

Auto-update to default

▼

Deployment type ⓘ *

Standard

▼

Current resource

packt-ai | eastus

ⓘ 150K tokens per minute quota available for your deployment

Tokens per Minute Rate Limit (thousands) ⓘ

10K

Corresponding requests per minute (RPM) = 60

Content Filter ⓘ

Default

▼

Enable Dynamic Quota ⓘ

☒ Enabled

Create

Cancel

Figure 11.4 – Configuring a model deployment

We'll not cover all of the options available here, but the most important settings are the deployment name, the model, and the rate limit.

The **deployment name** is the name we'll use in code later to refer to this specific model deployment.

The **model** is the specific model you wish to deploy. You can also select a specific current or past version of the model if you don't want the model to automatically update if a replacement model becomes available. Keep in mind that Microsoft will periodically retire old versions with some advance notice, so auto-updating is a good policy unless you need deeper control of upgrades.

Finally, the **rate limit** governs how frequently your model can be called. Each Azure subscription has a maximum quota of requests per minute for each type of model in an Azure region. You are able to request quota increases as needed, but if you expect to need multiple deployments of the same model, you should plan and calculate your quota allocations more carefully.

Quota tips

You can use the **Quota** blade in Azure OpenAI Studio to view and adjust your current quota allocations for a region. Keep in mind that if you delete Azure resources, it may take one to two days for their quota to be released.

Clicking **Create** will create your model deployment and make it available for use within the next five minutes.

For the content of this book, we'll need to create a model for chatting, a model for image generation, and a model for text embedding creation.

The book will use the following models:

- **Chat completions:** A `gpt-4o` model deployed as *gpt-4o-deployment*
- **Text embedding:** A `text-embedding-ada-002` model deployed as *text-embedding-ada-002*
- **Image generation:** A `dall-e-3` model deployed as *DALL-E*

The model details used in this book are shown in *Figure 11.5*:

Deployment name ▾	Model name ▾	Model version ▾	Deployment type ▾	Capacity	Status ▾	Model retirement date ▾	Content Filter ▾	Rate limit
text-embedding-ada-002	text-embedding-ada-002	2	Standard	120K TPM	✔ Succeeded	4/2/2025	Default	120000
DALL-E	dall-e-3	3.0	Standard	1 CU	✔ Succeeded	10/30/2024	Default	-
gpt-4o-deployment	gpt-4o	2024-05-13	Standard	10K TPM	✔ Succeeded	3/20/2025	Default	10000

Figure 11.5 – The models deployed in this book

If these models are not available, you may need to use a different Azure region or choose a different model that meets the same basic purpose.

With these models deployed, we're almost ready to start interacting with them from a Polyglot Notebook. But first, we'll need to get our API endpoint and an access key.

Getting access credentials for Azure OpenAI

Back in the Azure portal, find your Azure OpenAI Service resource and navigate to its **Keys and Endpoint** blade under **Resource Management**, as shown in *Figure 11.6*:

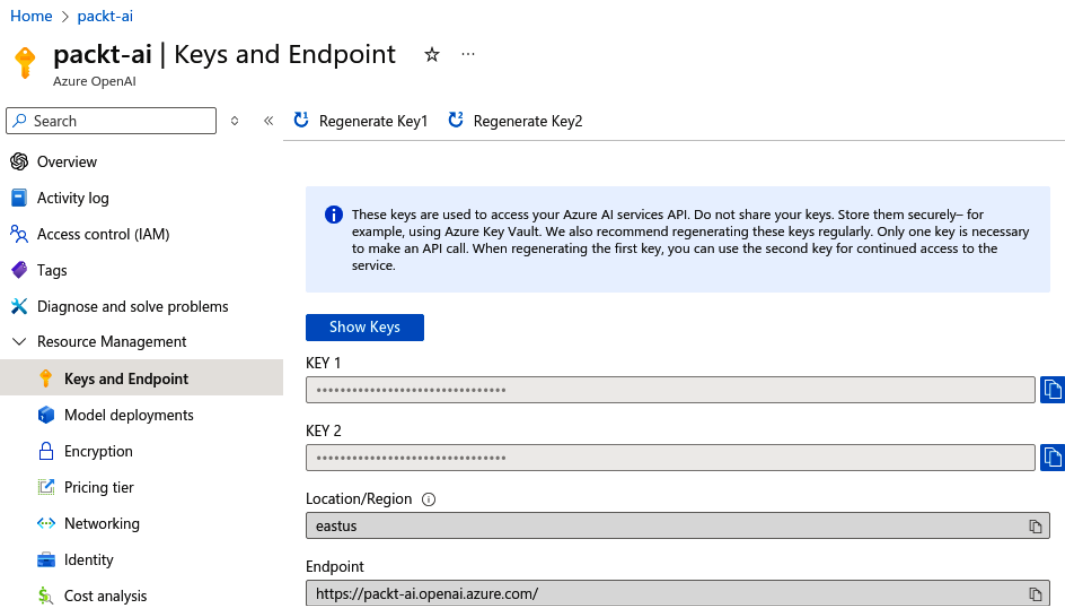


Figure 11.6 – The Keys and Endpoint blade of an Azure OpenAI Service

Like many other Azure resources, the Azure OpenAI Service allows you to authenticate using a key and endpoint combination and provides two keys to use for this purpose.

Typically, you have your applications using a single key for some period of time. Once that period elapses, you'll move your applications to use the other key. Once all applications have migrated, you'll regenerate the key that's no longer in use. The next time you want to regenerate your keys, you'll repeat the process, moving back to the original key slot.

Other authentication options

Azure OpenAI Service also supports other authentication options, including via managed identity, but key-based authentication is a good option for Polyglot Notebook experimentation.

Copy the endpoint and one of the keys to a temporary location, where you'll be able to refer to it as we connect a Polyglot Notebook to our Azure OpenAI Service in the next section.

Connecting to an Azure OpenAI Service

Now that we've set up our Azure OpenAI Service and deployed the models we'll be interacting with, let's open this chapter's Notebook and start chatting.

In the `Chapter11.dib` file, we'll start by installing the `Azure.AI.OpenAI` NuGet package with the following C# code cell:

```
#r "nuget:Azure.AI.OpenAI,2.0.0-beta.2"
using Azure;
using Azure.AI.OpenAI;
using Azure.AI.OpenAI.Chat;
```

This installs the various dependencies we'll need for our code in this chapter and allows us to easily reference types in those namespaces.

Next, we'll need to authenticate with `AzureOpenAIClient`. We'll do this by providing our endpoint and key as `Uri` and `AzureKeyCredential`, respectively.

Because these are secure values, we shouldn't store them in our Notebook.

Thankfully, we can prompt users to enter specific values via the `@input` directive, followed by a string prompt.

Here's an example that uses `@input` to store a user-provided value in the `url` variable in the C# kernel:

```
#!set --name url --value @input:"Enter your endpoint"
Uri endpoint = new Uri(url);
```

When this code executes, the user sees a prompt in the top center of VS Code, as shown in *Figure 11.7*:

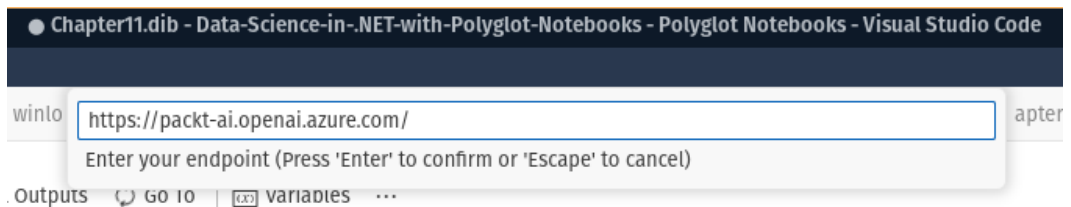


Figure 11.7 – VS Code prompting the user for their endpoint

The resulting prompt is then stored in the `url` variable, which is then used to create our `endpoint` variable on the following line.

By having the user enter sensitive information, we avoid the need to store that information in a shareable file or see it enter source control.

The `@input` directive prompts you to enter values in plain text. That may not be desirable for all types of information, such as passwords.

If you need to use a password-style prompt, you can interact with Polyglot Notebooks directly using its `Kernel` class. We'll do that now to accept the key:

```
using Microsoft.DotNet.Interactive;
string prompt = "Enter your key";
PasswordString key = await Kernel.GetPasswordAsync(prompt);
```

This works like the previous version, but when the user types in their value, asterisks' are shown instead of the raw value, as shown in *Figure 11.8*:

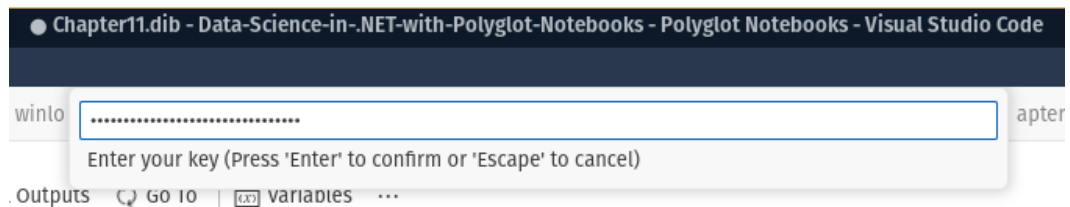


Figure 11.8: Using `Kernel.GetPasswordAsync` to get a secure value

You may have noticed that the result of `GetPasswordAsync` is a `PasswordString`. This is a more secure way of storing sensitive information in memory. We can still get the raw password out of a `PasswordString` object by calling its `GetClearTextPassword` method.

We'll do that next as we create our `AzureOpenAIClient` object:

```
AzureKeyCredential creds = new(key.GetClearTextPassword());
AzureOpenAIClient azureClient = new(endpoint, creds);
```

Here, we provided our key and endpoint, which resulted in a new `AzureOpenAIClient` object. This object will allow us to create other client objects for the various AI tasks we'll perform over the remainder of the chapter.

Chatting with a deployed model

Now that we have `AzureOpenAIClient`, we'll need to get a specific `ChatClient` to interact with our model.

To do this, we'll need the name of the deployment we're interacting with. For me, that will be *gpt-4o-deployment*, but for you, it will be whatever you named your chat model deployment earlier in this chapter.

We can get our `ChatClient` by calling `GetChatClient` with the name of the deployment we want to interact with, as shown here:

```
using OpenAI.Chat;
string chatModelName = "gpt-4o-deployment";
ChatClient chatClient =
    azureClient.GetChatClient(chatModelName);
```

Next, we'll need to call the `CompleteChat` or `CompleteChatAsync` method on our new `chatClient`.

Both methods require a collection of chat messages to give the model context in the conversation.

We'll provide a pair of chat messages:

- `SystemChatMessage`, which represents our AI agent's overall instructions. This is sometimes called the **system prompt**.
- `UserChatMessage`, which represents the **user prompt** we want.

We'll do this with the following code:

```
string systemPrompt = ""
You are a football (soccer) coach bot. Provide brief strategic
assistance to players for football-related questions.
"";
string userPrompt = ""
The opponents have one high-skilled attacker who is causing us
trouble. What should we do?
"";
var result = chatClient.CompleteChat([
    new SystemChatMessage(systemPrompt),
    new UserChatMessage(userPrompt)
]);
result.Value
```

Here, we do some basic prompt engineering to tell the system what its role is and how it should respond, and then we provide a short prompt to get responses from the system.

Finally, we end our cell with `result.Value`, which displays the result object below the cell, as shown in *Figure 11.9*:

CreatedAt	2024-07-14 19:23:53Z						
FinishReason	Stop						
ContentTokenLogProbabilities	▶ []						
Role	Assistant						
Content	<table> <tr> <th>index</th><th>value</th></tr> <tr> <td>0</td><td> ▼ To neutralize their high-skilled attacker, consider the following strategies:\n\n**Man-Marking:** Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This Kind ▶ text Text To neutralize their high-skilled attacker, consider the following strategies: 1. Man-Marking: Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This can restrict t 2. Double-Teaming: Use two players to challenge the attacker when they receive the ball. This can create pressure and limit t 3. Defensive Formation: Opt for a more defensive formation to provide extra cover at the back. For example, a 4-5-1 formation 4. Tactical Fouls: If necessary, use tactical fouls to disrupt the flow of play, though ensure it's done within the rules to 5. Communication and Coordination: Ensure your defenders communicate effectively and coordinate their positioning to cover fo 6. Video Analysis: Review previous matches to analyze the attacker's tendencies and preferred moves. Prepare your defenders t Implement these strategies to mitigate the threat posed by their skilled attacker. ImageUri <null> ImageBytes <null> ImageBytesMediaType <null> ImageDetail <null> </td></tr> </table>	index	value	0	▼ To neutralize their high-skilled attacker, consider the following strategies:\n\n**Man-Marking:** Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This Kind ▶ text Text To neutralize their high-skilled attacker, consider the following strategies: 1. Man-Marking: Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This can restrict t 2. Double-Teaming: Use two players to challenge the attacker when they receive the ball. This can create pressure and limit t 3. Defensive Formation: Opt for a more defensive formation to provide extra cover at the back. For example, a 4-5-1 formation 4. Tactical Fouls: If necessary, use tactical fouls to disrupt the flow of play, though ensure it's done within the rules to 5. Communication and Coordination: Ensure your defenders communicate effectively and coordinate their positioning to cover fo 6. Video Analysis: Review previous matches to analyze the attacker's tendencies and preferred moves. Prepare your defenders t Implement these strategies to mitigate the threat posed by their skilled attacker. ImageUri <null> ImageBytes <null> ImageBytesMediaType <null> ImageDetail <null>		
index	value						
0	▼ To neutralize their high-skilled attacker, consider the following strategies:\n\n**Man-Marking:** Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This Kind ▶ text Text To neutralize their high-skilled attacker, consider the following strategies: 1. Man-Marking: Assign a dedicated defender to follow and mark the attacker tightly throughout the match. This can restrict t 2. Double-Teaming: Use two players to challenge the attacker when they receive the ball. This can create pressure and limit t 3. Defensive Formation: Opt for a more defensive formation to provide extra cover at the back. For example, a 4-5-1 formation 4. Tactical Fouls: If necessary, use tactical fouls to disrupt the flow of play, though ensure it's done within the rules to 5. Communication and Coordination: Ensure your defenders communicate effectively and coordinate their positioning to cover fo 6. Video Analysis: Review previous matches to analyze the attacker's tendencies and preferred moves. Prepare your defenders t Implement these strategies to mitigate the threat posed by their skilled attacker. ImageUri <null> ImageBytes <null> ImageBytesMediaType <null> ImageDetail <null>						
ToolCalls	▶ []						
FunctionCall	<null>						
Id	chatcmpl-9kz73oN1viipxRoA1BXdSXBf1ZPk8						
Model	gpt-4o-2024-05-13						
SystemFingerprint	fp_abc28019ad						
Usage	▼ OpenAI.Chat.ChatTokenUsage <table> <tr> <td>OutputTokens</td><td>234</td></tr> <tr> <td>InputTokens</td><td>51</td></tr> <tr> <td>TotalTokens</td><td>285</td></tr> </table>	OutputTokens	234	InputTokens	51	TotalTokens	285
OutputTokens	234						
InputTokens	51						
TotalTokens	285						

Figure 11.9 – The results of our chat interaction

The ability to visualize complex and hierarchical objects is one of Polyglot Notebooks' strengths.

Visualizing complex objects

Being able to expand and collapse different nested objects is very helpful in analyzing the results of API calls in general. I find this particularly helpful when interacting with Azure OpenAI Services and Azure AI Services, such as the text, speech, and vision APIs. These specific APIs are not covered in this book; see the *Further reading* section of this chapter for additional information.

You'll notice from *Figure 11.9* that the `ChatCompletion` that came back from the API call contains a response message in its `Content` property, which appears to be an array or list. The actual response is in the only element of this collection's `Text` property.

You might also notice that the `Usage` property contains a listing of `OutputTokens`, `InputTokens`, and `TotalTokens`. Remember that LLMs measure, rate limit, and bill their interactions in terms of tokens. These properties allow you to see how many tokens your request and its response generate.

Based on what we've learned from the response, we can write the following code to display all text messages in that response:

```
var textMessages = result.Value.Content
    .Where(m => m.Kind == ChatMessageContentPartKind.Text);
```

```
foreach (var message in textMessages) {  
    Console.WriteLine(message.Text);  
}
```

This displays the full response to the user, which will vary from request to request and is too long to include in this book.

Now that we've seen how to generate prompt completions, let's talk briefly about an easy way of improving your AI agent's behavior – prompt engineering.

Customizing model behavior with prompt engineering

Prompt engineering refers to the art of customizing the system prompt that an AI agent receives and optionally providing it contextual examples, in order to more accurately respond to requests.

In our case, we've already performed some basic prompt engineering through our short prompt:

“You are a football (soccer) coach bot. Provide brief strategic assistance to players for football-related questions.”

When the system responded, it responded with far too many words for a coach to use in a live football match.

Let's tweak our system prompt to emphasize the importance of brevity of its responses, and also hint at a personality:

```
string systemPrompt2 = """  
You are an AI football (soccer) assistant with a cheerful and positive  
demeanor. Provide 1-2 sentence answers to strategic situations in a  
way that would be suitable to deliver to football players during a  
live match.  
""";  
ClientResult<ChatCompletion> result = chatClient.CompleteChat([  
    new SystemChatMessage(systemPrompt2),  
    new UserChatMessage(userPrompt)  
]);  
foreach (var message in result.Value.Content) {  
    Console.WriteLine(message.Text);  
}
```

Note how this new prompt is more explicit on the length of responses, the context in which the system should be used, and has a basic personality.

When I run this code, I get the following response:

```
Let's double-mark that attacker with our most reliable defenders! Keep  
the pressure tight and don't give them any space to work their magic.  
You got this!
```

While I'm no footballer, this response seems more usable in a live football match, and you can see a personality start to emerge.

While we could continue to tweak our system prompt, we also have another mechanism available to us to customize our agent's behavior – few-shot inferencing.

Zero-shot, one-shot, and few-shot inferencing

Previously, we gave our agent a system prompt, and then we gave it the user's prompt and had it generate a response.

This style of approach provides zero examples to the agent for how it should behave and is referred to as **zero-shot inferencing**.

Zero-shot inferencing is fine – unless you don't like the results your agent provides.

If you want more control of your agent's output, you can move to **one-shot inferencing**, where you provide a single example of how the agent should behave.

Let's do that now by giving the agent an example scenario in its chat history:

```
string ex1User = ""
We can't break out of our defensive end! What should we do?
"";
string ex1Assistant = ""
Stay calm and keep possession. Slow down the pace, spread out, and
look for opportunities to pass. You don't have to force the ball
forward.
"";
ClientResult<ChatCompletion> result = chatClient.CompleteChat([
    new SystemChatMessage(systemPrompt2),
    new UserChatMessage(ex1User),
    new SystemChatMessage(ex1Assistant),
    new UserChatMessage(userPrompt)
]);
foreach (var message in result.Value.Content) {
    Console.WriteLine(message.Text);
}
```

In this case, we provide a single sample question and the system's answer as a one-shot example. Although the user never sees this question and its response, the example influences the system's style and personality going forward.

In this case, we get a similar output to earlier, with a more consistent style and a slightly shorter response:

```
Double up on marking them and don't give them any space! Stay tight
and communicate with your teammates to limit their influence.
```


If a single example wasn't enough, you can move from one-shot inferencing to **few-shot** inferencing by providing additional examples.

By the way, this is how chat history works for completion – every request we send has the historical log of the conversation, including the user's message and the system's responses. Because this can add up over time, many systems limit the total number of interactions or discard older entries in chat history.

Now that we've covered the basics of chat, let's move on to a more complex form of representing text – embeddings.

Using text embeddings

Text embeddings are a critically important part of generative AI solutions. Yet embeddings are harder to understand because they do not translate easily to the ways we interact regularly.

Put simply, text embeddings are an array of numbers that are calculated from some string of text. These numbers represent the location of a specific concept using a complex, multidimensional way of representing concepts. To make things clearer, let's use a metaphor.

If two people are hiking in a large park and want to meet at a specific location, they could share GPS coordinates of the exact location. These GPS coordinates are an array of two numeric values representing a location somewhere in the world.

Alternatively, if those two people were referring to some concept in a book, they could share the name of the book, the chapter, the page, the paragraph, the sentence, and optionally, a word within that sentence. Those pieces of information are effectively a slightly larger array to refer to something known.

Text embeddings take this concept to an extreme degree. A simple sentence such as “*Our goalkeeper is out of position!*” can be represented using a text embedding model as a specific series of values in an array.

For the last few years, the most common text embedding model has been `text-embedding-ada-002`, although newer models are now available.

If you were to represent “*Our goalkeeper is out of position!*” using this embedding model, you'd get a series of 1,536 discrete numbers, representing the location of this unique concept in the embedding space.

If we then generated similar embeddings for “*Shoot the ball while the keeper is out of place!*”, those two locations should be close but not exactly the same, as they cover two similar but not identical concepts.

By calculating the distance between two sets of embeddings, you can see how similar or different two concepts are. This way, embeddings can be used to rank search results by relevance or cluster together related concepts. To generate embeddings, we first need an `EmbeddingClient` instance.

As before, we can get this client from our `AzureOpenAIClient` instance and the name of the specific deployment we're referencing:

```
using OpenAI.Embeddings;
string embedDeployment = "text-embedding-ada-002";
EmbeddingClient embedClient =
    azureClient.GetEmbeddingClient(embedDeployment);
```

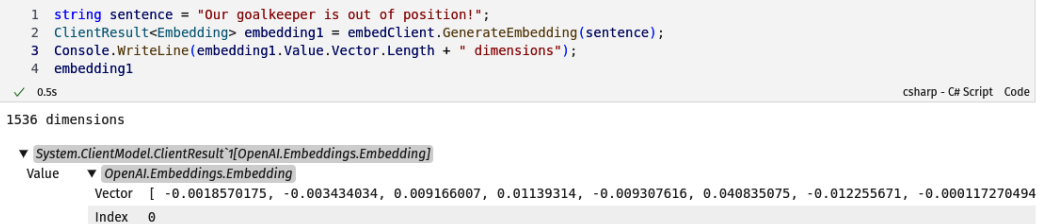
Next, we can call the `GenerateEmbedding` or `GenerateEmbeddingAsync` methods on `EmbeddingClient` to generate an embedding for our sentence:

```
string sentence = "Our goalkeeper is out of position!";
var response1 = embedClient.GenerateEmbedding(sentence);
Embedding embedding1 = response1.Value;
Console.WriteLine(embedding1.Vector.Length);
embedding1
```

When we run this, we can see the result of listing our embedding size:

```
1536
```

We also see a preview of some of the values in that embedding, as shown in *Figure 11.10*:



The screenshot shows a code editor with the following C# code:

```
1 string sentence = "Our goalkeeper is out of position!";
2 ClientResult<Embedding> embedding1 = embedClient.GenerateEmbedding(sentence);
3 Console.WriteLine(embedding1.Value.Vector.Length + " dimensions");
4 embedding1
```

Below the code, the debugger shows the value of `embedding1` as `System.ClientModel.ClientResult`1[OpenAI.Embeddings.Embedding]`. The `Value` property is expanded to show `OpenAI.Embeddings.Embedding`. The `Vector` property is expanded to show a list of 1536 dimensions, with the first few values displayed: `-0.0018570175, -0.003434034, 0.009166007, 0.01139314, -0.009307616, 0.040835075, -0.012255671, -0.000117270494`. The `Index` property is set to 0.

Figure 11.10 – The first few values from the generated embedding

These numbers aren't helpful on their own, so let's generate a second embedding and then calculate the cosine similarity between the two concepts.

We'll do this by using the `System.Numerics.Tensors` NuGet package, which we'll import here:

```
#r "nuget:System.Numerics.Tensors"
using System.Numerics.Tensors;
```

Next, we declare our second sentence, generate its embeddings, and then calculate the similarity using `TensorPrimitives.CosineSimilarity`:

```
string sentence2 =
    "Shoot the ball while the keeper is out of place!";
```

```
Embedding embedding2 =  
    embedClient.GenerateEmbedding(sentence2).Value;  
TensorPrimitives.CosineSimilarity(embedding1.Vector.Span,  
    embedding2.Vector.Span)
```

For me, this returns the following similarity score:

```
0.8986169
```

In other words, these two concepts are fairly similar.

Now, let's try comparing our first sentence to a completely random sentence on another topic:

```
string sentence3 = "I enjoy learning about AI.";  
Embedding e3 = embedClient.GenerateEmbedding(sentence3).Value;  
TensorPrimitives.CosineSimilarity(  
    embedding1.Vector.Span,  
    embedding3.Vector.Span)
```

This similarity score was significantly lower:

```
0.69371074
```

If you start to imagine using text embeddings on a larger scale, you could consider generating embeddings for paragraphs, sections of documents, blog posts, or even chat histories from interactions with AI agents.

With enough content encoded as embeddings, you would then be able to determine which entries were the most relevant for any search or identify content that is semantically similar to each other.

Such capabilities can power search results and article suggestions, or even identify useful information to provide to the AI orchestration systems that we'll introduce in *Chapter 12, AI Orchestration with Semantic Kernel*.

In summary, text embeddings may not be directly beneficial to your end users or easy to understand. However, the ability to compare the semantic meaning of two pieces of text can power features that rely on being able to identify and retrieve relevant pieces of information.

In the next section, we'll move away from text and show how generative AI can produce images from words.

Generating images with DALL-E

Image generation involves generating a digital image from a user-generated prompt.

The idea of image generation is a fairly straightforward one, but image generation relies on a complex pre-trained model and a large corpus of training images.

This is why image generation services such as Stable Diffusion, Midjourney, and DALL-E have proven to be powerful and effective tools.

The scenarios in which image generation is useful can vary – from entertainment at seeing the fantastical images that it generates and the professional use of generated imagery in articles or fliers to more nefarious purposes, such as propaganda or even disinformation campaigns.

Don't be evil. Seriously

Obviously, you are expected to only use these tools for good, and Microsoft takes this very seriously – to the point of gating access to OpenAI and making responsible use part of their service agreement through its responsible AI principles. See the *Further reading* section at the end of this chapter for more information.

It probably won't surprise you to learn that in order to generate images, we'll first need to create `ImageClient` from our `AzureOpenAIClient` instance and tell it which DALL-E deployment to use:

```
using OpenAI.Images;
string imageDeployment = "DALL-E";
ImageClient imageClient =
    azureClient.GetImageClient(imageDeployment);
```

Once we have the image client, we'll send a request to our DALL-E resource on Azure OpenAI Service. This can be done by calling the `GenerateImage` or `GenerateImageAsync` method, as shown here:

```
string imgPrompt = ""
A picture of a nerdy bald male author with glasses executing a bicycle
kick in a football match.
"";
var imgResult = imageClient.GenerateImage(imgPrompt);
imgResult.Value
```

This operation calls out to our DALL-E resource and returns a result with the generated image. Because we ended our cell with `imgResult.Value`, we can see its properties in our notebook, as shown in *Figure 11.11*:

```
OpenAI.Images.GeneratedImage
ImageBytes <null>
ImageUri ▼ https://dalleproduse.blob.core.windows.net/private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png?se=2024-07-16T02%3A00%3A4Z&sig=0MBpIj03eVLY2JmEgY9rG9KEAU330Cn8o7rf06IYCo8%3D&ske=2024-07-19T09%3A00%3A00%3A00
  AbsolutePath /private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png
  AbsoluteUri https://dalleproduse.blob.core.windows.net/private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png
  LocalPath /private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png
  Authority dalleproduse.blob.core.windows.net
  HostNameType Dns
  IsDefaultPort True
  IsFile False
  IsLoopback False
  PathAndQuery /private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png?se=2024-07-16T02%3A00%3A4Z&sig=0MBpIj03eVLY2JmEgY9rG9KEAU330Cn8o7rf06IYCo8%3D&ske=2024-07-19T09%3A00%3A00%3A00
  Segments [ /, private/, images/, b6280fe0-1e34-42a1-8552-357853e8ae97/, generated_00.png ]
  IsUnc False
  Host dalleproduse.blob.core.windows.net
  Port 443
  Query ?se=2024-07-16T02%3A00%3A4Z&sig=0MBpIj03eVLY2JmEgY9rG9KEAU330Cn8o7rf06IYCo8%3D&ske=2024-07-19T09%3A00%3A00%3A00
  Fragment
  Scheme https
  OriginalString https://dalleproduse.blob.core.windows.net/private/images/b6280fe0-1e34-42a1-8552-357853e8ae97/generated_00.png?se=2024-07-16T02%3A00%3A4Z&sig=0MBpIj03eVLY2JmEgY9rG9KEAU330Cn8o7rf06IYCo8%3D&ske=2024-07-19T09%3A00%3A00%3A00
  DnsSafeHost dalleproduse.blob.core.windows.net
  IdnHost dalleproduse.blob.core.windows.net
  IsAbsoluteUri True
  UserEscaped False
  UserInfo
```

Figure 11.11 – The output of our image generation request

As you can see from *Figure 11.11*, `GeneratedImage` has only three properties:

- `ImageBytes`, an array of bytes that will be set to null
- `ImageUri`, `Uri` pointing to the location of our generated image
- `RevisedPrompt`, an AI-generated prompt based on our original prompt

The RevisedPrompt property is particularly interesting.

What appears to be happening what appears to be happening here is that is that the prompt we provided is fed into a model that generates a revised prompt. This revised prompt is based on our prompt, but it fills in any missing details that would be relevant to the image generation model.

As image generation models continue to evolve and grow, researchers have determined that the details of the prompt matter quite a bit to get accurate and controlled images.

In our case, it took our original prompt of “A picture of a nerdy bald male author with glasses executing a bicycle kick in a football match.” and changed it to the following prompt:

Visualize a nerdy, Caucasian male author with bald head and glasses, in the middle of executing a bicycle kick in an intense football match. He wears the average football kit and the background is filled with green football pitch and cheering spectators blurred slightly to emphasize on his action.

Note how the revised prompt described the pitch, spectators, camera focus, and additional details we didn't specify.

Also, note how the prompt assumed the ethnicity of the subject of the prompt to be Caucasian. This was not in the original prompt, and while it could potentially be a safe default, it also could be considered a racial bias in the revised prompt model.

Prompt engineering tip

I consider it a best practice to specify the gender, rough age, race, and appearance of people in your images if you care about these factors. Image generation models are getting better at promoting diverse sets of individuals and combating gender and racial biases, but these are still issues we face based on biases present in historical images that our models have been trained on.

At this point, if we wanted to view the image the model generated, we could navigate to the image in the browser, but that defeats the point of having a notebook!

Polyglot Notebooks is a very customizable environment, and we can actually register a custom formatter just for `GeneratedImage` results.

We'll cover this syntax in more detail in *Chapter 14, Extending Polyglot Notebooks*, so for now, I'll just provide a cell that renders `GeneratedImages` as HTML image tags inside the notebook:

```
using Microsoft.DotNet.Interactive.Formatting;
Formatter.Register<GeneratedImage>((image, writer) =>
{
    writer.Write($"<img src=\"{image.ImageUri}\" >");
    writer.Write($"<p>{image.RevisedPrompt}</p>");
}, "text/html");
imgResult.Value
```

Now, with `imgResult.Value` as the last value in the cell, we can see the generated image in all its glory:



Figure 11.12 – DALL-E depicting the author performing a bicycle kick

Determining whether this image is an accurate representation of this author's athletic abilities is an exercise best left up to the reader.

Determinism and generative AI

Most forms of generative AI are non-deterministic, meaning that if we send the model the same request twice, we may get wildly different results. I'd encourage you to try the examples in this chapter multiple times without changes or with minor prompt adjustments to see how the results vary.

With that, we've now covered using generative AI to generate chat completions, text embeddings, and images. What's more, we used scalable and secure models hosted on Azure that can easily be replaced with more recent models as new models become available.

Summary

In this chapter, we saw how generative AI can involve using large pre-trained models to generate content from simple prompts.

We saw how chat completions allow you to generate text based on a prompt and conversation history, how image generation allows you to create serious and silly imagery, and how text embeddings help determine how similar or different the topics of two pieces of text are.

The models we interacted with in this chapter were all hosted on Azure in an Azure OpenAI Service, but generative AI can work with models directly on OpenAI, models hosted on another service (through a different set of APIs), or even models downloaded locally, using a library such as LLamaSharp. See the *Further reading* section for more information on these resources.

Working with generative AI does not require a notebook environment, but in this chapter, we saw how Notebooks can make the generative AI experience easier. Notebooks can specifically help you inspect complex result objects as cell outputs, allowing you to make minor adjustments to prompt cells and quickly evaluate your results.

Further reading

The following resources will help deepen your understanding of the topics covered in this chapter:

- *Attention is All You Need*: <https://research.google/pubs/attention-is-all-you-need/>
- *Create and deploy an Azure OpenAI Service resource*: <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/create-resource?pivots=web-portal>
- *Using Provision Deployments on Azure OpenAI Service*: <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/provisioned-get-started>
- *Azure AI Services*: <https://learn.microsoft.com/en-us/azure/ai-services/what-are-ai-services>
- *Bulk Text Analytics with Azure AI Language*: <https://accessibleai.dev/post/azureaitextanalytics/>
- *Applying Computer Vision with Azure AI Services*: <https://accessibleai.dev/post/azureaiimagev4/>
- *OpenAI Embeddings*: <https://platform.openai.com/docs/guides/embeddings>
- *LLamaSharp Repository*: <https://github.com/SciSharp/LLamaSharp>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/gen-ai/>

AI Orchestration with Semantic Kernel

In the previous chapter, we covered the basics of generative AI and how **large language models (LLMs)** can be used to generate chat responses, text embeddings, and images. We also saw how prompt engineering can help with customizing the behavior of an LLM.

In many applications, these tools will be all you need to build and deploy an effective application. However, sometimes you need the ability to integrate different data sources into your application.

In this chapter, we'll talk about AI orchestration and **retrieval-augmented generation (RAG)** and how they extend the capabilities of generative AI systems.

We'll specifically be focusing on Semantic Kernel, Microsoft's open source AI orchestration framework, and seeing how we can define simple functions that can work together to achieve complex results.

In this chapter, we'll cover the following topics:

- Understanding RAG and AI orchestration
- Introducing Semantic Kernel
- Chatting with Semantic Kernel functions
- Adding memory to Semantic Kernel
- Defining complex functions
- Calling multiple functions using plugins
- Handling complex requests with planners
- Knowing where to go from here

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the Chapter12 folder.

This chapter's code references Azure OpenAI chat and text embedding models defined in *Chapter 11*, *Generative AI in Polyglot Notebooks*.

Understanding RAG and AI orchestration

The process of training an LLM involves taking a massive amount of training data and building a mapping of how different tokens in the source data are associated with each other.

Because the training process uses the process of **self-attention**, the resulting models are able to have a larger context of the relationship between different words in sentences and even different sentences in a paragraph.

The model training process requires large scales of both data and computing resources – typically many **graphical processing units (GPUs)** – and a significant amount of time. This process is illustrated in *Figure 12.1*:

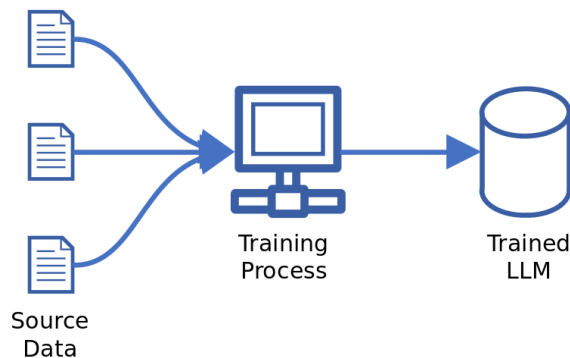


Figure 12.1 – Training an LLM on source data

As we've seen, these trained LLMs are very powerful, but they're limited to the data used in the training process.

This means that a model trained on news would not include any news stories published after the model began its training process. Consequently, you might find your model responding with out-of-date information.

For example, if you asked ChatGPT about the latest .NET version right after ChatGPT was released, it responded with an incorrect version because its training data didn't include the latest version information.

RAG attempts to address this gap by injecting additional context into the interaction with the LLM in order to expand its knowledge beyond its training data. This process is illustrated in *Figure 12.2*:

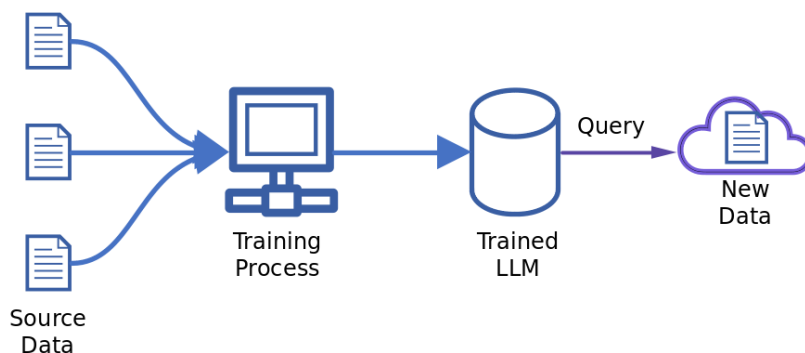


Figure 12.2 – RAG providing additional context to a model

A simple implementation of RAG would include the current date and time in the system prompt. A more complex and common implementation would be to perform a web search or database query based on the user's prompt and add relevant facts through that search.

RAG and Azure AI Search

For those looking for an easy way of getting started with RAG, I recommend investigating **Azure AI Search**. This service allows you to index different document sources on Azure and connect with Azure OpenAI Service in a single line of code and can be a great starting point when working with file-based data sources.

RAG is an effective way of augmenting an LLM, but in larger AI systems you'll find yourself wanting more than what RAG can deliver on its own.

For example, imagine a large enterprise wants to build an internal chat application to serve its users by providing information on product lines, key initiatives, sales, web analytic metrics, customers, frequently asked questions, and more.

In this scenario, it's unlikely a single data source exists that can be queried to provide information on all of these topics and systems.

To build such a system, you'd need to be able to query multiple internal and external systems and combine the results of your queries into a single coherent response.

AI orchestration systems aim to address these needs of orchestrating the querying of multiple data sources and combining the results into a single usable answer. This process is shown in *Figure 12.3*:

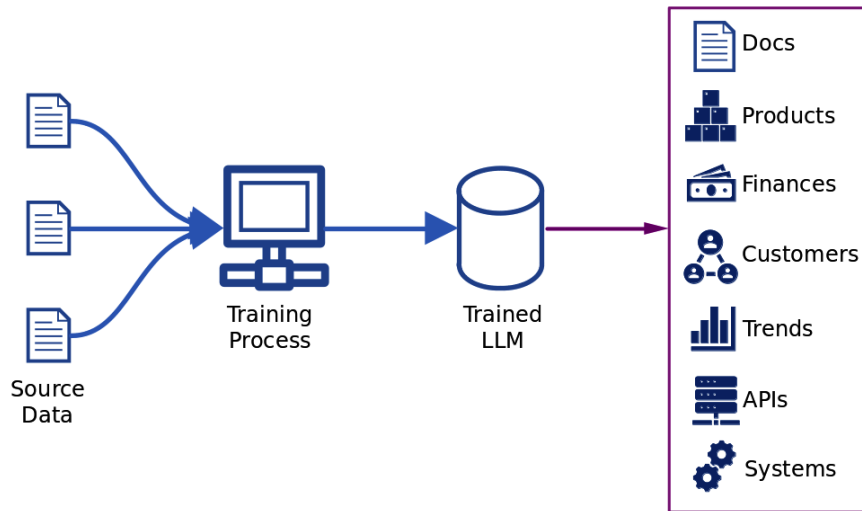


Figure 12.3 – AI orchestration coordinating multiple data sources

Not every LLM application will need RAG or AI orchestration, but once you reach a certain degree of complexity or diversity of data sources, you should be investigating AI orchestration.

There are currently two major AI orchestration solutions: Semantic Kernel and LangChain.

In this book, we'll be exploring Semantic Kernel, and specifically the **Semantic Kernel SDK for .NET**.

Introducing Semantic Kernel

Semantic Kernel is Microsoft's open source AI orchestration solution.

As with everything related to LLMs, and AI orchestration in particular, Semantic Kernel is rapidly evolving and continues to grow and change.

Semantic Kernel was first previewed in March 2023, entered beta in October 2023, and then was formally released in December of that same year.

A number of aspects of Semantic Kernel are still considered experimental, including some aspects presented in this chapter related to memory and planning. Keep in mind that by the time you read this chapter, the actual implementation details may be slightly different than what's outlined in this book.

That said, it's worth covering Semantic Kernel because Semantic Kernel and the generative AI capabilities we covered in the last chapter are the most exciting developments in the .NET AI ecosystem since ML.NET's AutoML capabilities were released in 2019.

Microsoft describes Semantic Kernel as an *enterprise-ready modular and extensible AI orchestration framework that works with existing APIs*

Using Semantic Kernel, developers can build agents with the mix of capabilities needed for their applications by using some or all of its capabilities, such as the following:

- **AI services** that connect Semantic Kernel to LLMs and other AI assets
- **Personas** that allow defining the style and personality of your system
- **Plugins** that define modular collections of functions to provide additional data to the agent
- **Planners** capable of responding to complex requests by using available capabilities
- **Semantic memories** that help with remembering conversational context and search existing knowledge
- **Integrations** and **connectors** to existing services, such as OpenAI, vector or relational databases, and Azure AI Search instances

Semantic Kernel is a huge topic and one deserving of its own book (such as *Building AI Applications with Microsoft Semantic Kernel* by Lucas A. Meyer).

While we don't have enough space in this chapter to cover all of these concepts, let's explore what components of a larger Semantic Kernel solution would look like.

We'll start with the basics: using Semantic Kernel to chat with an AI agent.

Chatting with Semantic Kernel functions

Semantic Kernel can be configured to use LLMs from OpenAI, Azure OpenAI, and locally hosted LLMs using LLamaSharp (see *Further reading* for resources on LLamaSharp).

For consistency, we'll use the same LLMs we used in the previous chapter: the chat and text embeddings we deployed on Azure OpenAI.

Building the Kernel

In order to reference our deployed models, we'll need the model name, endpoint, and a key from our Azure OpenAI Service.

We'll get these values after using C# in a similar manner to how we did in the last chapter:

```
using Microsoft.DotNet.Interactive;
string chatModelName = "gpt-4o-deployment";

string prompt = "Enter your Azure OpenAI Endpoint";
```

```
string url = await Kernel.GetInputAsync(prompt);

prompt = "Enter your Azure OpenAI key";
PasswordString key = await Kernel.GetPasswordAsync(prompt);
```

Next, we'll need to reference the Semantic Kernel SDK for .NET via its NuGet package and add using statements for its namespaces:

```
#r "nuget:Microsoft.SemanticKernel,1.16.0"
using Microsoft.SemanticKernel;
using Microsoft.SemanticKernel.Connectors.OpenAI;
using Kernel = Microsoft.SemanticKernel.Kernel;
```

Note here that we specify that the `Kernel` type we are interested in from this point forward is the `Microsoft.SemanticKernel.Kernel` type. This is necessary because `Microsoft.DotNet.Interactive` also declares a different `Kernel` type and we're importing both namespaces.

With these references in place, it's time to create our `Kernel` instance. We do this by creating an `IKernelBuilder` and then adding the various services we want to integrate before building the `Kernel` instance, as shown here:

```
IKernelBuilder builder = Kernel.CreateBuilder();
builder.AddAzureOpenAIChatCompletion(
    deploymentName: chatModelName,
    endpoint: url,
    apiKey: key.GetClearTextPassword());
Kernel kernel = builder.Build();
```

If you wanted to use LLMs that are not on Azure OpenAI Service, you'd use a different builder method to register those resources here.

Creating a prompt function

Now that we have our `Kernel` defined, let's create a `KernelFunction` that we can interact with.

`KernelFunction` is a function that Semantic Kernel can invoke and provide custom arguments to in order to get a response.

Our first function will be a prompt function that works by prompting an LLM to generate a response on our behalf.

Prompt functions require a prompt as well as some execution settings.

We'll define our prompt and settings now:

```
string coachPrompt = @"{{$input}}
Respond to the user as if you are a professional European football
```

```
coach giving players advice during a match. Focus on brief tidbits of
football wisdom.";
OpenAIPromptExecutionSettings settings = new() {
    MaxTokens = 100,
};
```

This prompt provides the user input (via the `$input` variable) and then uses a textual prompt below this to define the personality of our AI agent.

Next, we define an `OpenAIPromptExecutionSettings` object and specify that the max tokens we want to use is 100. This should keep our responses short, but may also result in the LLM abruptly cutting off long replies.

With these two variables in place, we can define our function:

```
var coach = kernel.CreateFunctionFromPrompt(coachPrompt,
    executionSettings: settings,
    functionName: "CoachPersonality",
    description: "A personality prompt for giving advice to players in
a football match");
```

The `CreateFunctionFromPrompt` method builds and returns a `KernelFunction` instance representing our new method. It requires our prompt and takes in the execution settings we defined earlier.

We're also providing a name for the function and a short description. These steps aren't necessary yet, but they'll help us later in this chapter when we add this function to a plugin.

With our function defined, let's invoke it with an input of "What is the time?":

```
KernelArguments data = new() {
    ["input"] = "What is the time?"
};
Console.WriteLine(await kernel.InvokeAsync(coach, data))
```

This code calls the `InvokeAsync` method on the kernel and passes it the `KernelFunction` we want it to invoke as well as additional `KernelArguments` that might be relevant to the function.

Note that the arguments we pass in define an input variable. This is what corresponds to the `$input` variable defined in our `coachPrompt` variable earlier.

When you run this, you should see a response similar to the following:

```
Lads, it's always time to stay sharp and alert! We'll worry about the
clock after the final whistle. Right now, keep your heads up, stay
compact in defense, and seize every moment. Possession is key—control
it, find space, and execute with precision. Remember, every second on
that pitch is a chance to make a difference. Let's push hard and play
with our hearts. Now, go out there and give it your all!
```


This reads as a classic football coach to me, but note that the response did not include the actual time. We'll talk more about this shortly.

For now, let's define another short function, this time with the personality of a butler to a crime-fighting masked vigilante:

```
string butlerPrompt = @"{{$input}}  
You are the butler to a crime-fighting vigilante. Keep your responses  
professional, but wry. Be helpful, encouraging, and encourage  
caution.";  
var butler = kernel.CreateFunctionFromPrompt(butlerPrompt,  
    executionSettings: settings,  
    functionName: "ButlerPersonality",  
    description: "A personality prompt for a butler to a crime-  
fighting vigilante");  
Console.WriteLine(await kernel.InvokeAsync(butler, data))
```

When we invoke this function with the exact same query about the current time, we get a different response:

```
Good day, Sir. The current time is precisely 2:45 PM. Might I suggest  
a brief respite for some much-needed sustenance? A vigilant mind,  
after all, requires a well-fueled body. And, as always, do proceed  
with caution. Shadowy figures cast longer silhouettes this time of  
day.
```

Clearly this agent has a different tone and style to our football coach personality.

But note that this response has a time embedded in it: 2:45 P.M. When I re-run the cell, I get a different response with 5:47 P.M. as the current time, and then 3:27 P.M. on the following invocation.

None of these times are correct, so why is our AI agent responding this way?

Keep in mind that LLMs are effectively frozen in time at the moment of their creation. They've been trained on many different documents containing times. As a result, when we request a time from the model, it thinks about what a time looks like and outputs a time like what it encountered in its training data.

Our LLM has no understanding of the *concept* of time and so this approach of generating something that looks like a time is perfectly reasonable as it doesn't understand that times need to correlate with real-world information!

We'll talk more about fixing this need later in the chapter as we cover creating functions from methods and building plugins and using planners.

For now, let's move on to adding text embeddings and memories to our agent.

Adding memory to Semantic Kernel

Semantic Kernel uses text embeddings to simulate memory. When an AI agent wants to see what it knows, it searches a database of text embeddings and pulls back the results with the largest similarity score.

Memory is available via a separate NuGet package. Because this capability is still early in development, we'll need to explicitly reference its version *and* we'll need to disable a series of SKEXP flags, as shown here:

```
#r "nuget: Microsoft.SemanticKernel.Plugins.Memory, 1.16.0-alpha"
using Microsoft.SemanticKernel.Memory;
#pragma warning disable SKEXP0001, SKEXP0010, SKEXP0050
```

These `#pragma` statements effectively acknowledge that we understand semantic memory is considered a preview feature, is subject to change, and should not yet be used for real-world production scenarios.

Reminder about preview features

As I mentioned earlier in this chapter, Semantic Kernel is still rapidly evolving. Expect that portions of this chapter will have changed significantly between when they were written and when you are reading them. Focus on the concepts and refer to the latest documentation (see links in the *Further reading* section) for the latest recommended ways of working with semantic memory and other preview features.

With our references defined, we'll now create a `MemoryBuilder` that will help define our text embedding source. We'll do that by referencing our text embedding deployment and the Azure OpenAI key and endpoint we defined earlier:

```
string embedDeployment = "text-embedding-ada-002";
MemoryBuilder memoryBuilder = new();
memoryBuilder.WithAzureOpenAITextEmbeddingGeneration(
    embedDeployment,
    url,
    key.GetClearTextPassword(),
    "model-id");
memoryBuilder.WithMemoryStore(new VolatileMemoryStore());
```

This code sets up our `MemoryBuilder` to use our text embedding deployment on Azure OpenAI Service whenever it needs to generate embedding vectors for new strings of text.

Note that we are defining a memory store at the end of this snippet. A memory store defines where these embeddings are stored and how searching for them works.

Memory stores are typically vector storage databases, but for simplicity, we're using a `VolatileMemoryStore`, which stores embeddings in memory inside of our .NET process.

`VolatileMemoryStore` is suitable for small proofs of concept and demos involving very low volumes of data and should not be used for any realistic scenarios.

With our `MemoryBuilder` configured, we can now create our memory and add some sample facts to it:

```
ISemanticTextMemory memory = memoryBuilder.Build();
string collection = "FootballFacts";
string[] facts = new[] {
    "Our next opponent is the Purple Cobras",
    "Our team is the Silver Robots.",
    "This author would make a terrible footballer.",
    "Our home pitch is at Packt Stadium",
    $"Today is {DateTime.Now.ToShortDateString()}"
};
for (int i = 0; i < facts.Length; i++)
{
    await memory.SaveInformationAsync(collection,
        id: $"Fact{i}",
        text: facts[i]);
}
```

This code loops over a series of strings representing facts the AI agent should know. Each fact is stored in our memory instance under a specific `FootballFacts` collection with a unique ID of `Fact0`, `Fact1`, `Fact2`, and so on.

Each fact also includes the text of the fact, which is sent on to our embedding model to generate the text embedding for that fact. The resulting fact is then indexed in our `VolatileMemoryStore` for later retrieval.

We can query our memory store via its `SearchAsync` method, as shown here:

```
string query = "Tell me about our team";
IEnumerable<MemoryQueryResult> results =
    memory.SearchAsync(collection, query, limit: 3);

await foreach (var result in results) {
    double relevance = result.Relevance;
    string text = result.Metadata.Text;
    string id = result.Metadata.Id;
    Console.WriteLine($"{id}: {text} ({relevance:P1})");
}
```

The `SearchAsync` method takes in the memory collection we're searching, the textual query, and the maximum number of results to return. Because the result of `SearchAsync` is `IEnumerable`,

we use `await foreach` to loop over its results, before displaying the resulting fact's `Id`, `Text`, and `Relevance`.

The “*Tell me about our team*” query produces the following output:

```
Fact1: Our team is the Silver Robots. (85.5%)
Fact3: Our home pitch is at Packt Stadium (81.9%)
Fact0: Our next opponent is the Purple Cobras (81.1%)
```

Note that our query is restricted to a maximum of three results because we specified a limit of 3. You can also specify a `minRelevanceScore` value if you want to increase or decrease the relevance required for a feature to be returned from your search results. `minRelevanceScore` defaults to 0.7 for 70% or higher.

With text embeddings and memory operational, let's introduce some more complex `KernelFunction` scenarios to start tying together the pieces of our AI orchestration system.

Defining complex functions

Earlier, we saw how we could define prompt functions that passed a textual prompt to an LLM and returned the output from that system.

It's also possible to define functions that don't involve LLMs at all and instead invoke a C# method.

Creating functions from methods

We'll demonstrate these more complex functions by defining a function that calls a simple C# method:

```
KernelFunction timeFunc = kernel.CreateFunctionFromMethod(
    () => DateTime.Now.ToShortTimeString(),
    "GetCurrentTime",
    "Retrieves the current time in the local time zone.");
```

This creates a `timeFunc` `KernelFunction` that can be invoked later.

Note how the first parameter of `CreateFunctionFromMethod` takes in a lambda expression. In this case, the expression takes in zero parameters and returns the current time as a short time string.

We also provide the function's name as `GetCurrentTime` and a short description that will be relevant later.

Now that we've defined our function, we can invoke it:

```
await kernel.InvokeAsync(timeFunc)
```

This produces the current time of 11:15 P.M. for me as well as some additional details on the response, as shown in *Figure 12.4*:

▼ 11:15 PM	
Function	► <code>GetCurrentTime</code>
Metadata	<null>
Culture	►
ValueType	System.String
RenderedPrompt	<null>

Figure 12.4 – The metadata from our function invocation

Note that the function call didn't result in a conversational string such as *"The time is now 11:15 PM"*, but was simply the *"11:15 PM"* string our method returned.

Functions created in this way can also take in parameters.

Accepting KernelFunction parameters

Let's define a sample function that returns the win/loss record of each team in the league:

```
Dictionary<string, string> teamRecords =
    new(StringComparer.OrdinalIgnoreCase) {
        { "Purple Cobras", "5 wins, 0 losses, 0 ties" },
        { "Orange Donkeys", "0 wins, 4 losses, 1 tie" },
        { "Silver Robots", "2 wins, 2 losses, 1 tie" },
        { "Maroon Baboons", "1 win, 2 losses, 2 ties" },
    };

KernelFunction teamInfoFunc =
    kernel.CreateFunctionFromMethod((string teamName) => {
        if (teamRecords.ContainsKey(teamName)) {
            return teamRecords[teamName];
        } else {
            return "There is no information on that team.";
        }
    },
    "GetTeamInsights",
    "Retrieves insights about a specific team");
```

This function takes in a `teamName` parameter, looks up the team by its name in a case-insensitive `Dictionary`, and returns its current win/loss record or a string indicating the team couldn't be found.

In a real-world scenario, this code would talk to a relational database, a file, or some external API to retrieve this information.

For simplicity of reading the chapter, the team records are currently hardcoded in a `Dictionary`, but nonetheless, this represents a RAG data source that can be connected to a chat agent, as we'll see in the next section.

For now, we'll test our function by invoking it and providing a value for the `teamName` parameter:

```
KernelArguments args = new KernelArguments {  
    ["teamName"] = "Silver Robots"  
};  
await kernel.InvokeAsync(teamInfoFunc, args)
```

This produces the expected win/loss record for the Silver Robots:

```
2 wins, 2 losses, 1 tie
```

Notice how Semantic Kernel passed the `teamName` value into our `teamInfoFunc` as the appropriate parameter.

If we had failed to provide a `teamName`, Semantic Kernel would have thrown a `KernelException` indicating that we did not supply an argument for this parameter.

Error-handling tip

When accepting parameters to kernel functions, never assume that these parameters will come in properly formatted or in an expected format. In some scenarios, Semantic Kernel will get input values for these parameters from an LLM and these values are often different than how you'd expect them to be formatted.

I consider it a good practice to handle unexpected inputs gracefully by returning a string indicating what was wrong with the input and what a valid input might look like.

Let's move on and see what a `KernelFunction` involving semantic memory might look like.

Defining a memory function

Although memory is still a concept in preview, there are already a number of ways of integrating memory into a Semantic Kernel agent.

For simplicity's sake, we'll cover the most straightforward approach: defining a `KernelFunction` that works with our `ISemanticTextMemory` instance:

```
KernelFunction searchFunc =  
    kernel.CreateFunctionFromMethod(async (string query) => {
```

```
var results = memory.SearchAsync(collection,
                                query,
                                limit: 1);
await foreach (var result in results) {
    return result.Metadata.Text;
}
return "No information found.";
},
"SearchMemory",
"Searches the memory for information");
```

This function is similar to our team information function except it is defined as an `async` method and its implementation searches the `ISemanticTextMemory` instance stored in our memory variable.

This code limits the search results to a single result and returns that result or alternatively a `no results found` string.

We can invoke the query in the traditional way:

```
KernelArguments args = new KernelArguments {
    ["query"] = "How good of a footballer would the author be?"
};
await kernel.InvokeAsync(searchFunc, args)
```

This returns the `Text` of the most relevant fact. In this case, that's the following string:

```
The author of this book would make a terrible footballer.
```

Now that we've seen how we can integrate custom code and memories into `KernelFunctions`, let's see how we can package together these functions into plugins and tie them all together into a cohesive experience.

Calling multiple functions using plugins

Semantic Kernel accomplishes AI orchestration by allowing you to integrate plugins into the kernel.

A Semantic Kernel plugin is a collection of one or more `KernelFunction` objects that can be discovered and invoked by the kernel in response to a specific input. This lets the kernel choose which function or functions might be relevant to responding to a specific prompt such as a message from a user.

Plugins can be defined by inheriting from the `KernelPlugin` class or via one of the simpler import methods built into `kernel` instances.

We'll take the latter approach because we want to create a simple plugin that combines some of the various functions we've created so far:

```
KernelPlugin plugin =  
    kernel.ImportPluginFromFunctions("FootballPlugin",  
        "A collection of functions related to football",  
        functions: [coach, timeFunc, teamInfoFunc, searchFunc]);
```

This defines and registers our plugin under the name `FootballPlugin` and provides a short description of the plugin's job. The plugin name and description will be used by Semantic Kernel to determine which plugins and functions will be most helpful in fulfilling a given request.

We also provide an array of functions, referencing `KernelFunction` instances we've created up to this point in the chapter.

With our plugin loaded into the kernel, we'll now ask the kernel a question:

```
OpenAIPromptExecutionSettings settings = new() {  
    MaxTokens=100,  
    ToolCallBehavior=ToolCallBehavior.AutoInvokeKernelFunctions  
};  
var result = await kernel.InvokePromptAsync(  
    "Are the orange donkeys any good?",  
    new KernelArguments(settings));  
Console.WriteLine(result.ToString());
```

Notice how we don't explicitly reference our plugin directly. Instead, we specify the `ToolCallBehavior` of automatically invoking kernel functions in order to fulfill a request. This setting tells our kernel that it's allowed to determine which plugins to use in fulfilling its requests.

When this cell executes, we get the following text returned from the function result's `ToString()` method:

```
The Orange Donkeys have had a tough season so far with 0 wins,  
4 losses, and 1 tie. It looks like they have some challenges to  
overcome.
```

That's a pretty accurate depiction of the facts.

In order to fulfill our request, the kernel executed the following sequence of steps:

1. Listed the plugins and kernel functions available to itself, including the name and description of each one.
2. Sent this list to the chat model and requested it flag methods that would be helpful.
3. Invoked those methods with the appropriate parameters. This means calling `teamInfoFunc` with a `teamName` of "Orange Donkeys".
4. Assembled the resulting text into a final response to the user via its chat model.

That's a pretty incredible sequence of operations. We'll look into that in more detail in our next section as we introduce planners.

Examining FunctionResult objects

Let's test out our system some more with a more complex query that formulates multiple questions:

```
FunctionResult result = await kernel.InvokePromptAsync(
    "Coach, what time is it? Who are we playing anyways? Are they good?",
    new KernelArguments(settings));
result
```

This time, instead of displaying the results in a `Console.WriteLine`, we're ending the cell with the returned `FunctionResult` object, which is then displayed in our Polyglot Notebook and shown in *Figure 12.5*:



Figure 12.5 – The FunctionResult from our query

That's a lot of information, but the most relevant piece of text is the response text.

The response text is not stored in a specific property but instead is returned from the `ToString()` method on our object.

For readability, the full text is as follows:

```
It's currently 12:01 AM. Our next opponent is the Purple Cobras, and
they're a tough team with 5 wins, 0 losses, and 0 ties so far. Let's
be prepared and bring our A-game!
```

This response appears to be completely correct in the following ways:

- It was 12:01 A.M. when I ran this cell
- According to our semantic memory, our next opponent is indeed the Purple Cobras
- 5 wins, 0 losses, and 0 ties is the correct record for the Purple Cobras according to our team information function
- A five wins/no losses record does indicate a tough opponent
- The final sentence seems to be evoking our coach personality

Clearly, AI orchestration is a powerful concept, but what else can the `FunctionResult` shown in *Figure 12.5* tell us?

If we expand the `Usage` node on the result's `Metadata` property, it lists the token usage, as shown in *Figure 12.6*:



▼ Azure.AI.OpenAI.CompletionsUsage	
CompletionTokens	48
PromptTokens	267
TotalTokens	315

Figure 12.6 – Token usage for the final rendered response

This tells us that the final prompt to generate our resulting text took 315 tokens, with 48 being used for the response text and 267 coming from the prompt that went to our LLM.

Caution

The `CompletionsUsage` metrics shown in *Figure 12.6* are potentially only a fraction of the total token usage that was used to fulfill our request. Semantic Kernel often makes multiple calls to the underlying model when generating plans and invoking functions and not all of these are included in the resulting metadata. Semantic Kernel offers flexible logging and telemetry features that can help you get a more accurate picture of token usage and exact prompts flowing to your LLM. See the *Further reading* section for additional resources on this topic.

When using Azure OpenAI Service as your LLM, you will also have a `ContentFilterResults` object in the response metadata.

Azure OpenAI content filtering

Content filtering is a safety feature built into Azure OpenAI Service that detects potentially unwanted content in either requests to the LLM or responses from the LLM.

If a request or a response is viewed as exceeding the sexual, violence, hate, or self-harm thresholds set up on your Azure OpenAI Service, the filter results will contain details on the aspect of content filtering that was flagged.

Our request and its response were safe and so the content filter results indicate safety in all four axes, as shown in *Figure 12.7*:

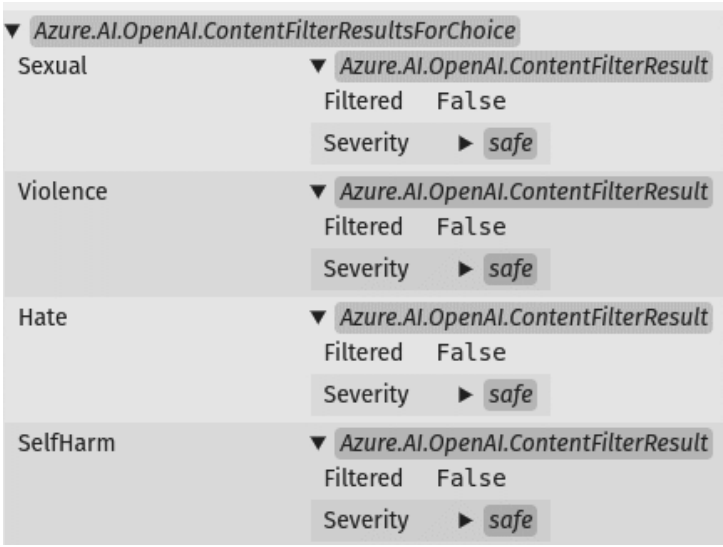


Figure 12.7 – Content filtering results

Azure OpenAI’s content filtering features are good ways for organizations to prevent undesired usage of its systems or undesirable responses from its agents.

This being said, Azure OpenAI’s content filtering can often be over-zealous in its classification of requests and responses in these areas.

Case in point, I was once presenting at a Women in Tech event and demonstrating a talking dog featuring a stuffed dog with a camera strapped to its head (see *Figure 12.8*).



Figure 12.8 – My cuddly “AI agent” for conference talks

This dog used Azure AI services for various aspects of its “intelligence.” Specifically, it used computer vision to detect objects in the crowd, Speech to Text to listen to my words, and Text to Speech to respond in a silly dog voice.

When I told the system, “You are interacting with a crowd of people and can see them with your webcam,” Azure OpenAI flagged this text as violating the sexual use policy because the word “webcam” has been used in less innocent ways before in its training data. As a result, Azure OpenAI refused to comply with my request and I was embarrassed in front of a crowd of women for violating Azure OpenAI’s sexual content filter! Thankfully, Azure OpenAI allows users to customize the thresholds for request and response filtering and even apply to be completely exempt from certain categories.

Microsoft is also continually working to improve the performance of its models and this should be less of an issue as time goes on.

Now that we’ve covered how plugins can handle simple requests by calling functions automatically, let’s cover planners, which can handle more complex plans while giving you increased visibility into what they’re doing.

Handling complex requests with planners

As we saw in the last section, the ability of Semantic Kernel to perform function calling by automatically invoking our functions can lead to powerful results.

However, function calling isn’t always enough.

If you have complex scenarios involving many different plugins and sources of data, the built-in function-calling capabilities may not perform adequately to meet your needs – or you may desire greater visibility into how different functions are chained together to generate your result.

Microsoft has an experimental `Microsoft.SemanticKernel.Planners.OpenAI` NuGet package that introduces a pair of automated planners to help with more complex tasks.

These planners, `FunctionCallingStepwisePlanner` and `HandlebarsPlanner`, may perform better with more complex tasks.

We'll see how a planner works by showing `FunctionCallingStepwisePlanner` in action:

```
#r "nuget:Microsoft.SemanticKernel.Planners.OpenAI,1.16.0-preview"
using Microsoft.SemanticKernel.Planning;
#pragma warning disable SKEXP0060
FunctionCallingStepwisePlanner planner = new();
await planner.ExecuteAsync(kernel, "Coach, what time is it? Who are we
playing anyways? Are they good?")
```

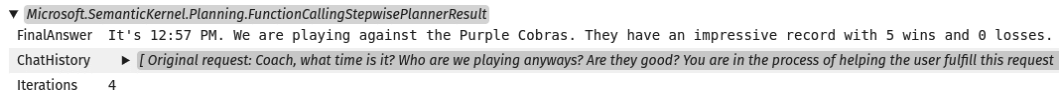
This code installs a preview version of the planners NuGet package, instantiates a planner, and then calls `ExecuteAsync` on the planner instance.

Caution

Planners, like memory, are experimental and intended for community evaluation. Because they may change or be removed entirely, planners are not yet intended for production usage and require a `#pragma` statement acknowledging this, just like we did with memory.

The planner's `ExecuteAsync` method takes in a kernel instance as well as a prompt and returns a `FunctionCallingStepwisePlannerResult`.

Because we ended our cell with the `ExecuteAsync` call, the output of this object is shown below the cell, as illustrated by *Figure 12.9*:



```
▼ Microsoft.SemanticKernel.Planning.FunctionCallingStepwisePlannerResult
FinalAnswer It's 12:57 PM. We are playing against the Purple Cobras. They have an impressive record with 5 wins and 0 losses.
ChatHistory ► [ Original request: Coach, what time is it? Who are we playing anyways? Are they good? You are in the process of helping the user fulfill this request
Iterations 4
```

Figure 12.9 – The results of chatting with the planner

Here, our planner result has three properties: a `FinalAnswer` representing the output of the request, the `ChatHistory` containing the steps executed in the final plan, and `Iterations`, which counts the number of plans that were evaluated before a final plan was selected.

The final answer the model returned in this case is as follows:

It's 12:57 PM. We are playing against the Purple Cobras. They have an impressive record with 5 wins and 0 losses.

This call generated the correct result but took 12 seconds to do so. This is significantly longer than other Semantic Kernel actions typically take.

Let's talk about why that is.

When we expand `ChatHistory` on the result object, we see that it is composed of 12 separate entries that are either `ChatMessageContent` or `OpenAIChatMessageContent` entries, as shown in *Figure 12.10*:

Count	12
(values)	index type value
	0 Microsoft.SemanticKernel.ChatMessageContent ▶ Original request: Coach, what time is it? Who are we playing anyways?
	1 Microsoft.SemanticKernel.ChatMessageContent ▶ Perform the next step of the plan if there is more work to do. When you
	2 Microsoft.SemanticKernel.Connectors.OpenAI.OpenAIChatMessageContent ▶
	3 Microsoft.SemanticKernel.ChatMessageContent ▶ 12:57 PM
	4 Microsoft.SemanticKernel.ChatMessageContent ▶ Perform the next step of the plan if there is more work to do. When you
	5 Microsoft.SemanticKernel.Connectors.OpenAI.OpenAIChatMessageContent ▶
	6 Microsoft.SemanticKernel.ChatMessageContent ▶ Our next opponent is the Purple Cobras
	7 Microsoft.SemanticKernel.ChatMessageContent ▶ Perform the next step of the plan if there is more work to do. When you
	8 Microsoft.SemanticKernel.Connectors.OpenAI.OpenAIChatMessageContent ▶
	9 Microsoft.SemanticKernel.ChatMessageContent ▶ 5 wins, 0 losses, 0 ties
	10 Microsoft.SemanticKernel.ChatMessageContent ▶ Perform the next step of the plan if there is more work to do. When you
	11 Microsoft.SemanticKernel.Connectors.OpenAI.OpenAIChatMessageContent ▶

Figure 12.10 – The chat messages composing the final plan

Each of these messages is a sequence in the larger plan.

The first message contains our request and some instructions to the LLM to generate a plan based on the available plugins and functions.

The LLM returns the following plaintext plan (abbreviated for length):

To address your questions, we need to determine the current time and gather insights about the team you are playing against. Here is the plan:

- 1. Retrieve the current time.**
- 2. Search memory to find out which team you are playing against.**
- 3. Retrieve insights about the team you are playing against.**

Let's execute this plan step by step.

The next major step the planner takes is to invoke `FootballPlugin-GetCurrentTime`, as shown by its `ToolCalls` section on the first `OpenAIChatMessageContent` and pictured in *Figure 12.11*:

ToolCalls	index value	
	0	▼ Azure.AI.OpenAI.ChatCompletionsFunctionToolCall
		Name FootballPlugin-GetCurrentTime
		Arguments {}
		Id call_AOM97JJ1SXk2PZNPBiWb4PyMZ

Figure 12.11 – The call to get the current time

The next message is just the raw string returned by the `GetCurrentTime` method: `12 : 57 PM`.

Next, the LLM is called again with the conversation history up to this point and an instruction to perform the next step or to mark the answer as our final response.

There's still more work to be done, so the next message is a request to call `FootballPlugin-SearchMemory` with a query of `"team we are playing against"`, as shown in *Figure 12.12*:

ToolCalls	index value	
	0	▼ Azure.AI.OpenAI.ChatCompletionsFunctionToolCall
		Name FootballPlugin-SearchMemory
		Arguments {"query":"team we are playing against"}
		Id call_C6KYq3oTHRqX9lTs3p1Bo5z6

Figure 12.12 – The call to search memory

Note that the arguments now specify a value for the method's query parameter. The query of `"team we are playing against"` came directly from the LLM in response to the original question we asked it.

Thankfully, our `SearchMemory` function could handle this query and returned the most relevant fact it knew:

Our next opponent is the Purple Cobras.

The next step in this process is to check whether we are ready to send the final answer. The LLM decides that we aren't because we don't have a record for the Purple Cobras and decides to call `GetTeamInsights` with a `teamName` of `Purple Cobras` to address this need, as shown in *Figure 12.13*:

ToolCalls	index value	
	0	▼ Azure.AI.OpenAI.ChatCompletionsFunctionToolCall
	Name	FootballPlugin-GetTeamInsights
	Arguments	{"teamName": "Purple Cobras"}
	Id	call_JmSmcNPX1g4HuVZuJLkAXK1E

Figure 12.13 – Querying for team win/loss records

This query returns the following response:

5 wins, 0 losses, 0 ties.

With information on the time, the next opponent, and their record, the LLM decides that its job is done and it sends on the final answer, which then sets the `FinalAnswer` string on the `FunctionCallingStepwisePlannerResult` object.

You'll note that the `Iterations` property had a value of 4. This tells me that the planner tried three other plans that didn't produce as favorable results. Maybe these plans tried to find team information for *The Purple Cobras* instead of *Purple Cobras* or they came up with a plan that wasn't effective.

Whatever the case, the results we saw from our planner were the fourth attempt at generating a valid output. Planners are incredibly sensitive to you adequately describing your plugins and functions and to the failure messages your functions return. Further, planners also will send significantly more requests to your LLM than not using a planner will. This can increase your costs as LLM usage is typically billed at a per-token level, and it can even lead to you encountering rate-limiting issues due to frequent requests.

While plugins still have their flaws and are being improved over time, they're one of the most interesting and promising aspects of AI orchestration because they allow your AI agents to tie disparate sources of data together to build and execute a plan.

Now that we've covered what AI orchestration is and how it currently works with Semantic Kernel, let's close this chapter with a discussion of the state of this technology.

Knowing where to go from here

AI orchestration is one of the new frontiers in AI. LLMs have made generative AI scenarios possible that were hard to imagine a decade ago. But, as we've seen, AI orchestration is not perfect. It can have a real financial cost, it can be sensitive to how you describe your capabilities, it can take additional time to execute, it can get answers to questions wrong, and it can be sensitive to change.

I believe we still need to see a few key innovations in AI orchestration.

First, we need to mature our testing practices for generative AI systems. Because the output of an LLM is non-deterministic and will change from request to request, interactions with LLMs are inherently hard to test in an automated manner.

There are some exciting innovations in this space, such as prompt flow, which helps automate the evaluation of LLM responses in an automated way. There are also a few LLM testing frameworks, such as DeepEval, which aims to test LLM outputs.

However, I've not yet seen anything purely .NET related that exists to evaluate LLM or AI orchestration outputs. Such a system could be designed to either use an LLM to grade the output of another LLM or by generating embeddings from an expected output and the actual output and comparing their similarity to see whether a specific threshold is reached.

Secondly, AI orchestration systems have trouble as they scale up in the capabilities they offer.

The more functions and plugins you add to an AI orchestration system, the more context needs to be sent to the planner. This increases the token usage and seems to exponentially increase the error rate planners encounter.

Once you get past a certain scale in terms of the number of functions or plugins, AI orchestration systems tend to hallucinate more and invoke functions that are not defined or make mistakes on the parameters to functions.

A more modular approach to plugins is likely the answer here.

Using a classification model or text embedding comparisons, you could potentially determine whether a plugin is relevant for addressing an incoming request.

Once you have a list of plugins that are relevant to the request, a new kernel could be created that only registers plugins that seem highly likely to be relevant in addressing the request.

Another approach might be to take on a more modular approach and segment your AI systems into multiple smaller systems that can interact with each other.

For example, you might have a kernel that exists just for answering questions about football clubs and another kernel that can handle questions related to individual players. You could then have a third kernel that would be able to communicate with the other two to fulfill complex requests.

In this approach, each kernel would own its own unique set of plugins that are specialized to the task the system is trying to accomplish.

While this approach would be more complex, it also would be more modular and testable. Changes to one system would be less likely to impact another, and the systems would scale and test better as a result.

Semantic Kernel also integrates with **OpenAI assistants**. This integration opens up new architectures for building AI orchestration systems and the modularity of such a system could help with some of these scalability concerns.

Covering the assistants API is beyond the scope of this chapter or book, but more resources are available in the *Further reading* section.

Summary

In this chapter, we explored the need for RAG and AI orchestration to address the natural limitations of LLMs.

We saw how AI orchestration frameworks such as Semantic Kernel connect a core kernel with specialized plugins and functions that can provide personalities, custom prompts, data from external sources, and knowledge from semantic memory.

We also saw how to create and invoke our own functions and plugins and how functions can work together to address complex requests.

This concludes *Part 3* of this book, focusing on exploring AI through Polyglot Notebooks.

In the final part of this book, we'll wrap everything up by showing how Polyglot Notebooks can integrate into the workflows of software engineering and data science teams.

Further reading

The following are some additional resources related to things that we covered in this chapter:

- *Semantic Kernel*: <https://aka.ms/semantic-kernel>
- *Semantic Kernel GitHub repository*: <https://github.com/microsoft/semantic-kernel>
- *Architecting AI Apps with Semantic Kernel*: <https://devblogs.microsoft.com/semantic-kernel/architecting-ai-apps-with-semantic-kernel/>
- *LLamaSharp Semantic Kernel integration*: <https://scisharp.github.io/LLamaSharp/latest/Integrations/semantic-kernel/>
- *Semantic Kernel logging and telemetry*: <https://github.com/microsoft/semantic-kernel/blob/main/dotnet/docs/TELEMETRY.md>
- *OpenAI Assistants: the future of Semantic Kernel*: <https://devblogs.microsoft.com/semantic-kernel/assistants-the-future-of-semantic-kernel/>
- Retrieval-Augmented Generation (RAG): <https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/>
- Azure AI Search and OpenAI: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/use-your-data>

- *Azure AI content safety*: <https://learn.microsoft.com/azure/ai-services/content-safety/overview>
- *Semantic Kernel planners*: <https://learn.microsoft.com/semantic-kernel/concepts/planning>
- *Prompt flow*: <https://github.com/microsoft/promptflow>
- *Evaluating LLMs with metrics*: <https://www.microsoft.com/research/group/experimentation-platform-exp/articles/how-to-evaluate-llms-a-complete-metric-framework/>
- *Unit Testing with Semantic Kernel*: <https://devblogs.microsoft.com/semantic-kernel/unit-testing-with-semantic-kernel/>
- *DeepEval*: <https://github.com/confident-ai/deepeval>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/ai-orchestration/>

Part 4:

Polyglot Notebooks in the Enterprise

We've now conducted a journey from data analytics to machine learning to generative artificial intelligence – all inside of the Polyglot Notebooks technology.

In this part, we'll bring things back home to see how Polyglot Notebooks can be used by all sorts of teams and individuals in tech, from software engineers to data analysts to data scientists.

We'll talk about how Polyglot Notebooks can create rich and interactive documentation using Mermaid diagrams. We'll explore how it can be extended by providing custom formatters and magic commands to deliver a richer user experience and new capabilities.

Your notebooks don't have to just live in VS Code, either. We'll see how .NET Interactive powers the core of the Polyglot Notebooks experience and how your notebooks can be embedded into a Jupyter Notebooks server or deployed to a GitHub Codespace.

We'll conclude the part and the book with an exploration of how Polyglot Notebooks can fit into your current and future workflows – regardless of your role – and how you might change your role if you're interested in moving into data science.

This part has the following chapters:

- *Chapter 13, Enriching Documentation with Mermaid Diagrams*
- *Chapter 14, Extending Polyglot Notebooks*
- *Chapter 15, Adopting and Deploying Polyglot Notebooks*

Enriching Documentation with Mermaid Diagrams

Notebooks gained popularity for their ability to intermix interactive code and rich documentation. The ability to add diagrams to notebooks further extends this capability.

In this chapter, we'll cover embedding Mermaid diagrams into your notebooks in order to provide simple, easy-to-maintain, minimal diagrams that support the rest of your notebook.

While there are many different types of Mermaid diagrams, this chapter focuses on those most relevant to software engineers and data scientists by covering the following topics:

- Introducing Mermaid diagrams
- Communicating logic with flowcharts
- Communicating structure with class diagrams
- Communicating data with Entity Relationship Diagrams
- Communicating behavior with state diagrams
- Communicating flow with sequence diagrams
- Communicating workflow with Git graphs

Technical requirements

This chapter assumes that you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter13` folder.

Introducing Mermaid diagrams

Part of what can make good documentation great is appropriate visuals, including charts, screenshots, and diagrams.

I often find myself thinking visually and naturally find myself drawn to communicating visually as a result. I have many vivid memories of systems analysis and design classes, spending hours in diagramming tools trying to design a “perfect” layout or wrangle little connectors. Invariably, I’d spend hours on small diagram quirks and focus far more on the presentation than the diagram’s content.

In many cases, when diagram software is used, not only do teams spend extra time unnecessarily on minor details but the people who later need to edit the diagram can’t update it because they lack the tools, licenses, or training needed to modify the files.

Mermaid diagrams avoid these problems by taking a different approach.

Mermaid diagrams use a specialized text-based format that doesn’t require an editor and focuses on simple visual presentation over offering extensive layout and styling customizations.

A sample Mermaid diagram is shown in *Figure 13.1*:



Figure 13.1 – A Mermaid flowchart diagram

This raw simplicity results in Mermaid diagrams being small, shareable, and easy to embed into other documents such as Polyglot Notebooks.

Mermaid provides a rapidly growing number of supported diagrams, including the following:

- Flowcharts and block diagrams for structural analysis
- Class diagrams and C4 diagrams for systems architecture
- Sequence and state diagrams for flow visualization
- **Entity Relationship Diagrams (ERDs)** for database design
- User journey and requirement diagrams for business analysis
- Gantt charts and timelines for project management
- Pie, quadrant, Sankey, and mindmaps for decision-making

Mermaid continues to grow and evolve with new diagram types being introduced periodically and refinements arriving for existing types.

Mermaid has grown significantly in popularity since its release and is supported by a number of different providers, including Polyglot Notebooks. Other notable environments that support Mermaid include Confluence, Azure DevOps, GitHub, GitLab, Obsidian, Notion, and various IDEs via plugin support.

In this chapter, we'll use Polyglot Notebooks to explore flowcharts, class diagrams, sequence diagrams, Git graphs, and state diagrams.

We'll introduce Mermaid by starting with basic flowcharts.

Communicating logic with flowcharts

Flowcharts are effective ways of communicating a process, such as code execution or how your code's domain logic should work.

In Polyglot Notebooks we can add a new code cell and then change the language to **mermaid** – **Mermaid** in order to work with Mermaid.

Next, let's add in some Mermaid notation specific to flowcharts:

```
flowchart LR
    load[Load Data] --> split[Split Data]
    split --> train[Train Model]
    train --> test[Test Model]
    split --> test --> predict[Predict New Values]
```

This notation starts with `flowchart LR`, which informs Mermaid of the specific diagram type – a flowchart. This is important because each diagram type in Mermaid has a different syntax, as you'll see in this chapter.

The LR indicator is optional but tells the flowchart diagram we want it to render in left-to-right mode instead of the default top-to-bottom mode.

Next, we define a series of steps, their names, and how they're related to each other. We'll talk through the specific syntax shortly, but let's look at the generated output in *Figure 13.2*:

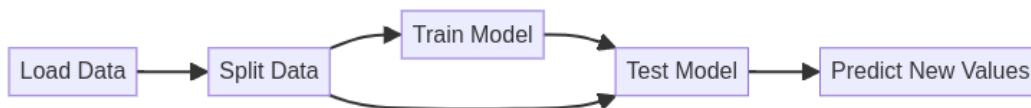


Figure 13.2 – Mermaid diagram of training a machine learning model

The steps inside our flowchart declare a sequence of steps with a short name such as *load* or *split*. Each step can optionally have a custom display name specified in brackets.

The `-->` arrow notation chains different steps together. Note that multiple steps can be chained together in an extended sequence as **Split Data**, **Test Data**, and **Predict New Values** are sequenced together.

Warning

Polyglot Notebooks unfortunately hides errors encountered with Mermaid syntax. If you run a Mermaid cell and see no output, it is likely that there's something wrong with your code. I'd encourage you to use the Mermaid Live Editor at <https://mermaid.live/> to diagnose your syntax.

Flowcharts can get a little more complicated. Let's look at the same flowchart, but with added labels on connectors, a new reverse connection from testing to re-training the model, and a group around the model training process:

```
graph LR
    load[Load Data] -- "DataFrame" --> split[Split Data]
    subgraph Training
        split -- "70-90%" --> train[Train Model]
        train -- "Trained Model" --> test[Evaluate Model]
        test -. "Performance insufficient..." .-> train
        split -- "10-30%" --> test
    end
    test -- "Performance insufficient..." --> train
    load -- "DataFrame" --> split
    test --> predict[Predict New Values]
```

Here, the added labels on connectors appear on the flowchart, along with a box for the training region containing the elements referenced in our markdown, as shown in *Figure 13.3*:

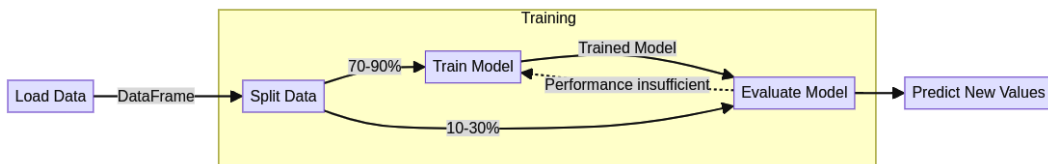


Figure 13.3 – Flowchart with connector labels and a subgraph region

Notice how the **Performance insufficient** connector was rendered with a dotted line. This is because we specified `- .` and `- .` around its connector text, denoting the style for a dotted line.

It's also possible to customize the shape styles by using supported additional shapes such as `[ ( ) ]` notation for a database, `( [ ] )` for a rounded box, and `[ [ ] ]` for a bracketed box.

The following code extends the diagram to cover inferencing in more detail and takes advantage of a few of these new shapes in the process:

```

flowchart TB
    load[Load Data]
    subgraph Training
        metrics[[[Evaluation Metrics]]]
        saved[[(Saved Model)]]
        split[Split Data] -- "70-90%" --> train[Train Model]
        train -- "Trained Model" --> test[Trained Model]
        test -- "10-30%" --> metrics
    end
    end
    subgraph Inferencing
        loaded[[(Loaded Model)]]
        saved -- "Load" --> loaded[Loaded Model]
        loaded -- "Load" --> predict([Predict New Values])
    end
    end
    test -- "Save" --> saved
    load -- "DataFrame" --> split
    test -- "Performance insufficient" --> train
    test --> metrics

```

This produces the output shown in *Figure 13.4*:

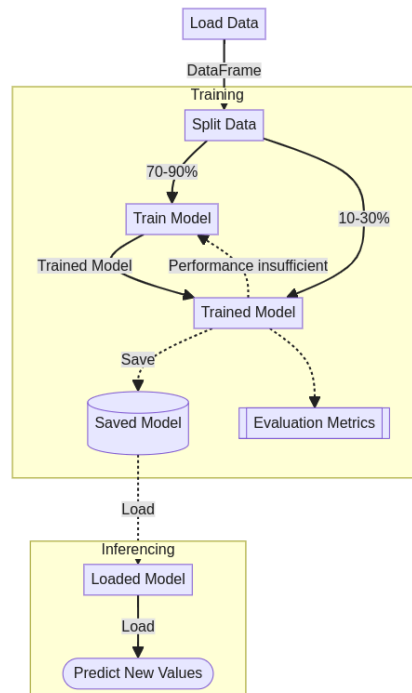


Figure 13.4 – A flowchart with customized shapes and connectors

Note that this diagram is tall instead of wide because it was defined as `flowchart TB` (top to bottom) instead of `flowchart LR` (left to right) in its first line.

There are many additional options for customizing connectors, shapes, and flowcharts in general, so I'd encourage you to visit Mermaid's official syntax documentation for more in-depth details.

See the *Further reading* section of this chapter for links to additional details on customizing flowcharts, but keep in mind that some of the latest features may not be available yet in Polyglot Notebooks as the Mermaid diagram version in Polyglot Notebooks doesn't always match the latest version of Mermaid.

Now that we've covered the basics of Mermaid diagrams via flowcharts, let's move on to one that will likely be familiar to many developers: **class diagrams**.

Communicating structure with class diagrams

Class diagrams are perhaps the most popular part of **Unified Modeling Language (UML)**, which emerged as a standardized way of communicating software designs in the late 1990's.

A class diagram contains a series of boxes representing software classes and lists the methods and attributes of each class, along with how classes relate to each other.

This makes class diagrams useful in illustrating how a set of classes within your application might interact.

We can see a simple class diagram from a game development project by adding a new Mermaid code cell with the following contents:

```
classDiagram
    direction LR
    GameObject <|-- Character
    Character *-- Weapon
    GameObject <|-- Projectile
    Weapon o-- Projectile
```

This defines a diagram of the `classDiagram` type that goes from left to right and defines relationships between the `GameObject`, `Character`, `Weapon`, and `Projectile` classes, as shown in *Figure 13.5*:

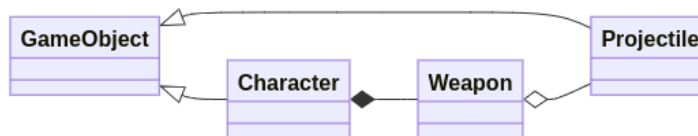


Figure 13.5 – A simple class diagram with four related classes

Note how we use `<|--` to indicate that the class on the right of the arrow inherits from the class on the left. In the same way, we use `*--` and `o--` to illustrate composition and aggregation respectively.

Composition versus aggregation

Composition and aggregation are both “has a” relationships with the class on the left of the arrow owning or controlling instances of the class on the right. The difference is that composition requires the other object to be present while aggregation indicates that it may be present but is not required. In this scenario, we’re saying that we cannot have a game character without at least one weapon, but those weapons don’t need to have fired any projectiles.

Right now, our classes lack methods and properties.

Let’s add our first class to the bottom of the Mermaid code cell:

```
class GameObject {
    Point Pos
    void Update()
    void Render()
}
```

Here, we use C#-like syntax to define our class and the attributes and methods inside of it.

However, this is not C# code and you can define any data type you’d like as the first word and any attribute or method name as the second word. You can also prefix lines with + or – to indicate public or private respectively, though this doesn’t impact the overall visual beyond changing the displayed text..

Let’s define our other three classes in the same cell:

```
class Character {
    int Health
    void Damage(int amount)
    bool IsAlive()
}
class Weapon {
    string Name
    int FireRate
    void Shoot()
}
class Projectile {
    Point Pos
    float Speed
    void Update()
    void OnCollision(GameObject other)
}
```

Notice how we can specify the parameter types and names for methods. When we run this, we see a much more realistic class diagram, as shown in *Figure 13.6*:

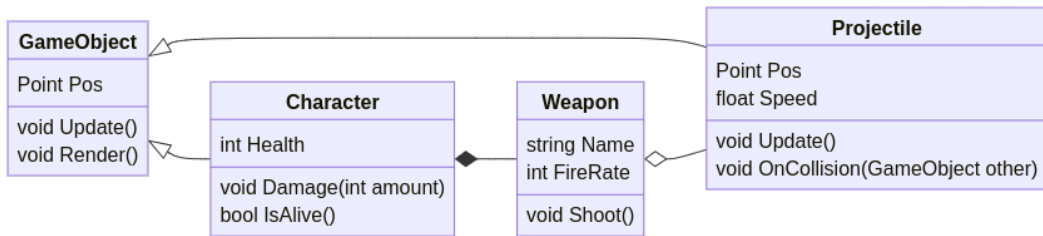


Figure 13.6 – A class diagram with methods and attributes

Some of you may be familiar with Visual Studio's native support for class diagrams. Those diagrams have additional visual polish and are automatically generated based on your code.

Compared to Visual Studio class diagrams, the Mermaid ones may seem minimal and tedious. That's true to a point, but keep in mind that the purpose of Mermaid diagrams is to allow you to embed something minimal into other documents without requiring additional compilation or analysis.

I view class diagrams in Mermaid as a simple way of communicating aspects of a larger system or a way of visualizing a proposed system on a small scale.

There are a number of additional features to class diagrams. We can't cover all of them here, but let's highlight two of interest.

First, we can indicate that a class is abstract by use of the `<<Abstract>>` annotation, as shown here for `GameObject`:

```
class GameObject {
  <<Abstract>>
  Point Pos
  void Update()
  void Render()
}
```

Additionally, we can add a note in our diagram via the `note` syntax. Let's add a note to the bottom of our cell to indicate that our game involves water balloons, not more serious forms of weaponry:

```
note for Projectile "All weapons launch water balloons"
```

With these two changes in place, we now see `GameObject` listed as abstract and a new note appearing near **Projectile**, as shown in *Figure 13.7*:

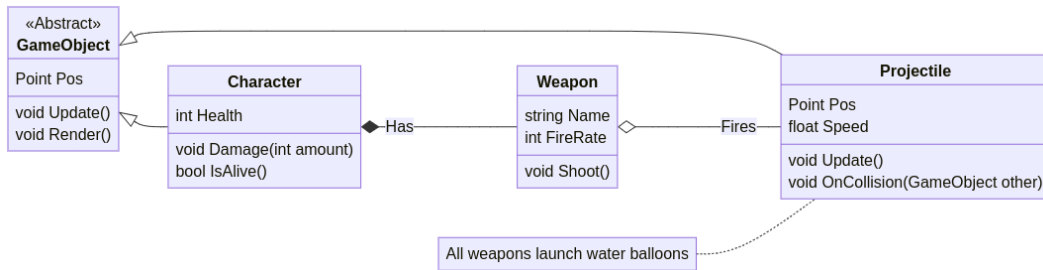


Figure 13.7 – The added annotations

There's a lot more you can do with class diagrams, including representing namespaces and even the cardinality of relationships such as one-to-one or one-to-many.

However, I tend to find myself using ERDs more to represent that information, so let's explore those next.

Communicating data with Entity Relationship Diagrams

Entity Relationship Diagrams (ERDs) are commonly used in communicating current or prospective database structures.

Using an ERD, you can show some of the tables in a relational database and how those tables relate to each other. This can help orient developers joining a project or find gaps in the systems you're designing.

Let's build a simple ERD showing how tables for football players, clubs, and matches might relate.

We'll start with another Mermaid code cell:

```

erDiagram
    Clubs ||--|{ Players : "Currently in"
    Players }|--o{ Matches : "Plays in"
    Matches }|--|| Clubs : "Home Club"
    Matches }|--|| Clubs : "Visiting Club"

```

The format of each line in our diagram starts with the table on the left, has a connector, then the table on the right, and then *must* have a colon and the name of the relationship.

The connectors follow the “crow’s foot” notation, whereby each side indicates the cardinality or multiplicity of the relationship.

While lines such as `Players }|--o{ Matches` may be daunting at first, they become easier if you focus on one side at a time. Here, each line is composed of the left entity, the left connector (or crow’s foot), a double dash, the right connector, and then the right entity.

The options for the left connector are as follows:

- $|o$ indicates that the entity on the right can have either zero or one entity in the entity on the left, but never more. In other words, that'd mean that a match could have at most one player.
- $||$ denotes that the entity on the right can have one and only one relationship with the entity on the left. Here, that'd mean that a match can have one player and one player only.
- $}\circ$ indicates that the entity on the right may have anywhere from zero to many different matches in the entity on the left. In other words, a match may have zero players, a large number of players, or something in between.
- $}\mid$ indicates that the entity on the right will have at least one match in the entity on the left, but may have more than one. For us, that'd mean that a match has at least one player and may have more than one player. This is the correct choice for our example relationship.

With the left connector covered, let's look at the right connector. Its logic works exactly the same as the left connector's, but the connections are reversed, so we write $|\{$ instead of $}\mid$. As an example, the $o\{$ in `Players }|--o{ Matches` indicates that a player may appear in any number of matches, including no matches at all.

You can read more about the different relationship types in the official documentation. For now, let's look at the output in *Figure 13.8*:

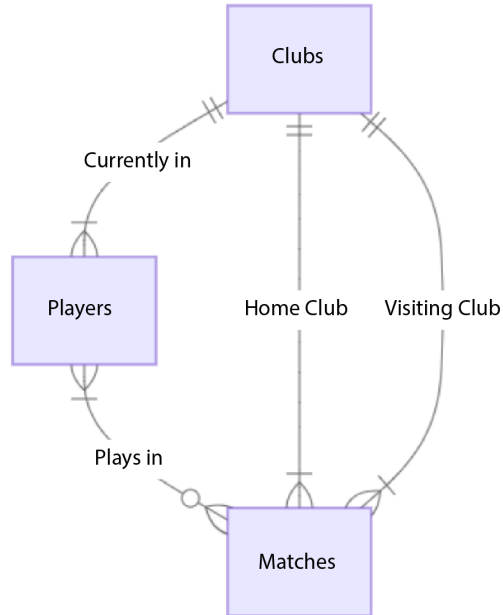


Figure 13.8 – A crow's foot entity relationship diagram

You may have noticed that our ERD only lists the entities, not the attributes associated with them. Mermaid offers the ability to keep ERDs simple like ours in *Figure 13.8* or list out the individual attributes associated with each entity.

The syntax for declaring attributes is similar to how we declared class details for class diagrams. Let's see it in action for our Clubs entity:

```
Clubs {  
    int ClubID PK  
    string Name  
    string Location  
}
```

Here, we define each attribute along with its type. Note that we may optionally add PK or FK after an attribute to indicate that the attribute is a primary key or foreign key respectively.

Let's expand the remaining entities in the same way:

```
Players {  
    int PlayerID PK  
    string Name  
    int CurrentClubID FK  
    string Position  
}  
Matches {  
    int MatchID PK  
    int HomeClubID FK  
    int VisitingClubID FK  
    int HomeScore  
    int VisitingScore  
    DateTime Date  
}
```


When we execute the ERD with these added details, we see a much richer picture, as shown in *Figure 13.9*:

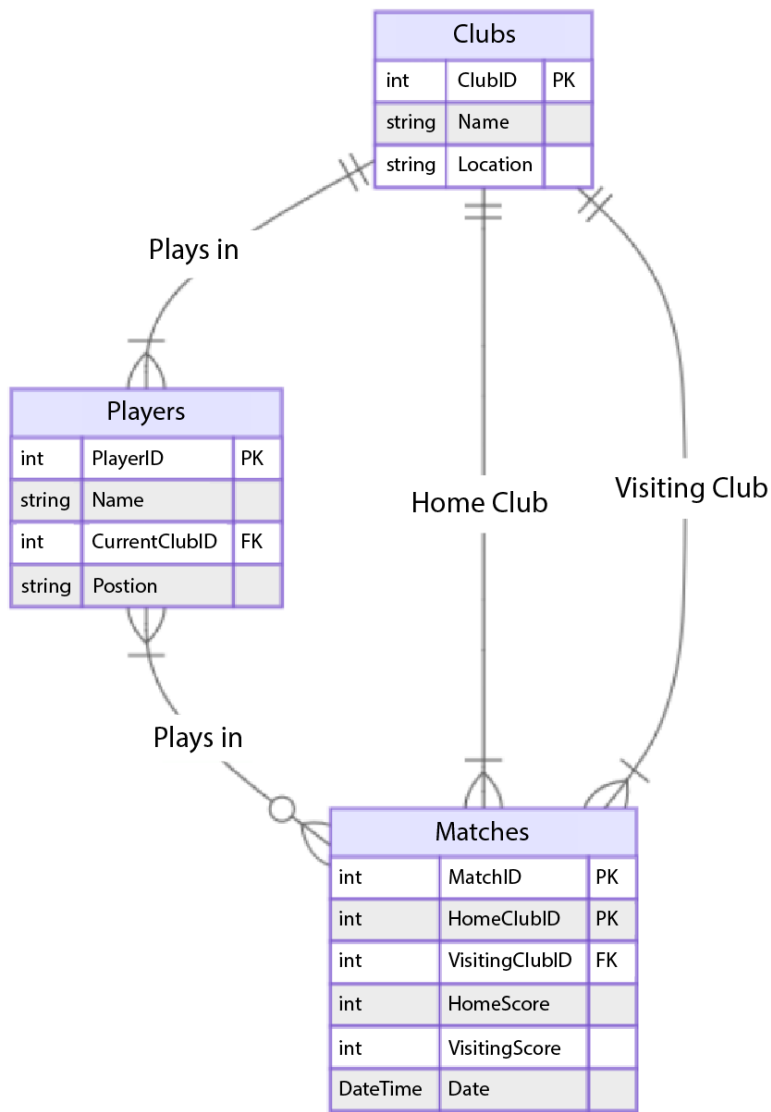


Figure 13.9 – An ERD with attributes and PKs/FKs shown

Note how the diagram now conveys data types and noteworthy keys.

While I often have to check the syntax for various Mermaid diagrams, ERDs tend to be one of the simpler diagram types, making them easier to remember.

I find ERDs to be valuable in describing proposed database changes and requesting feedback from my coworkers. You may find ERDs helpful in Polyglot Notebooks, particularly when paired together with the SQL kernel, as in *Chapter 3, Getting Data into Your Notebook*.

Let's move on to discuss how state diagrams can be particularly helpful in indicating the overall state of a complex system.

Communicating behavior with state diagrams

State diagrams are helpful when you want to communicate how specific parts of your system transition from one status to another. This can be helpful in modeling complex concepts such as the life cycle of a support ticket at your organization. I've also found state diagrams very helpful in defining how AI systems function because state diagrams pair well with the traditional AI concept of **Finite State Machines (FSMs)**, which seek to define systems in terms of states and transitions to and from those states.

FSMs are particularly popular in game development, so let's illustrate this with an example from my own experience.

A few years ago, I built a small game prototype with a more complicated "boss" monster at the end of the level. This monster would wait for the player to enter the room, then descend, roar, and begin attacking the player.

If the player hurt the boss enough, it would retreat. Additionally, if the boss was victorious or the player hid, the boss would go into an idle waiting mode and then retreat if it still couldn't find anything interesting to do. That's a complex sequence of events to describe, so it helps to diagram it using a state diagram using the following notation:

```
stateDiagram-v2
    direction LR
    Ready --> Combat
    Combat --> Retreat
    Combat --> Idle
    Idle --> Combat
    Idle --> Retreat
```

This should look very similar to a flowchart at this point, and it produces a similar output, as shown in *Figure 13.10*:

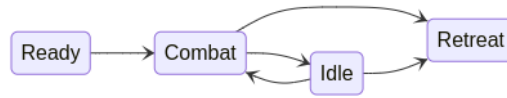


Figure 13.10 – A simple state diagram resembling a flowchart

State diagrams are more specialized than flowcharts and have a few extra shapes involved, such as a closed circle representing the start of the diagram and an open circle indicating its end.

Let's add in transition labels and our start and end states:

```

stateDiagram-v2
    direction LR
    [*] --> Combat : Player Detected
    Combat --> Retreat : Health < 10%
    Combat --> Idle : No Targets
    Idle --> Combat : Enemy Detected
    Idle --> Retreat : No Targets After 1 Minute
    Retreat --> [*]
  
```

This produces a more complex, but more descriptive, diagram as shown in *Figure 13.11*:

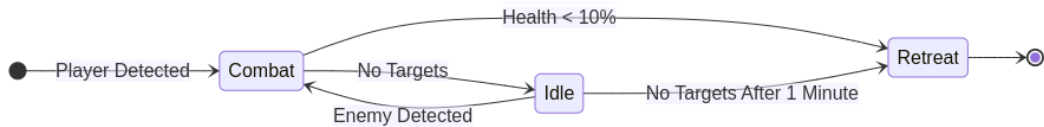


Figure 13.11 – A state diagram with transitions and a beginning and end

So far, this still looks a lot like a flowchart, so let's change it to take advantage of composite states. Think of this as a finite state machine within another finite state machine.

We can define a nested state by modifying our Mermaid syntax to define the `Combat` state in more detail:

```

stateDiagram-v2
    %% Other details omitted...
    [*] --> Combat : Player Detected
    state Combat {
        direction TB
        [*] --> Attacking
        Attacking --> Reloading : Ammo <= 0
    }
  
```

```

    Reloading --> Attacking : Ammo == Max
}

```

This effectively splits combat into a pair of Attacking and Reloading states that transition between each other, as shown in *Figure 13.12*:

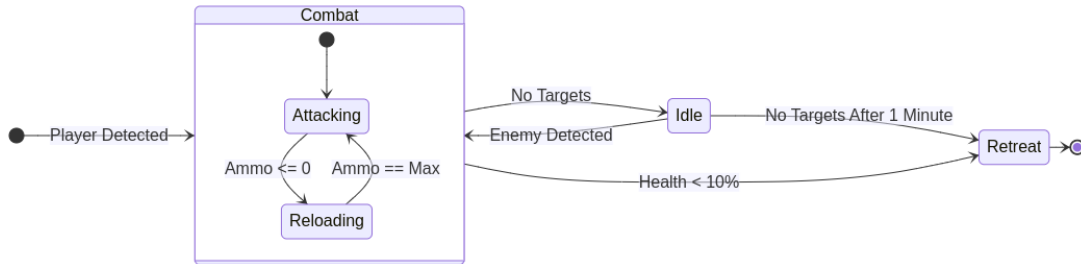


Figure 13.12 – Combat as a separate FSM

Note, in this scenario, that the Combat state has an entry point but no pre-defined exit. This means that Combat loops perpetually until the No Targets or Health < 10% events trigger to move out of that composite state.

State diagrams offer a number of interesting additional pieces of syntax, including operators for performing operations in parallel, synchronizing their results, and managing conditional logic.

Our example doesn't benefit well from parallel operations, but let's see how we could make the Combat state more explicit by adding a `<<choice>>` node to represent an if condition within the state transition:

```

state Combat {
    direction TB
    [*] --> Attacking
    state after_attack <<choice>>
    Attacking --> after_attack
    after_attack --> Reloading : Ammo <= 0
    after_attack --> Attacking : Ammo > 0
    Reloading --> Attacking
}

```

With this change, we now see a diamond representing a decision node (an `if` statement) after transitioning out of `Attacking` and then two conditional exits from that diamond. This is illustrated in *Figure 13.13*:

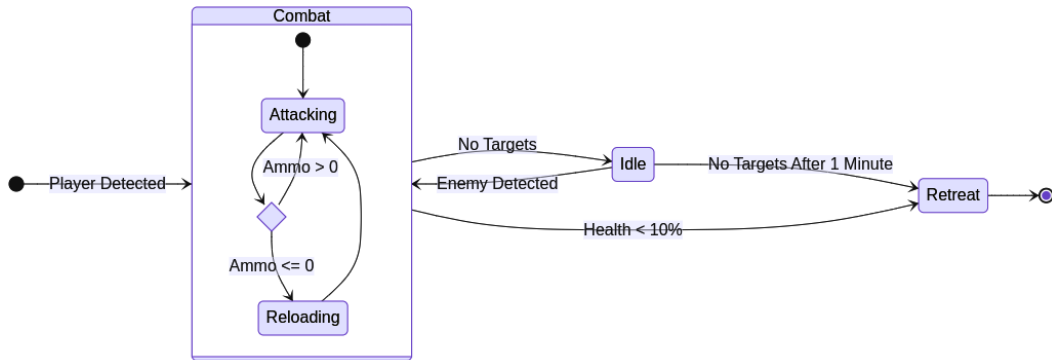


Figure 13.13 – An explicit decision on reloading vs attacking

State diagrams are more specialized, but are very helpful in visualizing transitions and the different key events in a larger system.

If you want to visualize how software components communicate instead, the sequence diagram is a great choice.

Communicating flow with sequence diagrams

Sequence diagrams are another popular UML diagram. Sequence diagrams show a top-to-bottom flow of events as various systems and components interact to carry out a task.

Let's add a Mermaid code cell to define a request from a client (such as a browser or desktop app) to a server and its underlying database:

```
sequenceDiagram
    Client->>Server: Request Data
    Server->>Database: SQL Query
    Database-->>Server: Query Results
    Server-->>Client: JSON Data
```

This syntax may not be exactly like others, but it should be familiar by this point in the chapter.

Notice how `->>` syntax indicates a request and `-->>` indicates a response. This syntax governs whether the dotted line is shown on the relationship as illustrated in *Figure 13.14*:

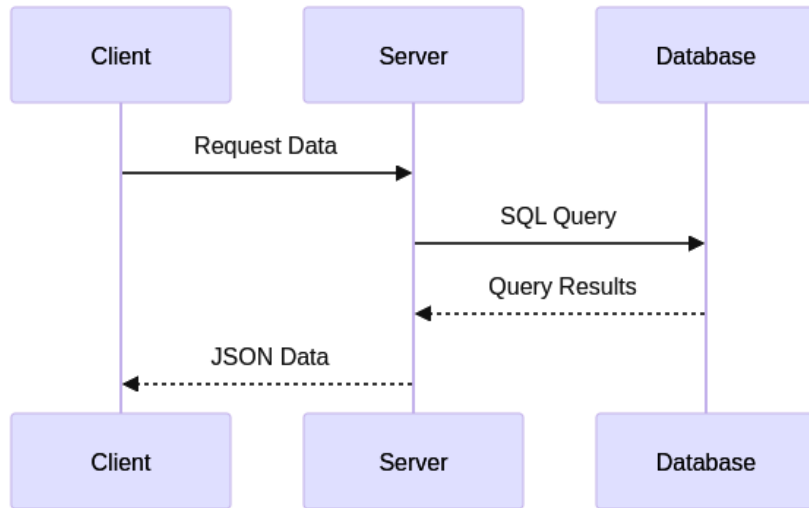


Figure 13.14 – A sequence diagram

Our sequence diagram is a little plain, so let's show some additional features by illustrating the following:

- Boxes around areas of interest
- Automatic sequential numbering of requests
- Specialized annotation notes on elements and interactions

These changes are highlighted here:

```

sequenceDiagram
    autonumber
    participant Client
    box transparent Azure
        participant Server
        participant Database
    end
    note right of Database: Azure SQL Instance
    Client->>Server: Request Data
    note over Client, Server: No auth required
    Server->>Database: SQL Query
    Database-->>Server: Query Results
    Server-->>Client: JSON Data
  
```

This produces a more detailed diagram, as shown in *Figure 13.15*:

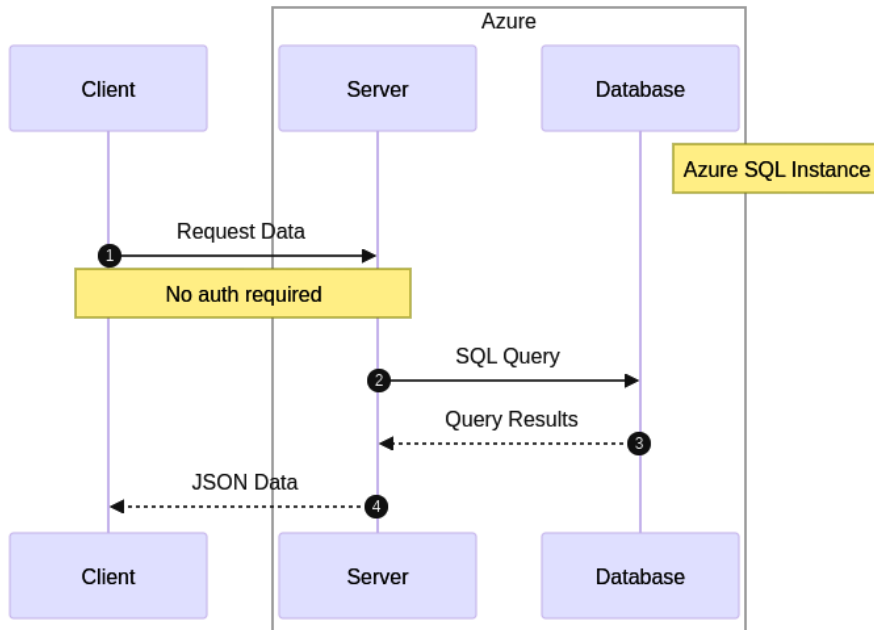


Figure 13.15 – A sequence diagram with additional annotations

Notice how each of the two notes pushes content further down in the chart. Also, note how notes can involve one participant (as in the database) or multiple (as in the client/server interaction).

Our box syntax is used here to specify that the server and database are hosted on Azure. Note that the syntax for declaring a box typically works by saying `box NameOfBox`. However, if that name is a color, the box will have no label and the color will be used. Since azure is a color, we say `box transparent azure` instead.

Let's show a few more features of sequence diagrams.

First, we can change a component from a box to a stick figure indicating an actor by using the `actor` keyword instead of `participant`.

If you're familiar with sequence diagrams, you may have noticed that the diagrams thus far haven't shown activation boxes indicating the life cycle of each component.

You can do this manually in Mermaid with `activate participantName` and `deactivate participantName` syntax, or you can add + or - (respectively) to the end of each request, as shown here:

```
sequenceDiagram
    actor Client
    Client->>+Controller: GET /football/clubs
    Controller->>+Service Layer: GetFootballClubs
    Service Layer->>+Database: SELECT * FROM Clubs
    Database-->>-Service Layer: Query Results
    Service Layer-->>-Controller: List<Club>
    Controller-->>-Client: 200 OK with JSON Data
```

This syntax uses +’s to start an activation box and - to tell the activation box that it should complete itself.

The net result of these changes is shown in *Figure 13.16*:

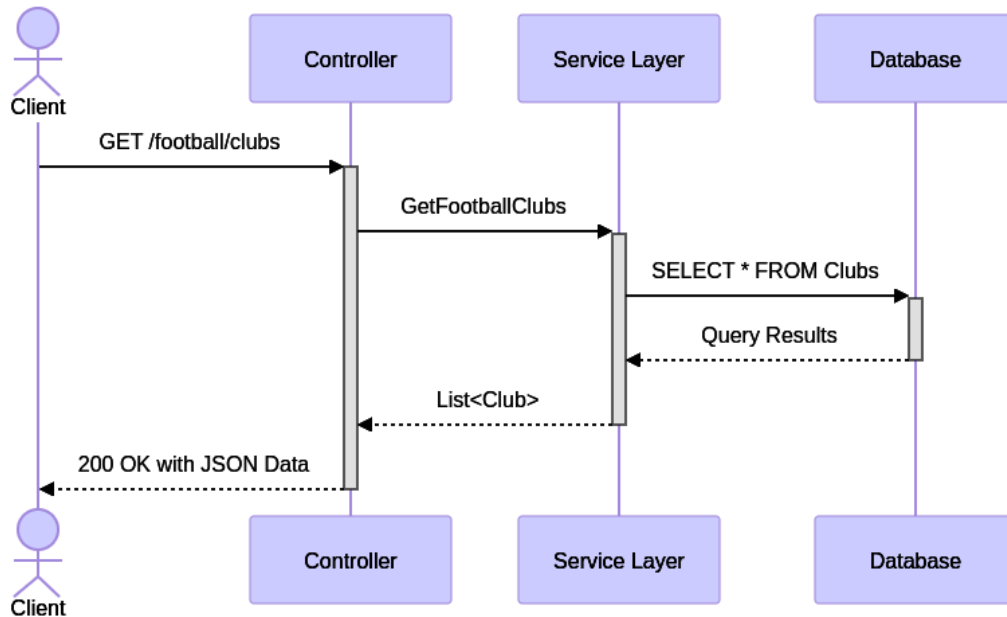


Figure 13.16 – Client as an actor with activation boxes present

Finally, you may be familiar with boxed regions in sequence diagrams such as those indicating a loop, exception handling, or conditional logic.

Sequence diagrams in Mermaid support these via keywords such as `alt` and `else`, mixed with the specialized logic inside those regions, as seen in this code sample:

```
sequenceDiagram
    actor Client
    Client->>Controller: GET /football/clubs/42
    Controller->>Service Layer: GetFootballClub(42)
    Service Layer->>Database: SQL Query
    Database-->>Service Layer: Query Results
    alt Club Found
        Service Layer-->>Controller: Club
        Controller-->>Client: 200 OK with JSON Data
    else Club Not Found
        Service Layer-->>Controller: NULL
        Controller-->>Client: 404 Not Found
    end
end
```

This produces a box labeled `alt` (sequence diagram notation for conditional logic) with a partition for our scenario, where we were able to find a football club by its ID and an alternative region where we were unable to do so. This is pictured in *Figure 13.17*:

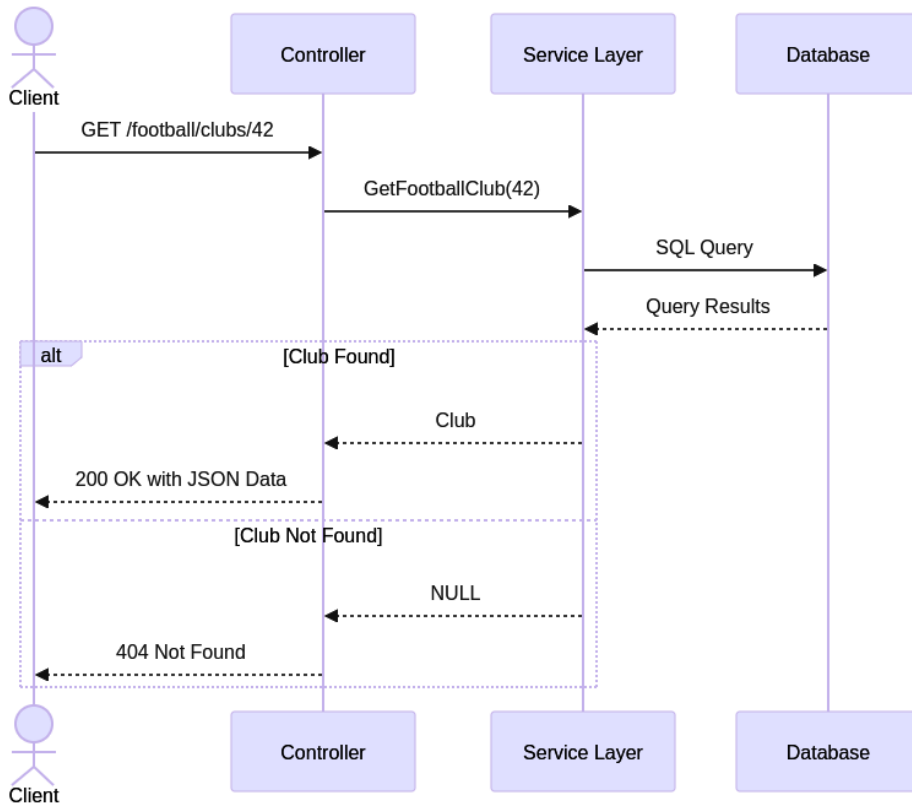


Figure 13.17 – A sequence diagram with `alt` and `else` regions

Sequence diagrams are helpful for illustrating how systems interact in sample interactions or in describing particularly complex scenarios.

Since sequence diagrams are so focused on individual component communications, they are very developer-oriented.

Let's close our chapter with a final featured diagram type that is also of particular relevance to developers: Git graphs.

Communicating workflow with Git graphs

The Git versioning system involves individual commits that modify one or more files on a specific branch. Most development teams work on more than one work item at the same time.

In order to keep code manageable and reduce the odds of shipping code on accident developers use branches for features, fixes, and even long-running software releases needing separate fixes.

This can make git repositories complex to understand without the right graphical tools. Thankfully, Git graphs help with this by allowing you to visualize different branches and how they interact with each other.

In Mermaid, Git graphs work by using commit lines to mark a commit on the current branch, the `branch` command to create a branch, `checkout` to switch to a branch, and `merge` to merge the contents of the target branch to the current branch.

All of this was designed to resemble how developers interact with Git normally, but in practice, it involves remembering which branch you last went to in your diagram, as shown here:

```
gitGraph LR:
    commit
    branch release
    checkout main
    branch "feat/5678"
    commit
    checkout main
    branch "fix/1234"
    commit
    commit
    checkout "release"
    merge "fix/1234"
    checkout main
    merge "fix/1234"
    checkout "feat/5678"
    commit
    checkout main
    merge "feat/5678"
```

This rather involved snippet illustrates a main branch, a release branch, a feature branch, a bugfix branch, and how these branches integrate together.

This effectively offers a picture of how a software team might choose to integrate its feature and fix branches into other branches, as shown in *Figure 13.18*:

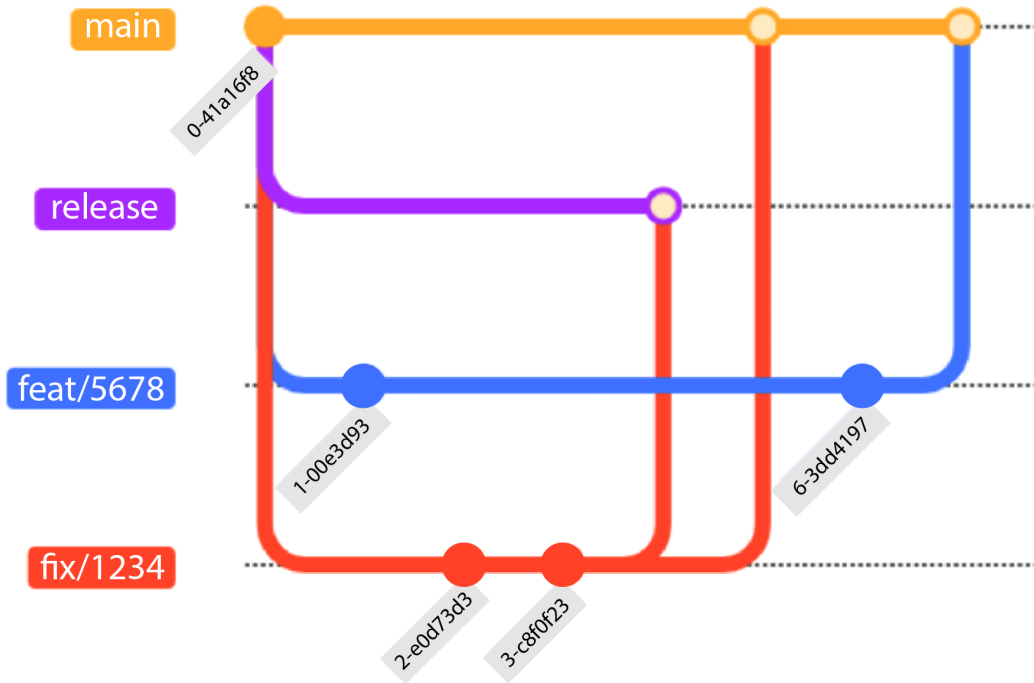


Figure 13.18 – A git graph showing how a team branches and merges code

If you're not familiar with reading a git graph, these diagrams represent individual commits as circles and branches as lines in a style resembling a subway map. If a commit represents a normal commit, its circle will appear filled while merge commits are rendered with an open center.

By default, each commit's ID is displayed as its commit number followed by a dash and some random character sequence resembling a hash.

Mermaid allows you to customize commits by specifying a custom ID instead. Additionally, you can denote specific tags such as release numbers and even flag specific commits as highlighted, or reverse commits as shown here:

```
gitGraph LR:
  commit id: "Initial Version" tag: "v1.0.0"
  branch release
  checkout main
```

```

branch "fix/1234"
  commit id: "Initial attempt"
  commit id: "Reverse initial attempt" type: reverse
  commit id: "Fixed Null Ref" type: highlight
checkout "release"
  merge "fix/1234" tag: "v1.0.1"
checkout main
  merge "fix/1234"

```

When you specify these values, you'll get a much more customized output, using helpful messages instead of random hashes, and calling attention to specific commits, as shown in *Figure 13.19*:

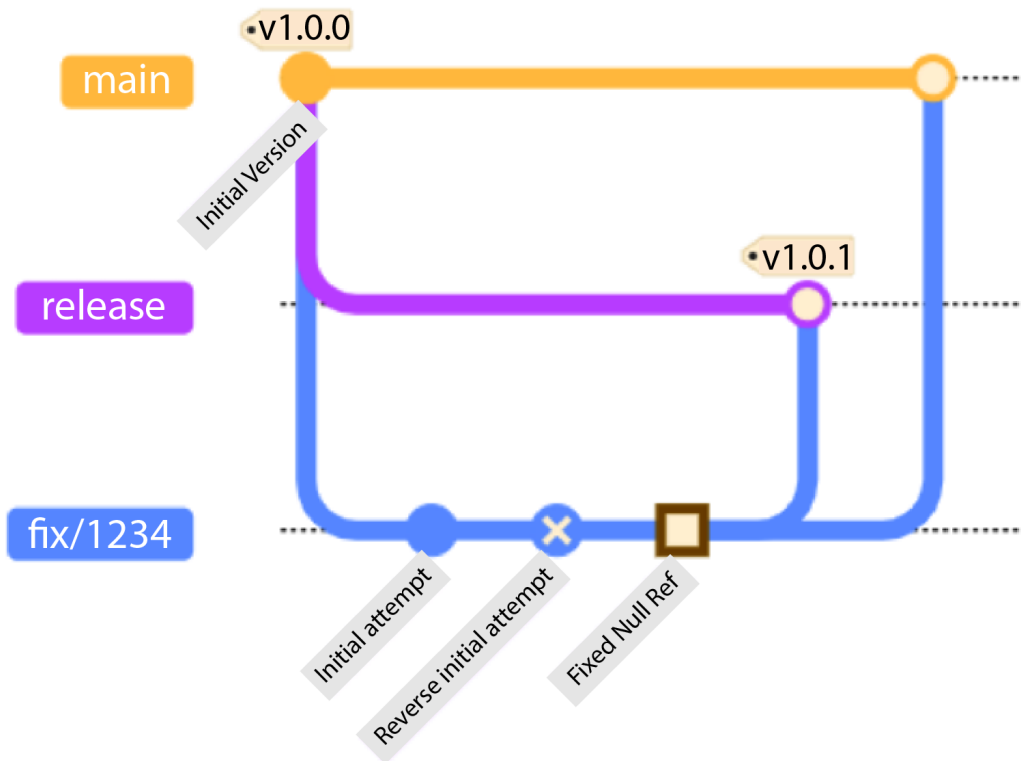


Figure 13.19 – A git graph with IDs, tags, and highlights

The ability to create small, specific samples of git branch interactions makes git graphs an ideal tool to illustrate specific branching strategies.

I've personally had good success in using this to propose and then later document Git branching and merging strategies. That gets back to the core value of Mermaid: the ability to succinctly communicate concepts in plain text that renders as a helpful diagram.

Summary

In this chapter, we explored how Mermaid diagrams rely on special text syntax to generate simple diagrams that can enrich your Polyglot Notebooks experience.

Since Mermaid diagrams are stored in plain text, they are small and don't involve storing image data. This makes Mermaid ideal for embedding in documents or storing in source control, as we learned in this chapter.

Additionally, we learned that Mermaid doesn't require any custom tooling to generate or edit diagrams, meaning anyone on your team who is familiar with the syntax can update the diagrams.

While Mermaid has a variety of diagram types, we saw how flowcharts, class diagrams, sequence diagrams, state diagrams, git graphs, ERDs, and communication diagrams can be particularly helpful.

In the next chapter, we'll take a deeper look into the internals of Polyglot Notebooks as we see how Polyglot Notebooks can be extended and customized.

Further reading

Here are some additional resources related to items discussed in this chapter:

- *Mermaid documentation*: <https://mermaid.js.org/>
- *Mermaid integrations*: <https://mermaid.js.org/ecosystem/integrations-community.html>
- *Mermaid Live Editor*: <https://mermaid.live/edit>
- *Mermaid flowchart documentation*: <https://mermaid.js.org/syntax/flowchart.html>
- *Mermaid class diagram documentation*: <https://mermaid.js.org/syntax/classDiagram.html>
- *Mermaid ERD documentation*: <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>
- *Mermaid state diagram documentation*: <https://mermaid.js.org/syntax/stateDiagram.html>
- *Mermaid sequence diagram documentation*: <https://mermaid.js.org/syntax/sequenceDiagram.html>
- *Mermaid Git graph documentation*: <https://mermaid.js.org/syntax/gitgraph.html>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/mermaid/>

Extending Polyglot Notebooks

Polyglot Notebooks offers a powerful editing environment, but sometimes, its default settings aren't enough. In this chapter, we'll look at customizing the Polyglot Notebooks experience by discussing formatters, the default formatting experience, and how to provide your own custom formatter.

We'll also revisit the concept of magic commands – those special directives such as `#!connect` and `#r` – and see how they work under the hood by building several extensions that provide their own magic commands.

By the time we're done, you'll understand how to customize Polyglot Notebooks and when you might want to do so.

In this chapter, we'll cover the following topics:

- Understanding default formatting behavior
- Styling output with custom formatters
- Exploring magic commands
- Creating a Polyglot Notebook extension
- Working with parameters
- Invoking code on kernels

Technical requirements

This chapter assumes you have installed Polyglot Notebooks and VS Code. See *Chapter 2* for additional details on this process.

You can find the notebook for this chapter on GitHub at <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> in the `Chapter14` folder.

Because this chapter is based on preview versions of the extensions API, the final shipped version may be slightly different than the code shown here. See the *Further reading* section for links to up-to-date documentation to help with any differences between the preview and final version.

Understanding default formatting behavior

One of Polyglot Notebook's best features for rapid development and prototyping is its ability to display objects below a cell. Objects are displayed if you call the `Display` extension method, available for every object, or end a cell with a variable declaration.

Polyglot Notebooks uses a series of **formatters** to determine how each object is represented in the notebook when displayed in this manner.

Objects can be displayed in a variety of ways, but each formatting method outputs objects in a specific **MIME type**.

Supported MIME types in Polyglot Notebooks include the following:

- `text/plain`: Plain text output with no formatting
- `text/html`: HTML output rendered as a fragment of a web page
- `application/json`: JSON output to render complex objects

The default behavior of Polyglot Notebooks is to show plain text values for simple variables, such as strings and integers.

For complex objects, Polyglot Notebooks typically renders an HTML table that displays values of public properties on the object.

When working with sequences of objects such as a list or an array, Polyglot Notebooks will render an HTML table, but each row represents a single object in that collection.

What about Mermaid?

As we saw in *Chapter 13, Enriching Documentation with Mermaid Diagrams*, Mermaid is a very powerful way to render data visually. Although Mermaid has a recommended MIME type of `text/vnd.mermaid`, this is not supported by all environments that a notebook could run in, including Polyglot Notebooks. We'll explore a workaround for this at the end of the chapter via a custom magic command.

To help us understand the default formatters, let's create a relatively simple C# object that represents a player in a football club:

```
public class Footballer
{
```

```

public string Name {get; set;}
public string Position {get; set;}
public decimal AverageGoals {get; set;}
public decimal AverageAssists {get; set;}
protected decimal? GoalToAssistRatio
    => AverageAssists > 0
        ? AverageGoals / AverageAssists
        : null;
}

```

This `Footballer` class is straightforward, except that it has one protected property –its `GoalToAssistRatio` property. Because this property is protected, it won't show up in the default formatter, as we'll see shortly.

Default object formatting

To see this in action, let's load up a JSON file from this chapter's repository that contains all the players in a football club:

```

using System.Text.Json;
using System.IO;
string json = File.ReadAllText("Footballers.json");
List<Footballer> team = JsonSerializer.
    Deserialize<List<Footballer>>(json);
Footballer bestPlayer =
    team.OrderByDescending(p => p.AverageGoals)
        .First();
bestPlayer

```

Here, we load the contents of our JSON file, deserialize it to a list of `Footballer` objects, find the footballer with the highest `AverageGoals` value, and then store it in the `bestPlayer` variable.

Because we end the cell with `bestPlayer`, Polyglot Notebook uses the default formatter to render that object, as shown in *Figure 14.1*:

▼ **Submission#1+Footballer**

Name	Damon Cormier
Position	Centre-Forward
AverageGoals	0.375
AverageAssists	0.075

Figure 14.1 – The default formatter rendering an object as a table

As you can see, all properties with a public getter appear in the results, but we don't see the `GoalToAssistRatio` property because it is protected.

Default collection formatting

Now, let's see the default formatting for `List<Footballer>` by displaying the `team` variable:

```
team
```

This produces *Figure 14.2*:

index	value
0	▶ Submission#1+Footballer
1	▶ Submission#1+Footballer
2	▶ Submission#1+Footballer
3	▶ Submission#1+Footballer
4	▶ Submission#1+Footballer
5	▶ Submission#1+Footballer
6	▶ Submission#1+Footballer
7	▶ Submission#1+Footballer
8	▶ Submission#1+Footballer
9	▶ Submission#1+Footballer
10	▶ Submission#1+Footballer
11	▶ Submission#1+Footballer
12	▶ Submission#1+Footballer
13	▶ Submission#1+Footballer
14	▶ Submission#1+Footballer
15	▶ Submission#1+Footballer
16	▶ Submission#1+Footballer
17	▶ Submission#1+Footballer
18	▶ Submission#1+Footballer
19	▶ Submission#1+Footballer
(8 more)	

Figure 14.2 – The formatter displaying the first 20 results

By default, .NET Interactive limits the number of rows you can see in the display to the first 20 rows, but you can modify this value by changing the `Formatter.ListExpansionLimit` property, as shown here:

```
using Microsoft.DotNet.Interactive.Formatting;  
Formatter.ListExpansionLimit = 5;  
team
```

With the `ListExpansionLimit` set to 5, we see only the first five results, as shown in *Figure 14.3*:

index	value
0	▶ Submission#1+Footballer
1	▶ Submission#1+Footballer
2	▶ Submission#1+Footballer
3	▶ Submission#1+Footballer
4	▶ Submission#1+Footballer
(23 more)	

Figure 14.3 – The formatter displaying only the first five results

This way, you can customize the number of results shown to meet your preferences.

The `Formatter` class provides a number of other interesting capabilities, including the following:

- Defining a string to use when representing `null` values
- Setting a recursion limit to limit how much objects can be expanded to form cyclical references
- Formatting an object to a specific MIME type
- Changing the default MIME type
- Resetting all formatting objects and custom formatters

If you're curious about these capabilities, I've included links to more documentation on formatters in .NET Interactive in the *Further reading* section of this chapter.

Let's explore formatters in more depth by creating our own custom formatter.

Styling output with custom formatters

As we saw in *Figures 14.2* and *14.3*, the default formatting for `List<Footballer>` is not very helpful in understanding our team at a glance. Thankfully, the `Formatter` class gives you a `Register` method that allows you to provide your own custom formatter.

The `Register` method takes in a specific type that the formatter should format, `FormatDelegate`, consisting of an instance of that type, and a writer that allows you to write content to the output. Let's see it in action by registering custom formatting for any `IEnumerable` of `Footballer` objects:

```
Formatter.Register<IEnumerable<Footballer>> (
(players, writer) => {
    writer.WriteLine("<h3>Team Scoring Stats</h3>");
    writer.WriteLine("<table>");
    writer.WriteLine("<thead><tr>");
    writer.WriteLine("<th>Name</th><th>Position</th>");
    writer.WriteLine("<th>Avg Goals</th><th>Avg Assists</th>");
    writer.WriteLine("</tr></thead><tbody>");
    foreach (var p in players) {
        writer.WriteLine("<tr>");
        writer.WriteLine($"<td>{p.Name}</td>");
        writer.WriteLine($"<td>{p.Position}</td>");
        writer.WriteLine($"<td>{p.AverageGoals}</td>");
        writer.WriteLine($"<td>{p.AverageAssists}</td>");
        writer.WriteLine("</tr>");
    }
    writer.WriteLine("</tbody></table>");
}, "text/html");
```

This is a lot of code, but let's focus first on the `Formatter.Register` call with its `IEnumerable<Footballer>` generic type and the specified MIME type of `text/html`.

This code tells .NET Interactive to use the method we registered whenever it needs to format any `IEnumerable<Footballer>`, which includes arrays, lists, or other collections.

Inside the delegate, the `players` parameter is `IEnumerable<Footballer>`, representing the object we're trying to format.

The method's implementation here writes out HTML that represents a header and a table with rows for each player. With our formatter registered, let's produce a short sequence of players:

```
team.Take(5)
```

Polyglot Notebooks sees that this sequence is `IEnumerable<Footballer>`, so it uses our custom formatter to generate HTML. Since the formatter specifies `text/html` as its MIME type, .NET Interactive tries to render out output as HTML, which produces *Figure 14.4*:

Team Scoring Stats

Name	Position	Avg Goals	Avg Assists
Moses Mitchell	Goalkeeper	0.0	0.0
Clark Casper	Centre-Back	0.0	0.05263158
Gerardo Kutch	Defensive Midfield	0.032258064	0.0
Earnest Lynch	Central Midfield	0.04347826	0.057971016
Gregory Ernser	Centre-Back	0.03846154	0.03846154

Figure 14.4 – Footballers rendered using our custom formatter

As you can see, a little custom code can drastically improve the overall notebook experience.

Formatter code too messy?

If you find the code in our custom formatter hard to read, there are a few options that can help. First, you can store the lambda expression in a variable or define it as a method. That step would simplify the code in the earlier listing and reduce indentation as well. Secondly, .NET Interactive offers a more object-oriented approach via `ITypeFormatterSource` implementations. See the *Further reading* section of this chapter for links to additional formatting documentation.

Usually, adding a custom formatter is enough to improve my Polyglot Notebooks experience when working with complex objects, but it's not always enough. Sometimes, you will want to extend Polyglot Notebooks even further. When that happens, you'll probably want to consider creating a custom magic command. Before we detail the process of creating a magic command, let's briefly recap what magic commands are.

Exploring magic commands

Throughout this book, we've seen a number of magic commands, such as `#r`, `#!connect`, and `#import`. In this section, we'll discuss a few other magic commands and talk about how they are implemented at a high level. Put simply, **magic commands** are special methods triggered by unusual syntax, referred to as **directives**.

When a kernel in .NET Interactive encounters a registered directive, it follows the following approximate sequence:

1. The directive is marked as ignored and not treated as code by the language kernel. In other words, the C# kernel won't attempt to compile a `#r` magic command.

2. Any relevant parameters are identified and extracted from the command. For example, the `#r "nuget:Microsoft.ML"` command extracts the NuGet string as a parameter and errors if it is not specified.
3. .NET Interactive creates an instance of a relevant `KernelCommand` object and sets properties as needed. We'll talk more about this later.
4. .NET Interactive invokes `KernelCommand`, passing in information about its current execution context.
5. The remainder of the cell executes.

This is a decent number of steps, and the details of this will be clearer once we create our own extensions. For now, let's start by using a helpful built-in diagnostic magic command:

```
#!about
```

Because `#!about` is registered as a directive in the .NET Interactive series of kernels, a registered handler is invoked, and we can see the following diagnostic output shown in *Figure 14.5*:

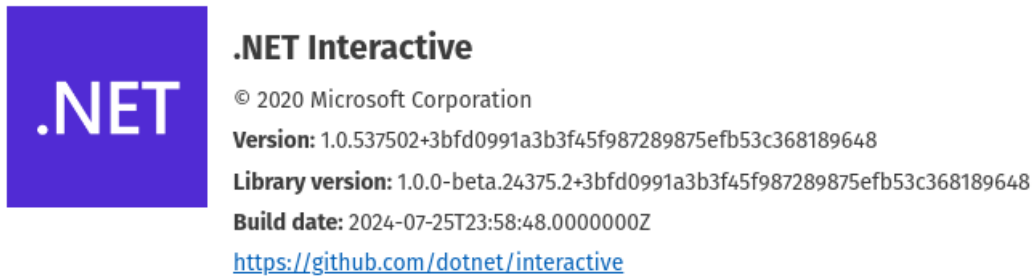


Figure 14.5 – The diagnostic output of the `#!about` magic command

Here's another example, this time of a `#!time` magic command that displays the execution time as a console message below the cell:

```
#!time
for (int i = 0; i < int.MaxValue; i++) {
    // Do nothing
}
```

In this case, we're looping from zero to the maximum integer value of a bit over 2.1 billion. Even though this loop does nothing internally, its execution takes some time. When I run this on my writing and presentation travel machine, I see the following output:

```
Wall time: 2178.4528ms
```

In other words, it took just over two seconds to run that cell on my machine when I ran it.

Over the course of the book, we've seen other magic commands, including the following:

- `#r` to import NuGet packages
- `#!import` to import code or notebook files
- `#!connect` to connect to a SQL or KQL data source
- `#!share` to share the current value of a variable from one kernel to another

There are many other magic commands built into .NET Interactive, including a magic command to list other magic commands:

```
#!lsmagic
```

When the `#!lsmagic` magic command executes, it looks at all available kernels and inspects them for registered directives. It then displays the magic command for that directive, along with its description. A small fragment of this can be seen in *Figure 14.6*:

.NET kernel

`#!about`

Show version and build information

`#!c#`

Run the code that follows using the csharp kernel.

`#!C#`

Run the code that follows using the csharp kernel.

`#!connect`

Connects additional subkernels

`signalr`

Connects to a kernel using SignalR

`stdio`

Connects to a kernel using the stdio protocol

`jupyter`

Connects a Jupyter kernel as a .NET Interactive subkernel.

Figure 14.6 – A listing of all magic commands

When you install a NuGet package, .NET Interactive will look for any custom extensions available in that package and register those magic commands. This is how SQL Server was able to register a magic command after we imported it.

Now that we've covered what magic commands are and what basically happens when you invoke one, let's see how to create one of our own.

Creating a Polyglot Notebook extension

We'll start by creating a simple extension that offers a `#!flip-coin` magic command. This magic command will simulate a coin flip and print either `Heads` or `Tails` below the cell when it is invoked.

Does this really need a magic command?

A simple coin flip isn't usually the type of thing you'd need a magic command for. Instead, defining a `CoinFlip` method that you could then call in a cell would be fine. Normally, I'd consider making a magic command if something needs to interact directly with the kernel in some way, as we'll see with our Mermaid example later. However, we need to start somewhere, so let's begin with a simple example and work our way up from there.

We'll start by defining `KernelActionDirective` for our new `#!flip-coin` magic command. This object will contain the magic command text we wish to reserve, as well as a description of the command, as shown here:

```
using Microsoft.DotNet.Interactive;
using Microsoft.DotNet.Interactive.Directives;

KernelActionDirective coinFlip = new("#!flip-coin")
{
    Description = "Flips a coin and displays the result.",
};
```

Next, we'll add our directive to .NET Interactive's root kernel, making it available in all other hosted kernels. We do this by providing the directive we just created, as well as an anonymous method to invoke when the magic command is invoked. This method takes in the instance of the `KernelCommand` class that was created when the command was invoked, as well as the kernel's execution context.

Our implementation will get a random number, which is either 0 or 1, and display `Heads` or `Tails`, respectively:

```
Random random = new();
Kernel.Root.AddDirective(coinFlip,
    (command, context) => {
        string result = random.Next(0, 2) == 0
            ? "Heads"
```

```
        : "Tails";  
        context.Display(result);  
        return Task.CompletedTask;  
    }  
);
```

Note that the anonymous method returns `Task`. Because we're not doing anything asynchronous in this method, we simply return `Task.CompletedTask` to keep the compiler happy. With the following command registered, you can now invoke it:

```
#!flip-coin
```

When I run this, I see the following output:

```
Heads
```

Re-running the cell a number of times produces `Heads` and `Tails` randomly, indicating that the magic command is running just fine.

Re-registering magic commands

One thing you'll encounter with magic commands is that if you try to run the cell that calls `AddDirective` multiple times, you'll get an error if the directive has already been registered. You can solve this by restarting the notebook or by using a different magic command name.

Now that our magic command has been registered, it will show up with its description when we run `#!lsmagic`, as shown here:

```
#!flip-coin  
Flips a coin and displays the result.
```

Now that we've covered a very simple magic command, let's see a more complex one that works with parameters.

Working with parameters

Remember how the default formatter for an object only showed the public properties?

This can be a real problem when working with ML.NET in particular, as many aspects of a machine learning pipeline have no public ways of inspecting their current configurations. This makes it hard to see what settings a pipeline step has, which, in turn, makes it hard to understand your overall processing pipeline.

While a more formal solution to this problem would be to have those objects expose property getters for relevant information, I've worked around this by using reflection to show the internal

state of objects. Reflection is .NET's built-in ability to dynamically look over the composition of an object or class in terms of its properties, methods, and fields. We're going to use reflection to build a Dictionary of each property or field on an object. We'll do that with this simplified `ReflectObject` method implementation:

```
using System.Reflection;
Dictionary<string, string> ReflectObject(object obj)
{
    Dictionary<string, string> properties = new();
    Type t = obj.GetType();
    BindingFlags flags = BindingFlags.Instance |
        BindingFlags.Public |
        BindingFlags.NonPublic;
    foreach (var prop in t.GetProperties(flags))
    {
        object? value = prop.GetValue(obj);
        properties[prop.Name] = value?.ToString() ?? "null";
    }
    return properties;
}
```

This code looks at the object coming in, and then it uses reflection to get public or private properties or fields belonging to that object from the object's overall type information. Next, the method loops over each of these, getting the current value of the reflected property or field, and then it stores it in the Dictionary. This code isn't very robust, particularly in how it represents object values, but it's simple enough for our demo scenario here. Let's see it in action by calling the new method on our `bestPlayer` variable from the earlier formatting section:

```
ReflectObject(bestPlayer)
```

This produces *Figure 14.7*, showing the full reflected state of the object:

key	value
Name	Damon Cormier
Position	Centre-Forward
AverageGoals	0.375
AverageAssists	0.075
GoalToAssistRatio	5

Figure 14.7 – A dictionary of reflected object values

Note how `GoalToAssistRatio` appears here, even though it's protected. That's reflection in action. Now that we've established a method to reflect things, let's start hooking it up to a magic command. We'll want to be able to run our magic command like this:

```
#!reflect bestPlayer
```

In order to support this, we'll need to have a parameter going into our magic command to represent the variable we want to reflect.

This means we'll need to instantiate a `KernelDirectiveParameter` object to store the name of the variable that we want to reflect:

```
KernelDirectiveParameter varName = new("--variable-name") {  
    Description = "The name of the variable",  
    Required = true,  
    AllowImplicitName = true  
};
```

This declaration contains the following pieces of information:

- `--variable-name`, which is the name of the parameter.
- `Description`, which is a short description of the parameter used in the help text.
- `Required`, indicating whether or not the parameter is required when invoking the command.
- `AllowImplicitName`, defining whether or not the parameter is assumed when a parameter name isn't specified but a parameter value is provided. There can be only one parameter with `AllowImplicitName` set to `true` for a given command.

Next, we'll create our directive as we did before, but this time, we'll specify a collection of parameters that includes this new parameter:

```
KernelActionDirective reflectDirective = new("#!reflect") {  
    Description = "Reflects the internal state of the object",  
    Parameters = { varName }  
};
```

Because we're working with parameters, there's now a new step we'll need to do to get the parameter value when the command is invoked – defining a `KernelCommand` class:

```
using Microsoft.DotNet.Interactive.Commands;  
using System.CommandLine;  
  
public class VariableNameCommand : KernelCommand  
{
```

```
public string VariableName { get; set; }
}
```

This `VariableNameCommand` class inherits from `KernelCommand`, representing a command that is invoked when a magic command is executed. This code defines a `VariableName` property that matches the `--variable-name` parameter name we declared earlier.

.NET Interactive uses conventions to connect parameter values to representative properties, much like how JSON deserialization works in other areas. As long as your property name matches the parameter name and is public with a public setter, it should be set by the kernel when your command is invoked.

Next, we're going to need a reference to the specific .NET interactive kernel used for the C# language to retrieve our variable later on by its name. We can do this via the `FindKernelByName` method on the current kernel:

```
using Microsoft.DotNet.Interactive.CSharp;
CSharpKernel csKernel =
    (CSharpKernel) Kernel.Current.FindKernelByName("csharp");
```

With this kernel in place, we can finally add our directive. This time around, we'll call `AddDirective<T>` instead of `AddDirective`. The difference between the two methods is that the generic type overload allows you to specify the `KernelCommand` type that gets instantiated when the command is invoked.

Since we need to be able to get our `VariableName` property from our command, we'll specify that the command should be the `VariableNameCommand` type we defined earlier:

```
csKernel.AddDirective<VariableNameCommand>(reflectDirective,
    (command, context) => {
        string name = command.VariableName;
        if (csKernel.TryGetValue(name, out object obj)) {
            var properties = ReflectObject(obj);
            properties.Display();
        } else {
            context.DisplayStandardError($"Can't find {name}");
        }
        return Task.CompletedTask;
    }
);
```

There are a few interesting things about this code that are different from our coin flip directive earlier:

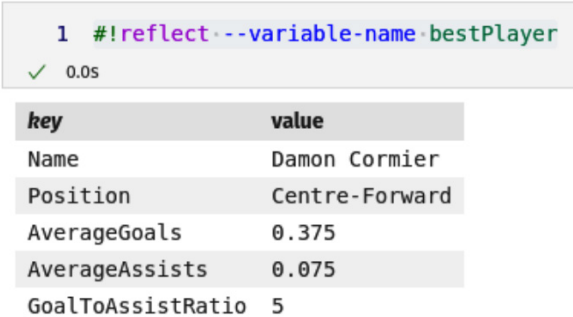
- We're registering this command specifically on the C# kernel because we are writing it to explicitly work in that kernel's context
- We use the `TryGetValue` on the C# kernel to try to find the variable by the name that the user supplied

- If we find the variable, we call the `ReflectObject` method we defined earlier and use the `Display` extension method to show it in the notebook
- If we don't find it, we use the context's `DisplayStandardError` method to indicate that the command failed

With the command registered, we can now invoke it and specify a value for `--variable-name`:

```
#!reflect --variable-name bestPlayer
```

This reflects the variable we provided and displays the result, as shown in *Figure 14.8*:



```
1 #!reflect --variable-name bestPlayer
```

✓ 0.0s

key	value
Name	Damon Cormier
Position	Centre-Forward
AverageGoals	0.375
AverageAssists	0.075
GoalToAssistRatio	5

Figure 14.8 – The results of our `#!reflect` magic command

Because we defined our parameter with the `AllowImplicitName` property set to `true`, we can also successfully invoke the command without the parameter name:

```
#!reflect bestPlayer
```

While this admittedly is pretty interesting, I'm not sure it's completely worth having a magic command for, since we could have just as easily simply called our `ReflectObject` method manually. With our added understanding of commands and parameters, let's build one last magic command that truly does need to work closely with the kernel.

Invoking code on kernels

In this final section of the chapter, we'll build a magic command that can take an `IEnumerable<string>` and represent it visually as a Mermaid flowchart, using a sequence of steps. This will work by taking in any `IEnumerable<string>` in the C# kernel, building a string of Mermaid code, and then sending that Mermaid code to the Mermaid kernel for rendering.

Let's start by defining a simple sequence of steps:

```
string[] steps = new[]
{
```

```

    "Load Data",
    "Train / Test Split",
    "Training",
    "Evaluation",
    "Inferencing"
};

```

These steps are hopefully familiar by this point in the book, as they represent the typical steps of a machine learning model training process. Next, let's build a C# method to take in a sequence of items and generate Mermaid syntax from them:

```

string GenerateMermaidFlowchart(IEnumerable<string> items)
{
    List<string> itemList = items?.ToList() ?? new();
    StringBuilder sb = new();
    sb.AppendLine("flowchart LR");
    string first = items.FirstOrDefault() ?? "No Items";
    sb.AppendLine($" s0[{first}]");
    for (int i = 1; i < itemList.Count; i++)
    {
        sb.AppendLine($" s{i - 1} --> s{i}[{itemList[i]}]");
    }
    return sb.ToString();
}

```

The details of this code aren't that important; we're essentially declaring flowchart nodes with IDs of s1, s2, s3, and so on and descriptions of the actual string that we want to render. We then make sure each step is linked to each other.

Let's test it out on our sequence of steps:

```
GenerateMermaidFlowchart(steps)
```

This produces the following Mermaid code:

```

flowchart LR
    s1[Load Data]
    s1 --> s2[Train / Test Split]
    s2 --> s3[Training]
    s3 --> s4[Evaluation]
    s4 --> s5[Inferencing]

```

While this doesn't render on its own, if you copy and paste the Mermaid syntax, you'll see it rendered as a flowchart, as shown in *Figure 14.9*:



Figure 14.9 – A flowchart of our steps

Being able to generate Mermaid on our own is neat, but having to put it into a new Mermaid cell is tedious, so let's make a magic command that can handle this in a single step.

In order to do this, we'll need a reference to both our C# kernel and our Mermaid kernel. We have the C# kernel already stored in `csKernel`, so let's find the Mermaid kernel by its name:

```
Kernel mermaidKernel =
    Kernel.Current.FindKernelByName("mermaid");
```

Next, let's define our directive. This directive will need a parameter sent to it, so let's reuse our `--variable-name` parameter from earlier as we define our new directive:

```
KernelActionDirective flowDir = new("#!flow")
{
    Description = "Generates a Mermaid flowchart",
    Parameters = { varName }
};
```

Since we have the same parameters, we can also reuse our `VariableNameCommand` class from earlier and don't need to redefine it.

Our next step is to register our directive in the C# kernel:

```
csKernel.AddDirective<VariableNameCommand>(flowDir,
    async (command, context) => {
        string name = command.VariableName;
        if (csKernel.TryGetValue(name, out object obj))
        {
            IEnumerable<string> items = (IEnumerable<string>) obj;

            string md = GenerateMermaidFlowchart(items.ToArray());
            context.DisplayStandardOut(md);

            SubmitCode submitCommand = new(md,
                targetKernelName: mermaidKernel.Name);

            var root = context.HandlingKernel.RootKernel;
```

```

    await root.SendAsync(submitCommand);
  } else {
    context.DisplayStandardError($"Can't find {name}");
  }
}
);

```

This code is similar to our `#!reflect` directive's code, with a few key differences:

- This method is asynchronous instead of synchronous, so its delegate starts with `async`.
- Instead of reflecting, we generate Markdown with our method from earlier and store it in the `md` variable.
- We use `DisplayStandardOut` to display the markdown below the cell. This step is not required, but it can be helpful for diagnostics when building magic commands like this.
- The code creates a `SubmitCode` object that contains the Markdown and tells it to target the Mermaid kernel by its name. This object is then executed on the root kernel.

The net effect of this code is that when the directive is invoked, we try to find the referenced object. Once we do, the code generates a string that represents our Mermaid syntax and sends it to the Mermaid kernel for execution.

We can see all this working in *Figure 14.10*:

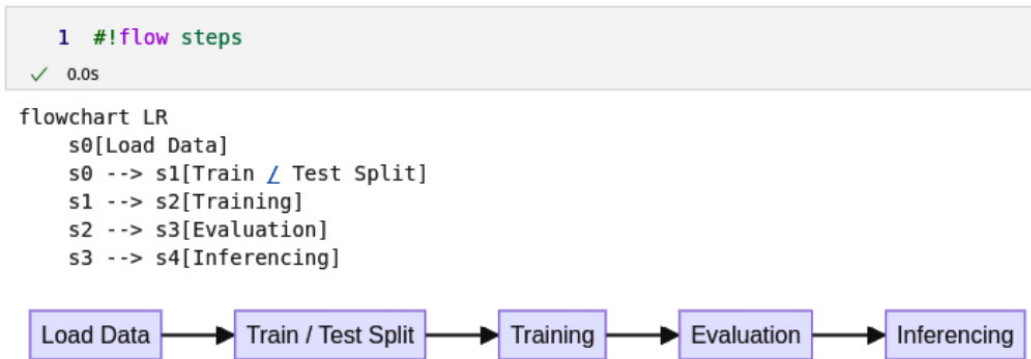


Figure 14.10 – The C# kernel sending Markdown to Mermaid

By building a custom magic command, we've given ourselves the ability to work directly with other kernels.

I tend to use magic commands mostly for cases like this – generating Mermaid diagrams from C# objects. For example, I recently built a magic command to visualize a complex `TransformerChain` in an ML.NET pipeline.

Even if you don't wind up building magic commands, understanding how they work internally is beneficial when interacting with code in a notebook environment.

Summary

In this chapter, we explored formatters and magic commands in .NET Interactive and saw how they can be used to extend and enrich the notebook experience.

We discussed the built-in formatters and a few ways of customizing their behavior. We also saw how you can register a method to format objects differently using plain text, HTML, or JSON output.

Each magic command involves a directive, an optional set of parameters, and `KernelCommand`, which is instantiated when the command is invoked.

Magic commands also have a command handler registered when the directive is added. These handlers can interact with various language kernels to display values, get or set variable values, or even execute code on different kernels.

While you won't always need to customize formatters or build your own magic commands, these capabilities can be invaluable in designing a better notebook experience.

In the final chapter of this book, we'll discuss where Polyglot Notebooks and .NET Interactive might fit into your team, ways of sharing notebooks, and ways of advancing into data science as a .NET developer.

Further reading

The following documentation sources may be helpful as you explore the concepts covered in this chapter:

- *Formatting objects in Polyglot Notebooks*: <https://github.com/dotnet/interactive/blob/main/docs/formatting.md>
- *Magic Commands*: <https://github.com/dotnet/interactive/blob/main/docs/magic-commands.md>
- *Extensions for Polyglot Notebooks*: <https://github.com/dotnet/interactive/blob/main/docs/extending-dotnet-interactive.md>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/extending-polyglot/>

Adopting and Deploying Polyglot Notebooks

In this chapter, we'll cover how Polyglot Notebooks can help you and your team succeed not just with exciting new machine learning tasks but also with the work you're already doing. We'll also cover how your notebooks can live outside of VS Code through hosting in a Jupyter notebook, JupyterLab, or GitHub codespace environment. Additionally, we'll cover some practical career concerns with how to add Artificial Intelligence (AI) and machine learning to your day job – or prepare to transition to a role that supports this.

In this chapter, we'll cover the following topics:

- Integrating Polyglot Notebooks into your day job
- Sharing Polyglot Notebooks with your team
- Deploying Polyglot Notebooks to GitHub codespaces
- Advancing into machine learning and AI

Technical requirements

This chapter has no code of its own but does discuss GitHub codespaces, which require an active GitHub account to use.

If you want to try out the concepts in this chapter, <https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks> holds the code for the previous chapters and is configured to support GitHub codespaces for individuals logged in to GitHub.

Integrating Polyglot Notebooks into your day job

Whenever I demonstrate Polyglot Notebooks at user groups and conferences, I inevitably see a number of people staring at the notebooks on the screen – appearing to be actively thinking through the implications of the technology. These are the people who usually find me after the session and talk to me more.

They're often excited about the capabilities of Polyglot Notebooks and how it opens up new modes of programming, but they're unsure of how to translate from the notebook experience I demonstrated to their day-to-day job.

Sometimes, this uncertainty is a tooling concern – particularly with developers who primarily develop in Visual Studio who aren't used to VS Code for .NET development. Other times, this is simply a radical difference between application types. Let's face it: notebooks are an entirely different type of application than enterprise web applications.

However, this is the point: Polyglot Notebooks aren't here to get rid of your existing applications, they're here to supplement them and make new types of workloads possible.

I believe that Polyglot Notebooks are ideal for new types of workloads your team hasn't considered yet, such as rapid experimentation, iterative and interactive testing of your objects, onboarding new team members, and even hosting public-facing interactive documentation.

Telemetry note

One potential issue for the use of Polyglot Notebooks is that the .NET Interactive kernels it sends information to Microsoft on your imported NuGet packages, as well as the languages involved with executing cells. This info could contain the name of your organization or project, which might be viewed as sensitive information. You can opt out of this telemetry by setting the `DOTNET_INTERACTIVE_CLI_TELEMETRY_OUTPUT` environment variable to 1. See <https://github.com/dotnet/interactive#telemetry> for more details on the data collection and opt-out process.

We've seen how Polyglot Notebooks fully supports end-to-end data analytics and machine learning tasks as well as making it easier to experiment with AI application development. Throughout the rest of this section, we'll explore each of these workflows in more detail.

Enabling rapid experimentation

One of the beauties of notebooks is they are lightweight and minimal. Before I discovered Polyglot Notebooks, if I wanted to run a small proof of concept application, I needed to open up my editor, create a new solution, specify what type of project I wanted to work with, name it, and then start coding. With Polyglot Notebooks, you simply open up VS Code and create a new `.dib` or `.ipynb` file and immediately have a cell available.

In a traditional editing experience, I'd need to declare my various classes, arrange my logic, and find ways of displaying or otherwise outputting the application's state. This could be using API methods, unit tests, `Console.WriteLine` statements, or even a graphical user interface. In order to verify my app worked, I'd need to compile it, launch it, and run it up until the point of interest.

With Polyglot Notebooks, you can instead write code in a cell, and then execute the cell. Variables are stored in your variables view and cell output appears immediately below the cell. Additionally, Polyglot Notebooks provides built-in formatters that can stylize your objects appropriately letting you focus on your core logic.

Formatters

If the built-in formatting behavior isn't enough, the custom formatters we introduced in *Chapter 14, Extending Polyglot Notebooks*, can significantly improve your notebook editing experience.

Polyglot Notebooks are intentionally lightweight. Instead of focusing on various files throughout a project, your focus is on the notebook and the information it conveys or the output of its cells. I personally find Polyglot Notebooks replacing the role that small console applications used to hold.

Polyglot Notebooks offer a good way of running arbitrary commands, calculating and displaying a value, or just experimenting with an API or NuGet package before integrating it into a production application.

Supporting AI and analytics workloads

Over the course of this book, we've seen how Polyglot Notebooks mixed with ML.NET and other libraries enable end-to-end data analytics workloads. Polyglot Notebooks give us an interactive environment where we can load, analyze, and transform data through the use of the `DataFrame` class.

Libraries such as `System.Numerics`, `ScottPlot`, and `Plotly.NET` enrich our data exploration experience with advanced descriptive statistics and data visualization capabilities.

Finally, ML.NET offers a high-performance .NET model training and inferencing environment. Using ML.NET data scientists can train, evaluate, and improve their models.

Using those trained models – or others imported using ONNX – developers can then embed machine learning models directly into their .NET applications.

Microsoft has a long-standing commitment to AI and machine learning, and that has only increased in recent years. When you combine ML.NET and .NET SDKs for services such as Azure OpenAI Service, Azure AI Service, and Azure Machine Learning, there's a lot you can do with machine learning on .NET directly through Microsoft.

The community also has some fantastic options such as GeneticSharp for genetic algorithms, Infer.NET for probability, Encog for neural networks and advanced machine learning, OpenCVDotNet for image processing, and others.

It's true that .NET is not currently as full-featured as other platforms in all aspects of machine learning and data analytics. For example, the `DataFrame` class needs some usability and capability improvements and ML.NET desperately needs a library with standardized data science charts such as the ROC curve or precision-recall curve.

For these reasons, data scientists and data analysts familiar with other technologies such as pandas or SciKit-Learn in Python may be more comfortable staying in those environments.

As an experienced developer and data scientist, when I start a new project, I tend to start it in .NET if I expect to do machine learning. If I'm only performing data analytics, I tend to go with Python 50% of the time and .NET the other 50%.

In fact, I believe so strongly in .NET's data science capabilities that I implemented my Master's project using Polyglot Notebooks and ML.NET while completing my Master's degree in data analytics. If you're more comfortable in .NET than Python or R, if you want high-performance computing, or if you simply want to work in the same languages as your production applications, .NET has you covered.

Assisting testing workloads

One of the benefits of Polyglot Notebooks is that it encourages experimentation by making it easy to make changes and see immediate results. This directly leads to more iterative development, tweaking, and experimentation in general. All of these things help developers interested in inspecting application behavior or finding and resolving bugs in their code.

With Polyglot Notebooks, if you're curious about how a method behaves with a certain set of parameters, you write a code cell, run it, and immediately see the results.

There's no need to trigger a build or wait for the application to launch; you just write code and observe the output. This encourages experimentation with different parameters and operation orders, which can help you find bugs.

Additionally, any uncaught exceptions will be displayed immediately below the cell, so that failure information is right there in front of you when you're going to fix the issue.

Plus, Polyglot Notebooks provides the variables view and helpful formatters for displaying objects. Both of these features help you inspect the state of your variables as your notebook runs.

All of these traits make Polyglot Notebooks a great tool for manual exploratory testing of your code.

Polyglot Notebooks isn't going to replace your unit or integration tests. Those tests are still vitally important to the long-term success of your application. However, Polyglot Notebooks encourages interactive exploration and *that* helps you identify weak areas of your application that need improvements and additional tests.

Training new team members with Polyglot Notebooks

When a developer joins a project for the first time, they may be unfamiliar with its classes, layers, systems, services, or database objects – or how these things all interact with each other.

As a result, these developers will be less effective while they are getting oriented and may make mistakes stemming from a lack of understanding of the code's architecture.

Polyglot Notebooks offer several ways of addressing this.

First, you can add code cells that illustrate how objects should be instantiated and how objects relate to each other. These cells can be augmented with either code comments or supplemental Markdown cells with additional documentation. Markdown allows for rich formatting with bold and italic formatting, making it more readable than long multi-line code comments.

Additionally, Markdown allows you to embed hyperlinks, images, tables, and more. This means you have many more tools available at your disposal to document how code works.

Don't forget that code cells can display their output below the code cells either by ending the cell with a variable or through explicit use of the `Display` extension method. This is particularly valuable if you are creating external-facing documentation for a library you're offering to the public. In this way, you can show how code should be invoked, include details on its results, and then illustrate decisions and relationships with tables and diagrams.

In *Chapter 13, Enriching Documentation with Mermaid Diagrams*, we saw how you can integrate Mermaid Markdown into a notebook to create simple visuals. Mermaid diagrams can be very effective at communicating system architecture, component communication, or even relevant parts of a data model, such as the entity relationship diagram pictured in *Figure 15.1*:

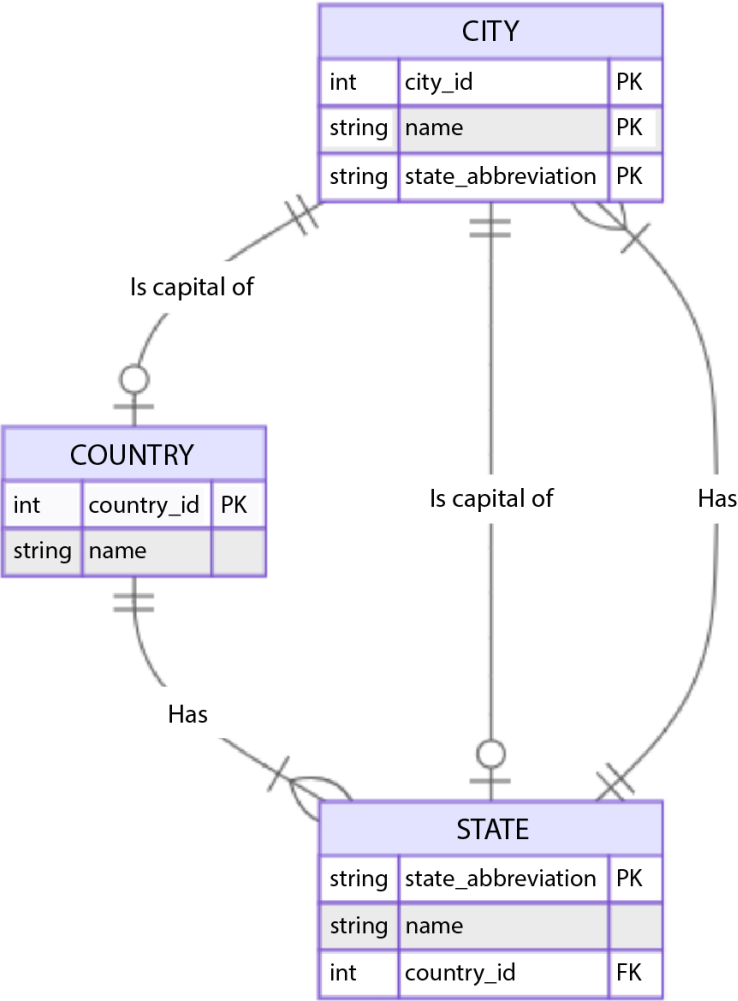


Figure 15.1 – An entity relationship diagram describing a database

Mermaid diagrams can also help illustrate processes and workflows as well, such as the Git branching example shown in *Figure 15.2*:

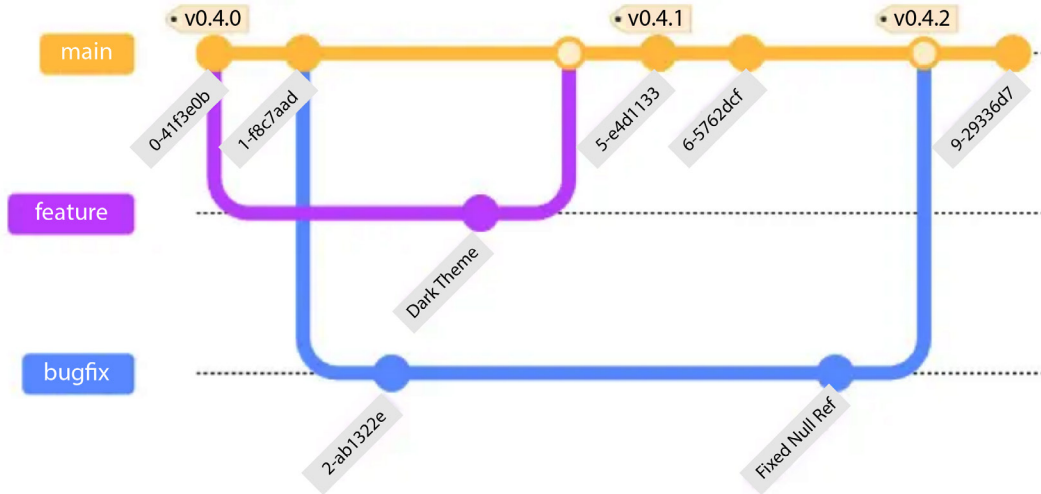


Figure 15.2 – A Git graph implemented in Mermaid

This type of diagram is very powerful because a single picture can help developers understand things such as the following:

- Branch naming policies
- Branching and tagging strategies
- Merge practices

I've used diagrams exactly like this to help propose and communicate git branching strategies to teams, get early feedback, and make adjustments. The final diagram then became part of our standard documentation and helped new developers orient themselves.

Additionally, because Mermaid diagrams are just Markdown, they're easy to update, change, and store in a Git repository.

Now that we've covered some sample scenarios of what notebooks can do, let's move on to how you actually share a notebook with your team members.

Sharing Polyglot Notebooks with your team

Working with a Polyglot notebook on your own is relatively straightforward: open VS Code, install the dependencies, create a notebook, and then start programming. This is the approach we've followed for much of the book and it works fine if you don't need to share a notebook with others.

When you *do* need to start sharing notebooks with other people on your team or stakeholders outside of your team, you'll need a way of coordinating your notebooks.

The simplest approach is to send the notebook file to another coworker as an email attachment or some other way of sharing a file. However, I don't recommend this for the same reason I don't recommend that developers pass code around in email: it's not efficient and is a good way of adding in extra communication and coordination overhead you don't need.

Instead, I recommend either adding your notebook to a version control system such as Git or integrating your code into a server already hosting notebooks via Jupyter or JupyterLab.

What is JupyterLab?

JupyterLab is an experimental new Jupyter Notebook experience. Think of JupyterLab as the next version of Jupyter Notebook with an improved user interface and additional features.

My overall recommendation is usually to add your notebooks to version control, but let's talk about the alternative first.

Integrating Polyglot Notebooks into Jupyter or JupyterLab

Project Jupyter and JupyterLab are free pieces of software that you can install onto any compatible host (almost anything that can run Python). Remember that Polyglot Notebooks is a set of kernels collectively referred to as .NET Interactive that integrate into the Jupyter environment.

Polyglot Notebooks versus .NET Interactive

Throughout this book, I've preferred the term Polyglot Notebooks when discussing notebook development using .NET, but there's an important distinction between Polyglot Notebooks and .NET Interactive.

Polyglot Notebooks is the specific VS Code extension that enables the rich Notebook editing and execution experience we've seen in this book. Polyglot Notebooks is by far the best way of experiencing .NET development in a notebook, but it relies on .NET Interactive to do its job.

.NET Interactive is the specific set of language kernels that allow running .NET code in a Jupyter Notebook environment. This section will be the only portion of this book focused on .NET Interactive without Polyglot Notebooks also in the picture.

This means that any host running Jupyter or JupyterLab could also run Polyglot Notebooks, assuming you have permission to install the appropriate dependencies and the host supports them. Installing Polyglot Notebooks into a Jupyter environment requires a compatible version of Python to be installed on the system, as well as the current version of .NET that Polyglot Notebooks requires.

This section is only a summary of the installation steps. I recommend you view the official documentation on this process at <https://github.com/dotnet/interactive/blob/main/docs/NotebookswithJupyter.md> before attempting to install anything. If you do encounter a problem, I recommend you open a GitHub issue in that repository in order to report bugs or get assistance with potential obstacles.

Once .NET is installed on the host, you can install Polyglot Notebooks globally via the following console command:

```
dotnet tool install -g Microsoft.dotnet-interactive
```

This will install Polyglot Notebooks *globally* on your machine.

You'll also need to run the following prompt in your Python or Anaconda environment:

```
dotnet interactive jupyter install
```

This will register your kernels into Jupyter.

Your installation can be verified by running the following command:

```
jupyter kernelspec list
```

If all is well, you should see something like the following output:

```
Available kernels:
.net-csharp      /home/jupyter/kernels/.net-csharp
.net-fsharp      /home/jupyter/kernels/.net-fsharp
.net-powershell /home/jupyter/kernels/.net-powershell
python3          /home/jupyter/kernels/python3
```

Next, you should be able to launch Jupyter via the `jupyter notebook` or `jupyter lab` commands. When you do so, the .NET kernels you saw listed earlier will be available for your notebook and you'll be able to use those kernels in your notebook, as shown in *Figure 15.3*:

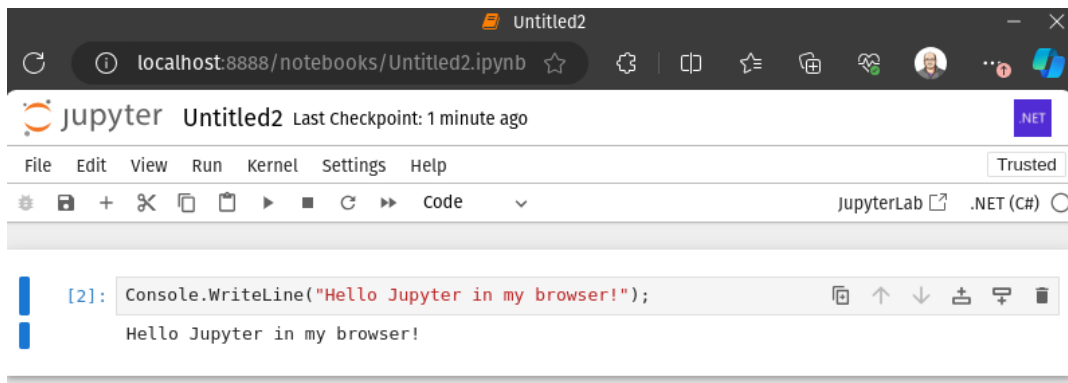


Figure 15.3 – A Jupyter Notebook using the .NET (C#) kernel

Integrating Polyglot Notebooks into Jupyter is an optional endeavor and one I'd only recommend if you want to use Jupyter Notebook or JupyterLab as your editing experience. You might prefer to install Polyglot Notebooks into Jupyter if the following things are true:

- The people you want to interact with your notebooks are not comfortable with tools such as VS Code or source control management software
- You absolutely need your notebooks to run in a browser
- You have a Jupyter Notebook or JupyterLab environment that you control and can install additional dependencies into

I would typically recommend hosting Polyglot Notebooks in Jupyter if you have stakeholders who want to be able to run reports themselves in the browser and experiment with different settings but aren't comfortable with a VS Code-style environment.

Most other times, I recommend checking your notebooks into source control and working either directly with VS Code or with GitHub Codespaces as we'll discuss later on in this chapter.

Storing Notebooks in source control

My default stance with Polyglot Notebooks is that the notebook files should be stored alongside your code as interactive documentation for that code.

Imagine your project has `docs`, `src`, and `tests` directories in the main project directory. The contents of these directories are typically tracked in version control as a single repository of information. In this way, your docs and tests match your source code at all times and you can easily look back and see how the documentation and tests changed from one major version of your software to the next.

With Polyglot Notebooks, you could add your notebooks either into the `docs` folder or add a new `notebooks` folder and store your notebooks there. This way, when any developer pulls down the latest code, they'll also pull down the latest notebooks that go with the code. The developer will then be able to open the notebook in VS Code and interact with it locally through the Polyglot Notebooks extension.

VS Code tip

In VS Code, you can mark an extension as part of your workplace recommendations by clicking the gear icon next to it and then choosing **Add to Workspace Recommendations**. This will prompt other developers to install the extension when they open the same workspace.

Additionally, when developers add new code, they can also add examples in an accompanying notebook file, which other developers will then be able to work with.

Unfortunately, as any developer who has worked on a team knows, source control systems sometimes encounter **merge conflicts** whereby one person modifies a file on their machine and another person

modifies the same person on their machine. This is true for code files, text files, and CSV data files; it's true for notebook files as well.

Merge conflicts will be different depending on which version control system you're working with. Resolving merge conflicts is beyond the scope of this book, but I bring them up because there's an important choice you make that impacts how difficult resolving merge conflicts is going to be with your notebooks.

If you recall from *Chapter 2, Exploring Polyglot Notebooks*, when we created our first notebook, we were prompted to choose whether the notebook should be a `.dib` file or an `.ipynb` file. This specific choice matters quite a bit for resolving merge conflicts.

`.dib` files are minimal and store only the code and markdown in the notebook. They store this data in as close to plain text as they can and do not store any outputs of notebook cells. This means that when you open a `.dib` file, you do not see any results until you run the notebook yourself locally. `.ipynb` files use a more custom file format that includes more details on the notebook, including the last output of a cell. This means that `.ipynb` files are immediately useful even without running them locally, but the custom format of `.ipynb` files makes resolving merge conflicts much harder. Since `.ipynb` files hold the output of their last execution, the file size of your notebooks will be larger. This effect is amplified if your notebooks use charts or other image-based visuals.

In general, I recommend using `.dib` files if you want to share your notebooks with other developers in source control. However, there are a few scenarios where I'd recommend that teams use `.ipynb` files:

- If your notebooks take a long time to run
- If your notebooks require a complex local setup
- If you need stakeholders to be able to open your notebooks and read them without executing them

If you go with the `.ipynb` file format, you may find yourself needing to coordinate edits to your notebooks in order to reduce the frequency of merge conflicts on those notebooks. This is never an ideal scenario, but it may be more manageable than regularly resolving complex merges in `.ipynb` files.

Either way, once your notebooks are in source control, they're there for your team to use and refer to.

Another nice aspect of having your notebooks in source control is that it can enable another way of sharing notebooks: using GitHub Codespaces.

Deploying Polyglot Notebooks to GitHub Codespaces

GitHub Codespaces allow you to use your browser to access a cloud-hosted instance of VS Code pointed at the code in a GitHub repository. This means that developers don't need to clone code locally or install dependencies on their machines in order to interact with the code.

As a result, developers can interact with code from a variety of different machines without needing to go through the installation and setup steps. If you'd like to experiment with pre-configured codespaces, this book's code is set up to run in your browser via GitHub Codespaces:

<https://github.com/PacktPublishing/Data-Science-with-.NET-and-Polyglot-Notebooks>.

Let's see how to configure GitHub Codespaces for Polyglot Notebooks

Configuring GitHub codespaces

Codespaces is a GitHub feature built on top of the concept of **dev containers**. Dev containers are disposable development environments that can be easily created from a pre-defined starting point and hosted using a service such as Docker.

With GitHub Codespaces, developers create a `devcontainer.json` file in the `.devcontainer` directory in the root of their repository. The `devcontainer.json` file tells GitHub how to create the codespace and what customizations need to be made to its execution environment.

Here's a sample minimal `devcontainer.json` file:

```
{
  "name": "Polyglot Sample",
  "build": {
    "dockerfile": "./Dockerfile",
    "context": ".",
    "args": {
      "VARIANT": "8.0"
    }
  },
  "forwardPorts": [],
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-dotnettools.dotnet-interactive-vscode"
      ],
      "settings": {
      }
    }
  }
}
```

This file sets up a container with the name **Polyglot Sample**. This container uses the extensionless `Dockerfile` configuration file in the same `.devcontainer` folder in order to define the execution environment. We'll examine this file shortly. Next, the file skips port forwarding. Port forwarding is often needed for web application development but is less relevant for Notebook development.

Finally, the file specifies a list of extensions that should be installed in VS Code once the container is set up. In our case, the container installs the `ms-dotnettools.dotnet-interactive-vscode` extension and does not make any additional settings changes.

Now that we've covered the `devcontainer.json` file, let's look at `Dockerfile`:

```
ARG VARIANT="8.0"
FROM mcr.microsoft.com/dotnet/sdk:${VARIANT}
```

This minimal `Dockerfile` uses the .NET SDK from Microsoft's container registry and specifies the variant to use – in this case, we'll use 8.0 since `VARIANT` defaults to it and since the `devcontainer.json` file specifies 8.0 in the `args` collection.

What to put in your files

Polyglot Notebooks continues to adapt with each new .NET release and so its .NET requirements will change over time. It's quite possible that by the time you read this book, Polyglot Notebooks might require .NET 9, 10, or beyond.

I recommend checking the `.devcontainer` files used by the .NET interactive repository itself, located at <https://github.com/dotnet/interactive>, whenever you are uncertain about what the current dependencies are. However, keep in mind that the configuration of that repository is fairly heavy and you likely won't need all of the extensions and dependencies defined in those files.

With these two files created in our `.devcontainer` folder, we should now be ready to create a codespace on GitHub.

Creating a codespace on GitHub

In order to create a codespace, you must be logged in to GitHub and looking at a repository with a `.devcontainer/devcontainer.json` file. Note that the repository doesn't have to be one of your own repositories, it just has to have the `devcontainer.json` file and you have to be authenticated with GitHub.

If these conditions are met, you should see a codespaces option under the **Code** button on the repository's home screen, as shown in *Figure 15.4*:

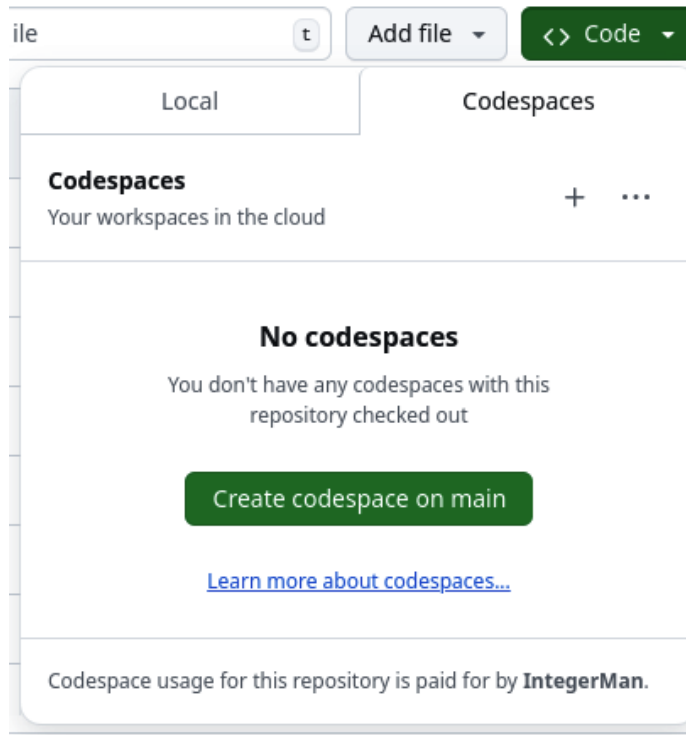


Figure 15.4 – The Codespaces tab in GitHub

This screen will show you any active or stopped codespaces you have access to for this repository.

Billing for codespaces

Note the message on codespace usage being paid by your user at the bottom of *Figure 15.4*. Each GitHub user has a certain amount of codespace usage per month associated with their account. Creating and using codespaces will typically come out of this time budget. However, organizations may also pay for codespaces at an organizational level. See *Further reading* for additional links on codespace usage and billing.

Once a codespace has been created, it will be assigned a random name and your browser will open VS Code in that codespace. This process will take some time to initialize. Once VS Code becomes active it will take additional time to install extensions into that instance.

Once extensions are installed, you should be able to open a `.dib` or `.ipynb` notebook file from the explorer and interact with code cells, as shown in *Figure 15.5*:

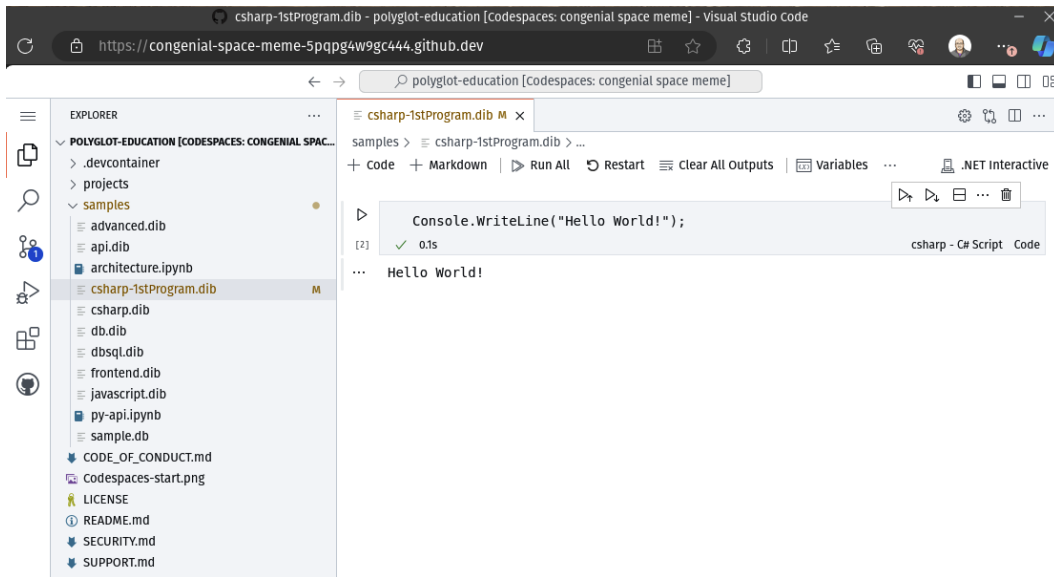


Figure 15.5 – Running Polyglot Notebooks in codespaces

By default, your codespace will automatically be stopped and deleted after inactivity. This is typically a good behavior because it stops you from accidentally using your monthly allotment of codespace time.

If you ever want to change this behavior, you can go back to your main repository, expand the **Code** button to show the codespaces list, and then view your codespaces, as shown in *Figure 15.6*:

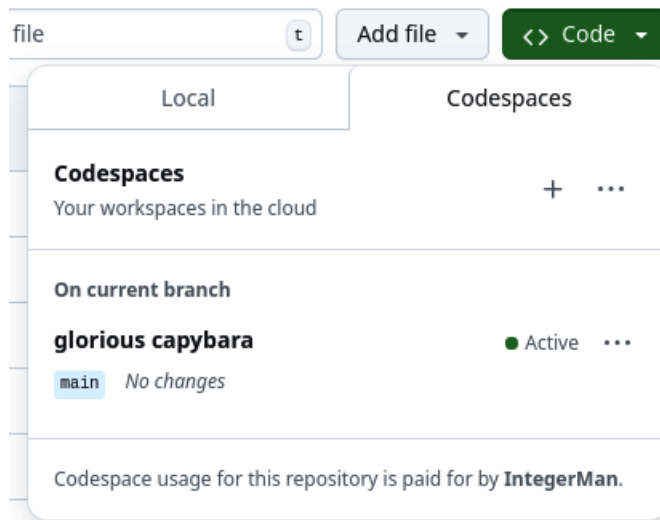


Figure 15.6 – Active and configured codespaces

Clicking on ... will open a context menu that gives you more options for managing your codespace or opening it in editors other than the web editor for VS Code.

The full details of managing codespaces or dev containers are outside of the scope of this book, but *Figure 15.7* should give you an appreciation of the currently supported features:

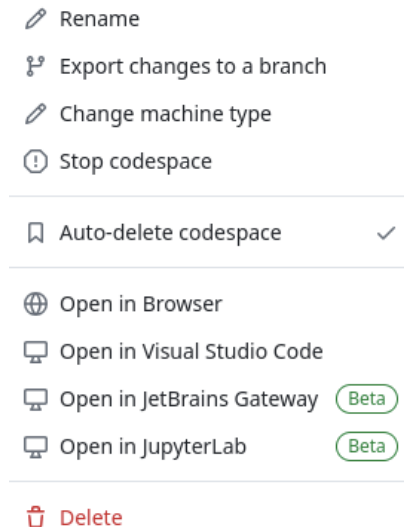


Figure 15.7 – Additional options for managing a codespace

If you're curious about GitHub Codespaces or dev containers, see the *Further reading* section of this chapter for additional details.

I view dev containers and GitHub Codespaces as a great way of hosting online notebooks. This hosting scenario is particularly compelling for people providing code libraries or APIs because you can offer an interactive notebook that a developer can quickly enter and use via the codespaces feature.

I hope to see more open source libraries offering interactive notebooks with samples of their libraries going forward, but I also acknowledge that the process for launching a new codespace will need to speed up slightly before this feels truly natural.

Now that we've covered a variety of ways of using and hosting notebooks, let's close with a discussion of how you can continue to learn and grow in your career as you advance into AI.

Advancing into machine learning and AI

I wrote this book for a few reasons. Firstly, I wanted to show people that .NET's data analysis, machine learning, and AI capabilities are ready for real-world usage. More importantly, I wrote this book to help people grow from being .NET developers to being .NET developers with skills in data science.

As software engineers, we are never done learning, and there is so very much to learn in the realm of data analytics, machine learning, and AI. It's possible to do both software engineering and data science work in the same role. It's also possible to transition from being a software engineer to being a data scientist.

I also am a firm believer in the idea that data analysis skills are valuable for any tech professional – from individual developers to managers, product owners, testers, and designers.

As we close this book, I want to leave you with some parting wisdom on how the skills and concepts we explored in this book can apply to your current or future jobs.

Adding data science to your day job

If you have a job in tech, it's likely that some of the contents of this book can expand the range of things you can do in your current job.

Data analysis

Data analysis is a specialized discipline involving analyzing and visualizing data to determine patterns and trends. This is such a dedicated discipline that there are data analyst jobs that exist solely for this purpose.

Data is relevant for all types of roles in tech, from developers to QA to product to management:

- As a software engineer, data analysis helped me understand and communicate the severity of bugs in tense situations.
- As a product owner, data showed me which features people were using the most and which ones could be improved or promoted more.
- As a team lead and engineering manager, data helped me identify the parts of our application that encountered the most bugs. As a software engineering manager, that data helped me convince upper management that additional time to remediate these issues was worth the investment.

Data analysis likely won't be your full-time job, though it can be a fulfilling one if you really enjoy it. My experience has shown that when you apply data analysis to other roles in key situations, it can lead to better decisions that are easier to explain and defend. This leads to greater freedom and effectiveness in actually improving things.

AI

Not long ago, AI was a specialized field that required formal study of the many different sub-disciplines in order to be effective at integrating AI into an application.

In the last decade, this has changed significantly, with the rise of **AI-as-a-Service** offerings such as Azure OpenAI Service for generative AI or Azure AI Service for speech, vision, language, document processing, and decision-making capabilities.

Large Language Models (LLMs) in particular offer developers the ability to download models from well-known model catalogs and run them locally in order to get language completions or image generation results. This means that you no longer need specialized study in order to add intelligence to your applications. In fact, the act of working with AI services and trained LLMs often resembles making an API call or working with a third-party library.

With these shifts in making AI accessible, everyday developers can add amazing capabilities to their applications simply by knowing the task they want to accomplish and the services or models that can help them with that task.

In this new paradigm, developers now need a baseline level of AI literacy in order to understand the tasks they're trying to accomplish and the potential weaknesses or difficulties that are likely to arise.

A growing number of online courses, books, and certifications exist that can help you achieve this baseline level of AI literacy and know where to go to learn more.

I'm personally a big proponent of Microsoft's **Azure AI Fundamentals** certification study path and the associated AI-900 exam. This is a lower-cost certification with no prerequisites and no expiration date. There are also a wide range of learning materials available on this exam and it serves as a good introduction to AI as a service and common AI obstacles.

Microsoft also offers an Azure AI Engineer Associate certification that adds additional depth and technical rigor for those wanting to further hone and prove their skills.

A key factor I want to stress with AI is that developers don't need to stop being developers in order to start working with AI. AI is an additional set of skills, knowledge, and tooling that developers can add to their skillset – much like other specializations such as frontend, backend, data, security, or DevOps.

This is partially the path my own journey took, with my first official AI-related role being that of a senior engineer and AI specialist.

Machine learning

We've covered data analysis and AI, but maybe you're fascinated by the field of machine learning and training models to generate predictions based on trends in the data.

If this is the case, it *may* be harder to add machine learning to an existing role. While there can be opportunities to add machine learning to applications as a software engineer, in my experience, these opportunities are very limited.

Additionally, training and evaluating machine learning models tends to be a separate enough discipline from software engineering that organizations will require proof of your competency before trusting you with a machine learning project. For this reason, I recommend that developers looking to add machine learning to their role at their organization have open conversations about this desire with their managers and coworkers. Engineers who do this should regularly share their learning efforts in the field of machine learning with their coworkers.

These efforts could include things like the following:

- Completing a course in machine learning (including smaller online courses on platforms such as Coursera, Udemy, or LinkedIn Learning)
- Training a model as a personal learning or hobby project
- Reading books focused on machine learning (this book counts!)
- Watching case study videos of machine learning projects
- Participating in machine learning competitions on sites such as Kaggle.com

There's a reason I want you to be open about both your desire to add machine learning to your work role and your learning activities: it helps change how people think of your skills. People likely know you as a software engineer, tester, or some other role at the moment.

Imagine that you spent 3 – 24 months in absolute secrecy studying machine learning intently, but quietly. You may emerge from that learning journey with fantastic knowledge and skills, but your coworkers still think of you in your current role's context. With that limited context, they're going to be hesitant to entrust you with data science projects.

On the other hand, if you openly share what you've been learning, your coworkers and managers will start to view you differently over time. Additionally, you sharing about your learning journey will improve *their* skills, which helps your organization mature into machine learning. Pretty soon, you'll earn the respect of your team and your team will be more comfortable with machine learning projects – and watch for opportunities to integrate them into their products.

Expanding into machine learning in your current organization won't always work, because machine learning isn't an ideal solution for every problem or organization. In such cases, if you really want to succeed, you'll need to leave your current role and enter a new data science role on another team.

Getting into data science

While adding data science to an existing role is possible, not every organization will have a need for data scientists or current openings on data science teams.

This means that if you want to move from a software engineer to a data scientist role, you'll likely need to move organizations – or at least move to another team within your current organization.

Data science is not better than software engineering

I want to stress that not every software engineer should aspire to get into data science. Software engineering and data science are separate fields with many skill overlaps. Pay levels are frequently roughly equivalent between the two fields and there tend to be more jobs for software engineers than for data scientists. I advise that you only shift to data science if you believe you'll find it more enjoyable or fulfilling than software engineering.

This means you'll need to successfully find, apply for, and pass the interview process for a data science role.

Finding a data science role

Finding roles is usually straightforward as there are only a few standardized AI titles out there on the job market at the moment, including the following:

- Data scientist
- Data engineer
- Machine learning engineer
- AI engineer

Each of these roles is slightly different from the others, with data scientists being primarily focused on model training and evaluation; data engineers having a focus on how data flows securely, repeatably, and reliably from system to system; AI engineers being focused on integrating AI into applications; and ML engineers being more of a hybrid of all roles mashed together.

Another path: consulting

Some consultancies hire full-time employees and move them from project to project between different employers. Each project a consultant takes on may involve different skills and technical stacks. As a result, if you can find the right consultancy that can market software engineers and data scientists to clients, there's a possibility that you may find yourself able to work as a software engineer on one project and a data scientist on the next, depending on the needs of your organization's clients. This is the path I'm following at Leading EDJE, and it's a lot of fun and a great way to make an impact.

If you are serious about getting into data science, I recommend that you look at a number of job listings for each of these roles and see how the responsibilities and qualifications align with your interests and backgrounds.

Some roles will require higher degrees of education, with Master's degrees or even doctorates occasionally being required for the more complex data scientist roles. However, this is not an industry-wide standard and you should be able to locate many jobs that require a bachelor's degree or equivalent experience.

Passing the data science interview

Once you've found a role of interest, you'll need to apply for it and successfully pass the interview process. For example, in the case of software engineering roles, the interview process is all about showing comfort and familiarity with the technologies and concepts relevant to the role.

You'll need to discuss your background and specifically your background with data science. This discussion should involve discussions of specific end-to-end data science projects you executed, the process you used, the decisions you made, and which high-level data processing and model training steps were involved.

You should also be prepared to talk about determining how effective your system was, why it made the decisions it made, and which areas it struggled in – and how you worked to improve those.

There will likely be “interview trivia” involved in the interview process, just like there usually is for software engineering roles. These questions can range from describing things like random forests to comparing logistic regression and linear regression to more advanced concepts, such as calculating machine learning metrics or specific advanced mathematical concepts.

You will likely also be asked how you would approach a hypothetical data science task, which decisions you would make, and which options you would consider. This book is not designed to help you pass the data science interview process, though additional reading materials are listed in this chapter’s *Further reading* section related to that topic.

You don’t need to be perfect in a data science interview any more than you need to be perfect to pass a programming interview. Instead, you need to give your interviewer a realistic understanding of your skills and experience and how your decision-making process works. You also need to show them your communication skills as these are vital in data science roles.

You may not perform well in your first few interviews, but these attempts should help you identify weak areas to work on. Practice your interviewing – and, more importantly, perform additional data science projects in areas of weakness.

Eventually, you should get to a point where you can clearly communicate the value you add in a data science role and you’ll get that first data science job.

Once you have that role, you need to succeed in it.

Succeeding in data science

Whatever your current field is, it’s unlikely that you started it fully competent and ready for any problem. Whether you’re a software engineer, a quality assurance professional, a database administrator, or something else – I don’t expect that you entered your professional field with perfect knowledge of the technologies involved or how to successfully deploy them.

All tech jobs involve learning and growth. All tech jobs involve uncertain challenges and unexpected obstacles and setbacks. These are the unavoidable facts of being a technology professional.

However, if you stay in a technical field for long enough, you’ll develop skills in dealing with these obstacles, learning new things, asking the right people for help or advice, and generally discovering how to succeed in scenarios you would have failed in previously.

As you find yourself adding aspects of data science to your current role or transitioning into a new data science role, it is highly likely that you will find yourself in uncertain and unfamiliar territory again. This unfamiliar territory may involve unfamiliar concepts such as the curse of dimensionality, prompt injection vulnerabilities, data drift, data poisoning, or stochastic dual coordinate ascent.

The early challenges you encounter may bring back those feelings of inadequacy, helplessness, confusion, and even despair that you may have encountered earlier in your career. When this occurs, I want you to remember two things:

- **You can do this.** You've overcome learning challenges before in a different field. I was once terrified of the idea of dependency injection. Now I help others understand it. You likely have similar victories in your past.
- **You are not alone.** You are entering a field with many established professionals and many others making the same transition. That means there are others you can reach out to for advice, as well as articles, books, courses, and YouTube videos to help you through those harder times.

Ultimately, you can succeed in the fields of AI and machine learning, and it can be very fulfilling and enjoyable when you do. Your success will come through continuing to learn, continuing to grow, and more than anything else – continuing to practice.

The technologies in this book and in the field will continue to grow and evolve. If you find these technologies interesting and compelling, you'll need to evolve and grow with them. You cannot learn everything or be good at everything, but you *can* be familiar with many things and do focused experimentation around the areas that are most likely to be valuable to you or your employer.

Keep learning. Keep doing small experiments on datasets or interacting with new models, algorithms, or services, and learn from the process. When you do learn new things, tell others so they can benefit as well on their own journeys.

Summary

In this chapter, we saw how your data science notebooks can move beyond VS Code and be integrated into Jupyter Notebooks or JupyterLab, or deployed to GitHub Codespaces.

We also discussed ways that notebooks can help various people inside and outside your organization with data analysis, data science, and software engineering tasks.

We closed the chapter – and the book – with discussions on where AI, machine learning, and data analytics can fit into existing tech careers – and some directions you can take as you explore how these concepts fit into your own interests and career direction.

As we now close this book, I'd like to leave you with some parting words.

I sincerely hope that you find data analysis, machine learning, and AI to be as interesting and fascinating as I do. These fields offer a multitude of interesting tools and approaches for solving unusual problems – and problems that are traditionally difficult for software engineers.

Using the tools and concepts in this book, you can build systems that determine how correlated two factors are, identify outliers, create compelling visuals to communicate data, categorize new data points automatically, make numerical predictions, generate new text or images, and even build systems that conversationally fetch data for the user from different data sources.

Perhaps the most interesting and exciting thing about all of this is that you don't need to leave .NET to do these things. If you are happy and comfortable in .NET and want to get into data analysis, machine learning, or AI, .NET has the tooling for this. Additionally, Microsoft's commitment to AI has been obvious and industry-leading. Every quarter, Microsoft continues to pave new ground in AI and .NET continues along this journey.

I fully expect Microsoft to continue expanding, improving, and polishing its AI and machine learning offerings and expect that we will continue to see improvements and innovations in these areas in every .NET release. I'm excited about the current state of AI in .NET and am eager to see the new features that will inevitably arrive in the second edition of this book, should I be privileged enough to get to revise it. In the meantime, I'm going to follow my own advice and continue to learn, build, and share – focusing on the intersection of .NET and AI / ML.

You can find my contact and connection information at `MattEland.dev` and I'd love to see the things you're up to.

Happy coding.

Further reading

Here are some additional resources related to items discussed in this chapter:

- *Project Jupyter / Jupyter Notebooks*: <https://jupyter.org/>
- *JupyterLab*: <https://github.com/jupyterlab/jupyterlab>
- *Integrating .NET Interactive and Jupyter Notebooks*: <https://github.com/dotnet/interactive/blob/main/docs/NotebookswithJupyter.md>
- *Dev containers overview*: <https://code.visualstudio.com/docs/devcontainers/containers>
- *GitHub Codespaces*: <https://docs.github.com/codespaces>
- *GitHub Codespace administration*: <https://docs.github.com/codespaces/managing-codespaces-for-your-organization/enabling-or-disabling-github-codespaces-for-your-organization>
- *GitHub Codespace billing*: <https://docs.github.com/billing/managing-billing-for-github-codespaces/about-billing-for-github-codespaces>

- *Dev container JSON formatting*: <https://aka.ms/devcontainer.json>
- *Adding a dev container*: <https://docs.github.com/en/codespaces/setting-up-your-project-for-codespaces/adding-a-dev-container-configuration/introduction-to-dev-containers>
- *What are dev containers*: <https://www.mikekasberg.com/blog/2021/11/06/what-are-dev-containers.html>
- *Acing a data science interview*: <https://hub.packtpub.com/how-to-ace-a-data-science-interview/>
- *Data science interview questions*: <https://www.springboard.com/blog/data-science/data-science-interview-questions/>
- *Cracking the Data Science Interview*: <https://www.packtpub.com/product/cracking-the-data-science-interview/9781805120506>
- *Additional online resources*: <https://MattOnML.NET/Books/DataScience/adopting-polyglot/>

Index

Symbols

3D scatter plots

with Plotly.NET 143, 144

.dib file

importing 52-54

`#!import` magic command 50

limitation 54-56

.ipynb file

importing 52-54

.NET Interactive 17-19, 352

versus Polyglot Notebooks 352

`#r` magic command 57, 58

A

absolute path

using 51

absolute values 171

AI agent 8

AI-as-a-Service 361

AI orchestration 276

AI services 277

Amazon SageMaker 156

artificial intelligence

(AI) 8-10, 345, 361, 362

deep learning 11, 12

neural networks 11

simulating, systems types 8

ASP.NET web applications

ML.NET models, hosting 235

assembly

defining 56

reference link 56

automated machine learning

(AutoML) 157, 177

used, for data pre-processing 158, 159

AutoML pipelines

controlling 209

creating 206-209

Featurizer, customizing 209-211

model trainer selector, customizing 211

Azure

generative AI models, deploying 251

Azure AI Search 275

Azure Data Explorer

URL 78

Azure Machine Learning Studio 156

Azure OpenAI

content filtering 290, 291

Azure OpenAI Service

- access credentials, obtaining 257, 258
- connecting to 258, 259
- creating 252-254
- models, deploying 254-256

B**big data**

- development 4

binary classification algorithms

- selecting 222-224

binary classification experiment 163**BinaryClassificationExperimentSettings**

- setting up 182

binary classification metrics

- accuracy 169
- area under the precision recall 169
- area under the ROC curve 169
- F1 Score 169

binary classification models 228

- evaluating 166-170
- feature importance, calculating 170, 171
- used, for predicting values 172-174

bins 129**boolean columns 147****box plot**

- creating, with ScottPlot 120-122
- in Plotly.NET 124-127
- using, for multivariate analysis 135

C**categorical data 94****cell sizes 31****class 223**

- declaring 36, 37

class diagrams 306

- structure, communicating with 306-309

classification 155, 156**classification model**

- training, with ML.NET Auto 163-166

classification model trainers, ML.NET

- AveragePerceptron 224
- FastForest 223
- FastTree 223
- FieldAwareFactorizationMachine (FFM) 224
- Gam 224
- LbfgsLogisticRegression 223
- LdSvm 223
- LightGbm 223
- LinearSvm 223
- SdcaLogisticRegression 223
- SdcaNonCalibrated 223
- SgdCalibrated 223
- SgdNonCalibrated 223
- SymbolicSgdLogisticRegression 223

class imbalance 167**cloud hosting**

- benefits 4

code

- invoking, on kernels 339-342

code cells 14, 15

- adding 29

code files

- .dib file, importing 52-54
- #!import magic command limitation 54-56
- .ipynb file, importing 52-54
- importing 50-52

columns

- adding 96-98
- analyzing 93
- information, displaying with info 94
- removing 95, 96

- renaming 96
- value occurrences, tabulating
 - with ValueCounts 94, 95
- values, getting from 95
- working with 91
- Command Palette 24**
- comma-separated values (CSV) 61**
- common language runtime (CLR) 38**
- compiler errors**
 - resolving 43
- complex functions**
 - creating, from methods 283, 284
 - defining 283
- complex requests**
 - handling, with planners 291-295
- concatenate step 204**
- confusion matrix 166**
- confusion table 231**
- connection string 72**
- connectors 277**
- content filtering 290**
- correlation 145**
- cross-validation 177, 185, 186**
 - results, interpreting 187-189
- CSV data 61, 62**
 - delimited file formats, loading 65
 - loading errors, troubleshooting 64, 65
 - reading 61
 - reading, into DataFrame 62, 63
 - specialized CSV loading scenarios 63
 - TSV files, loading 65
- custom formatters**
 - output, styling with 329-331

D

DALL-E

- images, generating with 266-270

- dall-e-3 model 256**

- data 71**

- data analysis 361**

- data analytics 4**

- data cleaning 82, 84**

- data drift 238, 239**

- DataFrames 62, 81**

- aggregating 101-103
 - building, from objects 69-71
 - Columns property 91, 92
 - CSV data, reading into 62, 63
 - filtering 106, 107
 - grouping 101-103
 - in other languages 108
 - merging 103-106
 - sorting 100, 101

- DataFrames, in C#**

- cell values, obtaining 89
 - cell values, setting 89
 - data sampling 86-88
 - data, viewing 86-88
 - iterating, over rows 90
 - Rows property 89
 - working with 85, 86

- data science 3**

- adding 361
 - artificial intelligence (AI) 361, 362
 - data analysis 361
 - field, exploring 3, 4
 - interview process 364, 365
 - machine learning 362, 363
 - obtaining 363
 - role, finding 364
 - success 365

- data visualization 7**

- role, in exploratory data analysis (EDA) 112, 113

- data wrangling** 84, 85
- DateTimeDataFrameColumn** 91
- decision trees** 213, 214
- deep learning (DL)** 11, 12
- default formatting behavior** 326, 327
 - default collection formatting 328, 329
 - default object formatting 327
- delimited file formats**
 - loading 65
- descriptive statistics**
 - for exploratory data analysis (EDA) 114
 - generating, with DataFrame.
 - Description 114-117
 - with MathNet.Numerics 117-120
- dev containers** 356
- DevOps** 240
- directives** 331
- discriminated unions** 39
- Dynamic-Link Library (DLLs)** 60

E

- Entity Relationship Diagrams (ERD)** 309
 - data, communicating with 309-312
- expected conditional improvement (ECI)** 217
- exploratory data analysis (EDA)** 112
 - data visualization role 112, 113
 - descriptive statistics 114
- Extensions Marketplace** 22

F

- F#** 37
- F1 score** 232
- false negative** 167
- false positive** 167

- Fast Forest regression trainer** 203
- feature correlation matrixes**
 - building 146-150
- feature engineering** 84
- features** 204
- few-shot inferencing** 264
- Finite State Machines (FSMs)** 313
- FizzBuzz** 38
- flowcharts, Mermaid diagrams** 303-306
- formatters** 326
- functional programming language** 38
- FunctionResult objects**
 - examining 288, 289

G

- generalized additive model (GAM)** 224
- generative AI** 250
- generative AI models**
 - behavior, customizing with
 - prompt engineering 262
 - chatting with 259-262
 - deploying, on Azure 251
- generative pre-trained transformers (GPTs)** 250
- Git graphs** 321
 - workflow, communicating with 321-323
- GitHub Codespaces**
 - configuring 356, 357
 - creating, on Github 357-360
 - used, for deploying Polyglot Notebooks 355
- gpt-4o model** 256
- gradient boosting** 221
- graphical processing units (GPUs)** 274

H

histograms

with Plotly.NET 129-131

hyperparameters 154, 205

hyperparameter tuner

customizing 216-219

hyperparameter tuning 205

customizing 212

I

image generation 266

with DALL-E 266-270

immutable objects 96

imputing missing values 158

init keyword 36

integrations 277

invalid data 82

J

JSON data

obtaining, with PowerShell 66-69

Jupyter 13

used, for integrating Polyglot
Notebooks 352-354

JupyterLab 352

used, for integrating Polyglot
Notebooks 352-354

Jupyter Notebooks 13, 15

extending, with kernels 16

K

KernelFunction parameters

accepting 284, 285

kernels

code, invoking 339-342

Jupyter Notebooks, extending with 16

K-Fold cross-validation 186

Kusto clusters

querying, with KQL 77-79

L

LangChain 276

languages

variables, sharing between 39, 40

large language models

(LLMs) 12, 156, 273, 362

LBFGS Poisson Regression model 219

linear regression trainer 183

local variables 34

logistic regression trainer 183

loss function 191

M

machine learning 5, 154, 360-363

classification 155, 156

regression 156

reinforcement learning 7, 8, 154

semi-supervised learning 154

supervised learning 5-7, 154, 155

unsupervised learning 7, 154

ways, for training model 156

Machine Learning Operations

(MLOps) 240, 241

machine learning tasks in ML.NET

reference link 242

MacroAccuracy 232

magic commands 41, 331

#!about 332

#!flip-coin 334

#!lsmagic 333, 334

- #!time 332
 - exploring 331-333
 - Markdown cells** 13, 14, 32, 33
 - benefits 14
 - masked correlation matrix** 149
 - MathNet.Numerics**
 - descriptive statistics with 117-120
 - URL 117
 - Mean Absolute Error (MAE)** 188, 189, 203
 - mean height** 99
 - Mean Squared Error (MSE)** 189
 - mean value** 116
 - median height** 99
 - memory**
 - adding, to Semantic Kernel 281-283
 - memory function**
 - defining 285, 286
 - Mermaid diagrams** 302
 - logic, communicating with flowcharts 303-306
 - Mermaid Live Editor**
 - URL 304
 - methods**
 - declaring 33-35
 - MicroAccuracy** 232
 - MIME type** 326
 - missing data** 82
 - missing values**
 - handling 98-100
 - ML.NET** 156, 157, 177
 - using, to work in ONNX model 243
 - using, to work in TensorFlow model 243
 - ML.NET AutoML**
 - classification model, training with 163-165
 - ML.NET capabilities**
 - surveying 242
 - ML.NET documentation**
 - reference link 242
 - ML.NET models**
 - exporting 234
 - hosting, in ASP.NET web applications 235
 - model drift** 239
 - detecting 240
 - model fitting** 83
 - model performance** 238, 239
 - models**
 - updating 240, 241
 - models & datasets, TensorFlow**
 - reference link 243
 - model's error** 189
 - model trainers** 183
 - versus algorithms 183
 - model trainer step** 204
 - model training pipeline**
 - example 203-206
 - multi-class classification** 163
 - multi-class classification model** 228, 229
 - evaluating 230-232
 - test predictions, generating 232, 233
 - training 230
 - multiple functions**
 - calling, with plugins 286, 287
 - multivariate analysis, with Plotly.NET**
 - 3D scatter plots, with Plotly.NET 142-144
 - color, adding to scatter plot 141, 142
 - data and dependencies, loading 134
 - multiple values, plotting with scatter plots 137-140
 - performing 134
 - performing, with box and violin plots 135, 136
- ## N
- natural language processing (NLP)** 207
 - neural networks** 11
 - normalization** 84

normalizing values 159

notebook 13

notebook cells

- code cells, adding 29
- executing 27
- Markdown cells 32, 33
- Variables view 31
- working, with variables 30

notebook execution

- compiler errors, resolving 43
- issues 44
- troubleshooting 43

NuGet feeds 57

NuGet Package Manager 57

NuGet packages

- #r magic command 57, 58
- default assemblies 56
- importing 56
- local packages, referencing 60, 61
- pre-release packages, referencing 58-60
- specialized NuGet feeds, referencing 60
- specific versions, referencing 58-60

numeric columns

- scaling 219, 220

O

object initializer 36

objects

- DataFrames, building from 69-71

one-hot encoding 158

one-shot inferencing 263

**Open Neural Network Exchange
(ONNX)** 156

- used, for working in ML.NET 243

ordinary least squares (OLS) 221

outliers 82

output

- styling, with custom formatters 329-331

P

parameters 154

- working with 335-339

pattern matching 39

Pearson correlation method 145

**permutation feature importance
(PFI)** 170, 194

personas 277

pipelines 203

pipng operator 118

planners 277

- complex requests, handling 291-295

Plotly 123, 124

Plotly.NET 123, 124

- box plots 124-127
- histograms with 129-131
- used, for performing multivariate analysis 134
- used, for performing univariate analysis 123
- violin plots with 127, 128

plugins 277

- multiple functions, calling 286, 287

polyglot

- defining 17

Polyglot Notebook extension

- creating 334, 335

Polyglot Notebooks 17-19, 325, 352

- AI and analytics workloads 347
- class names 37
- creating 24-26
- default formatting behavior 326
- deploying, to GitHub Codespaces 355
- experimentation, enabling 346
- installing 22-24
- integrating 346
- integrating, into Jupyter 352-354
- integrating, into JupyterLab 352-354
- sharing, with team 351
- storing, in source control 354, 355

- team members, training 349-351
- test workloads, assisting 348
- used, for performing tasks 19
- versus .NET Interactive 352
- working, with other languages 37-39

Polyglot Notebooks errors

- diagnostic output 45

Polyglot Notebooks repository

- issues 45, 46

Polyglot Sample 357**positive correlation 113****PowerShell**

- JSON data, obtaining with 66-69

PredictionEnginePool

- configuring 235, 236
- using 237, 238

prefix increment operator 30**project files**

- importing 61

Project Jupyter 13, 352**prompt engineering 262**

- model behavior, customizing with 262

Q

quantitative data 95

R

reflection feature 69**regression 156, 178**

- dataset 180, 181
- numerical formula 178-180
- performing, without AutoML 202, 203
- task 178

regression algorithms

- selecting 221

regression experiment 177

- performing 182-185

regression metrics

- evaluating 189-191
- Mean Absolute Error (MAE) 189
- Mean Squared Error (MSE) 189
- PFI, applying to regression models 194, 195
- Root Mean Squared Error (RMSE) 190
- values for outliers, predicting 191-193

RegressionMetrics class

- loss function 191
- R Squared 191

regression model

- applying 195-198

regression model trainers, ML.NET

- FastForest 221
- FastTree 221
- FastTreeTweedie 221
- LbfgsPoissonRegression 221
- LightGbm 221
- Ols 221
- OnlineGradientDescent 221
- Sdca 221

reinforcement learning 7, 8**relative path**

- using 51

required keyword 36**retrieval-augmented generation**

- (RAG) 273, 275

Root Mean Squared Error (RMSE) 190**R Squared 188, 191**

S

scatter plots

- color, adding 141, 142
- using, for multivariate analysis 138-140

ScottPlot

- used, for creating box plot 120-122

ScottPlot Dependencies

- reference link 121

search space 212, 213
 customizing 215, 216
self-attention 274
Semantic Kernel 276, 277
 building 277, 278
 functions 277
 memory, adding 281-283
 prompt function, creating 278-280
semantic memories 277
semi-supervised learning 8
sequence diagrams 316
 flow, communicating with 316-320
sequential model-based algorithm
 configuration (SMAC) tuner 217
SingleDataFrameColumn 91
SQL databases
 connecting, alternative ways 77
 connecting to 71-73
 executing, from SQL kernels 73, 74
 results, sharing with other kernels 74-77
standard deviation 119
state diagrams 313
 behavior, communicating with 313-316
stochastic dual coordinate
 ascent (SDCA) 221
stochastic gradient descent (SGD) 223
StringDataFrameColumn 91
supervised learning 5-7, 154, 155
support vector machines (SVMs) 223
switch expressions 39
SystemChatMessage 260
system prompt 260

T

tab-separated value (TSV) files 65
 loading 65

target 7
TensorFlow models
 used, for working in ML.NET 243
text-embedding-ada-002 model 256
text embeddings 264
 using 264-266
token 251
trained regression model 178
training and testing datasets
 creating 159-162
transforms 203
true negative 167
true positive 167
Tweedie distribution 221

U

unclean data
 impact 83
 sources 82
Unified Modeling Language (UML) 306
univariate analysis 123
 performing, with Plotly.NET 123
unsupervised learning 7

V

ValueCounts 94
variable correlations
 calculating 145
 feature correlation matrixes,
 building 146-149
 identifying 144
variables
 exporting 40-43
 sharing, between languages 39, 40
 working with 30

Variables view 34

violin plot

 using, for multivariate analysis 135-137

 with Plotly.NET 127, 128

voting ensemble 221

W

win/loss/draw classification model 240

Z

zero-shot inferencing 263



Packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

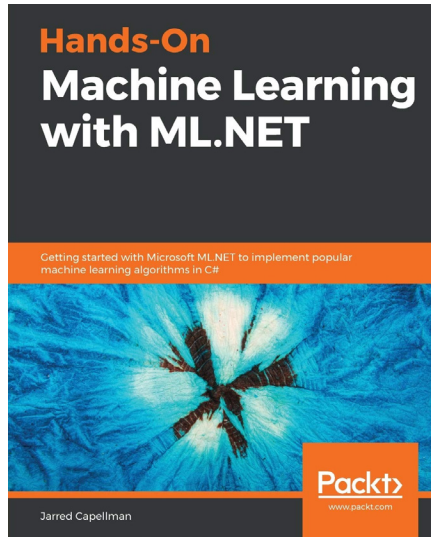
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Hands-On Machine Learning with ML.NET

Jarred Capellman

ISBN: 978-1-78980-178-1

- Understand the framework, components, and APIs of ML.NET using C#
- Develop regression models using ML.NET for employee attrition and file classification
- Evaluate classification models for sentiment prediction of restaurant reviews
- Work with clustering models for file type classifications
- Use anomaly detection to find anomalies in both network traffic and login history
- Work with ASP.NET Core Blazor to create an ML.NET enabled web application
- Integrate pre-trained TensorFlow and ONNX models in a WPF ML.NET application for image classification and object detection

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Data Science in .NET with Polyglot Notebooks*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/978-1-83588-296-2>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

